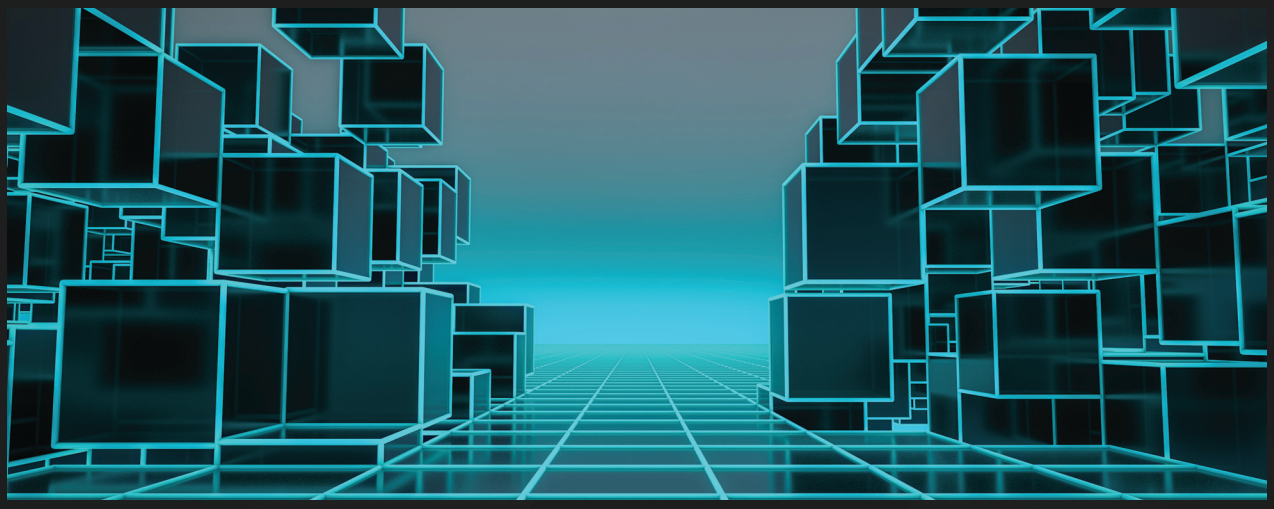
packt



1ST EDITION

# Data Engineering with Azure Databricks

Design, build, and optimize scalable data pipelines and analytics solutions with Azure Databricks

DMITRY FOSHIN | DMITRY ANOSHIN
TONYA CHERNYSHOVA | SERGII VOLODARSKYI

# Data Engineering with Azure Databricks

Design, build, and optimize scalable data pipelines and analytics solutions with Azure Databricks

**Dmitry Foshin**
**Dmitry Anoshin**
**Tonya Chernyshova**
**Sergii Volodarskyi**

‹packt›

# Data Engineering with Azure Databricks

Copyright © 2026 Packt Publishing

*All rights reserved*. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.
Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the authors, nor Packt Publishing or its dealers and distributors, will be held liable for any damages caused or alleged to have been caused directly or indirectly by this book.
Packt Publishing has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, Packt Publishing cannot guarantee the accuracy of this information.

**Portfolio Director:** Sunith Shetty
**Relationship Lead:** Sathya Mohan
**Project Manager:** Aniket Shetty
**Content Engineer:** Saba Umme Salma
**Technical Editors:** Sumant Jadhav
**Copy Editor:** Safis Editing
**Indexer:** Pratik Shirodkar
**Proofreader:** Saba Umme Salma
**Production Designer:** Shantanu Zagade
**Growth Lead:** Varun Nair

First published: April 2026
Production reference: 1230426

Published by Packt Publishing Ltd.
Grosvenor House
11 St Paul's Square
Birmingham
B3 1RB, UK.

ISBN 978-1-80610-637-0
www.packtpub.com

*To my radiant and beautiful wife Sveta, whose love is the quiet force behind every word on these pages — you are my anchor, my light, and my greatest adventure.*

*To Vasily — the smartest and kindest soul I have ever known. Your wisdom humbles me, your kindness inspires me, and I am endlessly proud to call you my son.*

*To Anna — whose grace upon the stage takes my breath away every single time. You dance not just with your feet, but with your whole heart. The world is your stage, my dear, and you were born to shine on it.*

*To Michael— my force of nature. Your optimism is contagious, your strength is boundless, and your joy is the kind that fills an entire room and spills out into the street. Never lose that light.*

*This book would not exist without your unwavering support, your patience, your laughter, and your absolute dedication to ensuring that every last kopek of my honorarium was wisely — and deliciously — invested in ice cream and roasted chicken.*

*I regret nothing. It was worth it.*

*All my love, always — Your husband and father – Dmitry Anoshin.*

*I would like to thank my family and friends for their continuous support and encouragement throughout my career and the writing of this book. A special thank you to the data engineering community and the incredible teams I've had the privilege to work with, whose collaboration and shared knowledge have been a constant source of inspiration. - Tonya Chernyshova*

*I would like to thank my wife Maria for her endless patience and support — especially during the late-night writing sessions. A special shout-out to my little son Miron, who at 1.5 years old has already mastered the art of closing my laptop at the worst possible moment. Thank you to my parents, my brother Ilia, and the rest of my family for their unwavering support, and my friends for always cheering me on. - Dmitry Foshin*

*To my beautiful, smart, and caring wife, Kazuki. I could not have done this without your support and patience.*

*To my curious, kind, and joyful daughter, Maya, you are only five, yet already so thoughtful and bright. Shine in whatever you choose to do and remember that practice makes you better.*

*To my parents and brother, who always support me.*

*You are my inspiration.*

*A loving husband and father, son and brother, Sergii Volodarskyi*

# Contributors

## About the authors

**Dmitry Foshin** is a lead data engineer with over 13 years of experience in IT and data, specializing in Azure, Databricks, Microsoft Fabric, with a recent focus on AI/ML data infrastructure. Throughout his career, he has designed and built scalable data platforms for global corporations across FMCG, jewelry, mining, and fintech industries, deploying solutions across 30+ markets. Dmitry has led globally distributed teams of data and BI engineers. He enjoys sharing his experience and is a coauthor of several tech titles, including the Azure Data Factory Cookbook and Jumpstart Snowflake. Based in picturesque Portugal, when not diving into data Dmitry is keen on sports and is an avid cyclist, yacht skipper, and cross-continental swimmer.

**Dmitry Anoshin** is a data engineering leader, serial entrepreneur, published author, educator, and space technology pioneer with over 15 years of experience across business intelligence, cloud analytics, big data, and machine learning — spanning North America and Europe.

Dmitry began his career in Moscow as a BI engineer working with Oracle Data Warehouse, Informatica, and SAP Business Objects, before joining Teradata. His journey then took him from Moscow to Montenegro to Canada, where he spent five years at Amazon across Alexa AI (Boston) and Customer Behavior Analytics (Seattle), followed by four years at Microsoft working on Xbox and Azure Data & AI — contributing directly to Azure Synapse, Microsoft Fabric, and Azure Databricks.

In 2017, he founded Rock Your Data, a Canadian cloud analytics consultancy, and later launched Surfalytics — a global, community-driven learning platform helping data professionals at every level launch and grow their careers. He teaches Cloud Computing at the University of Victoria, leads data communities with over 18,000 members, and speaks regularly at major conferences across North America.

Most recently, Dmitry founded Player One Space — a space tech startup building a LEO satellite network to deliver connectivity and real-time telemetry insights to remote assets worldwide, from pipelines to ships to mines.

He is the author of 8 books on analytics, including Jumpstart Snowflake, Azure Data Factory Cookbook (1st & 2nd editions), Tableau 2019.x Cookbook, and Mastering Business Intelligence with MicroStrategy.

**Tonya Chernyshova** is a Data Engineer with over 10 years of experience designing, optimizing, and scaling data platforms and analytics solutions. Specializing in the modern data stack—including Snowflake, AWS, Azure, and dbt—she has built high-performance data pipelines and foundational data products for high-growth technology companies like Amazon, Instacart, and 1Password. Throughout her career, Tonya has been passionate about leveraging data infrastructure to enable data-driven decision-making and build safe, beneficial AI systems. She is an active contributor to the data engineering community and a coauthor of the Azure Data Factory Cookbook. Based in Redmond, Washington, she enjoys mentoring fellow engineers and taking end-to-end ownership of complex data challenges.

**Sergii Volodarskyi** is a Data Engineer working daily on the Databricks ecosystem, across both platform and product data engineering. His expertise spans the full spectrum of modern data platform delivery, from designing Lakehouse architectures and building CI/CD pipelines to extracting data from APIs and shipping analytical products that drive business decisions.

This book is a reflection of experience and best practices built up across real projects. He actively shares his knowledge with the engineering community and is a builder with a deep interest in the intersection of data, AI, and software engineering..

# About the reviewer

**Alan Augustin** is a Senior Data Engineer with over five years of hands-on experience in Azure Databricks, building and optimizing data pipelines, Delta Lake architectures, and Lakehouse solutions for clients across diverse industries. Currently at Quartile, he works extensively with PySpark, SQL, and Python on Databricks, integrating these with Azure Data Factory, Synapse Analytics, and Microsoft Fabric to deliver end-to-end data engineering workflows. He holds certifications from both Databricks and Microsoft in data engineering and analytics. He is passionate about sharing knowledge and helping fellow data engineers navigate the Azure Databricks ecosystem with confidence.

# Table of Contents

**Preface** **xxiii**

**Free benefits with your book** **xxix**

**Chapter 1: The Role of Azure Databricks in Modern Data Engineering** **1**

**The evolution of data engineering** **2**

Rise of Apache Spark • 7

**Meet databricks** **9**

Azure Databricks key concepts • 10

**Databricks and competitors** **11**

**Reference architectures and use cases** **13**

**Summary** **16**

**Chapter 2: Setting up an End-To-End Azure Databricks Environment** **17**

**Governance foundation: Unity catalog** **18**

Why unity catalog was the need of the hour • 18

Metastore • 21

**Administrative access and workspace provisioning** **22**

Create an admin user • 25

Set up a workspace • 26

Continue building the platform: Introducing use cases • 27

**Connecting to cloud storage** **29**

Unity catalog storage integration overview • 29

Create the storage account • 31

Create containers in the storage account • 32

Create an access connector for Azure Databricks • 33

Create a Storage Credential Using a Managed Identity • 36

*Create an external location • 36*
Catalog design considerations and creation • 37
Creating a volume: Staging location for raw files • 39
Create an external volume for staging • 41
**Bronze, silver, gold tables ..... 42**
Data quality • 45
**Identity management ..... 45**
Identity provisioning • 46
Groups • 47
Create groups and assign to workspaces • 48
*Create groups at the account level (UI) • 48*
Assign groups to workspaces (UI) • 49
Entitlements, privileges and access control • 50
*Manage entitlements • 50*
*Manage access control lists • 51*
Privileges • 51
Permission matrix • 52
*Compute • 55*
*Serverless • 55*
*Classic compute access modes • 56*
*Standard (shared) access mode • 56*
*Dedicated (Single-User) access mode • 56*
*No Isolation Shared Access Mode • 57*
*Photon • 57*
SQL Warehouses • 57
*System Tables • 58*
*Deleting resources • 58*
**Summary ..... 59**

**Chapter 3: Data Ingestion Strategies for Azure Databricks 61**

**Technical requirements ..... 62**
**Understanding batch ingestion ..... 63**

Characteristics of batch ingestion • 64
Common Batch Ingestion Scenarios • 64
**Ingesting data from Azure Storage (ADLS and blob storage) ..... 65**
Authentication methods • 65
*Service principal with OAuth 2.0 • 65*
Reading different file formats • 66
CSV files • 67
*JSON Files • 68*
*Parquet files • 69*
Incremental loading pattern • 70
Working example: ADLS batch ingestion • 72
**Using Azure Data Factory for ingestion ..... 75**
Copy activity pattern • 76
ADF to Databricks patterns • 77
**Connecting to relational databases ..... 79**
JDBC connectivity basics • 79
Setting up sample data in Azure SQL • 80
Full table load example • 81
Incremental Loading Pattern • 83
**Performance optimization tips ..... 85**
**Azure Synapse Analytics ..... 86**
**Ingesting data from REST APIs ..... 87**
Authentication methods • 87
*API keys • 87*
*Handling pagination • 88*
*Rate limiting and error handling • 89*
Working example: JSONPlaceholder API ingestion • 90
**Other data sources ..... 94**
Azure cosmos DB • 94
SFTP/FTP sources • 95
**Summary ..... 97**

**Chapter 4: Data Engineering with Apache Spark 99**

**Apache Spark architecture and execution model ..... 100**

Core components and cluster architecture • 100

Applications of spark architecture • 101

*Spark driver • 101*

*Executors • 102*

*Cluster manager • 102*

Execution model and job scheduling • 103

Memory management and storage • 107

**Memory configuration and optimization ..... 108**

Azure Databricks runtime enhancements • 109

**Optimizing spark jobs with partitioning and caching ..... 110**

Data partitioning strategies • 111

Selecting optimal partitioning strategies • 113

Caching and persistence mechanisms • 113

Caching and persistence mechanisms • 115

Advanced optimization techniques • 116

**Writing efficient PySpark and scala code ..... 117**

DataFrame API best practices • 118

SQL optimization techniques • 119

User-Defined functions and performance • 121

**Handling Large-Scale data processing with spark ..... 122**

Scaling strategies and cluster management • 122

Data skew and resource optimization • 126

Production deployment patterns • 127

**Summary ..... 128**

**Chapter 5: Building Real-Time Data Pipelines 131**

**What is Data Streaming? ..... 132**

Streaming vs. Batch Processing • 132

Common Streaming Use Cases • 134

**Types of Streaming: Real-Time vs. Near Real-Time ..... 135**

Real-Time Streaming • 135

Near Real-Time Streaming • 136

*Choosing the Right Type • 137*

**Azure Databricks and Spark Streaming Capabilities ..... 139**

What is Apache Spark Structured Streaming? • 139

Azure Databricks Streaming Features • 141

Common Azure Streaming Patterns • 142

**Building Your First Streaming Pipeline..... 144**

Setting Up Your Azure Account • 144

*Prerequisites • 144*

*Creating an Azure Event Hub • 144*

*Setting Up Azure Blob Storage • 146*

*Writing Your First Streaming Job • 147*

*Using Autoloader for File Streaming • 151*

*Autoloader vs. Event Hub • 153*

Monitoring and Debugging • 154

*Using the Databricks UI • 154*

*Common Problems and Solutions • 155*

*Performance Tips • 155*

**Advanced Streaming Concepts ..... 156**

Managing Stateful Streaming • 156

*What is State in Streaming? • 156*

*When to Use Stateful Streaming • 157*

*Stateful Operations in Spark Structured Streaming • 157*

Handling Late Data and Fault Tolerance • 157

*Watermarks: Managing Late-Arriving Data • 157*

*Checkpointing: Ensuring Fault Tolerance • 158*

*Exactly-Once Processing • 158*

Performance Optimization • 158

*Partitioning for Performance • 158*

*Caching for Performance • 159*

*Resource Tuning for Streaming • 159*

*Autoloader Optimization • 159*

**Summary ..... 160**

**Chapter 6: Working with Delta Lake: ACID Transactions and Schema Evolution 163**

**Technical requirements ..... 164**

**Introduction to delta lake: Why it matters ..... 164**

**Enabling ACID transactions for reliable data processing ..... 165**

Introduction to delta log • 165

What do ACID guarantees mean? • 168

*What role does the _delta_log play? • 168*

*When are transactions written to the Delta Log? • 168*

*Are failed operations recorded? • 169*

*How does Delta Lake handle rollbacks? • 169*

*Why is delta log the single source of truth? • 169*

*How does reading the Delta Log scale? • 169*

How Delta Lake implements consistency • 170

How Delta Lake implements isolation • 171

How Delta Lake implements durability • 173

**Schema enforcement and evolution ..... 173**

**Why schema management matters ..... 173**

Delta Lake's approach to schema enforcement • 174

*Delta Lake's approach to schema evolution • 177*

*Schema evolution: Overwrite schema approach • 178*

*Schema evolution: Automated approach • 178*

*Schema evolution best practices • 179*

**Delta versioning and time travel ..... 180**

Why time travel matters • 180

How does the Delta Lake version data • 180

*Querying historical data with the SQL API • 181*

*Creating multiple table versions • 181*

*Querying by version number • 183*

*Querying by timestamp • 183*

*Restore the table state • 184*

Retention and data lifecycle • 185

**Change data capture in Delta Lake ..... 185**

Why CDC matters for analytical use cases • 185

Delta table as a target for CDC • 186

*Approach using MERGE INTO • 187*

*Approach using AUTO CDC • 187*

*Delta table as a source of CDC • 188*

*Enabling change data feed • 189*

*How to read changes • 189*

*Streaming queries • 190*

**How VACUUM affects change data feed ..... 190**

**Summary ..... 191**

**Chapter 7: Automating Data Systems with Lakeflow Spark Declarative Pipelines 193**

**Technical requirements ..... 194**

**Introduction to LakeFlow Spark Declarative Pipelines ..... 194**

**The Challenge with Traditional Pipelines ..... 194**

Key benefits of Lakeflow SDP • 195

When to use Lakeflow SD • 196

Core concepts • 196

**Building your first Lakeflow pipeline ..... 197**

The example scenario • 197

*Setup: Preparing your data • 198*

*The complete pipeline code • 199*

*Pipeline setup: Imports and configuration • 200*

Understanding the bronze layer • 200

Understanding the silver layer • 202

Understanding the gold layer • 206

Tables vs. materialized views in LakeFlow • 209
Creating and running the pipeline • 210
*Step 1: Create the pipeline • 210*
*Step 2: Configure the pipeline • 210*
*Step 3: Start the pipeline • 211*
*Step 4: Monitor execution • 211*
*Verifying pipeline results • 213*
*Viewing data quality metrics in the pipeline UI • 213*
**Managing data quality with expectations ..... 215**
Types of expectations • 215
Implementing effective quality rules • 216
**Monitoring quality metrics ..... 217**
**Best practices for data quality ..... 219**
**Optimizing Lakeflow pipeline performance ..... 220**
Automatic optimizations • 220
Pipeline configuration • 221
Clustering strategies • 221
Optimizing joins and aggregations • 222
*Monitoring Pipeline Performance • 223*
**Summary ..... 224**

**Chapter 8: Orchestrating Data Workflows: From Notebooks to Production 227**

**Technical Requirements ..... 228**
**Using Lakeflow Jobs for Task Scheduling ..... 229**
Understanding Lakeflow Jobs Architecture • 229
Hands-on Job Creation and Configuration • 229
Advanced Job Patterns and Optimization • 231
**Integrating Databricks with Azure Data Factory and Airflow ..... 232**
**Azure Data Factory Integration Architecture ..... 232**
Hands-on ADF Pipeline Development • 233
Apache Airflow Integration Patterns • 234

Orchestration Tool Comparison and Selection • 236
**Best Practices for Modularizing Notebooks ..... 238**
Notebook Modularization Principles • 238
Hands-on Notebook Refactoring Techniques • 239
Production Notebook Patterns and Standards • 241
Comprehensive Monitoring Infrastructure • 243
Hands-on Debugging Techniques and Tools • 243
Operational Excellence and Continuous Improvement • 245
**Summary ..... 246**

**Chapter 9: CI/CD and DevOps for Azure Databricks 249**

**Technical Requirements ..... 249**
**DevOps Practices in Databricks..... 250**
**Setting Up Git Integration for Databricks ..... 250**
Branching Strategy • 252
*Our Approach: Trunk-based Development with Environment Separation • 252*
Environment Separation • 254
*Our Approach • 255*
**Infrastructure as Code with Terraform..... 255**
Prerequisites • 256
What the Terraform Project Creates • 257
*Project Structure • 257*
*Configuration • 257*
*Deploying • 258*
**Declarative Automation Bundles..... 260**
Prerequisites • 260
*Databricks CLI commands • 260*
*Project Structure • 260*
*The Bundle Configuration: databricks.yml • 262*
Resource Definitions • 263
*Source Code • 266*
Tests • 266

*Deploying • 267*
Implementing CI/CD Pipelines with Azure DevOps • 268
*Prerequisites • 268*
*Requesting Free Parallel Jobs • 269*
CI/CD pipeline creation guide • 269
*Step 1: Copy project files from the book repo • 269*
*Step 2: Generate Databricks Personal Access Tokens • 269*
*Step 3: Create Variable Groups in Azure DevOps • 269*
*Step 4: Create the Dev Pipeline • 270*
*Step 5: Create the Staging and Production Pipeline • 271*
*Step 6: Create Environments with Approval Gates • 272*
*Step 7: Set Up Branch Policies • 272*
Observability • 275
**Summary ..... 276**

**Chapter 10: Optimizing Query Performance and Cost Management 279**

**Technical Requirements ..... 280**
**Performance Tuning for Spark and Delta Lake..... 280**
The Philosophy of Optimization • 281
Advanced Storage Layer Optimization with Delta Lake • 281
File Compaction and the Small File Problem • 281
Data Skipping • 282
Z-Ordering • 283
Partitioning Strategies • 283
Generated Columns for Partitioning • 284
**Liquid Clustering: The Modern Approach..... 284**
Advanced Processing Layer Tuning with Spark • 285
*Join and Shuffle Optimization • 285*
*Tuning Shuffle Partitions • 286*
The Perils of User-Defined Functions (UDFs) • 286
**Adaptive Query Execution and Caching Techniques..... 286**
The Power of Photon • 286

Dynamic Re-optimization with Adaptive Query Execution (AQE) • 287
A Deeper Look at AQE Features • 287
Accelerating Reads with Caching • 288
Spark Cache Storage Levels • 289
*When Not to Cache • 290*
**Managing Cluster Costs and Autoscaling Strategies ..... 290**
Cluster Configuration and Sizing • 291
*Instance Families • 291*
Enhanced Autoscaling • 292
Autoscaling for Streaming • 292
Using Spot Instances for Cost Savings • 292
Spot Instance Best Practices • 293
Cost Control with Cluster Policies and Tagging • 293
**Monitoring and Debugging Performance Bottlenecks ..... 294**
The Spark UI: A Deeper Dive • 294
Navigating the Spark UI • 294
Identifying Common Bottlenecks • 294
Other Monitoring Tools • 295
Real-World Example: Debugging a Skewed Join • 295
Example: Salting a Skewed Join Key • 296
**Summary ..... 297**

**Chapter 11: Security, Compliance, and Data Governance 299**

Technical requirements • 300
**Securing the Databricks environment ..... 300**
Network security with VNet injection • 301
Encryption: At rest and in transit • 304
Workspace-Level security settings • 305
**Fine-Grained access control with Unity Catalog ..... 306**
Column masking • 307
Row-Level security with row filters • 309
Dynamic views for complex access patterns • 310

*Working example: Securing a customer analytics table • 312*
**Auditing and monitoring data access ..... 314**
Unity catalog system tables • 314
Building an audit dashboard • 315
Monitoring with Azure diagnostic settings • 317
*Setting up alerts • 317*
**Ensuring compliance with regulatory standards ..... 318**
GDPR: Right to erasure with Delta Lake • 318
HIPAA: Protecting health information • 321
Building a compliance framework • 322
**Data lineage and impact analysis ..... 324**
How the Unity Catalog captures lineage • 324
Querying lineage programmatically • 324
Impact analysis of schema changes • 325
Lineage for compliance • 326
Secure data sharing with delta sharing • 326
*Internal sharing across workspaces • 327*
*External sharing with delta sharing • 327*
**Summary ..... 329**

**Chapter 12: Machine Learning and AI on Databricks 331**

**Technical Requirements ..... 331**
**Introduction to Databricks AI/ML innovation and capabilities..... 332**
The Databricks AI/ML Stack • 333
**ML and AI Capabilities Overview ..... 333**
Traditional Machine Learning • 333
Generative AI Platform: Mosaic AI • 334
Integration and Governance • 334
*Lineage Flow Example • 335*
*Key Integration Points • 335*
**MLflow: ML Experiment Tracking and Model Management..... 335**
What is MLflow? • 335

Core Concepts • 336
*Experiment Tracking • 336*
*Model Registry • 337*
*Try It: Tracking an Experiment • 338*
Model Registry and Deployment • 339
*Registering a Model • 339*
*Managing Model Versions with Aliases • 340*
*Deploying to Model Serving • 341*
**Business Use Case Ideas ..... 342**
**Feature Store: Centralized Feature Management ..... 342**
Feature Store Core Concepts • 343
*Try It: Create and Use Features • 344*
*Core Concepts • 346*
*Try It: Forecasting with AutoML • 347*
Using AutoML Results • 348
*When to Use AutoML • 348*
**Genie: Conversational Analytics for Business Users ..... 349**
What is Genie? • 349
Example and Use Cases • 349
Mosaic AI Platform Overview • 350
Common Use Cases • 350
**What is Vector Search? ..... 351**
Creating a Vector Index • 351
*Searching the Index • 352*
*When to Use Vector Search • 353*
**What is RAG? ..... 353**
RAG vs Fine-Tuning: Which Should You Use? • 354
Building a Simple RAG Function • 354
*Adding Conversation Memory • 355*
*Evaluating RAG Quality • 355*
**AI Gateway: Governance and Cost Control ..... 356**
What AI Gateway Does • 357

Setting Up AI Gateway • 357
Using the Gateway • 358
Deployment and Monitoring • 359
*Deploying AI Agents • 359*
**Summary ..... 360**
Next Steps • 361

**Chapter 13: Unlock Access to the Code Bundle and the PDF Version 363**

**Unlock this book's free benefits in three easy steps ..... 364**

**Other Books You May Enjoy 368**

**Index 373**

# Preface

Data engineering has changed fast in the last few years. Companies now deal with more data than ever before. They need systems that can handle batch and streaming data, enforce data quality, and scale without breaking. Azure Databricks has become one of the most popular platforms for building these systems.

Azure Databricks combines the power of Apache Spark with the convenience of a managed cloud service on Microsoft Azure. It gives you Delta Lake for reliable data storage, Structured Streaming for real-time processing, and Unity Catalog for data governance. Together, these tools let you build data pipelines that are fast, reliable, and secure.

This book takes you on a hands-on journey through data engineering on Azure Databricks. You will start with the basics: setting up your environment, understanding the platform, and learning how to bring data in. Then you will go deeper into Spark, streaming, and Delta Lake. You will learn how to automate pipelines with Spark Declarative Pipelines and orchestrate workflows for production.

The second half of the book focuses on real-world operations. You will set up CI/CD pipelines, optimize query performance, manage costs, and secure your data with Unity Catalog. The final chapter introduces machine learning and generative AI on Databricks, showing where data engineering meets AI.

We wrote this book to share what we have learned from building data solutions for companies in retail, finance, healthcare, and manufacturing. Every example in this book comes from real problems we have solved. Our goal is simple: help you build data pipelines that work well and grow with your needs.

Whether you are new to Azure Databricks or already use it in production, this book will give you practical knowledge you can apply right away.

# Who this book is for

This book is for data engineers, data architects, and developers who want to build scalable data pipelines on Azure Databricks. It is also useful for data analysts and data scientists working with data engineering teams who want to better understand the platform.

You should have a basic understanding of SQL and Python before you start. Some knowledge of cloud computing and Apache Spark will help, but it is not required. We explain core concepts as we go. No prior experience with Azure Databricks is needed.

# What this book covers

*Chapter 1*, *The Role of Azure Databricks in Modern Data Engineering*, introduces Azure Databricks and its place in the modern data stack. You will learn about the platform's core features, workspaces, clusters, and notebooks, and see common use cases for building scalable data solutions.

*Chapter 2*, *Setting Up an End-to-End Azure Databricks Environment*, walks you through creating and configuring Databricks workspaces, clusters, and access controls. You will also connect to Azure data services and set up Unity Catalog for centralized data governance.

*Chapter 3*, *Data Ingestion Strategies for Azure Databricks*, explores how to load data into Databricks using batch and streaming methods. You will learn to use Auto Loader for incremental ingestion and connect to external sources such as databases, APIs, and Event Hubs.

*Chapter 4*, *Deep Dive into Apache Spark on Azure Databricks*, gives you a solid understanding of Spark architecture and how to optimize performance. You will write efficient PySpark code and learn techniques like partitioning and caching for large-scale data processing.

*Chapter 5*, *Building Real-Time Data Pipelines*, teaches you how to build streaming data pipelines with Apache Spark Structured Streaming. You will handle stateful streaming, manage late-arriving data, and implement fault-tolerant real-time processing in Databricks.

*Chapter 6*, *Working with Delta Lake: ACID Transactions and Schema Evolution*, focuses on Delta Lake as a reliable storage layer. You will learn about ACID transactions, schema enforcement, time travel, and Change Data Capture for building consistent data pipelines.

*Chapter 7*, *Automating Data Pipelines with Spark Declarative Pipelines*, shows you how to build declarative data pipelines using Spark Declarative Pipelines. You will set up data quality expectations, monitor pipeline health, and optimize SDP performance.

*Chapter 8*, *Orchestrating Data Workflows: From Notebooks to Production*, covers end-to-end data workflow management. You will use Databricks Jobs for scheduling, integrate with Azure Data Factory and Apache Airflow, and learn best practices for production deployments.

*Chapter 9*, *CI/CD and DevOps for Azure Databricks*, explores continuous integration and deployment practices. You will set up Git integration, build CI/CD pipelines with Azure DevOps, use Declarative Automation Bundles, and automate testing and deployment.

*Chapter 10*, *Optimizing Query Performance and Cost Management*, teaches you how to tune Spark and Delta Lake queries for speed and efficiency. You will learn about Adaptive Query Execution, caching strategies, autoscaling clusters, and techniques to manage cloud costs.

*Chapter 11*, *Security, Compliance, and Data Governance*, covers securing your Databricks environment. You will implement role-based access control, data encryption, and auditing. You will also use Unity Catalog for column-level security, data lineage, and compliance with regulations such as GDPR and HIPAA.

*Chapter 12*, *Machine Learning and AI on Databricks*, introduces the platform's AI and ML capabilities. You will explore MLflow for experiment tracking, Feature Store for feature engineering, AutoML for automated model training, and generative AI applications, including RAG and LLM fine-tuning.

# To get the most out of this book

To follow the examples in this book, you will need the following:

1. An active Microsoft Azure subscription (a free trial account will work for most examples)
2. An Azure Databricks workspace (Standard or Premium tier)
3. Basic knowledge of Python and SQL
4. A web browser to access the Databricks workspace and Azure portal
5. Optionally, Visual Studio Code or another code editor for local development

All code examples in this book are designed to run on Azure Databricks. You do not need to install Apache Spark or any other big data tools on your local machine. Each chapter includes step-by-step instructions for setting up the required resources.

## Download the example code files

This book includes a complete downloadable code bundle containing all the example projects and files used throughout the chapters. We recommend downloading the bundle so you can follow along smoothly and experiment with the examples.

Use the bundle as a practical starting point. Modify it, extend it, and apply what you learn by creating your own variations as you progress through the chapters.

**Get the code bundle**

**If you bought the book directly from Packt:**

1. Go to **packtpub.com**
2. Click your **profile picture** and select **Your Orders**
3. Find this book and click **Download Code**

**If you bought this book from Amazon or any other channel partner:**

1. Go to `packtpub.com/unlock` or scan the following QR code:

2. Search for this book
3. Sign up or log in to your free Packt account
4. Upload your proof of purchase and download the code bundle locally

**Usage note:** You're free to use and modify this code for personal learning and non-commercial projects.

## Download the color images

We also provide a PDF file that has color images of the screenshots and diagrams used in this book. You can download it here:`https://packt.link/gbp/978-1-80610-637-0`.

## Conventions used

This book uses several text conventions.

`CodeInText`: Indicates code words in text, database table names, folder names, filenames, file extensions, pathnames, dummy URLs, user input, and Twitter handles. For example: "Use the `spark.read.format()` method to load data from Delta Lake."

A block of code is set as follows:

```
df = spark.read.format("delta").load("/mnt/data/sales")
df_filtered = df.filter(df.amount > 100).groupBy("region").sum("amount")
df_filtered.write.format("delta").mode("overwrite").save("/mnt/data/output")
```

Any command-line input or output is written as follows:

```
databricks clusters list --output TABLE
```

**Bold**: Indicates a new term, an important word, or words that you see on the screen. For instance, words in menus or dialog boxes appear in the text like this. For example: "Select **System info** from the **Administration** panel."

> **Note**
>
> Warnings or important notes appear like this.

> **Tip**
>
> Tips and tricks appear like this.

## Get in touch

Feedback from our readers is always welcome.

**General feedback**: If you have questions about any aspect of this book or have any general feedback, please email us at `customercare@packt.com` and mention the book's title in the subject of your message.

**Errata**: Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you have found a mistake in this book, we would be grateful if you reported this to us. Please visit `http://www.packt.com/submit-errata`, click **Submit Errata**, and fill in the form.

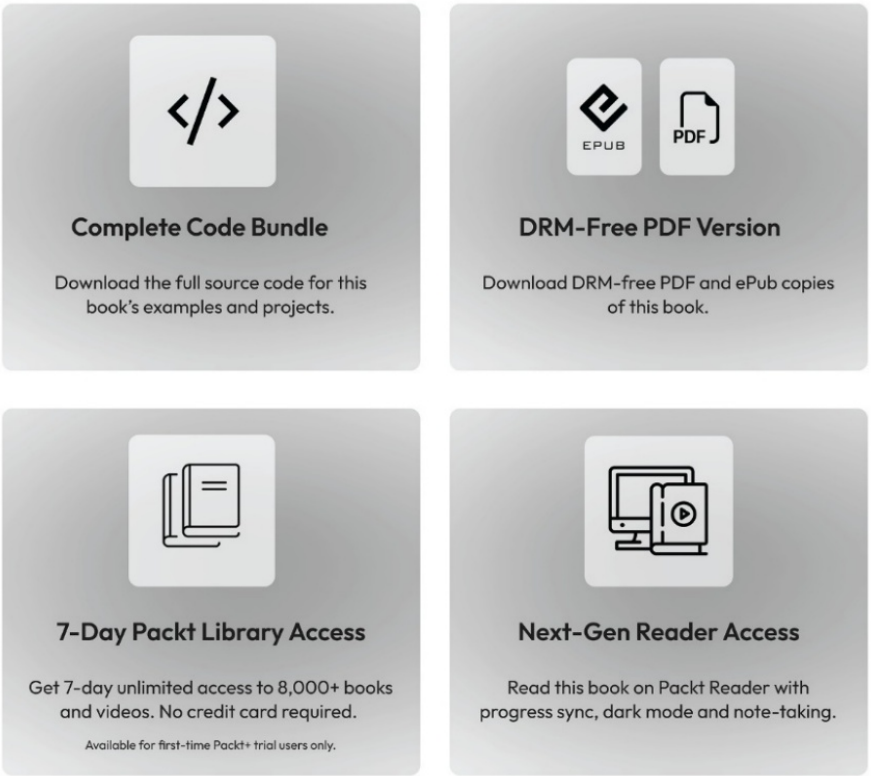
**Piracy**: If you come across any illegal copies of our works in any form on the internet, we would be grateful if you would provide us with the location address or website name. Please contact us at `copyright@packt.com` with a link to the material.

**If you are interested in becoming an author**: If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, please visit `http://authors.packt.com/`.

# Free benefits with your book

This book includes free benefits designed to support your learning and help you apply what you learn effectively. Activate them now for instant access (see the *How to unlock* section for instructions).

Here's a quick overview of what you can instantly unlock with your purchase:



## How to unlock

Scan the QR code (or go to `packtpub.com/unlock`). Search for this book by name, confirm the edition, and then follow the steps on the page.

*Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.*

# Share your thoughts

Once you've read *Data Engineering with Azure Databricks*, we'd love to hear your thoughts! Scan the QR code below to go straight to the Amazon review page for this book and share your feedback.

*https://packt.link/r/180610637X*

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

# 1

# The Role of Azure Databricks in Modern Data Engineering

In recent years, data engineering has become the fundamental pillar of analytics and **Artificial Intelligence** (**AI**). Every company in the world, from small startups to large enterprises, collects vast amounts of data and leverages it as a key competitive advantage to win customers and stay ahead of competitors.

Modern data engineering is a complex, constantly evolving process, with new products hitting the market monthly. However, fundamentally, data engineering remains the same – it makes data useful and accessible to consumers by building secure, scalable data infrastructure. There are several established patterns for data engineering system design built on public cloud infrastructure and, in rare cases, on-premises solutions.

When we design data engineering systems, we think about key areas:

- Source systems and how we want to extract data from them
- Data volume and data types
- Where we want to store data
- How we want to model data
- What tools we want to use to extract, process, and transform data
- How we want to access data and secure it
- And many other considerations

We had these questions years ago, and we will have the same questions in the future. However, with the rise of Generative AI, we are seeing significant changes in the market landscape and data engineering patterns. This means that each aspect of data engineering is being influenced by GenAI and LLMs, which are improving the quality and security of solutions.

Data Engineers are now using code assistants such as Cursor and Claude and will increasingly rely on them. GenAI allows us to build another layer of abstraction that assists during data engineering system implementation and design, providing access to best practices and automated reviews. At the same time, GenAI is becoming the new normal for organizations, and the speed of development matters more than ever, especially in the agentic AI space.

For the data engineering industry, it's crucial to follow these trends and leverage GenAI tools, because in 5-10 years, knowledge of AI tools and AI use cases will be essential for data engineering roles. Obviously, data itself has become even more important than before, as quality and speed directly influence business decisions and impact customer and product experiences.

This evolving landscape presents both opportunities and challenges. Traditional data engineering approaches, while still relevant, need to adapt to support AI workloads, real-time processing demands, and the growing complexity of data ecosystems. The bottlenecks we face today—from data silos and processing delays to integration complexities and scalability issues—require modern solutions that can bridge the gap between traditional data engineering and AI-driven futures.

This is where platforms like Databricks come into play, offering a unified approach to data engineering, data science, and machine learning that addresses these modern challenges while preparing organizations for the AI-driven future.

> **Note**
>
> **Your purchase includes a free PDF copy + code bundle**
>
> Your purchase includes a DRM-free PDF copy of this book, the code bundle, and additional exclusive extras. See the *Free benefits with your book* section in the *Preface* to unlock them instantly and maximize your learning.

## The evolution of data engineering

Before we dive deep into Azure Databricks, let's review the major milestones in data analytics history over the last several decades.

In *Figure 1.1*, you can see the key milestones in the last several decades of Databricks.

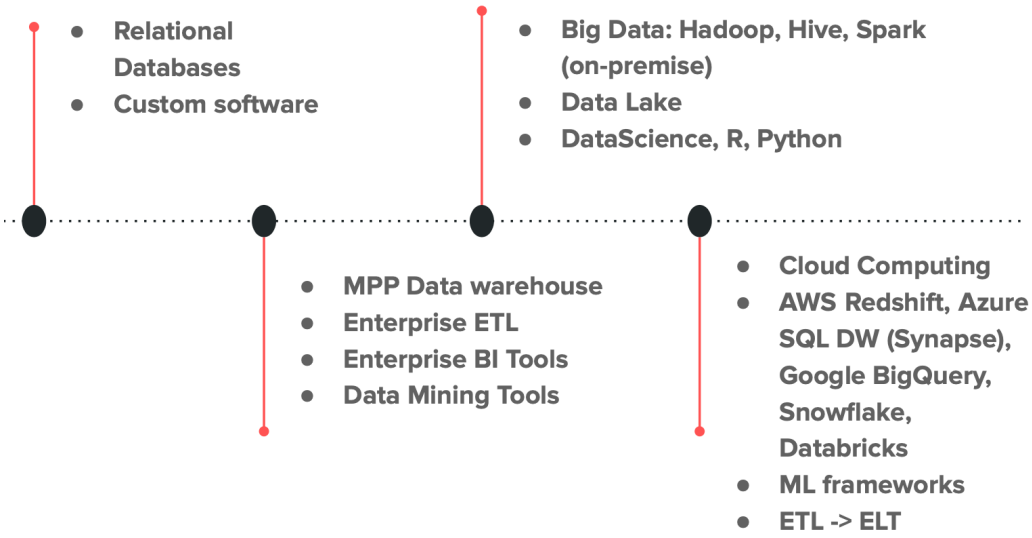



*Figure 1.1: Key milestones of data engineering evolution*

We can start our analytics journey from the relational databases that use Structured Query Language (SQL). It started in 1970, when Edgar F. Codd, then working at IBM as a computer scientist, published the paper "*A Relational Model of Data for Large Shared Data Banks*," introducing the concept of organizing data into tables (relations) using a declarative query language.

IBM began developing the System R project to demonstrate the practicality of Codd's ideas. A team of scientists who were working on the System R project developed a new query language. The language was originally called **Structured English Query Language** (**SEQUEL**) and later renamed to SQL due to a trademark issue with an aircraft company.

Donald D. Chamberlin and Raymond F. Boyce invented SQL.

The first commercial implementation of a relational database was released by Relational Software, Inc., which later became Oracle. Their product was Oracle Version 2 (there was no version 1). In the 1980s, relational databases gained widespread adoption in enterprises. Other major systems, such as IBM's DB2 and Microsoft's SQL Server, followed.

People began to use databases not only for business applications but also for analytics use cases. The idea of querying business data was very effective. However, it was challenging to build any visualizations, reports, and dashboards. Often, programmers had to create custom software applications for dashboards or data integrations on top of relational databases.

In the late 1980s, **Massively Parallel Processing** (**MPP**) data warehouses started to emerge as data volumes grew beyond what traditional single-node (**Symmetric multiprocessing** (**SMP**)) systems could efficiently handle.

One of the leaders was Teradata. It was founded in 1979 and delivered one of the first commercial MPP data warehouses. It could scale by distributing data and queries across many nodes. It exploded in popularity in the 2010s, driven by cloud-native platforms like Redshift, BigQuery, and Snowflake.

MPP data warehouses boosted a range of enterprise-grade tools, including ETL (Extract, Transform, Load), Business Intelligence, and Data Mining. These tools were expensive and were owned by big vendors such as Oracle, IBM, SAP, SAS, etc. Often, they just acquired the best products on the market.

As organizations continued to generate and store more data from business applications, websites, sensors, and user activity, the traditional data warehouse infrastructure started to show its limits. While MPP data warehouses offered significant performance improvements, they were still expensive to scale and rigid in structure. Data had to be modeled upfront and loaded through complex ETL processes before it could be queried. This led to challenges in working with semi-structured or unstructured data, such as logs, clickstreams, images, and JSON files.

In the early 2000s, a new paradigm began to emerge - Big Data. The term referred to datasets that were too large, fast, or varied for traditional systems to handle efficiently. The industry needed solutions that could store and process petabytes of data across clusters of commodity hardware, without relying on expensive proprietary systems.

A breakthrough came in 2004 when Google published a paper on MapReduce, a programming model for processing large datasets across distributed systems. Shortly after, the Hadoop project was born as an open-source implementation of MapReduce and the **Hadoop Distributed File System** (**HDFS**). This enabled organizations to build scalable, fault-tolerant data processing pipelines using affordable hardware and open-source tools.

As Hadoop gained traction, the concept of a Data Lake emerged. Unlike traditional data warehouses that required strict schema definitions and cleansing before ingestion, data lakes allowed organizations to store raw structured and unstructured data in its native format. This flexibility appealed to data engineers and scientists who wanted to explore, transform, and experiment with data without the overhead of rigid pipelines.

At the same time, Data Science began to rise as a distinct discipline. Analysts and statisticians moved beyond spreadsheets and SQL, adopting more powerful programming environments to

perform advanced analytics and machine learning. Two open-source languages stood out during this period: R and Python.

- **R**, a language designed for statistical computing, became popular in academia and among statisticians for its rich ecosystem of packages for modeling and visualization
- **Python**, a general-purpose language, gained momentum for its ease of use, readability, and growing collection of data-focused libraries like Pandas, NumPy, Scikit-learn, and Matplotlib

The combination of open-source tools, distributed computing frameworks, and flexible storage architectures gave rise to a new generation of data platforms capable of handling high-volume, high-variety data sources. Companies started building large-scale data infrastructure using Hadoop, Spark, Kafka, and other components of the open-source ecosystem. These systems formed the backbone of many on-premises Big Data platforms before the shift to public cloud services began.

As data volumes and variety continued to grow, and on-premises Big Data stacks became more complex and costly to maintain, the industry entered a new phase: the Cloud Computing era. This marked a significant shift in how organizations approached data storage, processing, and analytics. Instead of managing their own hardware and infrastructure, companies began moving workloads to public cloud platforms such as **Amazon Web Services** (**AWS**), **Microsoft Azure**, and **Google Cloud Platform** (**GCP**).

One of the first breakthroughs in cloud-based analytics came in 2012, when Amazon Redshift was launched. It was a fully managed, cloud-native MPP data warehouse that allowed organizations to scale compute and storage independently, with no hardware to maintain. Redshift significantly reduced the cost and complexity of building and operating data warehouses.

This shift to the cloud brought about key changes:

- **Elastic scalability**: Resources could be provisioned on demand, without upfront infrastructure investments
- **Serverless options**: Tools like BigQuery (GCP) and Azure Synapse Analytics allowed users to run massive SQL queries without managing clusters
- **Unified ecosystems**: Cloud providers offered native integrations between data warehouses, data lakes, storage, machine learning, and streaming services

This new environment catalyzed the rise of Data Engineering as a modern profession. The traditional roles of ETL developers and Big Data engineers began to merge into a single role focused on designing, building, and maintaining scalable data pipelines in the cloud. Data

engineers now need to master tools such as dbt, Airflow, and Spark on Databricks, as well as infrastructure-as-code tools such as Terraform.

At the same time, new vendors entered the scene, building cloud-native platforms from the ground up. One notable was Snowflake, founded in 2012. Unlike legacy tools, Snowflake was designed specifically for the cloud, offering features like:

- Instant, independent scaling of compute and storage
- Seamless data sharing and collaboration across organizations
- Unified support for both structured and semi-structured data

The popularity of **machine learning** (**ML**) also surged. Open-source frameworks like TensorFlow, PyTorch, and XGBoost became widely adopted, while cloud platforms introduced their own ML services (e.g., Amazon SageMaker, Azure ML, Google Vertex AI). Companies began investing heavily in data science and ML, aiming to extract predictive insights from their data.

This era also saw a gradual decline in traditional on-premises ETL and BI tools. Instead, organizations embraced self-service analytics platforms such as Looker, Tableau, Power BI, and Mode, empowering business users to explore and visualize data independently.

Vendors began offering end-to-end platforms that unified analytics, data engineering, machine learning, and data science into a single ecosystem. This convergence reflected the growing demand for real-time, scalable, and collaborative data-driven decision-making across entire organizations.

As this book is being written, we are witnessing the emergence of a new era - driven by Generative AI, **Large Language Models** (**LLMs**), and autonomous agents. These technologies are rapidly transforming the data and analytics landscape.

Analytics tools are beginning to integrate generative capabilities, allowing users to generate code, build queries, summarize insights, and even create entire dashboards through natural language. At the same time, businesses are actively developing new use cases that leverage these tools to gain a competitive advantage, from automating repetitive data tasks to enhancing decision-making with AI-powered recommendations.

This shift is not just about technology - it's about a new way of interacting with data. The boundary among data analysts, engineers, and business users is becoming more fluid as LLMs lower the barrier to working with complex data systems. We are entering a future where analytics, AI, and automation converge, opening new possibilities for how organizations understand and act on data.

## Rise of Apache Spark

Let's step back in our history and learn more about Apache Spark. It will help us better understand Databricks.

While Hadoop MapReduce enabled large-scale data processing, it suffered from slow performance, high latency, and complex programming models. Each job involved writing intermediate results to disk between every step, making iterative computations (such as machine learning or interactive queries) inefficient.

To address these issues, a new system was born: Apache Spark.

The story of Spark begins in 2009 at the AMPLab at UC Berkeley, where a team of researchers —including Matei Zaharia, the creator of Spark—set out to build a faster and more flexible data processing engine. In 2010, Zaharia published the paper "Spark: Cluster Computing with Working Sets," introducing a new abstraction called the Resilient Distributed Dataset (RDD). RDDs allowed Spark to keep data in memory across multiple stages of computation, dramatically improving performance compared to MapReduce.

Spark was designed with speed, simplicity, and versatility in mind. It was open-sourced in 2010, donated to the Apache Software Foundation, and officially became a top-level Apache project in 2014. The name "Spark" symbolized its goal: to provide a "spark of speed" in big data processing—a stark contrast to the slow-burning performance of MapReduce.

Key Problems Spark Solved Compared to Hadoop MapReduce:

- **In-memory processing**: Spark could cache datasets in memory, reducing the need to write and read from disk between every step
- **Ease of use**: Spark introduced concise APIs in Scala, Java, Python, and later R, making it easier to write data pipelines
- **Support for multiple workloads**: Spark unified batch processing, streaming, machine learning, and SQL into a single engine
- **Faster execution**: For many iterative and interactive workloads, Spark ran up to 100x faster than Hadoop MapReduce

Core Components of Apache Spark:

- **Spark Core**: The foundation of Spark, providing basic functionalities like task scheduling, memory management, fault recovery, and I/O
- **Spark SQL**: A module for working with structured data using SQL or the DataFrame API
- **Spark Streaming**: Enables scalable, high-throughput, fault-tolerant stream processing of live data

- **MLlib**: A scalable machine learning library with algorithms for classification, regression, clustering, and recommendation
- **GraphX**: A library for graph processing and computation
- **Structured Streaming**: A higher-level streaming API introduced later, unifying batch and streaming under one execution model

Apache Spark quickly became the de facto standard for distributed data processing. It was adopted widely by both enterprises and cloud providers. It became the core of platforms like Databricks—a company founded in 2013 by Spark's original creators, including Zaharia, to commercialize and evolve the Spark ecosystem.

In *Figure 1.2*, I've illustrated a simple view of Apache Spark (shown as the Spark Engine). The core idea is straightforward:

- The primary goal of Spark is to process data in memory, boosting speed and efficiency
- Spark can read raw data from a variety of sources - files (Parquet, CSV, ORC), APIs, JDBC, and more
- It transforms this data using its distributed engine and writes results back to the data lake or storage
- Spark also enables users to create tables on top of Data Lake files, registering them in a metastore or catalog—making them available for querying
- These tables and transformations can be accessed via SQL (Spark SQL), Python (PySpark), Scala, or other supported languages

This captures Spark's essence: flexible ingestion, high-speed in-memory processing, and seamless integration with table metadata and various programming interfaces.

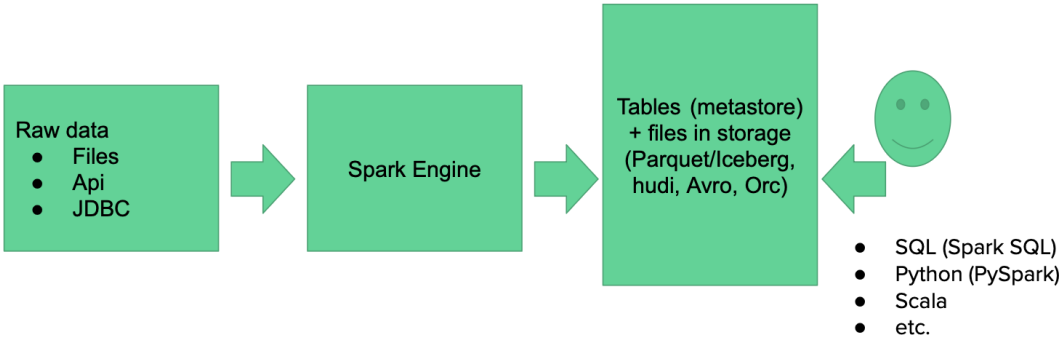



*Figure 1.2: Apache Spark in a nutshell*

Initially, Apache Spark was designed to work with Hadoop clusters and quickly proved its efficiency compared to engines like Hive, Impala, and Pig. It could run both on-premises and in

the public cloud, leveraging existing Hadoop compute (YARN/HDFS) while delivering significantly faster performance.

For data developers and analysts, Spark fits seamlessly into modern workflows. It integrates naturally with Jupyter Notebooks, enabling interactive exploration, visualization, and pipeline development in Python (via PySpark), Scala, SQL, and R. Integration with Git—using tools like Jupytext or platform-native features—allows teams to version-control notebooks and Spark scripts, embracing the practice of "data-as-code".

# Meet databricks

In 2013, a group of UC Berkeley researchers—including Ali Ghodsi, Matei Zaharia, Ion Stoica, Reynold Xin, Andy Konwinski, Patrick Wendell, and Arsalan Tavakoli—founded Databricks in San Francisco.

Their goal was simple: bring the power of Apache Spark to enterprises in the cloud, making it easier, faster, and more collaborative.

Databricks didn't just package Spark. It evolved it into a unified Lakehouse platform (a blend of data warehouse and data lake) with enterprise-grade features:

- **Automated cluster management**: No more manual spinning up Spark clusters. Infrastructure is provisioned and managed automatically, whether on AWS, Azure, or GCP
- **Unified workspace and collaboration**: It combines notebooks, version control (Git integration/Re), jobs, and visualizations into a single interface, ideal for data engineers, scientists, and analysts working together
- **Delta Lake & Delta Engine**: Databricks created Delta Lake, an open-source storage layer that adds ACID transactions, reliable schema enforcement, and time travel on top of cloud data lakes. The Delta Engine, introduced in 2020, dramatically improved query performance
- **Databricks SQL**: Launched in late 2020, this SQL analytics service enables data analysts to run BI-style queries directly on data lakes and integrate with popular visualization tools like Tableau, Looker, and Power BI
- **MLflow & feature store**: The platform includes MLflow for managing the machine learning lifecycle-tracking experiments, models, and deployments, as well as a centralized feature store for sharing and reusing ML features across teams
- **Generative AI & LLM integration**: Databricks continues to innovate with tools like Mosaic AI, built-in LLMs, vector search, and model serving capabilities, alongside open-sourcing its own DBRX model in 2024

The company grew rapidly alongside cloud adoption and AI/ML demand, attaining a 60% annual revenue growth and scaling to over 12,000 customers by early 2025. It also announced a massive $10 billion funding round in late 2024, raising its valuation to $62 billion. Source: `https://www.databricks.com/company/newsroom/press-releases/databricks-deepens-san-francisco-investment-new-office-and-multi`.

With its cloud-native, unified Lakehouse architecture, automatic infrastructure, and deep integrations across data engineering, analytics, and AI, Databricks has transformed Spark into a fully managed enterprise platform, setting the stage for modern data and AI workflows.

## Azure Databricks key concepts

In November 2017, Microsoft and Databricks launched Azure Databricks as a first-party service on Azure, integrating the collaborative and high-performance capabilities of Apache Spark directly into the Azure ecosystem. It simplified large-scale data and AI workflows with one-click setup, seamless integration with Azure services like Entra ID (ex. Active Directory), Data Lake Storage, Synapse Analytics (ex. SQL Data Warehouse), Cosmos DB, and Power BI, with enterprise-grade security.

Let's highlight key concepts and components:

1. **Hybrid Control Plane & Compute Plane**:
    - The control plane (managed by Databricks) hosts the workspace UI, metadata, and orchestration
    - The compute plane runs Spark clusters within your own Azure subscription. You can choose between classic clusters provisioned in your VNet and serverless compute, where the infrastructure is abstracted away.
2. **Azure-Native Setup**:
    - When you create a workspace, Azure Databricks deploys a managed resource group with a VNet, security groups, a storage account, and a Databricks "appliance"—while you keep control over VM size (F, M, D-series) and network configuration. All metadata is stored in a geo-replicated Azure SQL Server.
3. **Interactive Collaboration**:
    - Shared interactive notebooks provide real-time collaboration, integrated debugging, Spark job monitoring, and pre-installed Python/R machine-learning libraries—all within an Azure-native context
    - Git and Repos support means you can manage code and notebooks as versioned "data-as-code."

4. **Delta Lake & Lakehouse Foundation**:
    - Built on Delta Lake, Azure Databricks ensures ACID transactions, schema enforcement, and time travel over data lake files
    - Patterns like Auto Loader, Lakeflow pipelines, and SQL Warehouses support scalable ingestion, ETL, streaming, and BI workloads
5. **SQL Warehousing**:
    - Dedicated SQL endpoints provide BI-like query performance over data lakes. These integrate seamlessly with Power BI and other tools.
6. **Advanced Governance via Unity Catalog**:
    - Introduced in 2023, Unity Catalog brings centralized schema management, fine-grained access control, lineage tracking, and data discovery—all with ANSI SQL syntax across tables, views, and AI assets
7. **ML & AI Integration**:
    - Azure Databricks includes ML tools like MLflow and a feature store, plus built-in support for LLMs, Mosaic AI, model serving, and vector search

According to Microsoft, in 2024, Azure was deployed by approximately 348,000 organizations worldwide, ranging from startups to large enterprises. 95% of Fortune 500 companies use Azure in some capacity—whether for infrastructure, platform services, analytics, or identity/access management.

Almost all of these organizations are Microsoft Analytics shops, and many of them use Azure Databricks, Azure Synapse, or Microsoft Fabric. Based on my personal experience, Azure Databricks is the best available product for the Azure environment. Synapse and Fabric, while capable, have not yet reached the level of advancement that Azure Databricks provides.

## Databricks and competitors

In the 2024 Gartner Magic Quadrant for Cloud Database Management Systems, Databricks is recognized as a Leader, competing with major cloud providers such as AWS, Google, Microsoft, and Oracle, as well as other data platform vendors including Snowflake, MongoDB, and IBM.

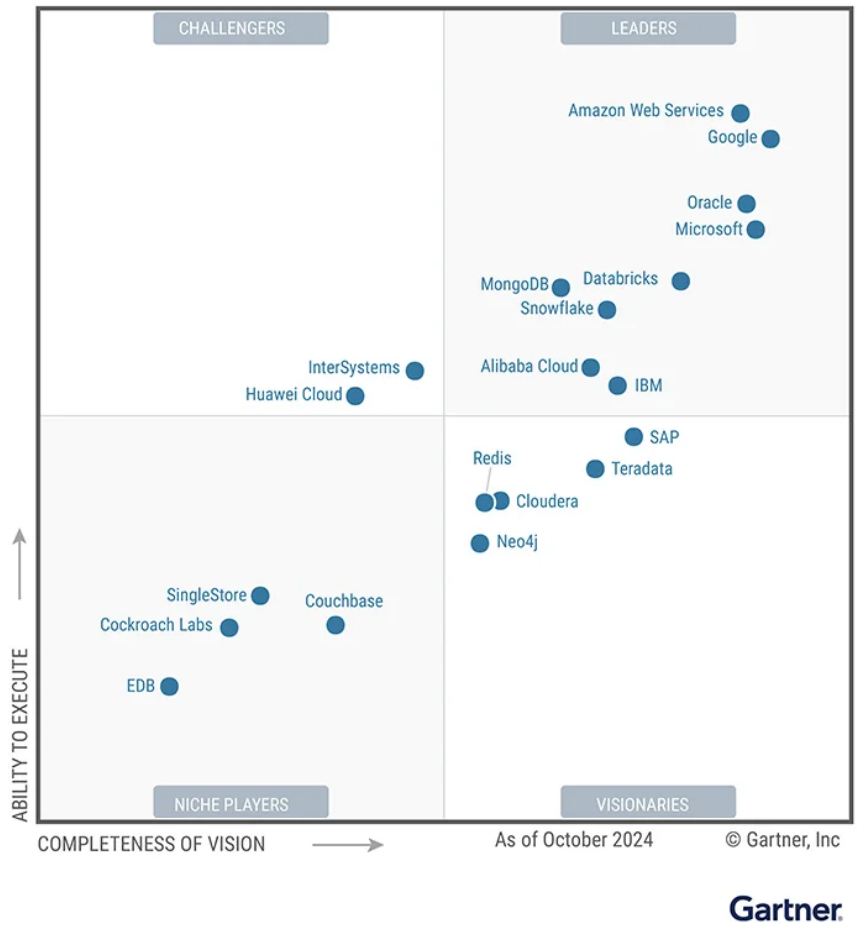



*Figure 1.3: 2024 Gartner Quadrant*

Databricks is a leader. However, let's review the closest competitors and their pros and cons in plain English.

We will focus on traditional Data Warehouse/Data Lake (Lake House) use cases. The closest independent competitor is Snowflake Data Cloud.

Snowflake is a managed data warehouse platform with rich functionality. It also decouples storage and compute; however, it stores data in its own format and applies its own performance optimization technique, also known as micro partitioning. It is truly a Data Warehouse-as-a-Service. Data engineers can ingest data, and it should "magically" distribute. The primary language for working with the platform is SQL. However, in competition with Databricks, Snowflake provides Snowpark, a Python DataFrame language. Moreover, it allows

users to work with container services and LLMs and to build data apps using Streamlit (a Python web app framework). The bottom line: it feels like an SQL data warehouse and is easy to start with, but hard to move out of (i.e., vendor lock-in).

Public cloud providers like AWS, Azure, and Google Cloud also have their own products for competing with Databricks:

1. In AWS, we have hosted or serverless options:
    - **Glue**: Managed Apache Spark
    - **Amazon Redshift**: MPP data warehouse like Snowflake idea, but much worse
    - **Elastic MapReduce**: Managed Hadoop with the option to deploy Apache Spark
2. In Azure, the primary platform would be legacy HDInsights (managed Hadoop) or Azure Synapse Analytics, which has:
    - **Dedicated Pools**: A kind of MPP data warehouse
    - **Spark pools**: Spark clusters
    - **Serverless SQL**: Pay-per-query SQL engine
3. In GCP:
    - **BigQuery**: Fully serverless, scalable MPP data warehouse (comparable to Snowflake; excellent for analytics)
    - **Dataproc**: Managed Spark and Hadoop clusters (more customizable than Glue or EMR)
    - **Dataflow**: Fully managed serverless stream and batch data processing (Apache Beam)
    - **Composer**: Managed Apache Airflow for orchestrating data pipelines

As you can see, there are a bunch of tools available at your disposal to build data analytics solutions. For some reason, I didn't highlight the AI and ML use cases much. As you might guess, public cloud vendors offer many tools for ML and Generative AI, such as AWS BedRock, GCP Vertex AI, Azure ML, and Azure OpenAI. These are building blocks for analytics solutions.

# Reference architectures and use cases

Now, we can review common Databricks reference architectures built from building blocks. All these blocks could be leveraged only by Databricks or replaced with alternatives.

One of the biggest advantages of the Databricks platform is its true unification, allowing you to combine workloads such as Data Engineering, ML, AI, and BI. Moreover, it allows you to leverage Software Engineering and DevOps approaches.

In the Azure documentation, we can find the best reference architecture for the Data Intelligence Platform: `https://learn.microsoft.com/en-us/azure/databricks/lakehouse-architecture/reference`.

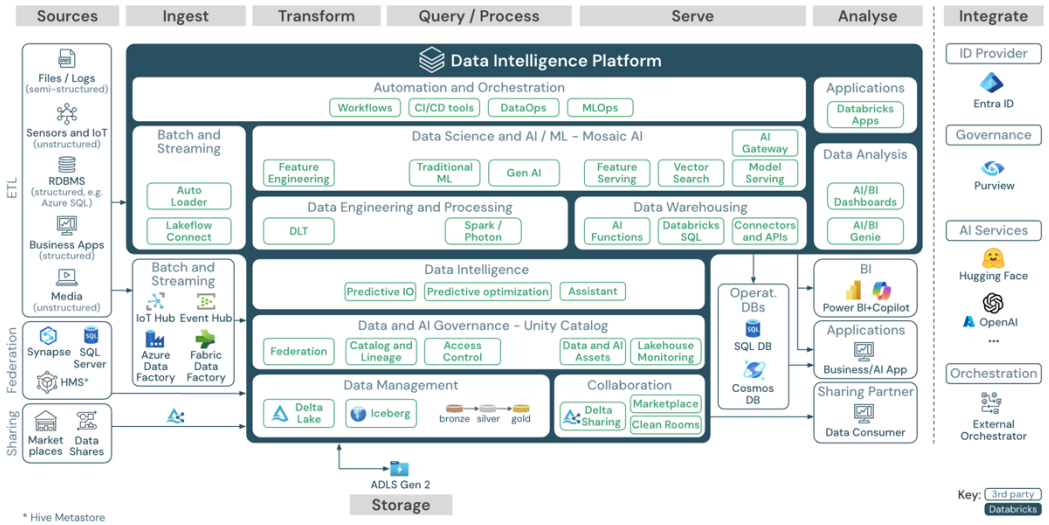



*Figure 1.4: Azure Databricks Reference Architectures*

> **Note**
>
> **Download the color images**
>
> Your purchase includes a color, DRM-free PDF copy of this book, ideal for viewing color images, screenshots, and diagrams. Refer to *Free benefits with your book section* at the end of the *Preface* to unlock your PDF copy.

Data Intelligence Platform is a buzzword. You can completely ignore it. You can use either the Data Platform or the Data Lake. In some cases, when you are comfortable with the Lakehouse approach, this will work too. Lakehouse is also a Databricks term, i.e., another buzzword.

In this diagram, multiple scenarios tailored for the Azure cloud are covered:

- **Data Ingestion**: Efficiently load raw data from diverse sources, including files, message streams, APIs, and databases, into the lakehouse architecture. Utilize Auto Loader for automated file processing or leverage partner connectors to handle both batch and real-time data ingestion seamlessly.
- **ETL/ELT Processing and Orchestration**: Transform and cleanse ingested data at scale using Spark-based pipelines or Spark Declarative Pipeline. Structure your data into

medallion architecture layers (Bronze, Silver, Gold) within Delta Lake to create curated datasets optimized for downstream consumption. Orchestration could be performed using Lakeflow Jobs (e.g., Databricks DLT jobs) or Azure Data Factory.

- **Streaming and Change Data Capture**: Process real-time data streams and change feeds from sources like Kafka, Azure Event Hubs, or database CDC feeds. Leverage Spark Structured Streaming and Spark Declarative Pipelines with built-in CDC capabilities for continuous data processing.
- **Machine Learning and AI**: Build, train, and deploy machine learning models using distributed Spark MLlib or integrate deep learning frameworks like TensorFlow and PyTorch. Track experiments and manage model lifecycles using MLflow for comprehensive ML operations.
- **Generative AI and AI Agents**: Accelerate generative AI development using Mosaic AI platform features, including built-in foundation models, vector search capabilities, and AI agent frameworks to build intelligent applications, chatbots, and autonomous systems on your data.
- **Business Intelligence and Analytics**: Query and visualize data using Databricks SQL warehouses with optimized compute or integrate seamlessly with external BI tools like Power BI and Tableau to create dashboards and reports directly over Delta Lake storage.
- **Data Governance and Cataloging**: Implement comprehensive data governance using Unity Catalog for centralized metadata management, fine-grained access control, automated data lineage tracking, and schema evolution across all data and AI assets.
- **Secure Data Sharing**: Share datasets, models, dashboards, and collaborative notebooks securely across teams and external partners using the Delta Sharing protocol, enabling data collaboration without physically moving or duplicating data.

Depending on your needs, you can use Azure Databricks for a range of use cases.

Historically, Databricks was positioned as a platform for building data lakes or lakehouses. Early marketing emphasized its strengths in handling flexible, unstructured datasets. As Databricks evolved—especially with the introduction of the Delta Lake format and its native SQL interface—it began addressing more traditional data warehouse workloads. Unity Catalog now supports data governance, access control, and schema management in ways that mirror conventional data warehouses.

Yet at its core, Databricks remains true to Spark's original philosophy: process data by reading from sources and writing to targets. Whether you're performing transformations, creating tables on Delta files, or running SQL queries, the underlying flow is consistent across all Spark under the hood.

## Summary

In this chapter, we explored the key milestones in the analytics landscape and delved into the rise of Apache Spark, which ultimately led to the founding of Databricks. We reviewed major leaders and competitors, examined the key features of Azure Databricks, and discussed its primary use cases. In the following chapters, we'll explore these concepts in greater depth.

## Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to `packtpub.com/unlock`). Search for this book by name, confirm the edition, and then follow the steps on the page.

*Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.*

# 2

# Setting up an End-To-End Azure Databricks Environment

Modern intelligence platforms include multiple capabilities such as data ingestion and processing, business intelligence, orchestration, CI/CD, machine learning, and AI. Bringing these capabilities together into a single, consistent platform requires coordination across shared data, identity, and security controls. Each component must remain usable across teams and workloads under centralized governance.

Azure Databricks addresses this challenge by providing a unified platform that tightly integrates all components with Azure services, including storage, identity, and networking. Features such as Unity Catalog, Microsoft Entra ID integration (formerly Azure Active Directory), flexible compute options, and built-in observability and cost control enable consistent governance and security across workspaces, data assets, and workloads. This integration, delivered as a **Platform-as-a-Service** (**PaaS**), eliminates the operational overhead of managing infrastructure, allows platform components to operate together as a unified environment, and enables teams to focus on data and analytics rather than infrastructure management.

The initial design decisions, setup, and configuration of platform components determine how easily the environment can be managed and extended as usage grows.

In this chapter, you will learn how to set up an end-to-end Azure Databricks environment, from region selection and workspace provisioning to data governance and access control with Unity Catalog. This chapter walks through the core components of the Databricks Data Intelligence Platform, from governance and storage to compute and cost management, showing how they fit together to form a scalable data environment.

By the end of this chapter, you should be able to:

- Understand key considerations when selecting an Azure region for Databricks deployment
- Set up Unity Catalog as the central governance and security layer across all workspaces
- Connect Databricks to Azure Data Lake Storage using storage credentials and external locations
- Create managed and external tables and volumes to organize data effectively
- Establish user groups and permissions following best practices for identity federation
- Evaluate and choose appropriate compute types, including serverless options
- Monitor usage and costs through Databricks system tables

Together, these steps form the groundwork for a robust, governed Databricks Lakehouse platform that future chapters will extend with ingestion, transformation, and CI/CD automation.

In the next section, we will introduce Unity Catalog early in the chapter, as it plays a vital role in establishing governance and security for the Databricks Intelligence Platform.

# Governance foundation: Unity catalog

With organizations managing an ever-growing set of assets - including data, pipelines, jobs, machine learning models, AI components, and other assets - the ability to govern these assets consistently and on a scale becomes a core platform concern. Unity Catalog emerged as Databricks' architectural response to this challenge, providing a centralized foundation for governing data and AI assets across the Databricks Intelligence Platform.

In this sense, Unity Catalog sits at the heart of the platform, forming the foundation for governance, lineage, access control, and other essential elements of the Databricks Intelligence Platform infrastructure.

## Why unity catalog was the need of the hour

Before Unity Catalog, governance in Databricks was centered around the Hive Metastore. While it served its purpose in earlier architectures, the growing complexity of platforms and the adoption of multi-workspace environments required governance changes. Understanding these challenges helps explain why Unity Catalog was introduced as a new governance paradigm rather than an extension of the old one.

Hive Metastore was the standard governance layer in earlier Databricks architectures, but it fundamentally differs from Unity Catalog in scope and capabilities. . Still, we believe it's

helpful to use it as a reference frame. This comparison helps clarify what Unity Catalog provides and why these capabilities are essential.

Well-known challenges of the Hive Metastore include:

- **Workspace-bound governance**: Governance was tied to an individual workspace and could not be shared across workspaces
- **Limited governance capabilities**: Lack of fine-grained access control, auditing, and centralized lineage
- **Narrow scope of securable objects**: Primarily standard database objects, with no native support for broader data and AI assets

Imagine an organization with many departments, each having several workspaces. You can easily end up managing hundreds of workspaces, each with its own Hive Metastore. There is no central place for managing user provisioning and access, making governance difficult to scale. Moreover, if you want to connect business intelligence tools or other downstream services to discover and explore data assets, there is no single point of integration. Unity Catalog addresses these challenges directly.

Unity Catalog was not just an incremental improvement of Hive Metastore—it represented a fundamental shift in architecture. *Figure 2.1* shows where Unity Catalog sits in relation to the Databricks workspace.

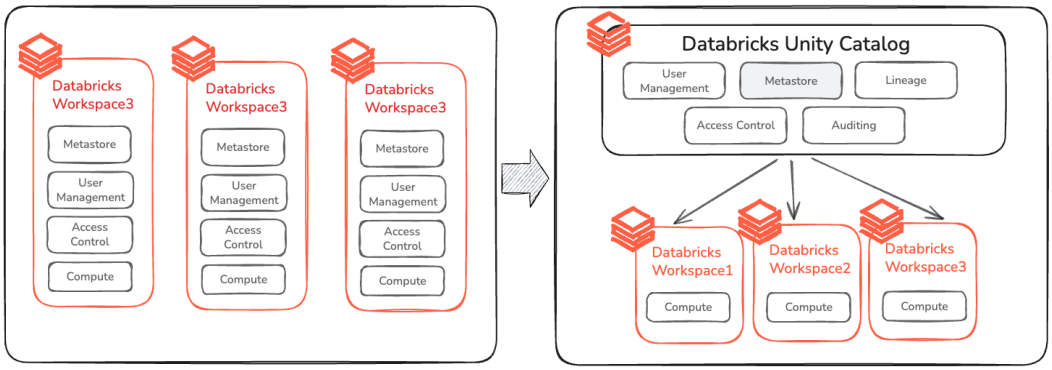



*Figure 2.1: Architectural change of Unity Catalog*

**Note**

**Download the color images**

Your purchase includes a color, DRM-free PDF copy of this book, ideal for viewing color images, screenshots, and diagrams. Refer to Free benefits with your book section at the end of the Preface to unlock your PDF copy.

As shown in *Figure 2.1*, one of the most significant architectural changes introduced by Unity Catalog is the move away from workspace-centric governance toward account-level governance. Instead of each workspace maintaining its own isolated metastore, governance is elevated to a shared, account-level service. This shift effectively addresses the core limitations of the legacy Hive Metastore, particularly when managing data and assets at scale.

Key features of the Unity Catalog:

- **Fine-grained access control**: Unity Catalog applies consistent, object-level permissions across tables, catalogs, schemas, and even cloud storage locations, overcoming the workspace-bound and table-only security model of Hive Metastore.
- **Lineage**: Unity Catalog captures end-to-end lineage across queries, notebooks, jobs, pipelines, and other assets, making it possible to understand data origins, dependencies, and the impact of change.
- **Auditing**: By centralizing user-level access and activity logs across all workspaces, Unity Catalog provides an enterprise-level of operational oversight.
- **Data discovery**: It allows users to search and explore governed assets across the whole Unity Catalog, preventing duplication and enabling reuse that was impossible with isolated metastores.
- **System tables**: Unity Catalog exposes governance and operational metadata as queryable tables for monitoring and cost control.
- **Three-level namespace for data assets**: By replacing Hive Metastore's flat, workspace-scoped structure, the catalog-schema-object hierarchy enables scalable organization of data assets aligned with domains, environments, and ownership boundaries.

As discussed earlier, instead of each workspace maintaining its own isolated metastore, governance shifts to a centralized, platform-wide service. Let's now take a closer look at the metastore itself—what it is, how it functions, and why it is foundational to Unity Catalog.

## Metastore

The metastore is Unity Catalog's top-level container for metadata and governance. It enforces access policies, ownership, and data lineage consistently across all workspaces and assets beneath it.

Within a single Databricks account, one metastore exists per Azure region and is shared across all workspaces in that region. Both the metastore and its associated storage container must reside in the same region to ensure data residency compliance.

The metastore does not store the actual data itself. Instead, it serves as a bridge:

- On one hand, it connects to data and AI assets. Technically speaking, these are securable objects such as catalogs, schemas, tables, views, volumes, connections, shares, and more. These objects are described by the Unity Catalog securable objects model shown in *Figure 2.2*.

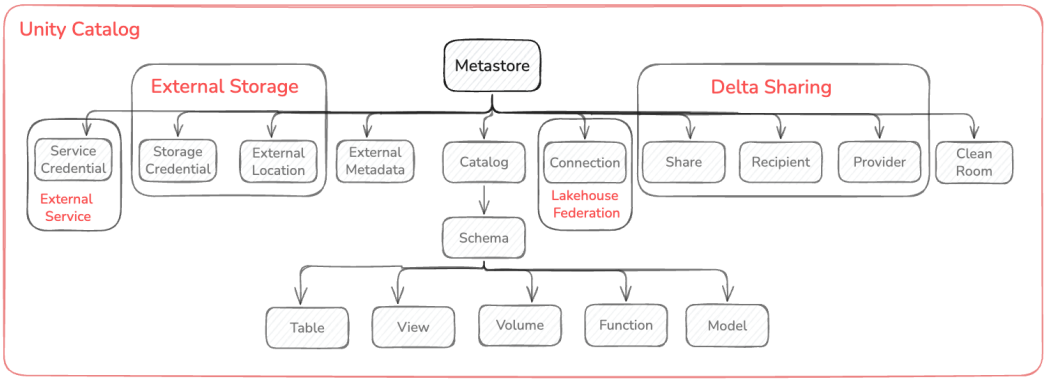



*Figure 2.2: Unity Catalog Securable Objects Model*

- On the other hand, the metastore governs access for principals - users, groups, and service principals. By default, most principals (except account and metastore admins) do not have inherent rights to access securable objects; privileges must be explicitly granted. Following the widely accepted security principle, Databricks recommends applying the principle of least privilege, granting only the access necessary for a principal to perform their role.

In this section, we will discuss and gain hands-on experience with catalogs, schemas, tables, volumes, storage credentials, external locations, and the users and groups that interact with them.

Next, we will cover the prerequisites for setting up a Databricks environment, with a focus on administrative access, and walk through workspace provisioning.

# Administrative access and workspace provisioning

Before creating a Databricks environment, several aspects must be considered. The following bullet list provides a high-level overview of Azure regions, subscriptions, Databricks accounts and workspaces, and administrative roles.

- **Databricks Subscription**: For this chapter, we recommend using the 14-day Databricks Premium trial. This trial allows you to explore features and capabilities that are not available in the Databricks Free edition, such as identity federation, access to the Databricks account console, and the ability to create connections and external locations backed by Azure ADLS.
  Remember to delete the trial workspace and associated resources after completing the chapter to avoid unexpected charges.
- **Regions**: A Databricks workspace can be provisioned only in specific cloud regions. Moreover, not all features are available in every region where Databricks can be deployed. You can check Azure Databricks regions on Microsoft Learn. It's very important to consider the entire cloud platform, not just Databricks, when selecting a deployment region. The location of upstream services (such as Azure Data Factory, Azure SQL Database, and CosmosDB) and downstream services (such as Power BI/ Fabric) relative to your Databricks workspace can affect infrastructure complexity and the throughput and efficiency of your data pipelines. Ideally, all services should be located in the same region to reduce data transfer time, network latency, and cross-regional egress costs. The complexity of the setup also affects the platform's maintainability, as each additional "hop" increases the time and effort required to troubleshoot issues.

- **Databricks account and workspace**: A Databricks account is the top-level administrative boundary that manages global settings such as identity, security, and governance across one or more workspaces. A Databricks workspace is an isolated environment where users develop notebooks, run clusters, execute jobs, and interact with data. Multiple workspaces can be created and centrally governed under a single Databricks account, as shown in *Figure 2.3*. When creating an Azure Databricks workspace from the Azure portal, the Databricks account is automatically provisioned and enabled for you.

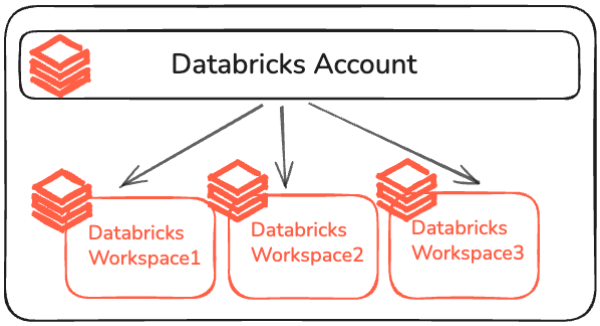



*Figure 2.3: Azure Databricks account and workspaces organization*

- **Databricks Account admin and Workspace admin**: As mentioned in the previous point, these roles have different privileges. A Workspace Admin is limited to managing workspace-specific identities, feature enablement, and access control-but only within the boundaries of a single workspace. In contrast, the Account Admin role is critical when building a platform from the ground up. It operates at the highest level and provides full visibility and control across the entire Databricks account. Access to the Databricks account console requires the Account Admin role. Workspace Admin permissions are not sufficient for account-level configuration.

*Figure 2.4* illustrates the steps to become a Databricks Account Admin.

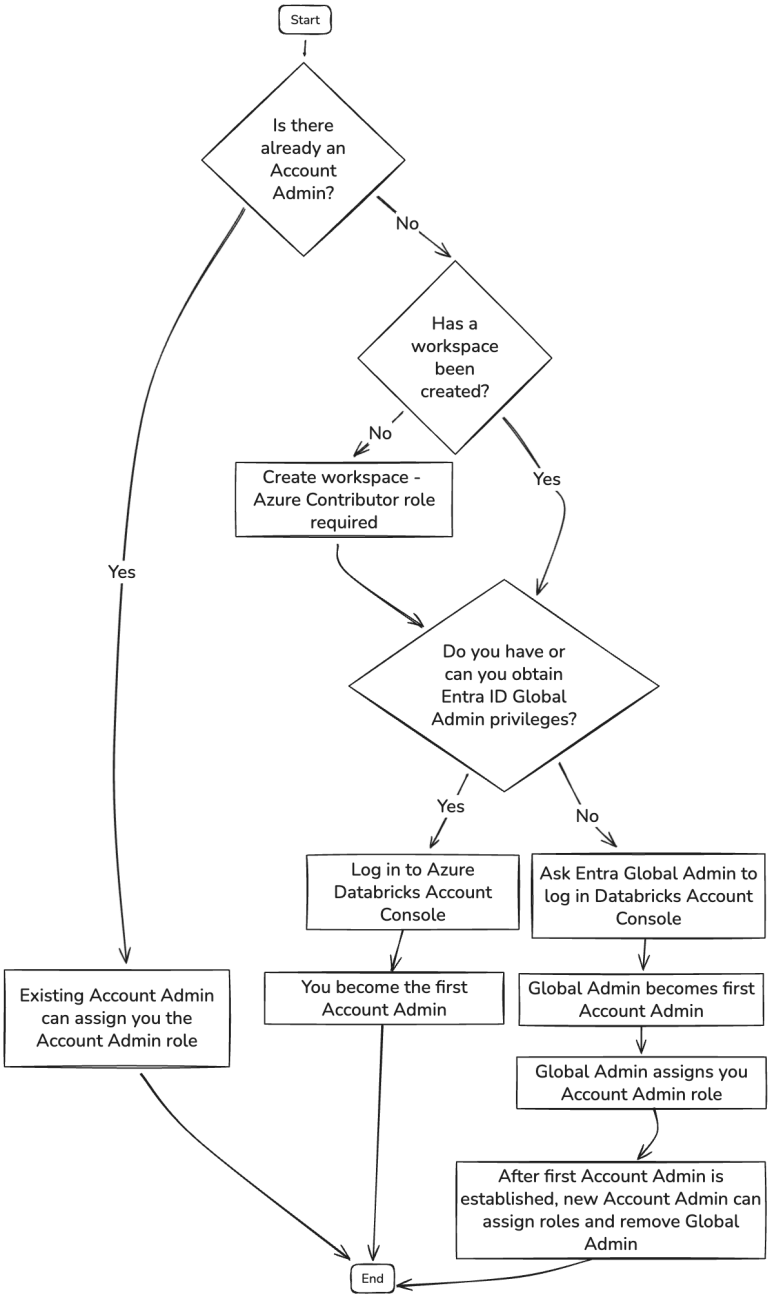



*Figure 2.4: Path to Establishing a Databricks Account Administrator*

If you are working in an enterprise Azure environment, you likely will not be able to obtain Microsoft Entra ID Global Administrator privileges. If a Databricks workspace has already been created, you will need to identify an existing Databricks Account Admin who can assign you the role. If no workspace exists yet, someone with Entra ID Global Administrator privileges must first become the initial Databricks Account Admin and then assign you the Account Admin role.

If you create a Databricks workspace in your personal Azure account, you may encounter issues accessing the Azure Databricks account console using the same identity that you use to sign in to the Azure portal. Common issues include being unable to log in to the account console or the identity not being recognized as an account admin despite having the correct Azure permissions.

To avoid this, we recommend creating an additional user identity specifically for accessing the Databricks account console, then assigning your personal identity the Account Admin role within it. (Account Admin privileges on your personal identity are also required when following the Terraform walkthrough in *Chapter 9*, *CI/CD and DevOps for Azure Databricks*.)

## Create an admin user

1. Sign in to the Microsoft Entra admin center using your existing account
2. Navigate to **Users** → **All users** → **+ New user**
3. Create the user: Enter the required names and a password
4. After creation, open the list of users and click on **User** profile → **Assigned roles** → **+ Add assignment**
5. Select and assign the *Global Administrator* role
6. Verify the role assignment under the user's *Assigned roles*

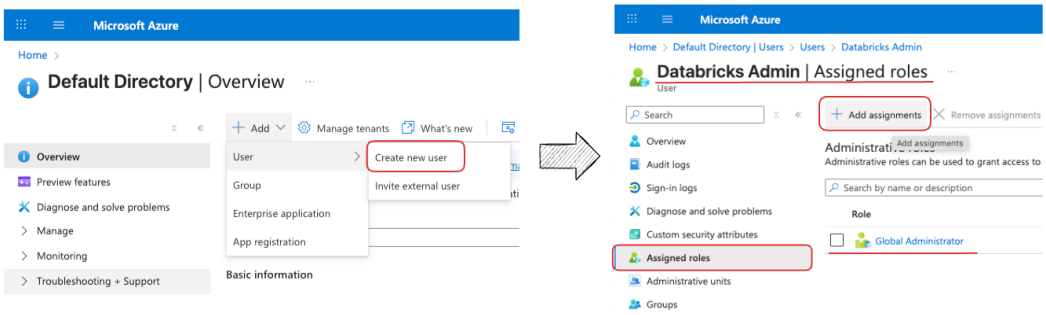



*Figure 2.5: Creating an admin user and assigning a privileged role*

Now that we have addressed the key organizational considerations, let's proceed with creating the Databricks workspace.

A Databricks workspace is an isolated environment for running workloads. In practical terms, it is where you create and execute data pipelines and jobs, build dashboards, and develop machine learning models. All development work and production workloads on the Databricks platform require a workspace.

## Set up a workspace

1. Go to Azure Portal `https://portal.azure.com`. Log in with the identity you intend to use to create the Azure Databricks Workspace.
2. In the Azure portal, search for **Azure Databricks** in the top search bar and select it from the results, then click **Create**. Alternatively, navigate to **Create a resource → Analytics → Azure Databricks → Create**.
3. Basics Tab - Fill in the Details

| Field | What to Enter |
|---|---|
| **Subscription** | Select your subscription from the dropdown. |
| **Resource Group** | Create a new RG or select an existing one. If new, base it on the general Azure naming convention: rg-analytics-dev-eastus-01 (rg = resource group, analytics = workload, dev = environment, eastus = region, 01 = instance number) |
| **Workspace Name** | dbw-analytics-dev-eastus-01 |
| **Region** | Select your region. We will use EastUS—most of the Databricks features should be enabled there. See discussion in the Regions section. |
| **Pricing Tier** | |
| **Managed Resource Group (optional)** | mrg-analytics-dev-eastus-01 |

*Table 2.1: Azure Databricks Workspace Creation Configuration*

The table above covers the Basics tab only. The remaining tabs (Networking, Tags) can be left at their defaults for this walkthrough. Production deployments may require additional configuration, particularly around networking.

About Managed Resource group:

4. Contains resources (VMs/classic clusters, storage accounts, vnets) required for the workspace to function
5. These resources are placed in a separate resource group which has restricted access, ensuring users can't accidentally delete or modify the resources
6. Select **Review + Create** —> **Create**

Now that we have explored Azure Databricks governance features and completed workspace creation, we can continue building out our platform. To do so, we will introduce a use case that allows us to explore the foundational components of a Databricks Intelligence Platform through hands-on exercises.

## Continue building the platform: Introducing use cases

> **Note**
>
> This is a fictional use case, which allows us to demonstrate Databricks features.

A retail company wants to modernize its analytics and data platform to enable better decision-making across multiple departments—starting with Sales, and expanding later to Marketing, Finance, and Operations.

Historically, data was siloed across systems such as CRM, ERP, and e-commerce platforms.

To translate this business objective into a concrete platform design, we define a set of technical and organizational requirements that will guide the architecture and hands-on implementation.

Here are some additional requirements:

- **Datasets**: customers, products, orders, order_items. Access to daily file dumps and exports will be provided.
- **Downstream consumers**: Power BI, Legacy ML workloads (external systems; not hosted on Databricks for the initial iteration).

- **Platform users and roles**: The following personas require access to the platform, with varying levels of access control: data engineers, data analysts, data scientists, QA/Testers, Product Owners, Business users, and CICD pipelines.
- **A multi-environment architecture**: The platform separates environments at three levels (workspace, catalog, and schema) to support a full development lifecycle from dev through staging to production.

The requirements outlined above reflect the first iteration of the data platform. In future iterations, additional departmental data will be onboarded, and machine learning workloads will be migrated to Databricks.

Based on these initial requirements, the proposed solution is a Databricks Lakehouse architecture governed by Unity Catalog. This design centralizes data, enforces consistent governance and data quality, and serves multiple downstream consumers, including Power BI dashboards and external legacy machine learning workloads.

*Figure 2.6* illustrates the high-level architecture of the proposed platform.

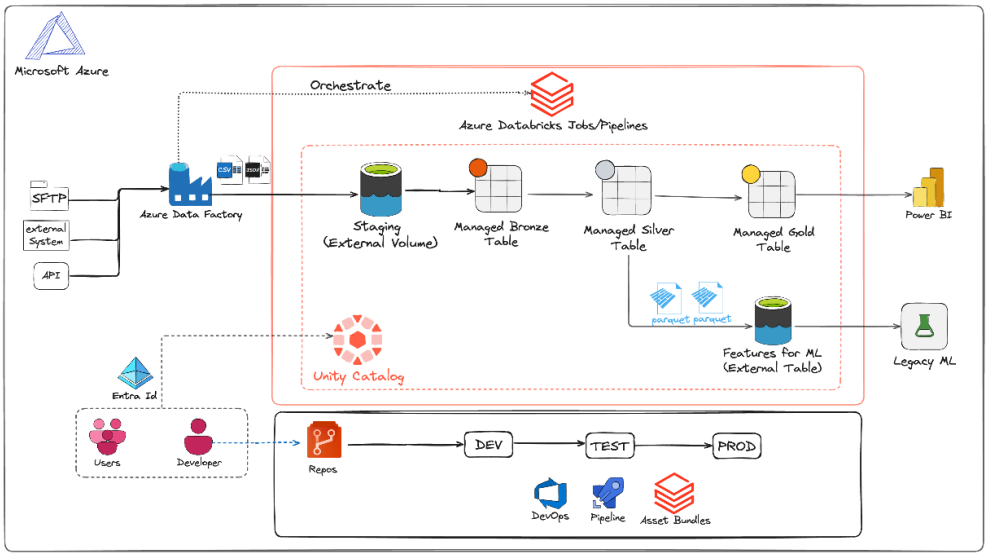



*Figure 2.6: Proposed Databricks Lakehouse Architecture Governed by Unity Catalog*

> **Note**
>
> **Download the color images**
>
> Your purchase includes a color, DRM-free PDF copy of this book, ideal for viewing color images, screenshots, and diagrams. Refer to *Free benefits with your book section* at the end of the *Preface* to unlock your PDF copy.

The architecture shown in *Figure 2.6* highlights the platform's core building blocks and how data flows from source systems to downstream consumers.

Raw data is ingested from external systems into a staging area and processed through managed. Bronze, Silver, and Gold tables following the Medallion architecture:

- **Bronze**: Raw, unprocessed data as ingested from sources
- **Silver**: Cleaned, validated, and enriched data
- **Gold**: Business-level aggregated data optimized for analytics and reporting

Unity Catalog provides centralized governance across all data assets, ensuring consistent access control.

Curated Gold datasets are exposed to downstream consumers such as Power BI, while selected feature datasets are shared with external legacy ML workloads that remain outside Databricks in the initial iteration.

Please note that several components shown in the diagram—such as data ingestion and orchestration with Azure Data Factory, workflow automation using Databricks jobs (Lakeflow), declarative pipelines, and CI/CD integration—will be introduced and explored in later chapters.

Let's discuss the key design decisions and their implications for the overall platform architecture.

# Connecting to cloud storage

The first step in setting up the platform is to connect Unity Catalog to **Azure Data Lake Storage (ADLS)**. This connection is required to configure volumes and to create both managed and external tables. After completing this setup, you will be able to build and manage data assets within Unity Catalog.

## Unity catalog storage integration overview

*Figure 2.7* provides a high-level view of the key components of the Unity Catalog storage and access model and how they relate to one another.

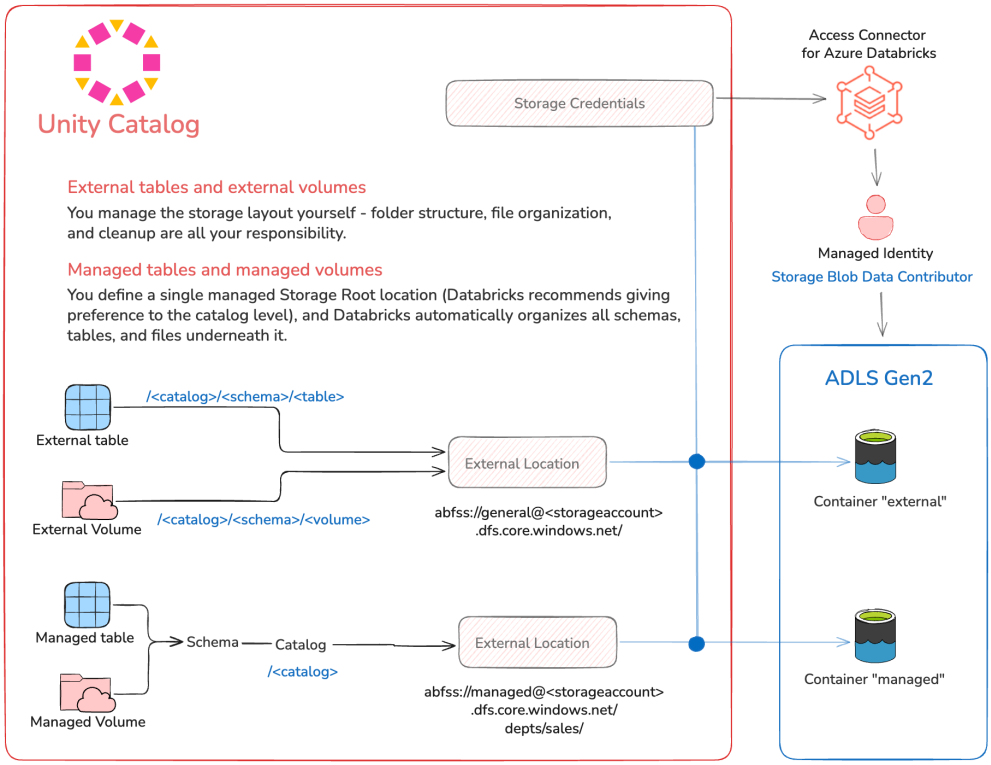



*Figure 2.7: Unity Catalog storage integration overview*

*Figure 2.7* also highlights the distinction between managed and external tables and volumes, and how each maps to its corresponding storage paths.

**Note**

**Cloud Storage Layout and Organization**

**External tables** and **volumes** require explicit layout management. Each time you create them, you must specify the full storage path where the data will reside.

**Managed data objects** are stored in a Managed Storage Location. You can define a managed storage location at different levels: metastore, catalog, and schema.

In the managed table use case, Databricks recommends assigning a managed storage location at the catalog level to ensure data isolation and consistency. When a catalog has a managed storage location, any schemas or tables created within it automatically inherit that location, eliminating the need to specify storage paths for each object

> individually. The same inheritance model applies when a managed storage location is defined at the schema level.
>
> This approach is flexible rather than prescriptive. Managed storage locations can be defined at any level of the hierarchy, and even when a catalog has its own managed location, you can override it by assigning a different managed storage location to specific schemas within that catalog.

Based on the Unity Catalog storage integration overview, we can now proceed with creating the Azure resources needed to support this setup.

## Create the storage account

We will create the Azure Data Lake Storage Gen2 account that will serve as the underlying storage layer for both managed and external data governed by Unity Catalog.

1. Go to Azure Portal → Create a resource → search for Storage account → Create
2. Fill out the Basics tab:
    - **Subscription**: Select your subscription
    - **Resource group**: Use the same resource group where you created your Databricks workspace
    - **Storage account name**: Must be globally unique (e.g. adlsdbriksdeveastus)
    - **Region**: Same as Databricks workspace
    - **Performance**: Standard
    - **Redundancy**: Choose LRS for now (Locally-redundant storage)
3. Under the **Advanced** tab:
    - Enable **Hierarchical namespace**
    - Leave default values for other settings
4. Click **Review + Create → Create**

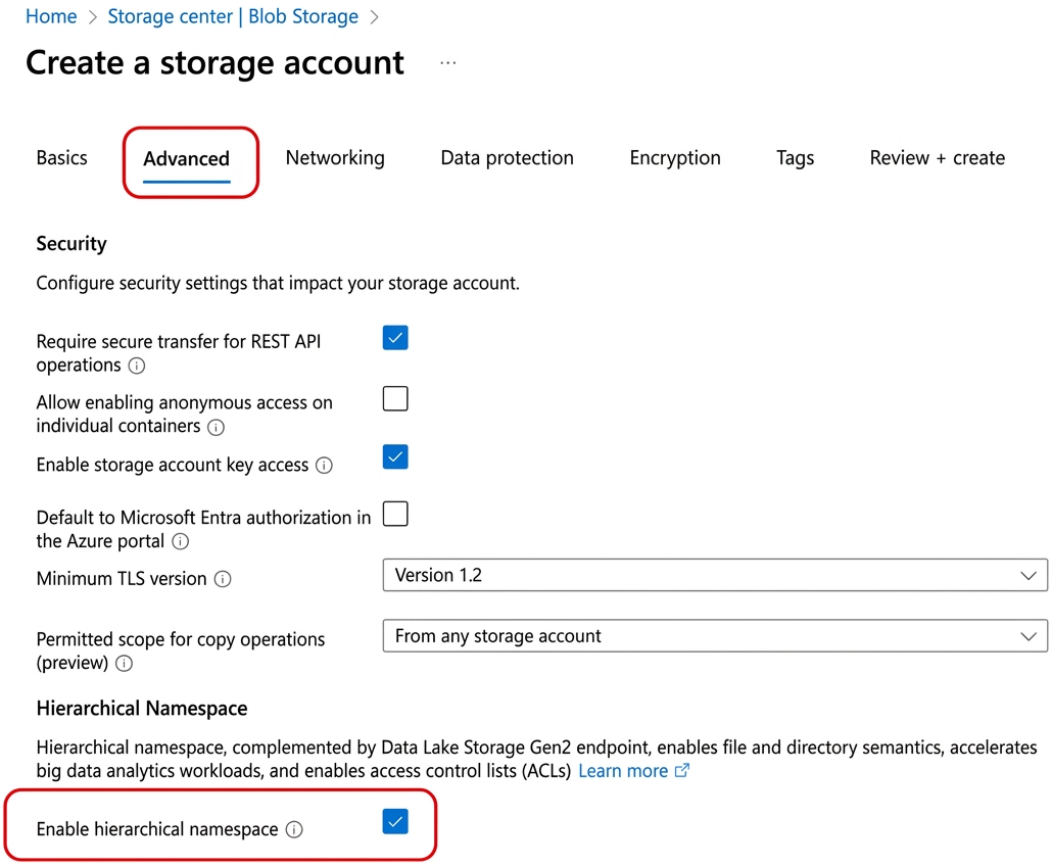



*Figure 2.8: Enabling namespace*

Before Unity Catalog can organize data within storage, we must first define clear physical boundaries for how data will be laid out.

## Create containers in the storage account

Let's create storage containers that logically separate managed data from externally managed data.

1. Go to the storage account you just created
2. Open the **Containers** view. Under **Data storage**, select **Containers**.
3. Create the managed container
    - Click **+ Container**
    - Enter a name: managed (or another name of choice)

- Set **Private** (no anonymous access)
- Click **Create**

4. Create the external container
    - Click **+ Add container** again
    - Enter a name: external (or other name of choice)
    - Set **Private** (no anonymous access)
    - Click **Create**

To enable secure, identity-based access between Databricks and cloud storage, Unity Catalog relies on a managed identity provided by Azure.

## Create an access connector for Azure Databricks

> **Note**
>
> When the workspace was created, Azure automatically provisioned an Access Connector in the Databricks-managed resource group. This connector cannot be modified or assigned roles, so we create our own to assign the required storage roles.

Now we will create an Access Connector for Azure Databricks, which supplies the managed identity used to access Azure Data Lake Storage.

1. In the Azure Portal, click **+ Create a resource**.
2. Search for Access Connector for Azure Databricks and select it.
3. On the Basics tab:
    - Choose your Subscription and Resource Group.
    - Enter a descriptive name: `dbac-analytics-dev-eastus-01` (dbac = resource type, analytics = workload, dev = environment, eastus = region, 01 = instance number).
    - Set the Region (same as your storage account).
4. On the Managed Identity tab:
    - Turn System-assigned identity-> On, or
    - Add one or more User-assigned identities.
5. Click **Review + Create**. After validation, click **Create**.

6. After deployment, click Go to resource and copy the Resource ID, e.g., /subscriptions/<subscription-id>/resourceGroups/<resource-group>/providers/Microsoft.Databricks/accessConnectors/<connector-name>

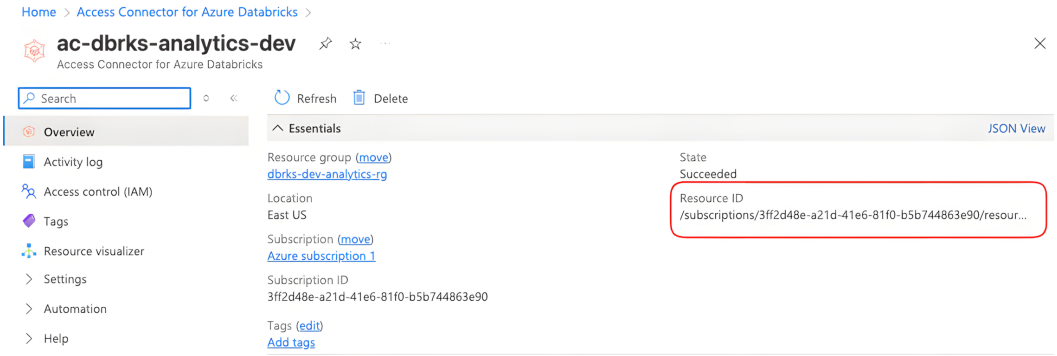



*Figure 2.9 Getting Resource ID of the access connector*

Creating the access connector alone is not sufficient; its identity must also be explicitly authorized to access storage.

Grant the Managed Identity access to Storage.

We will now grant the managed identity associated with the access connector the required permissions to read and write data in the storage account.

1. Open your **Azure Data Lake Storage** account in the portal
2. Go to **Access Control (IAM)** → **+ Add** → **Add role assignment**
3. Select the role **Storage Blob Data Contributor**. You can also assign **Storage Queue Data Contributor**, **EventGrid EventSubscription Contributor**, and **Storage Account Contributor** now for future use with Auto Loader File Notification mode.
4. Under **Assign access to**, choose **Managed identity**
5. Click **+ Select members**, search for your **Access Connector** (or user-assigned identity), and select it
6. Click **Review + Assign** to confirm

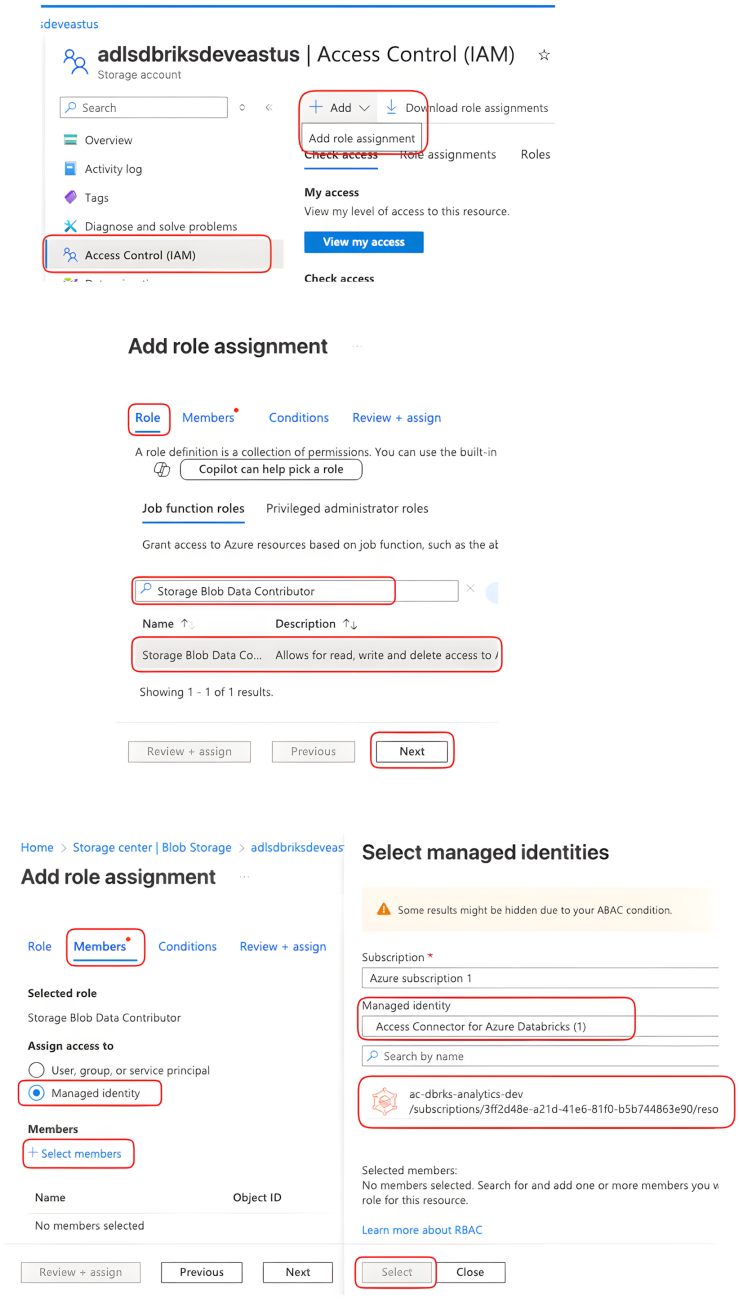



*Figure 2.10: Grant the managed identity access to storage*

Once Azure permissions are in place, the managed identity must be registered in Unity Catalog so it can be referenced by governed objects.

## Create a Storage Credential Using a Managed Identity

Storage credentials in Unity Catalog represent the managed identity used to access cloud storage.

1. In the Databricks workspace, go to **Catalog.**
2. Click **Connect ›**, go to the **Credentials** tab, and select **Create credential.**
3. Choose **Storage credential.**
4. For **Credential Type**, select **Azure Managed Identity.**
5. Enter a name for the credential, such as *adls_cred_sales_dev* (environment/department based).
6. In the **Access connector resource ID** field, provide the resource ID (see the last step in connector creation) in the following format: /subscriptions/<subscription-id>/resourceGroups/<resource-group>/providers/Microsoft.Databricks/accessConnectors/<connector-name>
7. Press **Create.**

With identity and permissions configured, we can now define how Unity Catalog maps storage paths to governed data objects.

## Create an external location

Here we will define external locations, which combine a storage credential with a storage path and are required for both managed and external tables and volumes.

Both managed and external data objects require storage credentials and a storage path, which together define an external location. We can use one storage credential for multiple external locations.

External locations can be created either through the UI by navigating to **Catalog → Connect → External Location**, or by using SQL in the editor.

We will use the SQL approach:

1. Go to **SQL Editor** and select **Serverless Starter Warehouse** (whichever default created for you during workspace creation)
2. Run the following command and add storage-specific names (account name and container)

```
CREATE EXTERNAL LOCATION if not exists uc_managed_sales_dev
URL "abfss://managed@<storage-accountname>.dfs.core.windows.net/"
WITH (STORAGE CREDENTIAL `adls_cred_sales_dev`);
```

We will create an external location for external tables/volumes later in the chapter.

With storage integration complete, we can now turn our attention to how data assets are logically organized within Unity Catalog.

## Catalog design considerations and creation

To keep your data organized, the first step is to create a catalog. You can think of a catalog as a container for related data assets—for example, those that belong to a specific product, team, business unit, environment (such as development, testing, or production), or a combination of these.

This kind of setup helps you separate data logically, gives clear context for how it's used, and makes it easier to manage permissions and access. When data is well organized, it's much easier to deal with growth. Since things can get messy quickly, taking the time to design a clear structure from the start really pays off later.

There is no single catalog design that fits all situations. However, two common approaches provide a solid foundation to get started, both offering logical organization and data isolation:

1. **Environment-driven design**: in this approach, catalogs are shared across environments, and schemas include the department or team name. Example:

```
analytics_dev.sales_raw/bronze/silver/gold
analytics_dev.finance_raw/bronze/silver/gold
```

```
analytics_test.sales_raw/bronze/silver/gold
analytics_test.finance_raw/bronze/silver/gold
```

2. **Domain-driven design**: Each department, product, or business unit has its own catalog, while still supporting environment separation. Example:

```
analytics_sales_dev.raw/bronze/silver/gold
analytics_finance_dev.finance_raw/bronze/silver/gold
analytics_sales_test.raw/bronze/silver/gold
analytics_finance_test.raw/bronze/silver/gold
```

For our case, we will use the environment-driven catalog design. It is simple, intuitive, and a practical choice when getting started.

We will use SQL to create a catalog.

Goto **SQL Editor** and select **Serverless Starter Warehouse** (whichever default created for you during workspace creation). Execute the following commands:

```
CREATE CATALOG analytics_dev MANAGED LOCATION 'abfss://
managed@<account_name>.dfs.core.windows.net/analytics_dev';
```

Execute the following command to check the details of the created catalog:

```
DESCRIBE CATALOG EXTENDED analytics_dev;
```

An example of the query's output is shown in *Figure 2.11*.

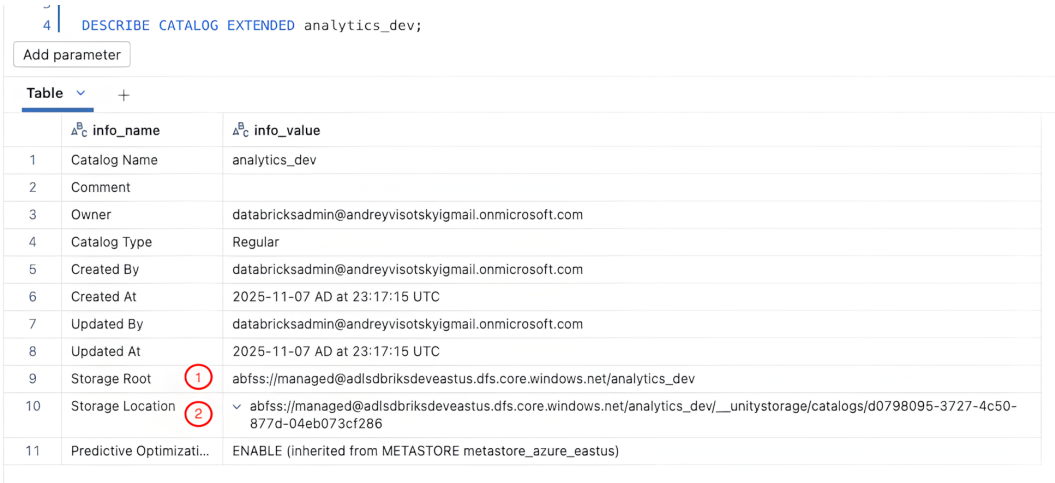



*Figure 2.11: Catalog Storage Metadata in Unity Catalog*

*Figure 2.11* highlights two important fields in the catalog metadata:

1. **Storage Root**: This matches the managed location specified during catalog creation and represents the root path under which all managed schemas and tables in the catalog are organized
2. **Storage Location**: This is the full physical path used internally by Databricks to store managed data. Part of the path is hashed to guarantee uniqueness and isolation for each catalog and its underlying data assets.

With the catalog and its managed storage location established, the next step is to define a governed staging area for raw files before they are processed into managed tables.

## Creating a volume: Staging location for raw files

Volumes are Unity Catalog objects that provide governed access to files stored in cloud storage. Like tables, volumes sit within the three-level Unity Catalog namespace (**catalog → schema → volume**). While tables store structured data that can be queried with SQL, volumes store raw files in any format, such as CSV, JSON, Parquet, images, or binary data. This makes them the natural entry point for raw data ingested from external systems, providing controlled file access before data is transformed into managed Bronze, Silver, and Gold tables following the Medallion architecture.

**Note**

Previously, to access files in Azure Data Lake storage, we relied on mount points via dbutils.fs.mount. A mount created an entry point into ADLS, accessible through **Databricks File System (DBFS)**. However, DBFS mounts are not governed by Unity Catalog and are no longer recommended by Databricks. Volumes, introduced as part of Unity Catalog, have de facto replaced mounts.

Choosing between managed and external volumes is a common design decision when working with Unity Catalog, and it depends largely on data ownership, access patterns, and integration requirements.

*Figure 2.12* contains a generalized decision chart and is helpful for choosing between managed and external volumes.

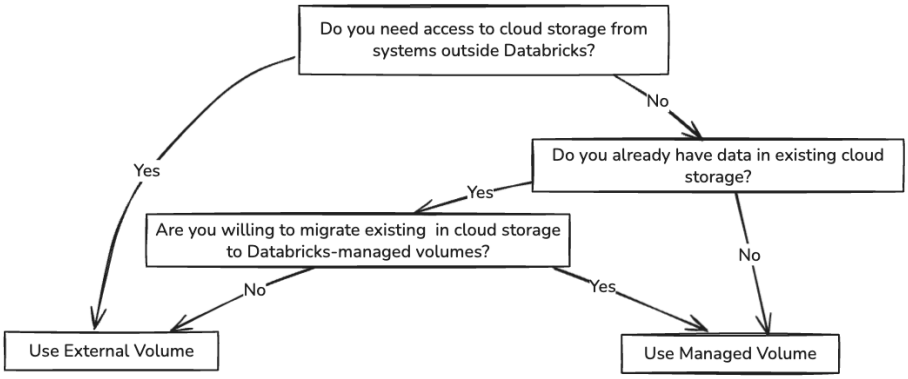



*Figure 2.12: Choosing Between Managed and External Volumes*

The decision chart in *Figure 2.12* provides a simplified framework for choosing between managed and external volumes. In general, if data is fully managed within Databricks and does not require direct access from external systems, choose a managed volume; otherwise, choose an external volume.

In our case, since files are expected to be copied into the staging area from outside Databricks, we will use an external volume.

Staging is only one of the possible use cases for volumes. Other common use cases include:

1. Exploratory storage for data scientists and analysts during analysis or experimentation
2. Library files
3. Logging/checkpointing

Now that we have chosen the Volume type, we are ready to configure the Volume for our project.

## Create an external volume for staging

We will create an external volume that serves as a staging area for raw files ingested from systems outside Databricks, allowing them to be accessed and managed under Unity Catalog governance before further processing.

Follow the steps below to create the staging area and copy raw files into it.

1. In the container **External** create directory path `analytics_dev/raw/sales_raw`. To create the path in the Azure portal, open your storage account, navigate to the external container, and use **+ Add Directory** to create each level in sequence: first `analytics_dev`, then raw inside it, then `sales_raw` inside that.
2. Create schema **raw** under catalog `analytics_dev`. Run in Databricks SQL Editor

```
CREATE SCHEMA analytics_dev.sales_raw
```

3. Create External Location for External Volume

```
CREATE EXTERNAL LOCATION if not exists uc_external_sales_raw_dev
URL "abfss://external@<storageaccount>.dfs.core.windows.net/analytics_dev/
raw/sales_raw"
WITH (STORAGE CREDENTIAL `adls_cred_sales_dev`);
```

4. Run in Sql Editor script to create external volume

```
CREATE EXTERNAL VOLUME analytics_dev.sales_raw.staging
LOCATION 'abfss://@<storage-account>.dfs.core.windows.net/'
COMMENT 'Staging location for data coming from external system.';
```

5. Verify the volume was created successfully:

```
DESCRIBE VOLUME analytics_dev.sales_raw.staging;
```

6. Copy data to staging table: Run the following Python script in Databricks notebook using serverless compute. The script copies raw files from the book's repository, simulating how files would be dropped into the staging area by external systems outside Databricks.

7. From the book repository for Chapter 2, in the folder scripts, run the script

```
Chapter02/scripts/00_copy_raw_file_sto_staging.py
```

8. List files loaded to the volume

```
LIST '/Volumes/analytics_dev/sales_raw/staging'
```

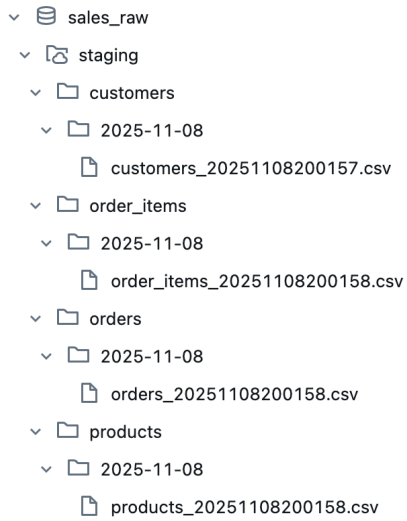



*Figure 2.13: External Volume Staging Layout for Raw Sales Data*

*Figure 2.13* shows the expected directory structure of the staging volume for sales_raw. Raw files are organized by dataset (customers, orders, order_items, and products), with each dataset further partitioned by ingestion date. Individual files include a timestamp in their name to ensure uniqueness and traceability.

This layout supports incremental ingestion, simplifies downstream processing, and aligns well with Medallion architecture pipelines.

Now that we have built the staging area and copied the raw files, let's build the pipeline for the Bronze, Silver, and Gold tables.

# Bronze, silver, gold tables

In this section, we ingest raw files from the staging area and build the Bronze, Silver, and Gold tables. The Bronze layer captures data as ingested, the Silver layer cleans and standardizes it,

and the Gold layer models it into analytics-ready structures. We also run data quality checks to validate the completeness, consistency, and freshness of the curated datasets.

A notable drawback of the Hive Metastore era is that Databricks-managed tables never gained widespread adoption. At the time, they offered little practical advantage over external tables, so most teams avoided them altogether.

In contrast, managed tables powered by Unity Catalog provide clear advantages. As a result, Databricks recommends using managed tables by default in most scenarios. We will discuss the trade-offs between managed and external tables in more detail later in this chapter.

Databricks outlines several advantages of using managed tables in Unity Catalog:

- Reduced storage and compute costs
- Improved query performance
- Automated table maintenance and optimization
- Seamless updates to the latest platform features
- Full support for Delta Lake and Apache Iceberg formats

Databricks recommends using external tables only in the following cases:

1. External systems cannot access Unity Catalog-managed tables via the supported access methods
2. The underlying data is stored in a format not supported by Unity Catalog managed tables. Supported formats include CSV, JSON, AVRO, PARQUET, ORC, and TEXT.

Given these considerations, we will use managed tables for our Bronze, Silver, and Gold layers. However, if we need to support external legacy ML workloads, we will need to create a Parquet-backed external table.

Follow the steps below to ingest raw files from the staging area and build the Bronze, Silver, and Gold tables.

1. **Create bronze, silver, and gold schemas**: As we discussed before, we will use environment-driven catalog names and append the name of the department to schema names. In the Databricks SQL Editor, execute the following commands:

```
CREATE SCHEMA analytics_dev.sales_bronze;
CREATE SCHEMA analytics_dev.sales_silver;
CREATE SCHEMA analytics_dev.sales_gold;
```

2. **Create Bronze tables**: We'll create Bronze tables using explicit DDL statements, and silver and gold tables using a simpler (CREATE OR REPLACE TABLE AS SELECT) approach for faster iteration during development
   In production environments, however, it's best practice to create tables first and then populate them using `MERGE`, `INSERT INTO`, or `INSERT OVERWRITE` statements. This approach provides better control over schema evolution, data quality, and incremental loading.

   From the book repository for *Chapter 2*, in the folder scripts, run the script `Chapter02/scripts/01_create_bronze_tables.sql`.

3. **Populate Bronze Tables**: The `COPY INTO` command provides incremental, idempotent loading by automatically skipping files that have already been processed. In addition, we insert the source filename from the _metadata column to support data lineage tracking and a load timestamp to enable deduplication in later stages.
   Incremental loading strategies will be discussed in more detail in the next chapter.

   From the book repository for *Chapter 2*, in the folder scripts, run the script `Chapter02/scripts/02_copy_into_bronze_tables.sql`.

4. **Create Silver tables**: In this step, we will further clean, cast, and process data from the bronze layer, ensuring idempotent processing
   From the book repository for *Chapter 2*, in the folder scripts, run the script `Chapter02/scripts/03_create_silver_tables.sql`.

5. **Create Gold Tables**: In this step, we model the gold layer using a star schema to support analytical queries and reporting
   From the book repository for *Chapter 2*, in the folder scripts, run the script `Chapter02/scripts/04_create_gold_tables.sql`.

With the tables in place, the next step is to validate that the curated datasets are complete, consistent, and ready for consumption.

## Data quality

Once your Gold tables are created, it's a good practice to run simple validation queries to check for data completeness and consistency. Below are a few examples of quality checks you can perform against your Sales dataset.

Data quality scripts are in the book repository for *Chapter 2*, in the folder scripts `Chapter02/scripts/05_data_quality_scripts.sql`.

1. **Check for missing dimension references**: This query identifies sales records that are missing links to dimension tables (for example, missing `product_sk` or `customer_sk` values)
2. **Check for incomplete dimension data**: This query finds product records that have missing key attributes such as product name, category, or price
3. **Check for duplicate sales records**: This query detects duplicate entries in the fact_sales table by grouping by sale_id
   These simple checks help validate data integrity in the Gold layer and are typically used in both ad hoc analysis and automated data quality pipelines.
4. **Data Freshness Test**: A freshness test helps verify that data in a specific table or pipeline is being updated regularly. One way is to check The recent modification time from system table and compare it with freshness metric.

With data quality and freshness validated, the next step is to control *who* can access these datasets and *how* they can interact with them.

# Identity management

Our use case states that the platform will host multiple user groups. Essentially, these groups will have different responsibilities and access requirements. In this section, we will guide you through best practices for managing users and groups within Databricks—from defining clear roles and responsibilities to implementing access control through Unity Catalog. We will also discuss how user provisioning and synchronization with Microsoft Entra ID can streamline onboarding and user management across the platform.

## Identity provisioning

Two key features play a significant role in user provisioning and management within Databricks:

- **Synchronization with the identity provider**: This allows Databricks to leverage existing users, groups, and service principals that are provisioned and managed in our case by Microsoft Entra ID.
- **Identity Federation**: Allows identities to be configured once at the account level rather than individually within each workspace. Workspaces then utilize these account-level identities, ensuring centralized management.

Automatic Identity Management is enabled for new accounts by default. In the rest of the cases, the account admin can enable it in **Account Console** (`https://accounts.azuredatabricks.net/`) → **Settings** → **User Provisioning** → **toggle Automatic identity management** to **Enabled**.

Identity Federation is automatically enabled once an account admin assigns a Unity Catalog metastore to a workspace. You can do this from the Account Console (`https://accounts.azuredatabricks.net/`) → **Catalog** → **<your metastore>** → **Assign to workspace**.

To make these concepts more concrete, *Figure 2.14* illustrates how identity flows through the Databricks platform when Automatic Identity Management is enabled.

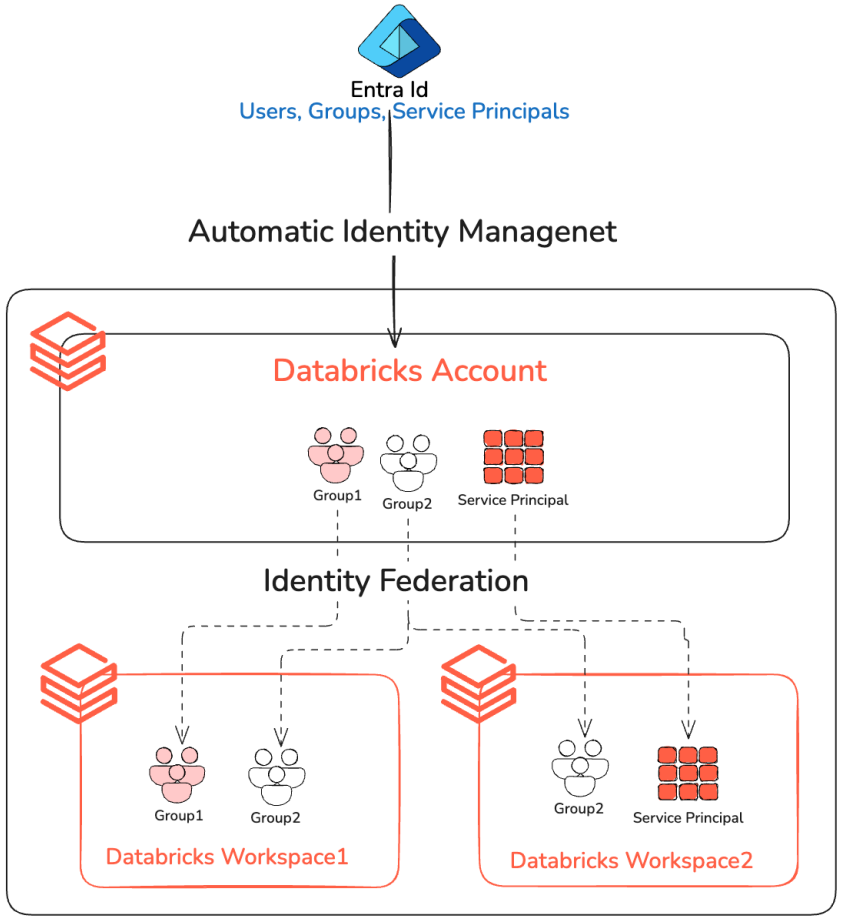



*Figure 2.14: Automatic Identity Management and Identity Federation in Databricks*

*Figure 2.14* shows how users, groups, and service principals flow from the enterprise identity provider (Entra ID) into the Databricks Account via Automatic Identity Management, and are then federated into individual workspaces, enabling centralized governance with workspace-level identity consumption.

With identities centrally managed and consistently propagated across the platform, the next design decision is how access should be granted and managed in practice.

## Groups

Databricks recommends assigning workspace access to groups rather than to individual users. This approach seems intuitive, and we will adopt it in our use case.

Based on our use case, *Table 2.2* outlines the groups we will use, their typical members, and the primary responsibilities each group handles on the platform.

| Group | Typical Members | Purpose |
|---|---|---|
| **Platform Admins** | Databricks administrators | Manage Unity Catalog, storage credentials, and security configuration |
| **Engineers** | Data Engineers, Analytics Engineers | Ingest, transform, and publish data; build data pipelines and dashboards |
| **Data Analysts** | Data Analysts, BI Developers | Query Gold data, create dashboards, and analytical views |
| **Advanced Analytics** | Data Scientists, ML, and AI Specialists | Work with data in files, experiment with data prepared by engineers, train models, and create model-serving endpoints |
| **QA Read Only** | QA Engineers, Testers | Validate data quality, logic, and transformations |
| **Business Users Read Only** | Product/Business Users | View dashboards, apps |

*Table 2.2: Platform User Groups*

## Create groups and assign to workspaces

With the user groups for our use case defined, the next step is to create these groups at the Databricks account level and make them available to the workspace. Creating groups centrally allows us to manage identities once and consistently apply access controls across multiple workspaces. Assigning groups to a workspace is a required step that enables those groups to be used for entitlements, SQL permissions, and object-level access within that workspace.

## Create groups at the account level (UI)

1. Log in to your **Databricks Account Console** (`https://accounts.azuredatabricks.net/`)
2. Navigate to **Account Console → User management → Groups**

3. Click **Create group** and add the following groups
    - `grp_admins`
    - `grp_data_engineers`
    - `grp_data_analysts`
    - `grp_advanced_analytics`
    - `grp_qa_readonly`
    - `grp_technical_business_readonly`
    - `grp_non_technical_business_consumer`

> **Note**
>
> We will reuse the same groups for all workspaces (when working on CICD). Depending on the isolation requirements, different workspaces/environments can have distinct group names. In such a case, the naming convention can include the environment, the workspace name, and even the level at which the group is provisioned (account or workspace).

4. **Add users** to each group(if any)

## Assign groups to workspaces (UI)

1. In the Account Console, go to **Workspaces → [Select your workspace] → Permissions**
2. Click **Assign group to workspace**
3. Choose which groups can access the workspace (e.g., assign all six groups to analytics-dev, analytics-prod, etc.)
4. Optionally, adjust workspace-specific roles:
    a. `platform_admins`: Workspace Admin
    b. **All others**: Workspace User

This step makes the groups visible inside the workspace for entitlements, SQL grants, and object-level permissions. If a group isn't assigned to a workspace, Databricks won't recognize it there—even if it exists at the account level.

For now, we have finished working on the account console and are moving the workspace.

## Entitlements, privileges and access control

Once groups have been assigned to the workspace, we can manage their access at three distinct levels: workspace features, workspace assets, and data assets.

Let's define each:

- **Workspace-level high-level permissions (Entitlements)**: Define what global features a group can access and use within a workspace. These govern access to capabilities such as *Data Engineering*, *Data Science and AI*, *Databricks SQL*, *Databricks Apps*, and *Genie Spaces*.
- **Workspace-level securable objects (Access Control Lists)**: Control what groups can do within the workspace and what non-data assets they have access to. These permissions determine who can use compute, create or manage jobs, run notebooks, manage clusters, or edit dashboards.
- **Unity Catalog securable objects (Privileges)**: Govern access to catalogs, schemas, tables, views, functions, and volumes. These privileges determine how users can interact with data (e.g., `SELECT`, `MODIFY`, `EXECUTE`, `CREATE`).

By default, when user groups are assigned to a workspace—either as Workspace Users or Workspace Admins—they are automatically assigned default permissions and entitlements.

It is important to review these default permissions to understand the level of access they provide and to determine whether you need to expand or restrict their scope based on your requirements.

Next, we walk through how access is managed within the workspace.

### Manage entitlements

You can manage entitlements from the **Workspace Admin Settings** page.

Here are the steps:

1. Click your username in the top-right corner of the Databricks workspace
2. Select **Settings → Identity and Access**
3. Choose the **User**, **Group**, or **Service Principal** you want to manage
4. From the list, click the specific Identity (User, Group, or Service Principal)
5. In the **Entitlements** section, select or deselect the desired entitlements

## Manage access control lists

In the Databricks UI, we manage **Access Control Lists** (**ACLs**) by setting permissions directly on the page of each individual asset, for example, notebooks, folders, jobs, pipelines, or SQL warehouses.

From the permissions dialog, we can assign access levels such as `CAN VIEW`, `CAN RUN`, `CAN EDIT`, or `CAN MANAGE` to specific users, groups, or service principals.

The highest permission level, `CAN MANAGE`, grants full administrative control over the asset and is automatically assigned to workspace administrators and the asset's creator.

We will explore how to set permissions programmatically in a YAML file when packaging a project into Declarative Automation Bundles.

# Privileges

Before setting up privileges, it's important to understand how the privilege model in Unity Catalog works.

One of the defining features of Unity Catalog's privilege model is its hierarchical inheritance.

Let's clarify what that means:

- Hierarchical refers to the structure of the securable object model reviewed earlier in this chapter: **Metastore → Catalog → Schema → Table/View/Volume**, and so on
- Inheritance means that once privileges are granted to a security principal at a higher level in this hierarchy, they are automatically applied to all objects contained within that level

This design makes access control very powerful; however, it could be more complex to manage.

For example, if a group is granted the `MODIFY` privilege at the catalog level, that privilege is automatically inherited by all schemas and tables within that catalog. This simplifies management by reducing the need to grant privileges repeatedly at each lower level. However, if a group such as Data Analysts should only have access to a specific Gold table, the top-down approach will not work.

This approach aligns with the principle of least privilege, ensuring that users have only the access necessary to perform their roles.

Let's now look at what fine-grained privileges the Data Analysts group needs to have in order to query Gold data:

1. `USE CATALOG`: On the catalog that contains the Gold data
    - This allows the group to browse and reference objects within the catalog

2. `USE SCHEMA`: On the schema that contains the Gold tables
    - This allows access to objects defined within that schema
3. `SELECT`: On the specific Gold tables or views
    - This allows reading and querying the data

Next we will create a permission matrix tailored to our use case, showing how scopes, entitlements, Unity Catalog privileges, and workspace permissions are combined for each group in our platform.

## Permission matrix

The table below shows an example permission matrix for our platform that maps user groups to their access scope, Unity Catalog privileges, workspace permissions, and entitlements.

This matrix reflects the needs of our specific use case and is not exhaustive. In real-world environments, these assignments may evolve as new assets are introduced, use cases expand, or Databricks adds new platform capabilities.

| **Group** | **Access Scope** | **Unity Catalog Privileges (SQL)** | **Workspace Permissions** | **Entitlements** |
|---|---|---|---|---|
| **Platform Admins** | Full (Account + All Catalogs) | CREATE CATALOG, CREATE EXTERNAL LOCATION, CREATE STORAGE CREDENTIAL, OWNERSHIP, MANAGE GRANTS | Full admin on clusters, jobs, repos, dashboards | All entitlements |

| Group | Access Scope | Unity Catalog Privileges (SQL) | Workspace Permissions | Entitlements |
|---|---|---|---|---|
| **Engineers** | Bronze/ Silver/Gold (read-write) | USE CATALOG, USE SCHEMA, CREATE TABLE, CREATE VIEW, CREATE MATERIALIZED VIEW, MODIFY, SELECT, EXECUTE, READ VOLUME | CAN ATTACH TO clusters and SQL warehouses, CAN MANAGE jobs/ pipelines, CAN EDIT METADATA on Feature Tables, CAN VIEW Files | Workspace Access, , Databricks SQL Access |
| **Advanced Analytics** | Silver/Gold (read-only), Sandbox (read-write) | WRITE VOLUME, USE CATALOG, USE SCHEMA, SELECT on Silver and Gold. CREATE TABLE, CREATE VIEW, MODIFY, SELECT on Sandbox. EXECUTE on shared functions | CAN ATTACH TO / CAN USE clusters, CAN EDIT Files, CAN EDIT METADATA on Feature Tables, CAN EDIT Experiments, and other ML/AI-specific permissions | Workspace Access, , Databricks SQL Access |

| Group | Access Scope | Unity Catalog Privileges (SQL) | Workspace Permissions | Entitlements |
|---|---|---|---|---|
| **Data Analysts** | Gold (read-only), Creating Dashboards | USE CATALOG, USE SCHEMA, SELECT on Gold tables/ views | CAN USE SQL warehouses, CAN EDIT dashboards | Databricks SQL Access |
| **QA Read Only** | Gold (read-only). Might need temporary access to other schemas depending on the scope of the QA task. | USE CATALOG, USE SCHEMA, SELECT on Gold tables/ views | CAN USE SQL warehouses and CAN VIEW dashboards | Databricks SQL Access |
| **Business Read Only** | Dashboards, apps | None, unless you need to refresh dashboards: USE CATALOG, USE SCHEMA, SELECT on Gold tables/ views | CAN VIEW dashboards,<br><br>if need to refresh dashboard: CAN USE SQL warehouses | Databricks SQ Access |

*Table 2.3: Permission matrix*

With identity, permissions, and data access clearly defined, the next step is to decide how work is executed on the platform. So far, we have used Databricks SQL Serverless and serverless compute in notebooks. Next, we explore the available compute options on Databricks and discuss how to choose the right compute type for different workloads.

## Compute

Databricks now offers a variety of compute options, which raises an important point about selecting the right compute type for each workload.

The introduction of serverless compute across all platform features—including notebooks, jobs, and pipelines—makes it the default choice for many scenarios, as it is always available once enabled at the account level.

However, it is still important to evaluate suitable compute options and determine which ones work best for your specific use case, considering factors such as cost, performance, and execution time, and the trade-offs among them.

## Serverless

So far, you have used serverless compute to run code in notebooks and SQL warehouses. Serverless compute in Databricks requires minimal setup and is optimized for quick startup, performance, and ease of use. Serverless compute now supports most major workload types in Databricks—including notebooks, SQL warehouses, Jobs, and Spark Declarative Pipelines.

You can generally use serverless compute when:

- You are running standard data engineering or ETL workloads
- You want to focus on development without managing cluster configuration
- Your workloads are mostly SQL-based
- You need compute that starts almost instantly

However, serverless compute has some limitations and may not be the right fit if:

- You need to use low-level APIs or custom libraries that are not supported in serverless environments
- You are running machine learning or AI workloads that require custom environments
- You need full control over compute configuration for performance tuning or cost optimization
- You need to write to external cloud systems such as relational databases, data warehouses, etc.

*Table 2.4* briefly compares Serverless and Classic computing.

| Feature | Serverless Compute | Classic Compute |
|---|---|---|
| **Setup** | No setup required; Databricks fully manages provisioning | Requires manual configuration (cluster size, runtime, termination, etc.) |
| **Startup Time** | Starts in seconds | Can take several minutes to start |
| **Scalability** | Scales automatically based on workload demand | Scaling must be configured (auto-scaling or fixed-size) |
| **Control** | Limited configuration options for compute resources | Full control over runtime, libraries, and environment |
| **Supported Workloads** | Notebooks, SQL warehouses, Jobs, Declarative Pipelines | All workloads, including ML, custom libraries, and GPU tasks |

*Table 2.4: Serverless and Classic compute comparison*

## Classic compute access modes

When using classic compute, it is not enough to choose the cluster size and runtime; you must also decide how compute resources are shared and isolated between users. Databricks provides multiple access modes for classic clusters, each offering different trade-offs between isolation, capability, and security. Selecting the appropriate access mode is essential to ensure workloads run correctly while aligning with governance and performance requirements.

## Standard (shared) access mode

This is the default and generally recommended access mode for most workloads. It simplifies setup and can reduce costs because a single cluster can be shared by multiple users running concurrent workloads. Each user's code runs in full isolation, ensuring workloads do not interfere with each other. Access to low-level system resources is not available in this mode.

## Dedicated (Single-User) access mode

This mode is used when compute resources must be restricted to a specific user or group. It supports workloads that require access to RDD APIs, R-based computations, operating system-

level resources, external systems through JDBC connectors, or custom or containerized environments not supported by Standard mode.

### No Isolation Shared Access Mode

This mode provides no isolation between users sharing the same cluster and is not supported by Unity Catalog. It is generally not recommended and should only be used for legacy workloads or for workloads that still rely on the Hive metastore.

With the right access mode selected, there is one additional configuration option worth considering: Photon, which can significantly improve performance for analytical workloads.

### Photon

Photon is a high-performance query engine in Databricks, designed to accelerate SQL and DataFrame API operations. It improves the execution speed of complex transformations, especially those involving joins, aggregations, and other compute-intensive operations.

Photon is most beneficial for long-running or large-scale workloads. For short or lightweight jobs, the performance gain may be minimal because startup overhead can outweigh the benefits. For classic compute, Photon must be enabled explicitly during cluster setup. In contrast, Photon is enabled by default for all SQL warehouse types in Databricks.

With query execution performance covered, we now briefly review SQL warehouses and how they fit into the overall compute landscape.

## SQL Warehouses

A SQL warehouse is a compute resource purpose-built for running SQL workloads in Databricks. SQL warehouses are optimized specifically for SQL execution and power a broad range of use cases: interactive queries in the SQL editor, BI dashboards and reports, streaming tables and materialized views, dbt transformations, SQL Statement API requests, and direct connections to tools like Power BI and Tableau.

There are three types of SQL warehouses:

- **Serverless**: Recommended for most use cases. It offers the best performance, fastest startup time, and rapid scaling.
- **Pro**: Recommended when serverless is not available or when custom networking or configuration is needed.
- **Classic**: Only recommended when Pro and Serverless options are unavailable.

With compute options in place and workloads running on SQL warehouses and clusters, the next important consideration is **observability**: understanding how the platform is being used, how resources are consumed, and what that usage costs.

Databricks exposes this operational insight through system tables, enabling you to analyze platform activity with the same SQL tools you use for business data.

## System Tables

**System tables** are Databricks-managed datasets that store operational information about your account and workspace activity.

They provide insights into areas such as:

- User and resource access
- Job execution history and associated costs
- Compute usage and consumption trends
- Monthly cost breakdowns
- Data quality and pipeline monitoring
- And much more

The examples above represent only a small portion of the insights that can be obtained by querying system tables.

If you have been running queries and workloads while following this book, your workspace should already contain accumulated usage data. You can run the following query to estimate compute consumption costs:

From the book repository for *Chapter 2*, in the folder **Scripts**, run the script `06_system_table_query.sql`.

Query explanation:

- `workspace_id`: Identifies the Databricks workspace where the usage occurred
- `billing_origin_product`: Indicates the product or service that generated the charge (for example, SQL Warehouse, Jobs or Interactive clusters)
- `sku_name`: Specifies the stock keeping unit (SKU), which represents a particular type of compute or storage resource
- `list_cost`: Calculates the estimated cost based on list prices for each SKU and usage type

You can further extend this query by filtering by date, SKU, or workspace to track monthly cost trends or identify which workloads consume the most resources.

## Deleting resources

If you used a Trial Premium subscription while working through this chapter, we recommend deleting the workspace and associated resources to avoid incurring additional costs.

## Summary

This chapter established the technical and governance foundation for building a modern, production-ready Azure Databricks Lakehouse. It guided you through the full setup process—from selecting an optimal Azure region and provisioning a workspace to configuring security, governance, and compute resources.

You were introduced to Unity Catalog as the central governance layer that unifies data and AI asset management across workspaces, replacing the legacy Hive Metastore with fine-grained access control, centralized metadata, and built-in lineage and auditing.

In a retail analytics use case, you configured an Azure Data Lake Storage integration using managed identities, storage credentials, and external locations, and designed a structured catalog and schema to organize data by environment and domain. You then created managed and external tables across the Bronze, Silver, and Gold layers, built volumes for raw-file staging, and performed basic data-quality and freshness checks to validate the pipeline's reliability.

The chapter also covered identity federation and group-based access control, showing how to synchronize users and roles from Microsoft Entra ID and assign appropriate entitlements in accordance with the principle of least privilege.

Finally, you explored compute options, including serverless and classic compute, learning when to use each for different workloads, and used system tables to monitor resource usage and costs.

Together, these steps provided a complete setup of a governed, scalable Databricks environment—the solid foundation upon which the following chapters will build data ingestion, transformation, and CI/CD automation workflows.

In the next chapter, we shift from platform setup to execution by focusing on data ingestion, the process through which data first enters the Lakehouse. Building on the governance, secure storage access, identity integration, and compute configuration established in this chapter, you will explore batch and streaming ingestion patterns and ingest data from Azure Data Lake Storage and external systems such as databases and APIs. By building directly on this platform foundation, ingestion pipelines become governed, secure, and production-ready components of the Data Intelligence Platform.

## Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to `packtpub.com/unlock`). Search for this book by name, confirm the edition, and then follow the steps on the page.

*Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.*

# 3

# Data Ingestion Strategies for Azure Databricks

Data ingestion is the foundation of any data engineering pipeline. It represents the critical first step in your data journey, bringing raw data from various sources into Azure Databricks, where it can be transformed, analyzed, and turned into valuable insights.

Azure Databricks provides a unified platform for data ingestion that can handle diverse data sources, from traditional relational databases to modern REST APIs, from cloud storage to NoSQL databases. Understanding how to ingest data from each source type effectively is essential for building efficient, scalable, and maintainable data pipelines.

This chapter focuses on batch data ingestion—the practice of loading data in scheduled, discrete chunks. Batch ingestion remains the most common pattern in enterprise environments, handling use cases such as daily reporting, historical analysis, regulatory compliance, and data-warehouse loading. You will learn practical techniques for ingesting data from Azure storage, relational databases, REST APIs, and other common sources, with working examples you can adapt for your own projects.

In this chapter, we will cover:

- Understanding batch ingestion patterns and when to use them
- Configuring authentication and connecting to **Azure Data Lake Storage** (**ADLS**)
- Reading data from various file formats, including CSV, JSON, and Parquet
- Implementing incremental loading patterns to process only new data
- Integrating Azure Data Factory for orchestrated data movement
- Connecting to relational databases using JDBC

- Ingesting data from REST APIs with authentication and pagination
- Writing data to Delta Lake tables with proper partitioning and optimization

By the end of this chapter, you will be able to design and implement robust batch ingestion pipelines that efficiently move data from diverse sources into Azure Databricks, ready for transformation and analysis in subsequent stages of your data engineering workflows.

> **Note**
>
> **Streaming**: For real-time data ingestion scenarios requiring continuous processing with low latency (seconds to minutes), Chapter 5 provides comprehensive coverage of streaming architectures using Apache Spark Structured Streaming, Event Hubs, and Auto Loader. This chapter focuses exclusively on batch patterns in which scheduled processing (hourly, daily, weekly) meets business requirements.

# Technical requirements

To follow the examples in this chapter, you will need:

1. **Azure Resources**:
    - An active Azure subscription with permissions to create resources
    - An Azure Databricks workspace (Premium tier required for Unity Catalog support)
    - An Azure Data Lake Storage Gen2 account with at least one container created
    - (Optional) An Azure SQL Database or SQL Server instance for database ingestion examples
    - (Optional) An Azure Data Factory instance for orchestration examples
2. **Databricks Configuration**:
    - A Databricks cluster running Databricks Runtime 13.3 LTS or later
    - Cluster configured with Unity Catalog access (standard access mode, formerly known as shared mode, or dedicated access mode, formerly known as single-user mode)
    - Permission to create schemas and tables in your Unity Catalog
3. **Authentication setup**:
    - Azure Service Principal with the following roles:
    - Storage Blob Data Contributor on your ADLS Gen2 storage account

- Databricks Secrets configured with your service principal credentials
- Access to create and manage Databricks secret scopes

4. **Code files:**
   - All code examples in this chapter are available in the book's GitHub repository
   - Throughout this chapter, replace placeholder values like `<storage-account>`, `<container>`, and `<scope-name>` with your actual Azure resource names and Databricks configuration values
5. **Catalog and Schema Setup:**
   - Throughout this chapter, we use a catalog called analytics_dev for our examples. To avoid repeating the full three-level namespace in every SQL statement, we set the default catalog at the start of each notebook:

```
# Set the default catalog for all SQL operations in this notebook
spark.sql("USE CATALOG analytics_dev")
```

   - With this default set, you can reference schemas and tables using two-part names (e.g., `control.ingestion_watermarks` instead of `analytics_dev.control.ingestion_watermarks`). If you are using a different catalog name in your environment, replace `analytics_dev` with your catalog name.

**Note**

Each Databricks notebook maintains its own session context, so you should include this `USE CATALOG` statement at the beginning of every notebook in this chapter.

## Understanding batch ingestion

Batch ingestion involves collecting data over a period of time and processing it all at once on a schedule. Think of it like doing laundry—you collect clothes throughout the week and wash them in batches rather than running the washing machine for every single item.

## Characteristics of batch ingestion

Batch ingestion is characterized by several key attributes that distinguish it from streaming or real-time processing. The following characteristics define how batch systems operate and why they remain foundational in data architecture.

- **Scheduled Processing**: Batch jobs run at predetermined intervals - every hour, every night at 2 AM, weekly on Sundays, monthly on the first of the month. This predictable schedule allows you to plan resources and coordinate with source systems.
- **Complete Datasets**: Batch processing works with complete, finite datasets. You know exactly how much data you are processing, making it easier to estimate runtime, allocate resources, and track progress.
- **Deliberate Latency**: There is an intentional delay between data generation and availability. For a nightly batch job, data from today becomes available tomorrow morning. This latency is acceptable - even preferable - for many business use cases.
- **Simplicity**: At their core, batch jobs are conceptually straightforward — load data, transform it, write results. If something fails, you can easily rerun the job. However, more advanced patterns, such as incremental loading (covered later in this chapter), do introduce state management complexity, specifically tracking which data has already been processed. Even so, this complexity is well-understood and manageable compared to the challenges of real-time streaming systems.

These characteristics make batch ingestion the preferred choice for many enterprise data engineering scenarios. However, the inherent latency makes batch ingestion unsuitable for use cases requiring real-time or near-real-time data availability.

## Common Batch Ingestion Scenarios

Batch ingestion is typically used when data does not need to be processed in real time and can be collected, transformed, and loaded at scheduled intervals.

Understanding where batch ingestion fits naturally helps you make appropriate architectural decisions. The following scenarios represent common patterns where batch processing provides the optimal balance of complexity, cost, and functionality:

- **Historical and trend analysis**: Monthly sales reports, quarterly performance analysis, yearly trend identification
- **Data Warehouse loading**: Daily fact and dimension table updates, slowly changing dimensions
- **Regulatory and compliance reporting**: Fixed-schedule reports for financial, healthcare, or government compliance

- **Large-scale ETL**: Moving and transforming large volumes of data from operational systems to analytical platforms
- **Cost optimization**: Running jobs during off-peak hours when cloud compute costs are lower
- **Archive and backup**: Periodic exports of operational data for long-term storage

Each of these scenarios benefits from the reliability, cost-effectiveness, and operational simplicity that batch ingestion provides. Now that we understand when and why to use batch ingestion, let's explore the technical implementation details, starting with one of the most common data sources in Azure: Azure Data Lake Storage.

# Ingesting data from Azure Storage (ADLS and blob storage)

**Azure Data Lake Storage** (**ADLS**) Gen2 and Azure Blob Storage are the primary cloud storage services for data lakes in Azure. ADLS Gen2 combines the scalability and cost-effectiveness of Blob Storage with a hierarchical file system optimized for analytics workloads.

## Authentication methods

Before reading data from Azure storage, you must configure authentication. Azure Databricks supports several methods:

## Service principal with OAuth 2.0

Run the following code in a Databricks notebook cell to configure service principal authentication:

```
# Configure service principal authentication
spark.conf.set("fs.azure.account.auth.type.<storage-
account>.dfs.core.windows.net", "OAuth")
spark.conf.set("fs.azure.account.oauth.provider.type.<storage-
account>.dfs.core.windows.net",
"org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider")
spark.conf.set("fs.azure.account.oauth2.client.id.<storage-
account>.dfs.core.windows.net",
               "<application-id>")
spark.conf.set("fs.azure.account.oauth2.client.secret.<storage-
account>.dfs.core.windows.net",
               dbutils.secrets.get(scope="<scope-name>", key="<service-principal-
key>"))
spark.conf.set("fs.azure.account.oauth2.client.endpoint.<storage-
```

```
account>.dfs.core.windows.net",
               "https://login.microsoftonline.com/<tenant-id>/oauth2/token")
```

Here's an important note:

- The service principal should be assigned the "Storage Blob Data Contributor" role at the Storage Account
- Always store credentials in Databricks Secrets rather than hardcoding them in notebooks

> **Note**
>
> While service principal authentication with OAuth 2.0 is widely used and secure, Microsoft and Databricks now recommend Managed Identities as the preferred authentication method for Azure resources. Managed Identities eliminate the need to manage secrets entirely, as Azure handles credential rotation automatically. Chapter 2 demonstrated how to set up storage access using Managed Identities. If your workspace is configured with Managed Identity access to ADLS (as shown in Chapter 2), you can skip the manual authentication configuration above - your cluster will automatically authenticate to storage. The service principal approach shown here remains useful for scenarios where Managed Identity is not available or when connecting to resources outside your Azure tenant.

- Access Keys (Simple but less secure):

```
spark.conf.set("fs.azure.account.key.<storage-
account>.dfs.core.windows.net",
               dbutils.secrets.get(scope="<scope-name>", key="<storage-
account-key>"))
```

## Reading different file formats

Apache Spark's ability to read data from various file formats is one of its core strengths. Different file formats have evolved to solve specific challenges in data storage and processing—from the universal simplicity of CSV files for human-readable data exchange, to the nested structures of JSON for semi-structured data, to the optimized columnar storage of Parquet for analytical workloads.

When ingesting data into Azure Databricks, choosing the right file format impacts not just storage efficiency, but also query performance, schema evolution capabilities, and overall data

pipeline complexity. Understanding these formats and Spark's support for each helps you make informed decisions about data storage strategies.

In this section, we'll explore how Apache Spark reads CSV, JSON, and Parquet files—the three most common formats you'll encounter in data engineering. Each format has distinct characteristics, optimal use cases, and specific options for handling various data scenarios. We'll examine not just how to read these formats, but when to use each one and what tradeoffs they involve. The reading code examples are provided as illustrative references showing the syntax and options for each format and are not intended to be executed—you will work with these formats in later chapters when the corresponding data files are available.

## CSV files

**Comma-Separated Values** (**CSV**) is the most widely compatible data format, storing data as plain text with values separated by commas. Its simplicity makes it ideal for data sharing between systems and human-readable inspection, but it lacks schema definition and struggles with nested data structures. Here's how to read CSV files with an explicit schema definition:

```
# Reading CSV files from ADLS with explicit schema
from pyspark.sql.types import StructType, StructField, IntegerType, DoubleType,
TimestampType
schema = StructType([
    StructField("sale_id", IntegerType(), False),
    StructField("product_id", IntegerType(), False),
    StructField("sale_date", TimestampType(), False),
    StructField("quantity", IntegerType(), True),
    StructField("unit_price", DoubleType(), True)
])
df = spark.read \
    .option("header", "true") \
    .schema(schema) \
.csv("abfss://<container>@<storageaccount>.dfs.core.windows.net/data/sales/*.csv")
display(df)
```

### Advantages of CSV

Universal compatibility with all tools, human-readable for debugging, simple integration with legacy systems, and no special dependencies required. However, for production data lakes, consider CSV as a source format—read it from external systems, then transform and store as Parquet or Delta Lake for efficient processing.

### Best practices

Always define an explicit schema rather than using `inferSchema` in production. Schema inference requires reading the entire file, which is expensive for large datasets.

## JSON Files

**JavaScript Object Notation (JSON)** naturally represents hierarchical data with nested objects and arrays, making it ideal for API responses, application logs, and semi-structured data. Unlike CSV, JSON is self-describing, with embedded field names, and supports flexible schemas in which records can have different fields. Here's how to read JSON files:

```
# Reading JSON files
df = spark.read \
    .option("multiLine", "true") \
    .json("abfss://container@storageaccount.dfs.core.windows.net/data/events/
*.json")
# Flatten nested structures
from pyspark.sql.functions import col
flattened_df = df.select(
    col("id"),
    col("name"),
    col("address.street").alias("street"),
    col("address.city").alias("city")
)
```

The `multiLine` option controls how Spark parses JSON files. By default (`multiLine` set to `false`), Spark expects each line of the file to be a separate, complete JSON record (JSON Lines format) - this is the most performant option for large files because records can be read in parallel. Setting `multiLine` to true tells Spark to read the entire file as a single JSON document, which is necessary when records span multiple lines (such as pretty-printed JSON or files containing a single JSON array). Use `multiLine` only when your JSON is not in line-delimited format, as it requires reading the entire file into a single partition and can impact performance on large files.

### Advantages of JSON

Natural representation of nested structures, self-describing data with embedded field names, flexible schema allowing variation between records, and native support from web APIs. Use JSON when ingesting from REST APIs or processing semi-structured data, but consider converting to Parquet for repeated analytical queries.

## Parquet files

Parquet is a columnar storage format designed for analytics workloads. Unlike row-oriented formats that store complete records sequentially, Parquet stores data by column, enabling excellent compression (typically 70-90% smaller than CSV) and fast query performance through column pruning and predicate pushdown. Parquet is the recommended format for data lakes and forms the foundation of Delta Lake tables. Here's how to read Parquet files:

```
# Reading Parquet files
df = (
    spark.read
.parquet("abfss://container@storageaccount.dfs.core.windows.net/data/warehouse/
sales/")
)
```

## Advantages of Parquet

Parquet's columnar design provides several critical benefits for analytics:

- **Efficient Storage**: Columnar compression achieves 70-90% storage reduction compared to CSV
- **Fast Queries**: Column pruning and predicate push down dramatically reduce I/O for selective queries
- **Schema Embedded**: No ambiguity in data types; schema is stored in file metadata
- **Schema Evolution**: Gracefully handles added or removed columns over time. These advantages make Parquet the standard for data lake storage and the foundation for Delta Lake

| Format | Storage Type | Compression | Best Use Case |
|---|---|---|---|
| **CSV** | Row-based | Poor | Data exchange, human-readable |
| **JSON** | Row-based | Moderate | Semi-structured data, APIs |
| **Parquet** | Columnar | Excellent | Analytics, data warehousing |

*Table 3.1: File Format Comparison – Key characteristics and recommendations for CSV, JSON, and Parquet formats*

## Incremental loading pattern

One of the most common challenges in batch ingestion is processing only new or changed data rather than reprocessing the entire dataset each time. This pattern, known as incremental loading or delta loading, is essential for production systems to maintain efficiency and reduce processing costs.

Full reprocessing of large datasets on every batch run wastes compute resources, increases processing time, and makes it difficult to meet service level agreements for data freshness. Imagine a daily batch job that must process sales data—reprocessing 10 years of historical sales records every day, when only today's records are new, would be tremendously wasteful.

Incremental loading addresses this by identifying and processing only the data that has arrived or changed since the last successful run. The most common approach uses a watermark—a timestamp or sequence number that tracks the last successfully processed record. Each batch runs queries for records beyond the watermark, processes them, and then updates the watermark for the next run.

This pattern provides several critical benefits: reduced processing time and compute costs by processing only new data, lower network bandwidth consumption when reading from remote sources, easier troubleshooting since each batch processes a smaller, discrete portion of data, and natural alignment with downstream incremental update patterns in data warehouses.

Here's a simple watermark-based incremental loading pattern you can use in your ingestion pipelines:

```
# Create the control schema (if it doesn't already exist)
spark.sql("CREATE SCHEMA IF NOT EXISTS control")
# Create watermark table to track last processed timestamp
spark.sql("""
    CREATE TABLE IF NOT EXISTS control.ingestion_watermarks (
        source_name STRING,
        last_processed_timestamp TIMESTAMP,
        updated_at TIMESTAMP
    ) USING DELTA
""")
# Initialize watermark
spark.sql("""
    INSERT INTO control.ingestion_watermarks
    SELECT 'sales_data', timestamp('2024-01-01 00:00:00'), current_timestamp()
    WHERE NOT EXISTS (
        SELECT 1 FROM control.ingestion_watermarks WHERE source_name =
```

```
'sales_data'
    )
""")
# Get the last watermark
last_watermark = spark.sql("""
    SELECT last_processed_timestamp
    FROM control.ingestion_watermarks
    WHERE source_name = 'sales_data'
""").first()[0]
# Read only records newer than the watermark
df = (
    spark.read
    .option("header", "true")
    .schema(schema)  # Uses the same sales schema
    .csv("abfss://<container>@<storageaccount>.dfs.core.windows.net/data/sales/
*.csv")
)
from pyspark.sql.functions import col, max as max_

# Filter for incremental data
incremental_df = df.filter(col("order_date") > last_watermark)
if incremental_df.count() > 0:
    incremental_df.write \
        .format("delta") \
        .mode("append") \
.save("abfss://<container>@<storageaccount>.dfs.core.windows.net/delta/sales")
    new_watermark = incremental_df.agg(max_("order_date")).first()[0]
    spark.sql(f"""
        UPDATE control.ingestion_watermarks
        SET last_processed_timestamp = timestamp('{new_watermark}'),
            updated_at = current_timestamp()
        WHERE source_name = 'sales_data'
    """)
    print(f"Processed incremental data up to {new_watermark}")
```

Here's the output:

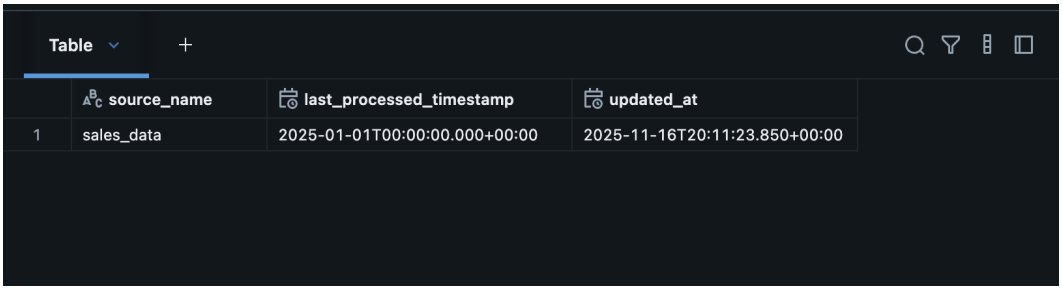



*Figure 3.1: The control.ingestion_watermarks table contents with columns for source_name, last_processed_timestamp, and updated_at*

**Note**

For continuous file monitoring and automatic processing, Auto Loader (covered in Chapter 5) provides a more robust streaming solution.

## Working example: ADLS batch ingestion

The concepts and code snippets we've covered so far demonstrate individual techniques for reading data from Azure storage. Now let's bring these together into a complete, production-ready example that demonstrates a realistic batch ingestion scenario.

This working example illustrates a typical pattern: reading CSV data from ADLS, applying schema validation and data quality checks, implementing incremental loading using watermarks, partitioning data for query performance, and writing to a Delta Lake table with proper metadata. The code combines authentication, file format handling, incremental loading, and Delta Lake best practices into a cohesive pipeline.

You can run this example in your Azure Databricks workspace using the sample data files from the chapter's GitHub repository. The code is fully commented and includes error handling and logging to demonstrate production-quality patterns.

**Note**

This working example uses service principal authentication for illustrative purposes. If you configured Managed Identity access to your storage account in Chapter 2, you can skip the authentication configuration block below and access storage directly without additional setup.

Here's where you can find the code: `01_adls_batch_ingestion.py` in the GitHub repository.

```
# Complete ADLS Batch Ingestion Example
from pyspark.sql.functions import col, current_timestamp, year, month
from pyspark.sql.types import StructType, StructField, StringType, IntegerType,
DoubleType, TimestampType
# Configure authentication (using service principal)
storage_account = "yourstorageaccount"
container = "rawdata"
spark.conf.set(f"fs.azure.account.auth.type.
{storage_account}.dfs.core.windows.net", "OAuth")
spark.conf.set(
    f"fs.azure.account.oauth.provider.type.
{storage_account}.dfs.core.windows.net",
    "org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider"
)
spark.conf.set(
    f"fs.azure.account.oauth2.client.id.{storage_account}.dfs.core.windows.net",
    dbutils.secrets.get(scope="azure-key-vault", key="sp-client-id")
)
spark.conf.set(
    f"fs.azure.account.oauth2.client.secret.
{storage_account}.dfs.core.windows.net",
    dbutils.secrets.get(scope="azure-key-vault", key="sp-client-secret")
)
spark.conf.set(
    f"fs.azure.account.oauth2.client.endpoint.
{storage_account}.dfs.core.windows.net",
    f"https://login.microsoftonline.com/{dbutils.secrets.get(scope='azure-key-
vault', key='tenant-id')}/oauth2/token"
)
# Define paths
base_path = f"abfss://{container}@{storage_account}.dfs.core.windows.net"
sales_csv_path = f"{base_path}/sales/csv/"
products_json_path = f"{base_path}/products/json/"
delta_output_path = f"{base_path}/delta/sales_enriched/"
# Define schema for CSV
sales_schema = StructType([
    StructField("sale_id", IntegerType(), False),
    StructField("product_id", IntegerType(), False),
    StructField("sale_date", TimestampType(), False),
```

```
    StructField("quantity", IntegerType(), False),
    StructField("unit_price", DoubleType(), False)
])
# Read CSV sales data
print("Reading CSV sales data...")
sales_df = (
    spark.read
    .option("header", "true")
    .schema(sales_schema)
    .csv(sales_csv_path)
)
# Read JSON product data
print("Reading JSON product data...")
products_df = (
    spark.read
    .option("multiLine", "true")
    .json(products_json_path)
)
# Enrich and transform
sales_enriched = (
    sales_df
    .withColumn("total_amount", col("quantity") * col("unit_price"))
    .withColumn("sale_year", year("sale_date"))
    .withColumn("sale_month", month("sale_date"))
    .withColumn("ingestion_timestamp", current_timestamp())
)
# Join with product information
sales_with_products = sales_enriched.join(products_df, "product_id", "left")
# Create schema if it doesn't exist
spark.sql("CREATE SCHEMA IF NOT EXISTS sales_db")
# Write to Delta Lake as a managed table with partitioning
print("Writing to Delta Lake...")
sales_with_products.write \
    .format("delta") \
    .mode("overwrite") \
    .partitionBy("sale_year", "sale_month") \
    .option("overwriteSchema", "true") \
    .saveAsTable("sales_db.sales_enriched")
print("Data loaded successfully!")
display(spark.sql("SELECT * FROM sales_db.sales_enriched LIMIT 10"))
```

## Using Azure Data Factory for ingestion

Azure Data Factory (ADF) is Microsoft's cloud-based data integration service and one of the most well-established tools for orchestrating data movement in Azure. ADF is particularly strong for connectivity to legacy and on-premises data sources through its Self-hosted Integration Runtime, which enables secure data transfer from systems behind corporate firewalls. With over 90 built-in connectors and a visual, low-code pipeline builder, ADF has long been the default choice for data movement in Azure-based architectures.

It is worth noting that Databricks has introduced Lakeflow Connect as a native alternative for ingesting data from modern SaaS sources directly into the lakehouse. Lakeflow Connect provides managed connectors for popular platforms such as Salesforce, Workday, NetSuite, and others, eliminating the need for an external orchestration tool for these sources. For new projects, consider using Lakeflow Connect for SaaS ingestion and reserving ADF for scenarios where its strengths are most relevant — namely, legacy system connectivity, on-premises data movement, and complex multi-step orchestration workflows.

While Chapter 8 covers orchestration patterns in depth, understanding ADF's role in ingestion is important for designing complete data pipelines. Its key capabilities include:

- **Connector Library**: Over 90 built-in connectors to various data sources
- **Visual Interface**: Drag-and-drop pipeline building
- **Scheduling**: Built-in scheduling and monitoring
- **Staging**: Moving data from legacy systems to ADLS before Databricks processes it

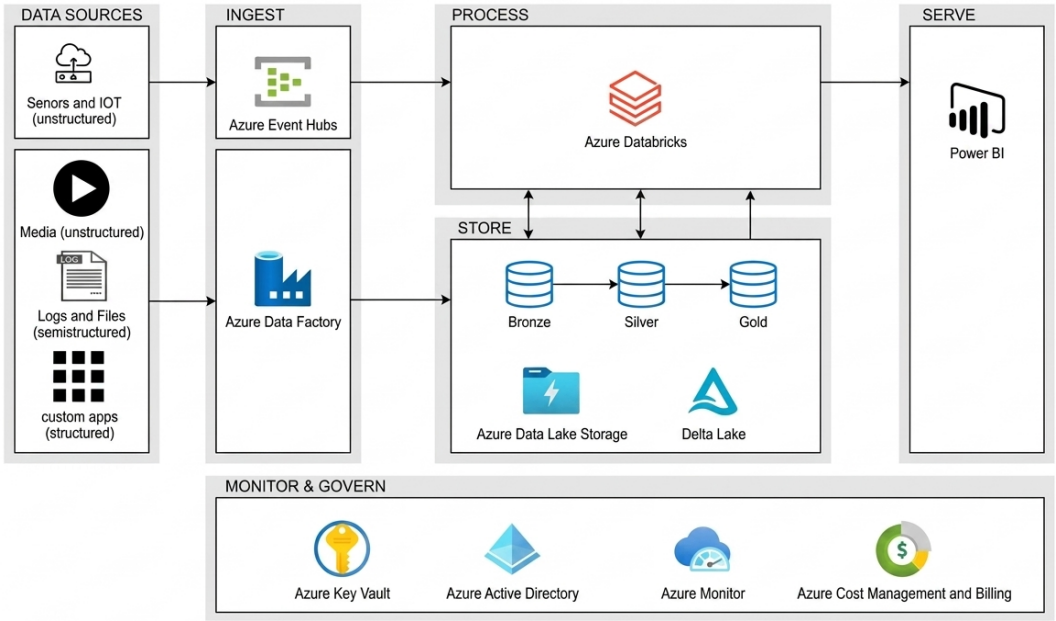



*Figure 3.2: Common ADF ingestion pattern – extract data from sources, load into Azure Storage, and trigger Databricks for transformation*

In this reference architecture, Azure Data Factory serves as the ingestion layer, connecting to various data sources and loading raw data into Azure Data Lake Storage. Once data is staged, Databricks reads from these staging locations, processes and transforms the data, and stores the results in Delta Lake format for downstream analytics and BI tools such as Power BI.

## Copy activity pattern

The ADF Copy Activity pattern shown here is an example of reference architecture. Setting up ADF requires creating an Azure Data Factory instance in the Azure portal, configuring linked services for your source and sink, and building a pipeline—steps outside the scope of this chapter. For detailed, step-by-step ADF setup instructions, refer to Microsoft's documentation at `https://learn.microsoft.com/en-us/azure/data-factory/quickstart-create-data-factory`. The code snippets below show the JSON pipeline configuration to illustrate the pattern conceptually.

**Basic Copy Activity Configuration**

1. Create a pipeline in ADF Studio
2. Add Copy Activity with source (e.g., SQL Database) and sink (ADLS)

3. Configure to write as Parquet for optimal Databricks performance
4. Add Databricks Notebook activity to process the staged data

```
// Example Copy Activity configuration (simplified)
{
    "name": "CopySQLToADLS",
    "type": "Copy",
    "source": {
        "type": "AzureSqlSource",
        "sqlReaderQuery": "SELECT * FROM sales.orders WHERE order_date >=
'@{formatDateTime(addDays(utcNow(), -1), 'yyyy-MM-dd')}'"
    },
    "sink": {
        "type": "ParquetSink",
        "storeSettings": {
            "type": "AzureBlobFSWriteSettings"
        }
    }
}
```

## ADF to Databricks patterns

The most common integration pattern follows a staged approach: ADF extracts data from source systems and stages it in ADLS, then triggers a Databricks notebook to transform the data into Delta Lake tables. This provides separation of concerns—ADF handles data movement complexity, Databricks handles transformation logic. The ADLS staging layer serves as a recovery point if transformations need to be rerun.

Here's a typical ADF pipeline that stages data from a source to ADLS, then triggers a Databricks notebook to process it:

```
// ADF Pipeline with Databricks Notebook Activity
{
    "activities": [
        {
            "name": "CopyToADLS",
            "type": "Copy",
            "dependsOn": []
        },
        {
            "name": "ProcessWithDatabricks",
```

```
            "type": "DatabricksNotebook",
            "dependsOn": [{"activity": "CopyToADLS"}],
            "typeProperties": {
                "notebookPath": "/Shared/ingestion/process_staged_data",
                "baseParameters": {
                    "source_path": "@activity('CopyToADLS').output.files[0].path"
                }
            }
        }
    ]
}
```

**Note**

The example above uses the `DatabricksNotebook` activity for simplicity. However, the current recommended approach is to use the native **Databricks Job activity** (`DatabricksJob`) in ADF, which triggers Databricks Workflows jobs instead of running notebooks directly. The Databricks Job activity provides better monitoring, retry capabilities, and parameter passing. See the Databricks blog post "Best Practices for Kicking Off Databricks Workflows Natively from Azure Data Factory" (`https://www.databricks.com/blog/best-practices-kicking-databricks-workflows-natively-azure-data-factory`) for details on this pattern. Chapter 8 covers orchestration integration in more depth.

The Databricks notebook then reads from the staged location:

```
# Databricks notebook: process_staged_data
dbutils.widgets.text("source_path", "")
source_path = dbutils.widgets.get("source_path")
# Read staged data
df = spark.read.parquet(source_path)
# Write to Delta Lake
df.write \
    .format("delta") \
    .mode("append") \
    .saveAsTable("sales_db.orders")
```

**Key Takeaway**: Use ADF for data movement and scheduling, and use Databricks for data transformation and processing.

# Connecting to relational databases

Relational databases remain the backbone of transactional systems, storing operational data for customer records, financial transactions, inventory, orders, and countless other business-critical workloads. Azure SQL Database and SQL Server are Microsoft's primary relational database offerings, powering millions of applications across enterprises.

Ingesting data from these databases into Azure Databricks is one of the most fundamental patterns in data engineering. This supports building data warehouses from operational databases, creating analytical datasets for business intelligence, performing historical trend analysis, and feeding machine learning models with features from transactional systems. The challenge is balancing completeness with performance—extracting necessary data without overloading operational databases with heavy analytical queries.

Databricks also offers Lakehouse Federation, which lets you query external databases directly from Databricks without moving data. With Lakehouse Federation, you create a connection and a foreign catalog in Unity Catalog, then query external database tables as if they were native Databricks tables using standard SQL. This is the recommended approach when you need to query external data on-site rather than ingest it. Lakehouse Federation supports Azure SQL Database, SQL Server, PostgreSQL, MySQL, and other common databases. The JDBC approach shown in this section remains relevant for batch ingestion scenarios that involve physically copying data into Delta Lake for transformation and analytics.

## JDBC connectivity basics

> **Note**
>
> JDBC connectivity requires a classic compute cluster (standard or dedicated access mode). Serverless compute, which is the only option available in trial workspaces, does not support the JDBC connector. If you are using a trial workspace, you will need to upgrade to a paid workspace to run the JDBC examples in this chapter. Alternatively, consider using the Lakehouse Federation (described above), which supports serverless compute.

**Java Database Connectivity (JDBC)** is the standard API for connecting to relational databases from Spark:

```
# Basic JDBC connection to Azure SQL Database
jdbc_url = "jdbc:sqlserver://yourserver.database.windows.net:
1433;database=yourdatabase;encrypt=true"
```

```
connection_properties = {
    "user": dbutils.secrets.get(scope="azure-key-vault", key="sql-username"),
    "password": dbutils.secrets.get(scope="azure-key-vault", key="sql-password"),
    "driver": "com.microsoft.sqlserver.jdbc.SQLServerDriver"
}
# Read entire table
df = spark.read.jdbc(
    url=jdbc_url,
    table="sales.orders",
    properties=connection_properties
)
display(df)
```

Always remember to use Databricks Secrets for credentials; never hardcode passwords.

## Setting up sample data in Azure SQL

To run the database examples in this chapter, you need an Azure SQL Database with the sample tables described below. If you do not already have one, follow Microsoft's quickstart guide to create an Azure SQL Database: `https://learn.microsoft.com/en-us/azure/azure-sql/database/single-database-create-quickstart`. When configuring networking, ensure you enable "Allow Azure services and resources to access this server" so that Databricks can connect. Once the database is provisioned, execute the setup script provided in `sql_scripts/setup_azure_sql_sample.sql` from the chapter's GitHub repository to create the sample schema and tables.

```
-- Create sample schema and tables
CREATE SCHEMA sales;
GO
-- Create customers table
CREATE TABLE sales.customers (
    customer_id INT PRIMARY KEY,
    customer_name NVARCHAR(100) NOT NULL,
    email NVARCHAR(100),
    city NVARCHAR(50),
    state NVARCHAR(50),
    created_date DATETIME2 DEFAULT GETDATE(),
    updated_date DATETIME2 DEFAULT GETDATE()
);
-- Create orders table
CREATE TABLE sales.orders (
```

```
    order_id INT PRIMARY KEY,
    customer_id INT NOT NULL,
    order_date DATETIME2 NOT NULL,
    total_amount DECIMAL(18, 2) NOT NULL,
    order_status NVARCHAR(20) NOT NULL,
    updated_date DATETIME2 DEFAULT GETDATE(),
    FOREIGN KEY (customer_id) REFERENCES sales.customers(customer_id)
);
-- Insert sample data
INSERT INTO sales.customers (customer_id, customer_name, email, city, state)
VALUES
    (1, 'John Smith', 'john.smith@email.com', 'Seattle', 'WA'),
    (2, 'Jane Doe', 'jane.doe@email.com', 'Portland', 'OR'),
    (3, 'Bob Johnson', 'bob.johnson@email.com', 'San Francisco', 'CA');
INSERT INTO sales.orders (order_id, customer_id, order_date, total_amount,
order_status)
VALUES
    (1001, 1, '2024-01-15 10:30:00', 1250.00, 'Completed'),
    (1002, 2, '2024-01-16 14:20:00', 750.50, 'Completed'),
    (1003, 1, '2024-01-17 09:15:00', 2100.00, 'Completed');
-- Create index for better performance
CREATE INDEX idx_orders_updated_date ON sales.orders(updated_date);
GO
```

**Note**

We create an index on the `updated_date` column because it serves as the watermark column for incremental loading. When the Databricks incremental load job queries for records where `updated_date` > `last_watermark`, this index allows SQL Server to efficiently locate only the new or modified records rather than scanning the entire table.

## Full table load example

Complete example of loading data from Azure SQL Database:

Here's where you can find the code in the GitHub repository: `02_azure_sql_full_load.py`

```
# Complete Azure SQL Database to Databricks Batch Load
from pyspark.sql.functions import col, current_timestamp
# Database connection configuration
jdbc_url = "jdbc:sqlserver://<yourserver>.database.windows.net:
```

```
1433;database=salesdb;encrypt=true"
connection_properties = {
    "user": dbutils.secrets.get(scope="azure-key-vault", key="sql-username"),
    "password": dbutils.secrets.get(scope="azure-key-vault", key="sql-password"),
    "driver": "com.microsoft.sqlserver.jdbc.SQLServerDriver"
}
# Create schema in Unity Catalog
spark.sql("CREATE SCHEMA IF NOT EXISTS sales_db")
print("Loading customers table...")
# Read customers table
customers_df = spark.read.jdbc(
    url=jdbc_url,
    table="sales.customers",
    properties=connection_properties
)
# Add ingestion metadata.
customers_enriched = customers_df.withColumn("ingestion_timestamp",
current_timestamp())
# Write to Delta Lake
customers_enriched.write \
    .format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable("sales_db.customers")
print(f"Loaded {customers_enriched.count()} customers")
print("Loading orders table...")
# Read orders table
orders_df = spark.read.jdbc(
    url=jdbc_url,
    table="sales.orders",
    properties=connection_properties
)
# Add metadata
orders_enriched = orders_df.withColumn("ingestion_timestamp", current_timestamp())

# Write to Delta Lake
orders_enriched.write \
    .format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable("sales_db.orders")
```

```
print(f"Loaded {orders_enriched.count()} orders")
# Verify the load
print("\nData verification:")
display(spark.sql("""
SELECT o.order_id, o.order_date, c.customer_name, o.total_amount, o.order_status
FROM sales_db.orders o
JOIN sales_db.customers c ON o.customer_id = c.customer_id
ORDER BY o.order_date DESC
"""))
```

## Incremental Loading Pattern

For large tables that change slowly, incremental loading is much more efficient.

Code reference: `03_azure_sql_incremental_load.py` in the GitHub repository

```
# Incremental Load using Timestamp Watermark
from pyspark.sql.functions import col, max as max_, current_timestamp
from delta.tables import DeltaTable
jdbc_url = "jdbc:sqlserver://yourserver.database.windows.net:
1433;database=salesdb"
connection_properties = {
    "user": dbutils.secrets.get(scope="azure-key-vault", key="sql-username"),
    "password": dbutils.secrets.get(scope="azure-key-vault", key="sql-password"),
    "driver": "com.microsoft.sqlserver.jdbc.SQLServerDriver"
}
# Create watermark table
spark.sql("CREATE SCHEMA IF NOT EXISTS control")
spark.sql("""
CREATE TABLE IF NOT EXISTS control.watermarks (
    source_table STRING,
    last_watermark_value TIMESTAMP,
    updated_at TIMESTAMP
) USING DELTA
""")
# Initialize watermark if needed
source_table = "sales.orders"
spark.sql(f"""
MERGE INTO control.watermarks AS target
USING (SELECT '{source_table}' AS source_table) AS source
ON target.source_table = source.source_table
WHEN NOT MATCHED THEN
```

```
INSERT (source_table, last_watermark_value, updated_at)
VALUES ('{source_table}', timestamp('1900-01-01'), current_timestamp())
""")
# Get last watermark
last_watermark = spark.sql(f"""
SELECT last_watermark_value
FROM control.watermarks
WHERE source_table = '{source_table}'
""").first()[0]
print(f"Last watermark: {last_watermark}")
# Read incremental data from source
incremental_query = f"""
(SELECT *
FROM {source_table}
WHERE updated_date > '{last_watermark}') AS incremental_data
"""
incremental_df = spark.read.jdbc(
    url=jdbc_url,
    table=incremental_query,
    properties=connection_properties
)
record_count = incremental_df.count()
if record_count > 0:
    print(f"Found {record_count} new/updated records")
    # Add ingestion metadata
    incremental_enriched = incremental_df.withColumn("ingestion_timestamp",
current_timestamp())
    # Check if target table exists
    if not spark.catalog.tableExists("sales_db.orders"):
        # First load
        incremental_enriched.write \
            .format("delta") \
            .mode("overwrite") \
            .saveAsTable("sales_db.orders")
    else:
        # Merge into existing table
        delta_table = DeltaTable.forName(spark, "sales_db.orders")

        delta_table.alias("target").merge(
            incremental_enriched.alias("source"),
            "target.order_id = source.order_id"
```

```
        ).whenMatchedUpdateAll() \
         .whenNotMatchedInsertAll() \
         .execute()
    # Update watermark.
    new_watermark = incremental_enriched.agg(max_("updated_date")).first()[0]
    spark.sql(f"""
    UPDATE control.watermarks
    SET last_watermark_value = timestamp('{new_watermark}'),
        updated_at = current_timestamp()
    WHERE source_table = '{source_table}'
    """)
    print(f"Incremental load completed. New watermark: {new_watermark}")
else:
    print("No new records to process")
```

# Performance optimization tips

1. **Predicate Pushdown and Query Pushdown**
   When reading from databases via JDBC, Spark automatically applies predicate pushdown—an optimization in which filter conditions are sent to the source database rather than applied in Spark after all data has been transferred. For example:

   ```
   # Spark automatically pushes the filter to the database
   df = spark.read.jdbc(url =jdbc_url, table="sales.orders",
   properties=connection_properties)
   filtered_df = df.filter("order_date >= '2024-01-01'")
   ```

   Even though the filter is applied after the read call, Spark's query optimizer recognizes that the filter can be evaluated at the source. You can verify this by calling `filtered_df.explain(True)` — the physical plan will show `PushedFilters: [GreaterThanOrEqual(order_date, 2024-01-01)]`, confirming that the filter is being pushed down to the database.

   For more complex filtering logic, you can use query pushdown by passing a SQL subquery as the table parameter. This gives you explicit control over what data the database returns:

   ```
   # Explicit query pushdown using a SQL subquery
   query = "(SELECT * FROM sales.orders WHERE order_date >= '2024-01-01') AS
   filtered_orders"
   ```

```
df = spark.read.jdbc(url=jdbc_url, table=query,
properties=connection_properties)
```

Query pushdown is useful when you need to apply joins, aggregations, or complex logic at the source—operations that the Spark optimizer cannot automatically push down.

2. **Partitioned Parallel Reads: For large tables, parallelize the read:**

```
# Parallel read using numeric partition column
df = spark.read.jdbc(
    url=jdbc_url,
    table="sales.orders",
    column="order_id",       # Partition column (must be numeric)
    lowerBound=1000,         # Minimum value
    upperBound=10000,        # Maximum value
    numPartitions=4,         # Number of parallel connections
    properties=connection_properties
)
```

# Azure Synapse Analytics

Azure Synapse Analytics is Microsoft's enterprise data warehouse solution that combines big data and data warehousing capabilities. When ingesting data from Synapse into Databricks, use the optimized Synapse connector that leverages Azure Blob Storage for efficient staging and data transfer:

```
# Reading from Azure Synapse Analytics
synapse_df = spark.read \
    .format("com.databricks.spark.sqldw") \
    .option("url", "jdbc:sqlserver://yoursynapse.sql.azuresynapse.net:
1433;database=datawarehouse") \
    .option("user", dbutils.secrets.get(scope="azure-key-vault", key="synapse-
username")) \
    .option("password", dbutils.secrets.get(scope="azure-key-vault", key="synapse-
password")) \
    .option("tempDir", "abfss://temp@yourstorage.dfs.core.windows.net/synapse-
temp") \
    .option("forwardSparkAzureStorageCredentials", "true") \
    .option("dbTable", "dbo.FactSales") \
    .load()
```

The Synapse connector is more efficient than standard JDBC for large data volumes because it uses staging storage for parallel data transfer.

Databricks now recommends using Lakehouse Federation for integrating with Azure Synapse Analytics. With Lakehouse Federation, you can create a connection to your Synapse workspace in Unity Catalog and query Synapse tables directly using standard SQL, without the need for a specialized connector or staging storage. The Synapse-specific connector shown above remains functional but is considered a legacy approach. For new implementations, evaluate Lakehouse Federation as the preferred alternative.

# Ingesting data from REST APIs

Modern applications increasingly expose data through REST APIs, making API ingestion a critical skill for data engineers. Unlike file-based or database sources, where data sits waiting to be read, APIs require active requests with proper authentication, pagination of results, rate-limit handling, and robust error recovery.

REST API ingestion is common for SaaS platforms (Salesforce, HubSpot, Stripe), social media data, IoT device management platforms, and external data providers (weather, financial markets, geolocation). The challenge lies in translating HTTP-based interactions into reliable batch ingestion patterns suitable for data pipelines. This section demonstrates authentication methods, pagination strategies, and error handling patterns for production API ingestion.

## Authentication methods

Different APIs use different authentication mechanisms.

### API keys

```
import requests
# API key in header
headers = {
    "X-API-Key": dbutils.secrets.get(scope="azure-key-vault", key="api-key")
}
response = requests.get("https://api.example.com/v1/users", headers=headers)
```

### Bearer tokens

```
# Bearer token authentication
token = dbutils.secrets.get(scope="azure-key-vault", key="bearer-token")
headers = {
    "Authorization": f"Bearer {token}"
```

```
}
response = requests.get(api_url, headers=headers)
```

## OAuth 2.0

```
# OAuth 2.0 token acquisition
def get_oauth_token():
    token_url = "https://login.provider.com/oauth/token"
    client_id = dbutils.secrets.get(scope="azure-key-vault", key="oauth-client-
id")
    client_secret = dbutils.secrets.get(scope="azure-key-vault", key="oauth-
client-secret")
    data = {
        "grant_type": "client_credentials",
        "client_id": client_id,
        "client_secret": client_secret
    }
    response = requests.post(token_url, data=data)
    if response.status_code == 200:
        return response.json()["access_token"]
    else:
        raise Exception(f"Failed to get OAuth token: {response.text}")
access_token = get_oauth_token()
headers = {"Authorization": f"Bearer {access_token}"}
```

## Handling pagination

Most APIs limit results per request. Common pagination patterns:

```
#Offset-Based Pagination:
def fetch_all_pages_offset(base_url, headers, page_size=100):
    all_data = []
    offset = 0
    while True:
        params = {"limit": page_size, "offset": offset}
        response = requests.get(base_url, headers=headers, params=params)
        if response.status_code == 200:
            page_data = response.json()
            if not page_data or len(page_data) == 0:
                break
            all_data.extend(page_data)
            offset += page_size
```

```
            print(f"Retrieved {len(all_data)} records so far...")
        else:
            print(f"Error: {response.status_code}")
            break
    return all_data
```

### Cursor-Based pagination

```
def fetch_all_pages_cursor(base_url, headers):
    all_data = []
    next_cursor = None
    while True:
        params = {"cursor": next_cursor} if next_cursor else {}
        response = requests.get(base_url, headers=headers, params=params)
        if response.status_code == 200:
            result = response.json()
            all_data.extend(result.get("data", []))
            next_cursor = result.get("next_cursor")
            if not next_cursor:
                break
        else:
            break
    return all_data
```

## Rate limiting and error handling

Implement retry logic and rate limiting to avoid API throttling:

```
import time
from requests.adapters import HTTPAdapter
from requests.packages.urllib3.util.retry import Retry
def create_session_with_retries():
    """Create a requests session with automatic retries"""
    session = requests.Session()
    retry_strategy = Retry(
        total=5,
        backoff_factor=1,
        status_forcelist=[429, 500, 502, 503, 504]
    )
    adapter = HTTPAdapter(max_retries=retry_strategy)
    session.mount("http://", adapter)
    session.mount("https://", adapter)
```

```
    return session
def fetch_with_rate_limiting(url, headers, rate_limit_per_second=10):
    """Fetch data with rate limiting"""
    session = create_session_with_retries()
    try:
        response = session.get(url, headers=headers, timeout=30)
        # Check rate limit headers
        if "X-RateLimit-Remaining" in response.headers:
            remaining = int(response.headers["X-RateLimit-Remaining"])
            if remaining < 5:
                reset_time = int(response.headers.get("X-RateLimit-Reset", 60))
                print(f"Approaching rate limit. Waiting {reset_time} seconds...")
                time.sleep(reset_time)
        # Standard rate limiting
        time.sleep(1.0 / rate_limit_per_second)
        return response
    except requests.exceptions.RequestException as e:
        print(f"Request failed: {e}")
        return None
```

## Working example: JSONPlaceholder API ingestion

The previous sections covered individual API ingestion concepts. Let's bring these together into a complete, working example that you can run in your environment. This example uses JSONPlaceholder (jsonplaceholder.typicode.com), a free fake REST API designed for testing and prototyping. While the data is fake, the patterns demonstrated here apply directly to real-world API ingestion scenarios.

This example shows authentication, pagination, error handling, rate limiting, response parsing, and Delta Lake writing in a production-ready pattern.

Here's where you can find the code in the GitHub repository: `04_rest_api_ingestion.py`

```
# Complete REST API to Databricks Pipeline
# Using JSONPlaceholder - a free fake REST API (no API key needed!)
import requests
import json
from datetime import datetime
from pyspark.sql.functions import col, current_timestamp
import pandas as pd
# JSONPlaceholder API - No authentication required
BASE_URL = "https://jsonplaceholder.typicode.com"
```

```
def fetch_users():
    """Fetch all users from the API"""
    url = f"{BASE_URL}/users"
    try:
        response = requests.get(url, timeout=10)
        if response.status_code == 200:
            users = response.json()
            # Add query timestamp to each record
            for user in users:
                user["query_timestamp"] = datetime.now().isoformat()
            return users
        else:
            print(f"Error fetching users: {response.status_code}")
            return []
    except Exception as e:
        print(f"Exception fetching users: {e}")
        return []
def fetch_posts():
    """Fetch all posts from the API"""
    url = f"{BASE_URL}/posts"
    try:
        response = requests.get(url, timeout=10)
        if response.status_code == 200:
            posts = response.json()
            for post in posts:
                post["query_timestamp"] = datetime.now().isoformat()
            return posts
        else:
            print(f"Error fetching posts: {response.status_code}")
            return []
    except Exception as e:
        print(f"Exception fetching posts: {e}")
        return []
# Fetch users data
print("Fetching users from API...")
users_data = fetch_users()
print(f"Retrieved {len(users_data)} users")
# Fetch posts data
print("Fetching posts from API...")
posts_data = fetch_posts()
print(f"Retrieved {len(posts_data)} posts")
```

```
# Convert users to Spark DataFrame
# Use pandas as intermediate format (works on shared clusters)
if users_data:
    # Convert to pandas DataFrame first
    users_pdf = pd.DataFrame(users_data)
    # Convert pandas to Spark DataFrame
    users_df_raw = spark.createDataFrame(users_pdf)
    # Flatten nested address and company structures
    users_df = users_df_raw.select(
        col("id").alias("user_id"),
        col("name"),
        col("username"),
        col("email"),
        col("phone"),
        col("address.city").alias("city"),
        col("address.geo.lat").alias("latitude"),
        col("address.geo.lng").alias("longitude"),
        col("company.name").alias("company_name")
    ).withColumn("ingestion_timestamp", current_timestamp())
    # Create schema in Unity Catalog
    spark.sql("CREATE SCHEMA IF NOT EXISTS sandbox")
    # Write to Delta Lake
    users_df.write \
        .format("delta") \
        .mode("overwrite") \
        .saveAsTable("sandbox.users")
    print(f"Loaded {users_df.count()} users to Delta Lake")
    display(users_df.limit(5))
# Convert posts to Spark DataFrame
if posts_data:
    # Convert to pandas DataFrame first
    posts_pdf = pd.DataFrame(posts_data)
    # Convert pandas to Spark DataFrame
    posts_df_raw = spark.createDataFrame(posts_pdf)
    posts_df = posts_df_raw.select(
        col("id").alias("post_id"),
        col("userId").alias("user_id"),
        col("title"),
        col("body").alias("content")
    ).withColumn("ingestion_timestamp", current_timestamp())
    # Write to Delta Lake
```

```
    posts_df.write \
        .format("delta") \
        .mode("overwrite") \
        .saveAsTable("sandbox.posts")
    print(f"Loaded {posts_df.count()} posts to Delta Lake")
# Create an enriched view joining users and posts
print("\nCreating enriched view...")
spark.sql("""
    CREATE OR REPLACE VIEW sandbox.posts_with_authors AS
    SELECT
        p.post_id,
        p.title,
        u.name as author_name,
        u.company_name,
        u.city
    FROM sandbox.posts p
    JOIN sandbox.users u ON p.user_id = u.user_id
""")
print("\nAPI data ingestion completed successfully!")
display(spark.sql("SELECT * FROM sandbox.posts_with_authors LIMIT 10"))
```

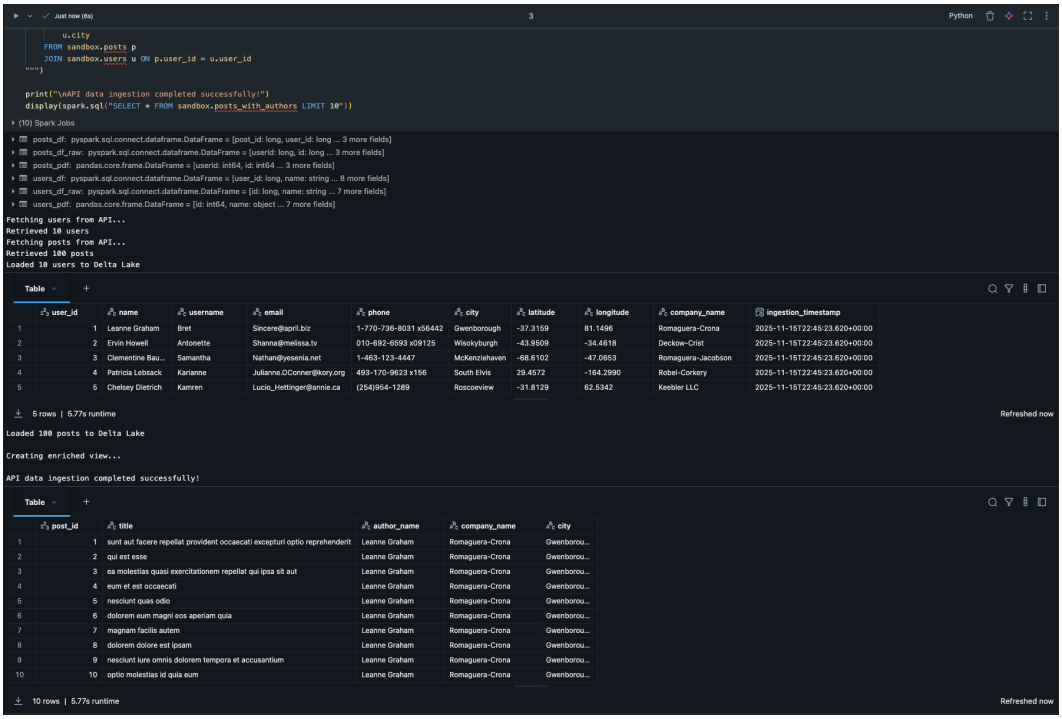



*Figure 3.3: Databricks notebook execution showing successful API calls with output*

**Key Takeaways for API Ingestion:**

- Always use authentication secrets; never hardcode API keys
- Implement pagination for APIs that limit results
- Add rate limiting to respect API quotas
- Use retry logic for transient failures
- Flatten nested JSON structures for a tabular format
- Add ingestion timestamps for tracking

# Other data sources

Beyond Azure storage, databases, and APIs, Databricks can ingest data from several other sources.

## Azure cosmos DB

Azure Cosmos DB is Microsoft's globally distributed NoSQL database service, designed for applications requiring low-latency access to data across multiple regions. While relational

databases excel at structured, transactional data, Cosmos DB handles semi-structured documents, key-value pairs, graph data, and other NoSQL scenarios.

Common use cases for ingesting Cosmos DB data into Databricks include analyzing application telemetry and user behavior stored in Cosmos DB, aggregating IoT sensor data for historical trend analysis, and joining operational Cosmos DB data with analytical datasets in your data lake. Databricks provides a native connector optimized for Cosmos DB's partitioning model:

```
# Reading from Cosmos DB SQL API
cosmos_config = {
    "spark.cosmos.accountEndpoint": "https://
yourcosmosaccount.documents.azure.com:443/",
    "spark.cosmos.accountKey": dbutils.secrets.get(scope="azure-key-vault",
key="cosmos-key"),
    "spark.cosmos.database": "ecommerce",
    "spark.cosmos.container": "products"
}
cosmos_df = (
    spark.read.format("cosmos.oltp")
    .options(**cosmos_config)
    .option(
        "spark.cosmos.read.customQuery",
        "SELECT * FROM c WHERE c.status = 'Active' AND c.created_date >=
'2024-01-01'"
    )
    .load()
)
# Add ingestion metadata and write to Delta
cosmos_enriched = cosmos_df.withColumn("ingestion_timestamp", current_timestamp())
cosmos_enriched.write \
    .format("delta") \
    .mode("overwrite") \
    .saveAsTable("catalog.ecommerce.products")
```

## SFTP/FTP sources

Despite the dominance of cloud storage and APIs, many legacy systems and external partners still exchange data via SFTP (SSH File Transfer Protocol) or FTP (File Transfer Protocol). Banks, government agencies, healthcare systems, and established enterprises often mandate SFTP for secure file transfer.

Ingesting from SFTP requires a two-step process: first download files from SFTP to ADLS using Python libraries, then process them using standard Databricks file ingestion patterns. This separation allows you to leverage Databricks' distributed processing while handling the connection-oriented nature of SFTP. Here's how to download files from SFTP servers:

```
import paramiko
def download_sftp_files(hostname, username, password, remote_path, local_path):
    """Download files from SFTP server"""
    transport = paramiko.Transport((hostname, 22))
    transport.connect(username=username, password=password)
    sftp = paramiko.SFTPClient.from_transport(transport)
    try:
        files = sftp.listdir(remote_path)
        for filename in files:
            if filename.endswith('.csv'):
                remote_file = f"{remote_path}/{filename}"
                local_file = f"{local_path}/{filename}"
                sftp.get(remote_file, local_file)
                print(f"Downloaded {filename}")
        sftp.close()
        transport.close()
    except Exception as e:
        print(f"Error: {e}")
# Download to DBFS
sftp_host = "sftp.example.com"
sftp_user = dbutils.secrets.get(scope="azure-key-vault", key="sftp-username")
sftp_pass = dbutils.secrets.get(scope="azure-key-vault", key="sftp-password")
download_sftp_files(sftp_host, sftp_user, sftp_pass, "/exports", "/dbfs/mnt/sftp-
data")
# Read downloaded files
df = spark.read.option("header", "true").csv("/mnt/sftp-data/*.csv")
```

**Note**

For production workloads, consider having data providers use Azure Blob Storage or ADLS instead of SFTP for better integration and performance.

## Summary

In this chapter, we explored essential data ingestion strategies for Azure Databricks, focusing on batch ingestion patterns that underpin enterprise data platforms.

We began by understanding the differences between batch and streaming ingestion. Batch ingestion processes data in scheduled chunks and is ideal for historical analysis, regulatory reporting, and cost-sensitive workloads. Streaming ingestion (covered in detail in Chapter 5) handles real-time data processing with Event Hubs and Auto Loader.

We examined ingesting data from Azure Storage (ADLS Gen2 and Blob Storage), and learned about authentication methods, including Managed Identities (the recommended approach, as set up in Chapter 2), service principals with OAuth 2.0, and access keys. We explored reading key file formats—CSV for data exchange, JSON for semi-structured data, and Parquet for optimal analytical performance. We implemented a simple watermark-based incremental loading pattern for processing only new data.

Azure Data Factory was covered as a data movement tool, using Copy Activity to stage data from various sources into ADLS, where Databricks then processes it. We distinguished between ADF's strengths in data movement and Databricks' capabilities in transformation, understanding that orchestration patterns are covered in Chapter 8.

We dove deep into JDBC-based relational database connectivity, building complete, working examples for Azure SQL Database. We implemented both full table loads and incremental loading patterns using watermark tracking and Delta Lake MERGE operations. We covered performance optimization through predicate pushdown and partitioned parallel reads, and introduced the Azure Synapse Analytics connector for large-scale data-warehouse ingestion.

REST API ingestion was explored comprehensively, including authentication methods (API keys, Bearer tokens, OAuth 2.0), pagination patterns (offset and cursor-based), and rate limiting with retry logic. The working example demonstrated a complete API-to-Delta-Lake pipeline using the JSONPlaceholder API, covering data fetching, JSON flattening, and writing to managed Delta tables.

Finally, we covered additional data sources, including Azure Cosmos DB with its native Spark connector and SFTP for legacy system integration.

Throughout this chapter, we emphasized security best practices by consistently storing credentials in Databricks Secrets and recommending using explicit schemas rather than schema inference for production reliability. We wrote all data to Delta Lake format, preparing for the deep dive into Delta Lake's ACID transactions and schema evolution features in Chapter 6.

With these ingestion strategies, you are now equipped to bring data from diverse sources into Azure Databricks, setting the foundation for building robust data platforms that can transform raw data into valuable business insights.

The ingestion patterns covered in this chapter form the foundation for the data transformation workflows we'll explore next. Chapter 4, "Data Transformation with Apache Spark," builds directly on these ingestion strategies by showing you how to clean, validate, and transform the raw data you've ingested into high-quality, analytics-ready datasets.

Where this chapter focused on getting data into Databricks from various sources, Chapter 4 will teach you what to do with that data once it arrives - from handling data quality issues and managing schema evolution, to implementing complex business logic and optimizing transformation performance. Together, these chapters provide the complete ingestion-to-transformation pipeline that forms the core of modern data engineering.

## Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to `packtpub.com/unlock`). Search for this book by name, confirm the edition, and then follow the steps on the page.

UNLOCK NOW

*Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.*

# 4

# Data Engineering with Apache Spark

Organizations today face enormous challenges when processing and analyzing large-scale datasets. The sheer volume, velocity, and variety of data can overwhelm traditional data processing systems, leading to issues with complexity, performance, scalability, and operational management. Apache Spark has emerged as a leading solution to these challenges, providing a powerful, unified platform for batch processing, real-time streaming, machine learning, and interactive analytics. Azure Databricks enhances Spark's capabilities by offering an enterprise-grade, fully managed cloud platform with advanced features for security, cluster management, and team collaboration.

This chapter provides a comprehensive exploration of Spark's core architecture and its synergistic relationship with Azure Databricks. We will delve into key performance optimization techniques, establish best practices for writing reliable and efficient code, and outline effective strategies for managing large-scale data operations. A thorough understanding of Spark's internal execution model is essential for developing high-quality applications, and this chapter will equip you with the knowledge to build data processing systems that are not only performant but also scalable and resilient.

Here's what we're going to cover in this chapter:

- Spark's Core Architecture: A detailed look at the fundamental components of Spark's architecture, including the driver, executors, and cluster manager
- Execution Model and Job Scheduling: An exploration of Spark's execution model, lazy evaluation, and the role of the Catalyst Optimizer
- Memory Management and Storage: A deep dive into Spark's memory management system and various storage levels
- Writing Efficient **PySpark** and Scala Code: Best practices for writing clean, efficient, and maintainable Spark code
- Advanced Optimization Techniques: An overview of advanced optimization strategies, such as broadcast joins, bucketing, and dynamic partition pruning
- Handling Large-Scale Data Processing: Techniques for scaling Spark applications to handle massive datasets

# Apache Spark architecture and execution model

Apache Spark is a powerful open-source distributed computing system that has become the de facto standard for big data processing. It was introduced to address the limitations of Hadoop MapReduce, offering significant performance improvements and a more flexible programming model. Spark provides a unified platform for a wide range of data processing tasks, including batch processing, real-time streaming, machine learning, and graph analytics. Its architecture is designed for speed, scalability, and ease of use, enabling developers to build complex data pipelines with high-level APIs in Scala, Python, R, and Java.

## Core components and cluster architecture

Apache Spark implements a controller-worker architecture that distributes computational workloads across multiple machines while maintaining a unified programming interface. This architectural approach enables efficient processing of large datasets by leveraging parallel computation and distributed storage capabilities inherent in modern cluster computing environments.

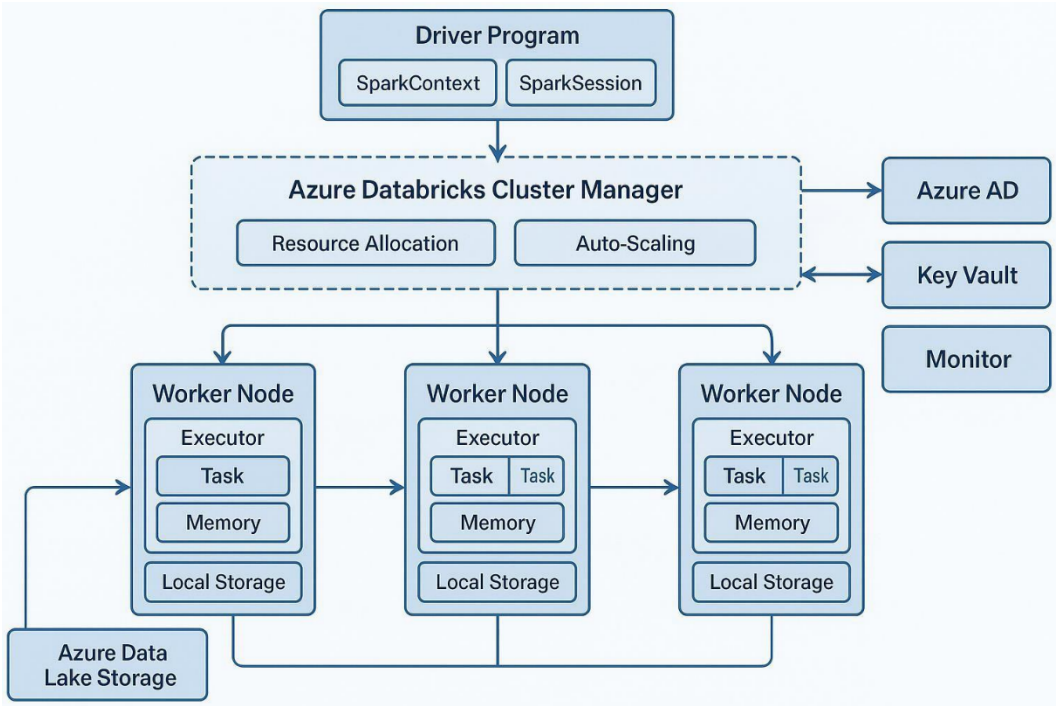



*Figure 4.1: Apache Spark Cluster Architecture on Azure Databricks showing the relationship between Driver, Executors, and Cluster Manager*

## Applications of spark architecture

Understanding the roles of the Spark Driver, Executors, and Cluster Manager is fundamental to writing efficient and robust Spark code.

## Spark driver

**Spark Driver** acts as the command center for your Spark applications. It creates **SparkContext**, converts your code into **directed acyclic graphs** (**DAGs**), and manages execution across the cluster. The driver runs your application's main function and keeps track of everything - task scheduling, collecting results, and deciding how to use resources.

When you submit an application, the driver examines your transformations and actions, builds an optimized execution plan using the Catalyst optimizer, and sends tasks to executor processes on worker nodes. The driver also collects results from distributed computations and sends them back to your client application.

## Executors

**Executors** are the worker processes that actually run your computational tasks and store data. Each executor runs in its own **Java Virtual Machine** (**JVM**) and can handle multiple tasks simultaneously using different threads. Executors communicate directly with the driver for task coordination and with each other when they need to exchange data during shuffle operations, which is the process of redistributing data across partitions, often between stages of a Spark job.

Executor memory management is quite sophisticated. It dynamically splits resources between execution memory (for operations like joins and aggregations) and storage memory (for caching **DataFrames** and **RDDs**). This unified memory system automatically adapts to different workload patterns, allowing memory regions to expand and contract as your application's needs change. An executor also includes Reserved memory (300 Mb) and User Memory (40%).

## Cluster manager

The **Cluster Manager** handles resource allocation across your distributed infrastructure. Azure Databricks provides an advanced cluster management system that goes beyond basic resource allocation. It includes smart features such as Enhanced Autoscaling, automatic node replacement upon failures, and deep integration with Azure services for improved reliability and performance.

Databricks' cluster manager is reactive, not predictive. It responds to the current workload to scale the cluster up or down. This is different from Predictive Optimization, which uses runtime statistics to improve the query plan, not to scale the cluster.

The following code example shows how to configure a Spark session. In a Databricks notebook environment, the spark session is automatically created for you. The following code is for demonstration purposes, showing how you would configure these settings in a standalone Spark application. In Databricks, these configurations are typically set at the cluster level in the Spark Configuration.

```
from pyspark.sql import SparkSession
# Create a Spark session with Azure Databricks optimizations
spark = (
    SparkSession.builder.appName("DatabricksOptimizedSession")
    .config("spark.databricks.adaptive.enabled", "true")
    .config("spark.databricks.delta.optimizeWrite.enabled", "true")
    .config("spark.databricks.delta.autoCompact.enabled", "true")
    .getOrCreate()
)
# Your Spark application code here
```

This example shows how to enable several key Databricks optimizations. While it's good to be aware of these settings, in a modern Databricks environment, many of these, like **Adaptive Query Execution** (**AQE**), are enabled by default. It is best practice to manage these configurations in the cluster settings rather than within the notebook code itself to ensure consistency and avoid overriding important defaults.

Having explored the fundamental components of Spark's architecture, we now turn our attention to how Spark actually executes your code. Understanding the execution model is crucial for writing efficient applications and diagnosing performance issues when they arise.

## Execution model and job scheduling

Understanding Spark's execution model is key to unlocking its full potential. The model is built on the principle of lazy evaluation, in which transformations (e.g., `filter()`, `select()`, `withColumn()`) are not executed immediately but are compiled into a logical execution plan. This plan is then optimized by the Catalyst Optimizer and executed only when an action (e.g., `collect()`, `count()`, `show()`, `display()`) requires the results to be materialized. This approach enables Spark to perform sophisticated optimizations, yielding significant performance gains.

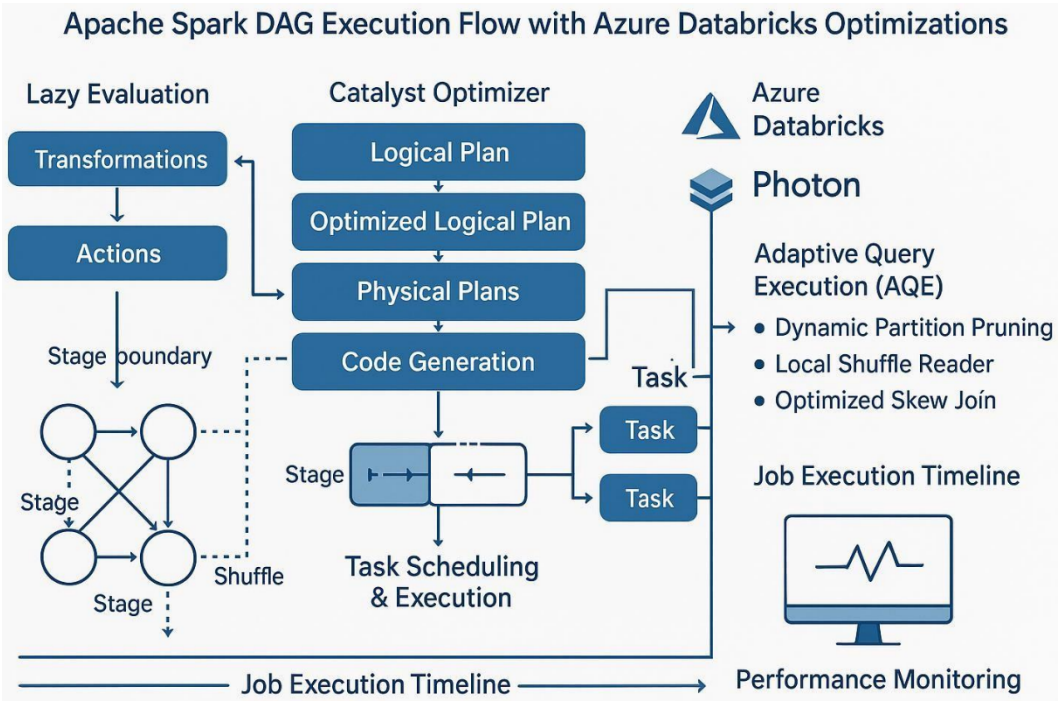



*Figure 4.2: Spark DAG Execution Flow showing lazy evaluation, transformations, actions, and stage boundaries*

Lazy evaluation gives Spark a major advantage by letting the Catalyst optimizer examine your entire sequence of operations before executing anything. The optimizer can remove unnecessary steps, reorder transformations for better performance, push filters down to the data source level, and combine multiple operations into a single execution stage when appropriate.

Spark follows a structured execution process that balances performance with reliability. When you trigger an action on a DataFrame or RDD, the execution engine examines the entire chain of transformations and builds an optimized execution plan. This plan minimizes data movement, maximizes parallel processing, and leverages data locality wherever possible.

The following example illustrates how lazy evaluation and the construction of a DAG work in practice:

```
from pyspark.sql.functions import count
# In Databricks, the 'spark' session is already available.
# Load a dataset
df = spark.read.format("csv").option("header", "true").load("/Volumes/
analytics_dev/sales_raw/staging/dataset.csv")
```

```
# Define a series of transformations
transformed_df = (
    df.filter(df["age"] > 30)
    .groupBy("department")
    .agg(count("*").alias("employee_count"))
)
# At this point, no computation has occurred. We can view the plan.
transformed_df.explain(True)
# Now, trigger an action to execute the plan
results = transformed_df.collect()
# The results are now materialized and can be displayed
for row in results:
    print(row)
```

This code demonstrates how Spark builds execution plans, applies Catalyst optimizations, and runs operations across your cluster. It clearly shows the difference between defining transformations and their actual execution, highlighting the performance gains from lazy evaluation.

**Job scheduling** in Spark uses a hierarchical structure that efficiently uses resources and provides fault tolerance. Your Spark applications contain one or more jobs, where each job matches an action in your code. Jobs break down into stages, which are groups of tasks that can run together without shuffling data between cluster nodes.

Stage boundaries are determined by the type of transformation. Narrow transformations, such as select or filter, can be performed on a single partition without requiring data from other partitions. These are efficient and don't trigger a shuffle. Wide transformations, like `groupBy` or `join`, require data from multiple partitions to be exchanged across the cluster. This data exchange is called a shuffle, and it marks the boundary between stages. Shuffles are expensive operations as they involve network I/O and disk I/O, so minimizing them is key to performance. Within each stage, Spark creates individual tasks that represent the actual work units executed on specific data partitions. The number of tasks in a stage usually matches the number of input partitions, though various configuration settings and optimizations can change this.

Azure Databricks improves Spark's job scheduling with several smart features. The platform includes **adaptive query execution** (**AQE**), which adjusts execution plans based on runtime statistics, automatic optimization of join strategies using actual data characteristics, and query re-optimization that adapts to changing workload needs during execution. For example, AQE can dynamically change join strategies or coalesce partitions based on the actual data sizes it observes at runtime, which is different from resource allocation.

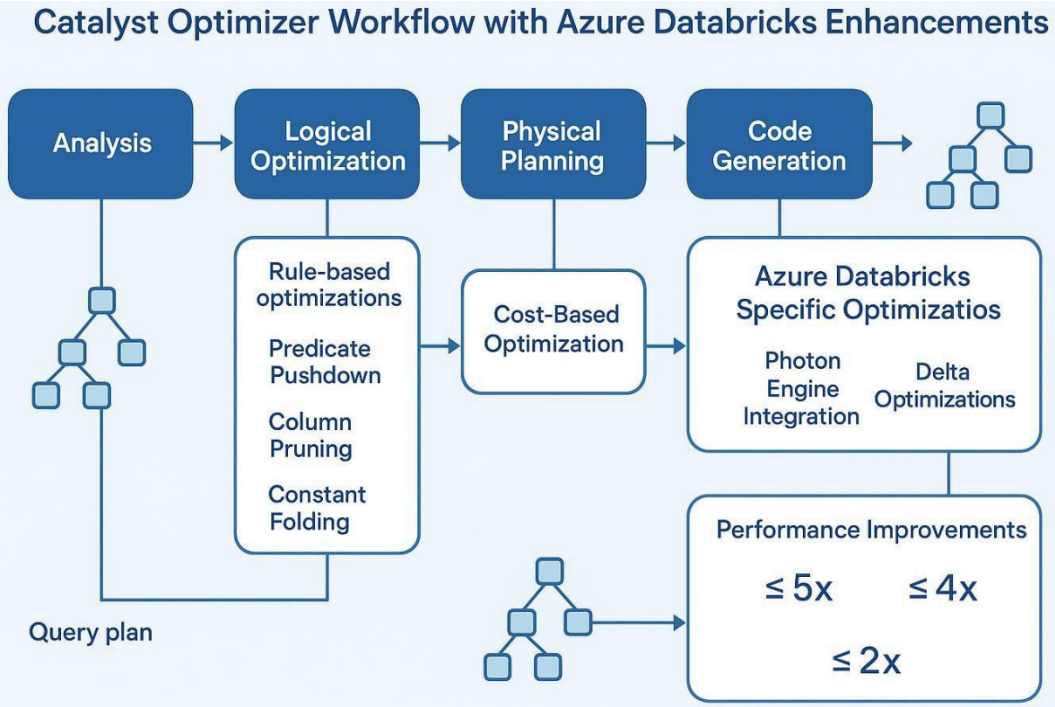



*Figure 4.3: Catalyst Optimizer workflow showing the four main optimization phases and Azure Databricks enhancements*

The **Catalyst Optimizer** is one of Spark's most powerful features. It takes your high-level DataFrame or SQL operations and transforms them into optimized physical execution plans. Catalyst combines rule-based and cost-based optimization techniques to create execution plans that outperform straightforward implementations.

The optimization process occurs in several phases. Initially, Catalyst analyzes the logical plan to understand the query structure and applies rule-based optimizations, such as predicate pushdown, column pruning, and constant folding. Following this, it generates multiple candidate physical execution plans and employs cost-based optimization to select the most efficient option based on statistical information about the data.

Azure Databricks further enhances the Catalyst Optimizer with additional optimizations tailored for cloud environments and Delta Lake storage. These enhancements include optimizations for cloud storage access patterns, intelligent caching decisions based on data access analytics, and integration with the Photon engine. Photon provides highly optimized, vectorized execution for SQL and equivalent DataFrame operations on Delta and Parquet tables, often leading to significant performance gains.

## Memory management and storage

Spark's memory management system uses smart allocation strategies that adjust automatically to different workload patterns while giving administrators the controls they need to optimize performance for specific use cases. The system splits executor memory into separate regions, each designed for specific parts of the data processing pipeline.

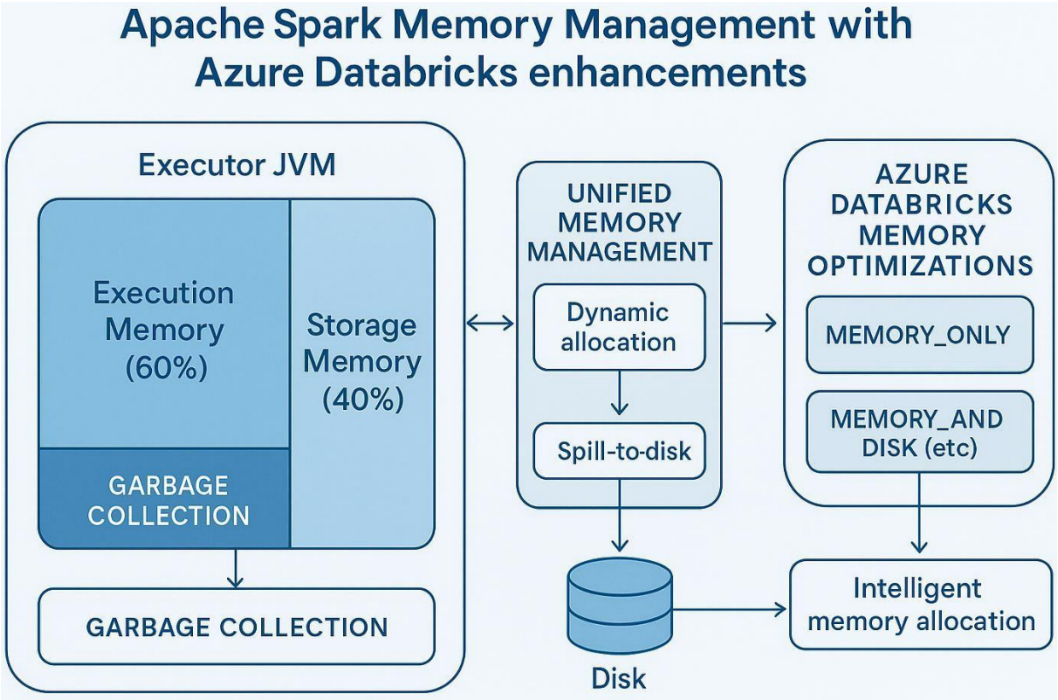



*Figure 4.4: Apache Spark Memory Management showing executor JVM layout, storage levels, and spill-to-disk mechanisms*

**Execution memory** represents the largest allocation within executor memory, dedicated to computational operations such as joins, aggregations, and sorting. This memory region stores intermediate results during complex operations and serves as the workspace for in-memory sorting and hashing, which significantly improves performance compared to disk-based alternatives.

**Storage memory** is allocated for caching DataFrames and RDDs that are explicitly cached or persisted through application code. When applications invoke `cache()` or `persist()` operations on DataFrames, Spark stores the data in memory to enable subsequent operations to access it without recomputation or re-reading from the original data sources. This capability

can provide substantial performance improvements for iterative algorithms and scenarios where identical data is accessed multiple times.

Spark's **unified memory management** system represents an elegant solution that allows execution and storage memory to dynamically borrow from each other based on current application requirements. When applications perform extensive caching with minimal computation, storage memory can expand into execution memory space. Conversely, when applications execute complex aggregations without significant caching requirements, execution memory can utilize space typically allocated to storage. Note that execution memory has priority over storage memory; if an operation needs more memory for execution, it can evict data from the storage cache.

## Memory configuration and optimization

Effective memory configuration is critical for achieving optimal performance in Spark applications, particularly when processing large datasets or performing memory-intensive operations.

The following code example demonstrates essential memory configuration parameters and their performance implications:

```
from pyspark.sql import SparkSession
# Create a Spark session with custom memory configurations
spark = (
    SparkSession.builder.appName("MemoryManagementDemo")
    .config("spark.driver.memory", "4g")
    .config("spark.executor.memory", "8g")
    .config("spark.memory.fraction", "0.8")
    .config("spark.memory.storageFraction", "0.3")
    .getOrCreate()
)
# Your Spark application code here
# Stop the Spark session
spark.stop()
```

Azure Databricks adds several improvements to Spark's memory management that boost both performance and reliability. The platform includes smart memory allocation algorithms that automatically adjust memory distribution based on your workload characteristics, advanced garbage collection tuning that reduces pause times, and integration with Azure's high-performance storage systems that handle efficient data spilling when memory runs out.

**Garbage collection** performance is critical for Spark's memory management, especially for applications that process large datasets or run for long periods. Poor garbage collection can cause significant application pauses, reducing overall throughput and potentially causing timeouts in distributed operations.

Azure Databricks uses optimized garbage collection settings tuned for typical Spark workloads. The platform uses the G1 garbage collector by default, which provides consistent pause times even with large heap sizes. Databricks also offers comprehensive monitoring and alerting to help you spot garbage collection issues before they affect application performance.

**Off-heap storage** is an important part of Spark's memory management approach. While most memory usage occurs within the JVM heap, certain operations benefit from storing data outside the heap to reduce garbage-collection pressure and improve memory efficiency. Azure Databricks supports off-heap storage configurations that work especially well for applications with large caching needs or memory-intensive operations.

Good memory management in Spark requires understanding the trade-offs between different configuration options and continuously monitoring how your application uses memory. Azure Databricks provides comprehensive monitoring tools that help you understand application memory usage and find optimization opportunities.

## Azure Databricks runtime enhancements

Azure Databricks Runtime builds on Apache Spark with a comprehensive set of improvements that boost performance, reliability, and ease of use. These enhancements provide significant value, setting Azure Databricks apart from standard Apache Spark distributions.

The **Photon Engine** is one of the biggest performance improvements in Azure Databricks Runtime. This vectorized query engine, built in C++, can dramatically speed up SQL workloads and DataFrame operations. Photon works particularly well for queries with filters, joins, and aggregations on large datasets, often delivering 2-3x performance gains over standard Spark execution engines.

Photon's vectorized processing operates on batches of rows at once rather than processing individual rows one by one. This approach leverages modern CPU architectures and significantly reduces the overhead of function calls and data movement. Photon automatically kicks in for supported operations and data types, so you don't need to change any existing application code.

**Delta Lake integration** is another major enhancement in Azure Databricks Runtime. While Delta Lake is available as an open-source project, the integration within Azure Databricks includes additional optimizations and features you won't find elsewhere. These include automatic optimization features like auto-compaction and vacuum, enhanced statistics collection for better query planning, and seamless integration with other Azure services.

The runtime includes **adaptive query execution** (**AQE**) improvements that go beyond standard Spark capabilities. Azure Databricks' AQE can dynamically adjust partition sizes, optimize join strategies based on runtime statistics, and modify physical execution plans during query execution when it finds better optimization opportunities.

**Disk Caching** in Azure Databricks Runtime automatically caches data on the local storage of worker nodes. When a file is read from a remote source, a copy is placed in the disk cache. Subsequent reads of the same data can then be served from the local disk, which is significantly faster than reading from remote storage. This process is transparent to the user and does not require any code changes. It is important to note that disk caching is reactive; it caches data after it has been read for the first time and does not pre-cache data based on query analytics.

**Enhanced security features** are built throughout the runtime, including automatic encryption of data in transit and at rest, integration with **Microsoft Entra ID** (formerly Azure Active Directory) for authentication and authorization, and comprehensive audit logging that tracks all data access and modifications. These security capabilities meet enterprise compliance requirements while maintaining high performance.

The runtime includes **optimized connectors** for Azure services like Azure Data Lake Storage, Azure SQL Database, and Azure Cosmos DB. These connectors are specifically tuned for optimal performance and include features like automatic retry logic, intelligent connection pooling, and optimized data transfer protocols that significantly improve data ingestion and export operations.

**Automatic cluster management** features in Azure Databricks Runtime include intelligent auto-scaling. It is a reactive mechanism that adjusts the number of worker nodes in a cluster based on the current workload. It does not use predictive algorithms but rather responds to metrics like the number of pending tasks to scale up and idleness to scale down. This ensures that your applications have the right amount of computational power without manual intervention, providing a balance between performance and cost.

These enhancements work together to create a platform that not only outperforms standard Spark distributions but also provides better reliability, security, and operational efficiency. Understanding how to effectively use these enhancements is essential for getting the most value from your Azure Databricks investment and building data processing solutions that meet enterprise requirements for performance, reliability, and security.

# Optimizing spark jobs with partitioning and caching

Optimizing Spark jobs requires a deep understanding of how data is distributed and accessed across your cluster. This section explores two fundamental optimization techniques: data

partitioning and caching. Effective partitioning strategies ensure that data is distributed evenly across nodes, enabling maximum parallelism and minimizing data movement. Caching allows frequently accessed data to be stored in memory for rapid retrieval. Together, these techniques form the foundation of high-performance Spark applications.

## Data partitioning strategies

Data partitioning represents one of the most critical optimization techniques in Apache Spark, fundamentally determining how data is distributed across cluster nodes and directly impacting performance, resource utilization, and parallel processing capabilities. Effective partitioning strategies enable Spark to maximize parallelism while minimizing data movement, thereby achieving optimal performance for diverse workload patterns.

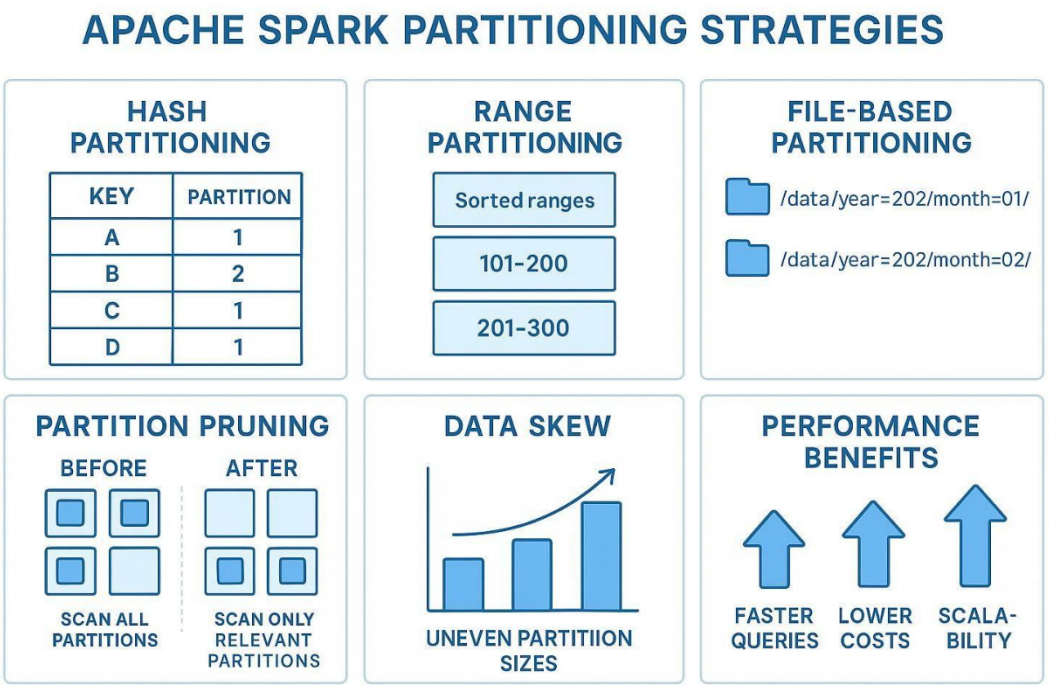



*Figure 4.5: Different Apache Spark partitioning strategies, including hash, range, file-based, and custom partitioning, with performance optimization examples*

Data partitioning makes or breaks performance in distributed systems. Good partitioning lets Spark process different chunks of data simultaneously across your cluster, making full use of available resources and keeping execution times low. Bad partitioning creates data skew—some nodes finish quickly while others struggle with massive workloads, creating bottlenecks that slow everything down.

**Hash partitioning** is the default for most Spark operations. It applies a hash function to one or more columns to determine which partition each row is assigned to. This usually spreads data evenly across partitions, provided your hash keys are well-distributed. Hash partitioning shines with join operations because it puts rows with matching keys on the same node, avoiding expensive data shuffles.

The beauty of hash partitioning is its simplicity and versatility. For joins, Spark groups all rows with the same join keys on a single node, eliminating costly shuffle operations. This can dramatically speed up large-scale joins.

**Range partitioning** works differently—it sorts your data and splits it into partitions based on value ranges. This approach excels when you frequently query data using range conditions, like time-based queries or filters on sorted columns. Range partitioning allows partition pruning, where Spark skips entire partitions that don't have the data you need.

Consider a dataset partitioned by date ranges. A query filtering for a specific month can ignore all partitions outside that timeframe. This drastically reduces data processing, especially for time-series analytics.

**File-based partitioning** (sometimes called Hive-style partitioning) stores data in separate directories based on partition values. This works great with columnar formats like Parquet or Delta Lake, where the directory structure helps with partition pruning. Queries filtering on partition columns can skip entire directories, reducing both disk reads and processing overhead.

Spark's default partitioning works reasonably well for most cases but knowing how it works helps you optimize for specific scenarios. Spark creates partitions based on your input data characteristics, cluster resources, and settings like `spark.sql.files.maxPartitionBytes` and `spark.default.parallelism`.

Here's an example that shows different partitioning strategies in action:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col
# Create a Spark session
spark = SparkSession.builder.appName("PartitioningStrategiesDemo").getOrCreate()
# Load a dataset
df = spark.read.format("parquet").load("path/to/dataset.parquet")
# Hash partitioning (default for most operations)
hash_partitioned_df = df.repartition(10, "user_id")
# Range partitioning
range_partitioned_df = df.repartitionByRange(10, "timestamp")
# Write with file-based partitioning
```

```
df.write.partitionBy("year", "month").parquet("path/to/output")
# Analyze partition distribution
print(f"Number of partitions: {hash_partitioned_df.rdd.getNumPartitions()}")
# Stop the Spark session
spark.stop()
```

This code explores hash partitioning, range partitioning, and file-based partitioning with partition pruning. It demonstrates how to analyze partition distribution, measure performance differences, and apply best practices for various data access patterns.

## Selecting optimal partitioning strategies

Picking the best partitioning strategy depends on understanding your data characteristics, query patterns, and performance needs. There's no one-size-fits-all solution, but some basic principles can help guide your decisions.

For **analytical workloads** that often filter on date or time columns, range partitioning by date or timestamp usually works best. This lets you skip irrelevant partitions and can dramatically cut down the amount of data you need to process for time-based queries.

For **join-heavy workloads**, hash partitioning on join keys can eliminate expensive shuffle operations and significantly boost performance. This works especially well when you're using the same join keys repeatedly across multiple operations in your application.

For **mixed workloads** that combine analytical queries with join operations, hybrid approaches often work best. You might use file-based partitioning for efficient analytical queries while making sure frequently joined data sits together through hash partitioning.

The secret to good partitioning is understanding how your data actually gets used and picking strategies that match real usage patterns. Monitoring query performance and reviewing execution plans provides valuable insights into how well your partitioning works and where you can improve. For Delta Lake tables, Databricks now recommends **Liquid Clustering** as a more flexible and efficient alternative to traditional partitioning. Liquid clustering can be defined on any column and is not limited to a fixed set of partition columns, simplifying data layout management and improving query performance.

## Caching and persistence mechanisms

Caching is one of the best optimization tricks in Apache Spark. It lets you store frequently used data in memory so you can access it quickly in later operations. When done right, caching can dramatically speed up applications that reuse the same data, especially in iterative algorithms and interactive analytics.

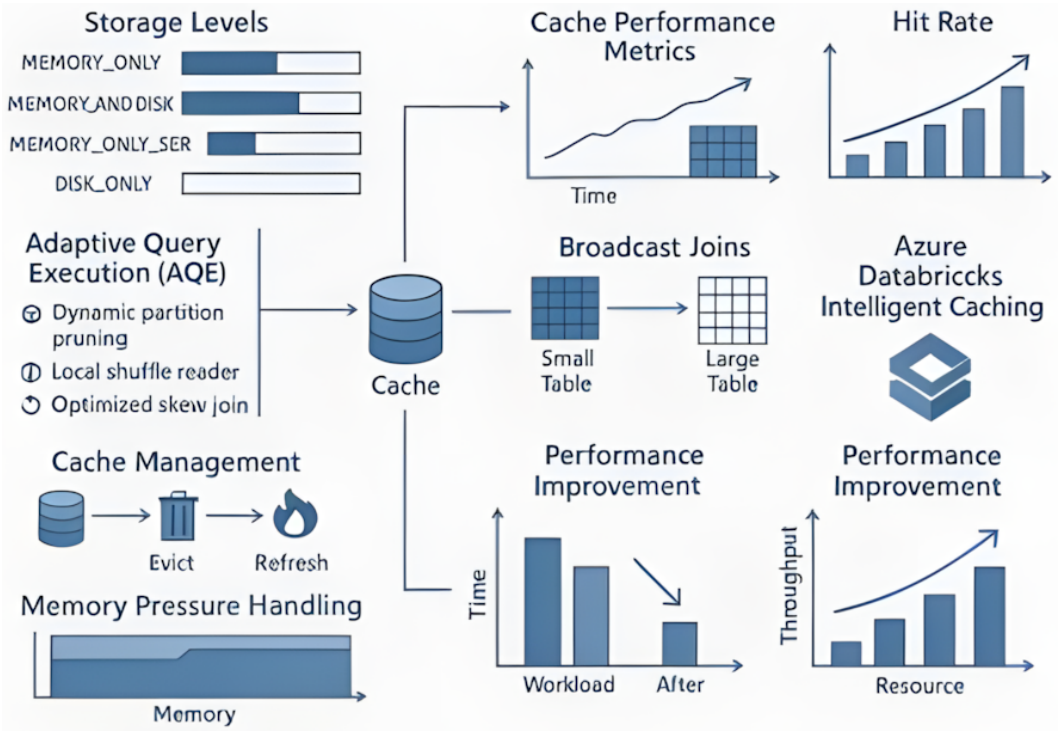



*Figure 4.6: Spark caching and optimization strategies showing storage levels, cache performance, broadcast joins, and Adaptive Query Execution features*

Spark offers several storage levels that balance memory usage, serialization overhead, and fault tolerance differently. Understanding these options and when to use them is key to building effective caching strategies that boost performance while making good use of your resources.

The following example shows different caching strategies in action:

```
from pyspark.sql import SparkSession
from pyspark.storagelevel import StorageLevel
# Create a Spark session
spark = SparkSession.builder.appName("CachingStrategiesDemo").getOrCreate()
# Load a large dataset
df = spark.read.format("parquet").load("path/to/large/dataset.parquet")
# Cache the DataFrame in memory only
df.persist(StorageLevel.MEMORY_ONLY)
# Perform some operations on the cached DataFrame
result1 = df.filter(df["column1"] > 100).count()
result2 = df.groupBy("column2").count()
```

```
# The DataFrame is now in memory, so these operations will be faster
# Unpersist the DataFrame to free up memory
df.unpersist()
# Now, let's cache another DataFrame with a different storage level
df2 = spark.read.format("csv").load("path/to/another/dataset.csv")
df2.persist(StorageLevel.MEMORY_AND_DISK_SER)
# This will store the
DataFrame in memory as serialized objects, and spill to disk if memory is full
# ... perform operations on df2 ...
df2.unpersist()
# Stop the Spark session
spark.stop()
```

This code demonstrates various storage levels, how to measure caching benefits, and advanced techniques like broadcast joins and adaptive query execution. It shows effective cache management strategies and how to monitor cache performance in production.

`MEMORY_ONLY` is the fastest storage level available. It keeps cached data in memory as regular Java objects. This gives you the best access times but uses the most memory and doesn't protect against failures—if a node crashes, you lose the cached data and have to rebuild it. Use this for frequently accessed data that fits easily in memory when speed matters more than fault tolerance.

`MEMORY_AND_DISK` balances performance with fault tolerance by storing data in memory when possible but spilling to disk when memory runs out. This is often the best choice for production applications because it gives good performance while ensuring cached data doesn't disappear when memory gets tight.

`MEMORY_ONLY_SER` stores data in memory in compressed form, which uses less memory than MEMORY_ONLY but requires CPU time to compress and decompress. This works well when memory is limited, but you want to avoid disk I/O.

`DISK_ONLY` stores cached data only on disk, providing fault tolerance without using memory but with much slower access times than memory-based options. This is rarely used but can work for very large datasets that you don't access often.

## Caching and persistence mechanisms

Good cache management goes beyond just picking storage levels. You need to think about when to cache data, when to remove it, and how to monitor whether caching actually improves performance rather than just consuming resources.

When it comes to caching, it's about caching data you'll access multiple times during your application run. The basic rule is: if you'll use a DataFrame or RDD more than once, caching probably helps. But watch your memory usage carefully and focus on caching the data that gives you the biggest performance boost.

Monitoring cache hit rates helps you know if your caching strategy actually works. Azure Databricks provides monitoring tools that show cache hit rates, memory usage, and the impact of caching on performance. You can find these tools in the Spark UI, which is accessible directly from the cluster details page in your Databricks workspace. Navigate to the Storage tab in the Spark UI to see a list of all currently cached RDDs and DataFrames, along with information about their size, the number of partitions cached, and the fraction of data cached in memory versus on disk. The Executors tab also provides memory usage metrics for each executor, which can help you determine whether your cache is causing memory pressure. Low hit rates might mean you're caching the wrong data or your cache gets cleared too often.

**Smart cache eviction** keeps the most valuable data in cache when memory runs low. Spark uses a **Least Recently Used** (**LRU**) policy by default, which works well for most cases. But understanding how your data gets accessed can help you make better decisions about cache sizing and management.

**Weighing caching costs and benefits** means considering both memory usage and the time it takes to populate the cache. Caching huge datasets might not help if it takes longer to cache the data than you save on later accesses. And caching data you only use once just wastes memory.

## Advanced optimization techniques

Advanced Spark optimization requires a deeper understanding of the framework's internals but delivers significant performance gains for demanding workloads. These techniques build on basic partitioning and caching to tackle specific performance challenges in complex data processing.

**Broadcast joins** are highly effective for joins involving small tables. When one table fits in memory across all executors, Spark broadcasts it to every node, eliminating expensive shuffle operations. This dramatically improves performance, especially for star schema queries joining large fact tables with smaller dimension tables.

Azure Databricks automatically detects broadcast opportunities based on table statistics. You can use explicit hints with the `broadcast()` function or SQL hints. The key is ensuring broadcasted tables actually fit in memory—broadcasting large tables hurts performance by overwhelming the network and memory.

**Bucketing** improves join performance by pre-partitioning data based on join keys. When tables are bucketed on identical columns with the same bucket count, joins happen without data shuffling. This works especially well for data warehouse scenarios with repeated joins on the same keys. It is important to note that bucketing is not supported for Delta Lake tables.

**Dynamic partition pruning** dramatically improves star schema query performance by using dimension table filters to prune fact table partitions at runtime. For example, when joining the sales and product tables and filtering by product categories, it uses the filtered product IDs to skip irrelevant fact table partitions.

**AQE** represents a major advancement, letting the optimizer make decisions based on runtime statistics rather than just static analysis. AQE dynamically adjusts partition sizes, optimizes join strategies, and modifies execution plans mid-query when it finds better opportunities.

Azure Databricks enhances AQE with cloud-specific optimizations, including intelligent partition coalescing to reduce small-file overhead, dynamic join-strategy selection based on actual data sizes, and automatic shuffle optimization.

Liquid Clustering is the recommended replacement for Z-ordering and partitioning for Delta tables. It is a more flexible and efficient way to organize data. Z-ordering is a Delta Lake-specific technique that can improve performance for queries filtering on multiple columns by reorganizing data within files to co-locate related data. However, Liquid Clustering provides a more adaptive and automated way to achieve similar benefits without the need for manual tuning.

This works particularly well for analytical workloads filtering on column combinations, like queries filtering on both date and customer ID, or product category and region. Z-ordering significantly improves these query types while maintaining good single-column filter performance.

Successfully implementing these techniques requires understanding your specific workload characteristics and performance needs. Not every technique helps every workload, and some optimizations can hurt performance if applied incorrectly.

The best approach is implementing optimizations incrementally, measuring their impact, and adjusting based on actual results. Azure Databricks monitoring tools, Spark UI, and query execution plans help identify bottlenecks and optimization opportunities for data-driven decisions.

# Writing efficient PySpark and scala code

Writing efficient and maintainable code is crucial for building robust and scalable Spark applications. While Spark's optimizer can perform many optimizations automatically, the

structure and quality of your code can have a significant impact on performance and reliability. This section provides best practices for writing high-quality PySpark and Scala code, covering topics such as DataFrame API best practices, SQL optimization techniques, and the use of **user-defined functions** (**UDFs**).

## DataFrame API best practices

The DataFrame API is the foundation of modern Spark development. It combines Spark's Catalyst optimizer performance with accessible programming interfaces. Writing efficient DataFrame code means understanding how the Catalyst optimizer works and structuring your code to help it generate the best execution plans.

DataFrame operations give the Catalyst optimizer hints to apply sophisticated optimizations. When you structure your code accordingly, you can achieve significant performance improvements. Azure Databricks enhances this with vectorized execution through Photon, improved memory management, and seamless Delta Lake integration.

Good filtering is one of the most important optimizations in Spark. The order and structure of your filters dramatically impact performance by determining how much data subsequent operations process. The optimizer can push filters down to the data source, potentially eliminating the need to read large portions of the dataset.

**Early filtering** reduces data volume for later operations. Apply filters before joins to reduce dataset sizes being joined. **Column pruning** means selecting only the needed columns to reduce memory usage, network traffic, and I/O. **Transformation chaining** should balance conciseness with clarity.

Spark's built-in functions provide highly optimized implementations that outperform custom code. They're compiled into efficient bytecode and leverage Catalyst optimizations that user-defined functions can't achieve.

Use built-in functions for string processing (`regexp_replace`, `split`, `concat`), date/time operations (`date_add`, `datediff`, `window`), and mathematical functions (`round`, `sum`, `avg`).

These handle complexities automatically and provide significant performance benefits over custom implementations.

Window functions enable sophisticated analytics that consider related rows based on window specifications. Performance depends on partitioning and ordering clauses—good designs enable parallel execution while poor designs force expensive shuffles.

Partition by natural data divisions (customer ID for customer analytics, time periods for temporal analytics). Common patterns include running totals, moving averages, ranking functions, and lag/lead comparisons.

Joins are often the most expensive Spark operations. **Broadcast joins** work best for small tables that fit in memory—Spark broadcasts them to all nodes, eliminating shuffles. **Sort-merge joins** handle larger tables by shuffling and sorting both datasets. **Hash joins** use hash tables and can be more efficient for certain patterns.

Optimize joins by ordering smaller tables first, filtering before joining, and avoiding skewed join keys. The Catalyst optimizer typically automatically selects the best strategy based on data characteristics.

## SQL optimization techniques

Spark SQL provides a familiar interface for data analysts and engineers comfortable with SQL, while still enabling access to all of Spark's optimization capabilities through the Catalyst optimizer. Writing efficient Spark SQL requires understanding how SQL constructs translate into DataFrame operations and how to structure queries to leverage available optimizations.

Azure Databricks enhances Spark SQL with cost-based optimization, enhanced statistics collection, and Photon engine integration for vectorized SQL execution.

The Catalyst optimizer uses both rule-based and cost-based techniques to generate efficient execution plans. **Rule-based optimizations** include predicate pushdown, column pruning, and join reordering. **Cost-Based Optimization** (**CBO**) is enabled by default in Azure Databricks. It uses statistical information about data distributions (such as table size and the number of distinct values per column) to estimate the cost of different query plans and choose the most efficient one. This allows it to make intelligent decisions about join strategies, join order, and other execution choices. You can collect the statistics needed by CBO by running `ANALYZE TABLE my_table COMPUTE STATISTICS FOR ALL COLUMNS`.

Understanding how the optimizer works lets you write queries that maximize available optimizations. This includes structuring WHERE clauses for predicate pushdown, organizing JOINs for optimal ordering, and using appropriate aggregate functions for efficient aggregation.

**Predicate pushdown** applies filters as early as possible in query execution, often at the data source level. This dramatically reduces the data volume that needs to be read and processed, especially for columnar formats like Parquet and Delta Lake.

When filters are pushed down, entire files or row groups can be skipped when they don't contain matching data. This provides significant performance improvements for queries filtering on indexed or partitioned columns.

**Filter ordering** impacts performance since some filters are more selective than others. Highly selective filters should be applied early to minimize data processed in subsequent operations.

SQL join optimization is critical for query performance. Join algorithm choice depends on table sizes, join key distribution, and available memory and network resources.

**Join hints** can be valuable when automatic optimization doesn't achieve optimal results. The BROADCAST hint forces broadcast joins for small tables, while MERGE forces sort-merge joins for larger tables. Use hints judiciously since they prevent the optimizer from adapting to changing data characteristics.

**Join ordering** is important for multi-join queries. Generally, join smaller tables first and apply highly selective filters before join operations.

**Subqueries and Common Table Expressions (CTEs)** structure complex queries but must be used thoughtfully. Correlated subqueries referencing outer query columns can be expensive since they evaluate for each outer row. Non-correlated subqueries can be optimized more effectively.

**CTEs** improve the readability and maintainability of complex queries. While they don't necessarily provide performance benefits over subqueries, they can make queries easier to understand and sometimes enable optimizations not possible with nested subqueries.

The choice among EXISTS, IN, and join operations significantly impacts performance. EXISTS is often more efficient than IN for large result sets since it can short-circuit once a match is found.

Here is an example of SQL optimization techniques:

```
-- Using a Common Table Expression (CTE) for readability
WITH filtered_sales AS (
  SELECT
    product_id,
    amount
  FROM
    sales
  WHERE
    sale_date > '2023-01-01'
)
-- Joining with a dimension table using a broadcast hint
SELECT
  /*+ BROADCAST(p) */
  p.product_name,
  SUM(s.amount) AS total_sales
FROM
  filtered_sales s
```

```
  JOIN products p ON s.product_id = p.product_id
GROUP BY
  p.product_name
ORDER BY
  total_sales DESC;
```

This example demonstrates several SQL optimization techniques in action, including the use of CTEs for improved readability, broadcast hints for small table joins, and proper filter placement. By structuring your SQL queries with these principles in mind, you can achieve significant performance improvements. However, there are times when SQL and built-in functions aren't enough, and you need custom logic. The next section explores user-defined functions and their performance implications.

## User-Defined functions and performance

User-Defined Functions let you extend Spark with custom logic that built-in functions can't handle. However, UDFs carry significant performance costs that require careful consideration. The main issue is that UDFs break the optimization boundary—the Catalyst optimizer can't see inside UDF implementations to understand their behavior or apply optimizations.

UDFs execute in Python processes separate from JVM-based Spark executors, which requires data serialization, creating substantial overhead. Azure Databricks provides enhancements like vectorized UDFs that process multiple rows simultaneously, Arrow integration for efficient data transfer, and optimized Python environments.

Use UDFs carefully, weighing functional needs against performance costs. UDFs work best when you need functionality that built-in functions can't provide, when integrating with external libraries, or when implementing complex business logic that's difficult to express in SQL or DataFrame operations.

Performance considerations include serialization overhead between JVM and Python processes, Python execution costs compared to optimized built-in functions, and the optimizer's inability to optimize UDF operations. These factors can cause 10x or more performance degradation compared to built-in functions.

**Vectorized UDFs (Pandas UDFs)** provide significant improvements over traditional row-at-a-time UDFs by processing entire columns using vectorized operations. They use Apache Arrow for efficient data transfer and leverage NumPy and Pandas optimizations.

The key decision is whether the benefits of custom logic outweigh the performance costs. For simple transformations that can be performed with built-in functions, UDFs usually aren't worth the penalty. For complex business logic or external system integration, UDFs might be your only option.

When UDFs are necessary, several strategies minimize performance impact:

**Batch processing** with vectorized UDFs amortizes serialization overhead across multiple rows. **Caching UDF results** avoids recomputation when applying the same UDF multiple times. **Selective application** only to data subsets requiring custom processing minimizes overall impact.

**Code optimization** within UDFs helps significantly. Use efficient libraries such as NumPy and Pandas, avoid unnecessary object creation, minimize external dependencies, and structure the logic to fail fast on invalid inputs. Consider data types, null handling, and error propagation carefully. Proper error handling maintains reliability while efficient null handling prevents unnecessary computation.

**Pandas UDFs** are the most efficient approach for custom logic in PySpark, providing vectorized execution that can approach built-in function performance. They operate on Pandas Series or DataFrames, enabling vectorized operations and efficient numerical libraries.

**Grouped map UDFs** enable custom aggregation logic operating on entire data groups, implementing complex analytical functions that standard aggregation can't handle. They receive Pandas DataFrames with all group rows and can return arbitrary results.

**Iterator UDFs** process large datasets efficiently by operating on data iterators rather than materializing entire partitions in memory. This works well for UDFs processing large amounts of data or integrating with external systems having their own batching requirements.

Choose UDF types based on specific use case requirements, data characteristics, and performance needs. Understanding trade-offs between different approaches helps you make informed decisions about when and how to use custom logic in Spark applications.

# Handling Large-Scale data processing with spark

As datasets continue to grow in size and complexity, scaling Spark applications effectively becomes a critical challenge. This section introduces the core principles and strategies for handling large-scale data processing with Spark, with a focus on scaling and cluster management. A comprehensive understanding of these concepts is essential for building robust, performant Spark applications capable of handling the demands of modern data workloads.

## Scaling strategies and cluster management

Scaling Apache Spark applications effectively requires a comprehensive understanding of distributed computing principles and the ability to orchestrate computational resources across multiple machines while maintaining optimal performance characteristics. The complexity of large-scale data processing extends beyond simply adding computational power; it

encompasses the sophisticated orchestration of resources to maximize throughput while minimizing operational costs.

Modern data workloads exhibit significant variation in resource requirements, ranging from CPU-intensive transformations to memory-heavy aggregations to I/O-bound data loading operations. Effective scaling strategies must accommodate these diverse workload characteristics and provide the flexibility to adapt to changing computational requirements throughout application execution.

Azure Databricks provides sophisticated scaling capabilities that go beyond traditional horizontal scaling. The platform incorporates Enhanced Autoscaling, which is a reactive mechanism that adjusts cluster size based on the current workload, and deep integration with Azure's infrastructure services, enabling rapid scaling without the operational overhead of manual cluster management.

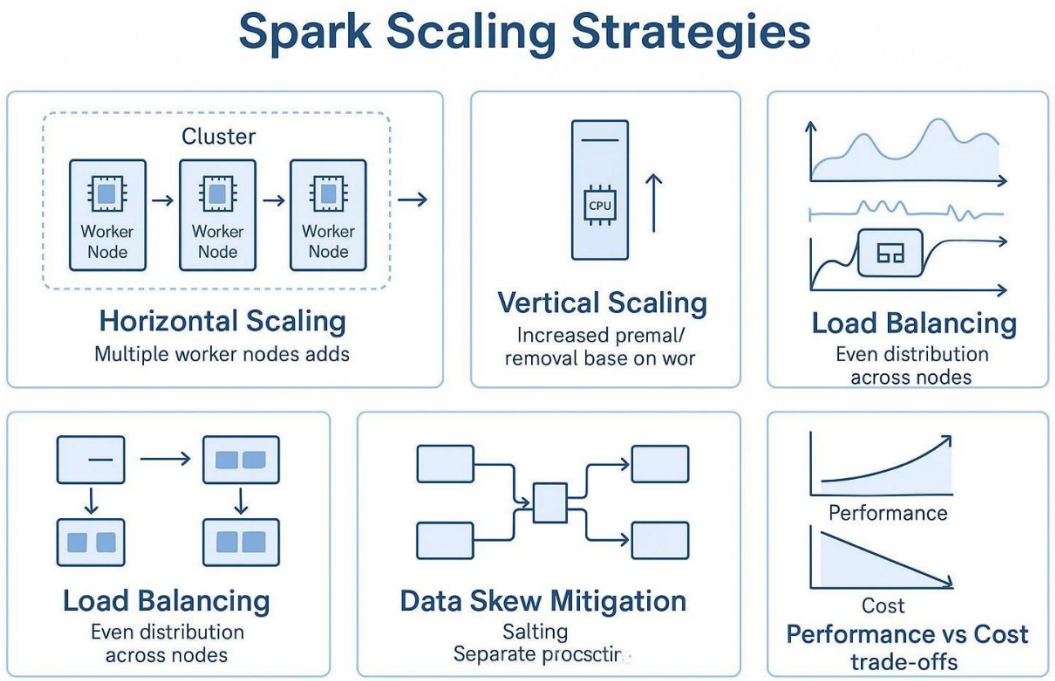



*Figure 4.7: Apache Spark scaling strategies showing horizontal scaling, vertical scaling, auto-scaling patterns, load balancing, and data skew mitigation techniques*

**Horizontal scaling** adds more worker nodes to increase overall processing capacity. This works particularly well for embarrassingly parallel workloads where data can be processed independently across multiple nodes. Well-partitioned workloads can achieve nearly linear performance improvements by simply adding more nodes.

Horizontal scaling effectiveness depends on workload characteristics and the quality of data partitioning. Workloads with independent tasks requiring minimal inter-node communication scale horizontally very well. However, workloads with frequent data shuffling or significant data skew may not benefit proportionally from additional nodes.

**Vertical scaling** increases computational resources (CPU, memory, storage) of existing nodes rather than adding more nodes. This works better for workloads constrained by individual node capacity rather than overall cluster capacity. Vertical scaling is valuable for memory-intensive operations such as large joins and complex aggregations, which benefit from more memory per node.

Choosing between horizontal and vertical scaling depends on specific workload characteristics and cloud environment costs. Azure Databricks provides flexible cluster configurations, including memory-optimized instances for analytical workloads and compute-optimized instances for CPU-intensive processing.

The following comprehensive example demonstrates scaling strategies and cluster management techniques:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, count, rand, when, concat, lit
# Create optimized Spark session for scaling
spark = (
    SparkSession.builder
    .appName("ScalingDemo")
    .config("spark.sql.adaptive.enabled", "true")
    .config("spark.sql.adaptive.skewJoin.enabled", "true")
    .config("spark.executor.memory", "8g")
    .getOrCreate()
)
# Load dataset and optimize partitioning
df = spark.read.format("parquet").load("path/to/dataset.parquet")
optimal_partitions = spark.sparkContext.defaultParallelism * 3
df = df.repartition(optimal_partitions, "customer_id")
# Handle data skew with salting
skewed_customers = (
    df.groupBy("customer_id")
    .agg(count("*").alias("cnt"))
    .filter("cnt > 1000")
)
result = (
    df.join(skewed_customers.select("customer_id"), "customer_id", "left")
```

```
    .withColumn("salt", when(col("customer_id").isNotNull(), (rand() *
10).cast("int")).otherwise(0))
    .withColumn(
        "key",
        when(col("salt") > 0, concat(col("customer_id"), lit("_"), col("salt")))
        .otherwise(col("customer_id"))
    )
    .groupBy("key")
    .agg({"amount": "sum"})
)
result.show()
spark.stop()
```

**Auto-scaling** is one of Azure Databricks' most valuable features, automatically adjusting cluster size based on workload demands. This provides significant cost savings by ensuring you pay only for resources actually needed, while ensuring applications have sufficient resources to meet performance requirements.

Azure Databricks auto-scaling is a **reactive** process. It monitors the cluster's workload and scales the number of worker nodes up or down based on current demand, not on predictive algorithms. This ensures that you have enough resources to meet performance targets while minimizing costs by removing idle resources.

Auto-scaling effectiveness depends on proper configuration of scaling policies, continuous monitoring, and a comprehensive understanding of workload patterns. **Scale-out policies** determine when and how quickly nodes are added, while **scale-in policies** determine when nodes are removed. Balancing these requires understanding trade-offs between performance, cost, and stability.

Key scaling metrics include cluster utilization rates, scaling frequency, performance impact of scaling events, and cost implications. Monitoring these metrics enables data-driven optimization of scaling policies.

Effective resource allocation goes beyond determining node count—it requires configuring nodes for optimal performance. This includes memory allocation, CPU utilization, storage configuration, and network optimization decisions that significantly impact application performance.

**Memory configuration** is critical for Spark applications. Improper memory allocation leads to out-of-memory errors, excessive garbage collection, or inefficient resource utilization. Azure Databricks provides intelligent memory allocation algorithms that automatically optimize

memory distribution, but understanding memory management principles enables fine-tuning for specific use cases.

**CPU optimization** ensures computational resources are used efficiently across the cluster. This includes task parallelism, CPU affinity, and balancing CPU and I/O operations. Modern Spark applications often benefit from configurations that maximize CPU utilization while ensuring I/O operations don't become bottlenecks.

**Storage optimization** covers both local storage configuration on cluster nodes and access to external storage systems, such as Azure Data Lake Storage. This includes storage caching, data locality, and optimizing data transfer operations, all of which significantly impact overall application performance.

## Data skew and resource optimization

Data skew is one of the biggest challenges in distributed data processing. It happens when data is unevenly distributed across partitions, causing some tasks to process substantially more data than others. This creates bottlenecks, with most of your cluster sitting idle while a few nodes struggle with oversized partitions. For Delta tables, you can use the `OPTIMIZE` command to compact small files into larger ones, which can help to improve query performance. Additionally, you can enable `autoOptimize.optimizeWrite` to automatically compact files as they are written.

Data skew can cause individual tasks to run significantly longer than others, limiting overall performance. In extreme cases, it can cause out-of-memory errors or task failures that halt entire job execution.

Azure Databricks provides features to detect and mitigate data skew, including adaptive query execution that dynamically adjusts partition sizes, intelligent join strategies for skewed keys, and monitoring tools that identify skew-related performance issues.

**Partition size analysis** reveals significant variations in data volume processed by different tasks. When some partitions are orders of magnitude larger than others, you have data skew.

**Task execution time analysis** shows that some tasks take substantially longer than others. The Spark UI provides detailed task execution times that help identify skew-related issues.

**Resource utilization monitoring** reveals uneven usage across the cluster. With data skew, some nodes run at high utilization while others are underutilized.

Data skew should be identified manually through the Spark UI. By inspecting the task execution times and the amount of data processed by each task, you can identify partitions that are significantly larger than others.

**Salting** adds a random component to skewed keys, distributing them more evenly across partitions. While it requires additional processing, it can dramatically improve performance for severely skewed datasets. Choose appropriate salt ranges and balance skew reduction with processing overhead.

**Separate processing paths** for skewed and non-skewed data involves identifying skewed keys and processing them separately using different strategies optimized for their characteristics.

**Adaptive query execution (AQE)** in Azure Databricks automatically detects and mitigates certain types of data skew during query execution. AQE dynamically adjusts partition sizes, changes join strategies, and splits large partitions to improve performance.

**Custom partitioning strategies** address skew at the source by ensuring more even data distribution from the beginning using composite partition keys or custom partitioning logic.

Processing large datasets efficiently requires careful memory management. **Columnar storage formats**, such as Parquet and Delta Lake, provide significant memory efficiency through compression and encoding, reducing memory usage by 5-10x compared to row-based formats.

**Lazy evaluation optimization** minimizes memory usage by ensuring that only necessary data materializes. **Garbage collection optimization** becomes important for applications that process large datasets or run for extended periods.

## Production deployment patterns

Deploying Spark applications to production requires comprehensive planning, testing, and monitoring. Production involves larger datasets, stricter performance requirements, higher reliability expectations, and more complex operational environments.

Azure Databricks provides enterprise-grade security, automated cluster management, comprehensive monitoring, and integration with Azure operational tools.

**Production cluster sizing** balances performance requirements with cost considerations. Choose **instance types** based on workload characteristics: memory-optimized for analytical workloads with large joins, compute-optimized for CPU-intensive tasks, storage-optimized for high I/O throughput.

**Auto-scaling configuration** for production requires conservative settings to ensure stability. Account for node startup time, scaling impact on running jobs, and cost implications.

**High-availability configuration** ensures applications continue to operate when nodes fail through multi-zone deployment, automatic node replacement, and checkpoint configuration.

**Comprehensive monitoring** tracks application performance metrics, cluster resource utilization, data quality indicators, and business metrics. Performance monitoring should track job execution times, task failure rates, memory utilization, and throughput.

**Robust error handling** includes retry strategies for transient errors, circuit breaker patterns to prevent cascading failures, checkpoint-and-recovery mechanisms, and dead-letter-queue patterns for unprocessable data.

**Automated deployment pipelines** include automated testing, configuration validation, and rollback capabilities. **Infrastructure as Code** enables consistent, repeatable deployments with version control.

# Summary

This chapter has provided a comprehensive overview of Apache Spark's architecture, execution model, and key optimization techniques. We have explored the fundamental components of Spark, including the driver, executors, and cluster manager, and examined how they work together to enable large-scale data processing. We have also delved into advanced topics such as memory management, caching strategies, and the Catalyst Optimizer, providing you with the knowledge and tools to write high-performance Spark applications. By following the best practices and strategies outlined in this chapter, you can build robust, scalable, and efficient data pipelines that can handle the challenges of modern data workloads. The combination of Spark's powerful distributed computing capabilities and Azure Databricks' enterprise-grade features provides a formidable platform for tackling the most demanding data processing tasks. While this chapter focused on batch processing and optimization techniques, modern data architectures increasingly require the ability to process data in real time as it arrives.

In the next chapter, we will explore how to build real-time data pipelines using Apache Spark Structured Streaming in Databricks. You will learn how to implement streaming data processing, manage stateful streaming workloads, and handle the unique challenges of real-time data, such as late-arriving events and fault tolerance. We will also cover optimization strategies specifically tailored for streaming applications, enabling you to build high-performance, reliable real-time data pipelines that complement the batch processing capabilities you have learned in this chapter.

## Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to `packtpub.com/unlock`). Search for this book by name, confirm the edition, and then follow the steps on the page.

*Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.*

# 5

# Building Real-Time Data Pipelines

Data streaming has become a critical capability for modern organizations that need to process and analyze data as it arrives, rather than waiting for batch processing windows. Unlike traditional batch processing, which operates on static datasets at scheduled intervals, streaming enables continuous processing of data in motion, allowing businesses to detect patterns, respond to events, and make decisions in real-time or near real-time. Whether you're monitoring IoT sensors, processing financial transactions, analyzing clickstream data, or tracking supply chain events, streaming pipelines provide the foundation for building responsive, event-driven applications that can act on insights in real time.

In this chapter, we will cover the following topics:

- Understanding streaming data and how it differs from traditional batch processing
- Examining common use cases and when to choose streaming over batch approaches
- Exploring real-time versus near real-time streaming and selecting the right approach for your needs
- Apache Spark Structured Streaming capabilities and Azure Databricks enhancements, including cloud-native integrations, auto-scaling, and enterprise security
- Building your first streaming pipeline with hands-on examples using Azure Event Hubs, Blob Storage, and Autoloader for automatic file processing
- Advanced streaming concepts, including stateful processing and handling late-arriving data
- Fault tolerance and performance optimization techniques for production-ready streaming applications

# What is Data Streaming?

Streaming data is data that comes in small pieces over time. Think of it like water flowing from a tap. The data keeps coming, one piece after another, without stopping. This continuous flow of data is what makes streaming different from other ways of handling information, such as batch processing.

The reason that streaming data is so important today is that it allows us to process information as it happens, rather than waiting to collect large amounts of data before we can work with it. This means we can make decisions faster, respond to problems quickly, and provide better experiences to users who expect real-time information.

Every day, you interact with streaming data without even realizing it. When you check your social media feed, you see new posts appearing as they are created. When you look at stock prices on your phone, those numbers update every few seconds during trading hours. When you use a fitness tracker, it continuously sends your heart rate and step count to an app on your phone. All of these are examples of streaming data in action.

The key advantage of streaming data is speed. Instead of waiting for a batch process to run at the end of the day or week, you can see what is happening right now. This real-time visibility allows businesses to detect problems quickly, respond to customer needs immediately, and make better decisions based on current information rather than outdated reports.

## Streaming vs. Batch Processing

There are two main ways to process data: streaming and batch. Understanding the difference between these approaches is crucial for choosing the right solution for your needs.

Batch processing handles large amounts of data all at once. It is like doing all your laundry on Sunday instead of washing clothes as you wear them. The system collects data over time, stores it, and then processes it in a single operation. This approach has been around for decades and is still widely used today.

The main advantage of batch processing is simplicity. You can process huge amounts of data without worrying about real-time performance. The system can take its time to do complex calculations, and you do not need to worry about handling data that arrives late or out of order. Batch processing is also typically less expensive because you can use cheaper hardware and run jobs during off-peak hours when computing resources cost less.

However, batch processing has a significant drawback: the data you are working with is always old. If you run a batch job every night, you are always looking at yesterday's information. This delay can be problematic when you need to make quick decisions or respond to problems as they happen.

Streaming processing, on the other hand, handles data as it arrives. It is like washing dishes as you use them, rather than letting them pile up in the sink. The system processes each piece of data immediately, or at least very quickly, so you can see results almost instantly.

The main advantage of streaming processing is speed. You can see what is happening right now, not what happened hours or days ago. This real-time capability is essential for applications such as fraud detection, where you need to stop suspicious transactions before they complete, and for live dashboards, where managers need current information to make decisions.

*Table 5.1* shows the key differences between batch and streaming processing.

| **Feature** | **Batch Processing** | **Streaming Processing** |
|---|---|---|
| **Data size** | Large chunks | Small pieces |
| **Speed** | Slow (hours/days) | Fast (seconds/milliseconds) |
| **Complexity** | Simple | More complex |
| **Cost** | Low | Higher |
| **Use cases** | Reports, analytics | Alerts, dashboards |
| **Data freshness** | Old data | Current data |

*Table 5.1: Key differences between batch and streaming processing.*

Below, *Figure 5.1* illustrates batch processing as a large container that fills up over time and gets processed all at once, while streaming processing shows data flowing continuously through a pipe and being processed immediately as it passes through.

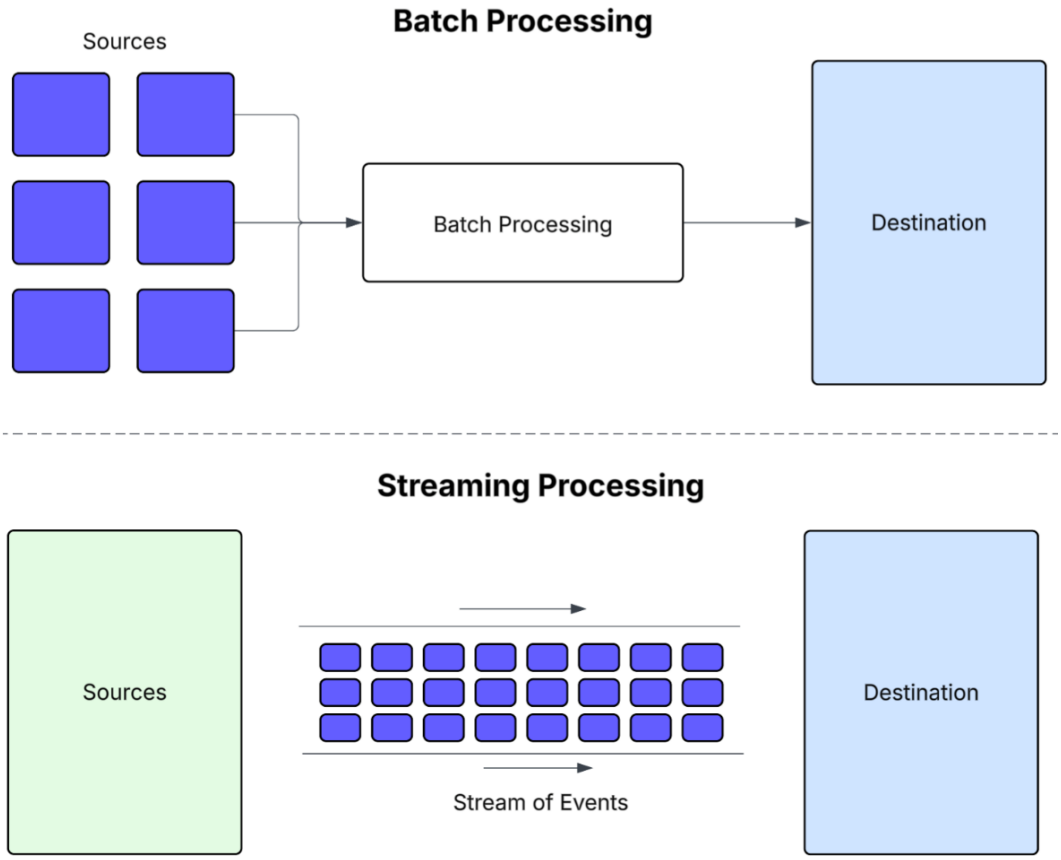



*Figure 5.1: Batch vs. Streaming Processing*

The choice between batch and streaming processing depends on your specific needs. If you are generating monthly reports or doing historical analysis, batch processing is usually the better choice. If you need real-time alerts, live dashboards, or immediate responses to events, streaming processing is the way to go.

## Common Streaming Use Cases

Now that we understand what streaming data is and how it differs from batch processing, let us look at the most common ways organizations use streaming data in practice.

Real-time analytics is one of the most popular use cases for streaming. Companies use streaming data to create live dashboards that show current performance metrics, website traffic patterns, or sales figures, all of which update in real time. Instead of waiting for a daily report to see how the business is performing, managers can check a dashboard to see what is

happening right now. This immediate visibility allows them to spot problems quickly and make adjustments before small issues become big problems.

Data ingestion is another common use case for streaming. Many organizations need to move data from various sources into their data warehouses or data lakes as quickly as possible. Streaming data pipelines can automatically detect new files in cloud storage, process them as they arrive, and load them into the appropriate destination. This approach ensures that data is available for analysis as soon as possible, rather than waiting for a scheduled batch job to run.

Event processing is where streaming really shines. Modern applications generate thousands or millions of events every second. These events might be user clicks on a website, sensor readings from IoT devices, or financial transactions. Streaming systems can process these events in real-time, triggering alerts when something unusual happens, updating user interfaces with new information, or automatically taking actions based on predefined rules.

Machine learning is becoming an increasingly important use case for streaming. Instead of training models on historical data and using them to make predictions on new data later, organizations are now using streaming data to make predictions in real-time. For example, e-commerce sites use streaming data to provide product recommendations as users browse, while financial institutions use it to detect fraudulent transactions in real time.

The key to successful streaming implementations is understanding that these use cases often overlap and work together. A single streaming system might handle real-time analytics, data ingestion, and event processing simultaneously. The data that flows through the system can serve multiple purposes, making streaming systems highly efficient and cost-effective when properly designed.

In the next section, we will explore the different types of streaming and help you choose the right approach for your specific needs.

# Types of Streaming: Real-Time vs. Near Real-Time

Let's understand the difference between Real-Time and Near Real-Time data streams. It is crucial for understanding data engineering.

## Real-Time Streaming

Real-time streaming processes data in milliseconds or seconds. This type of streaming is designed for applications where any delay is unacceptable and could result in significant problems or missed opportunities. The goal is to process and respond to data as quickly as possible, often within the same second it arrives.

The most common use cases for real-time streaming include financial trading systems, where stock prices change constantly, and traders need to make split-second decisions. Emergency

alert systems also require real-time processing, because even a few seconds' delay could mean the difference between preventing a disaster and responding to one. Live video streaming is another example in which viewers expect smooth playback without buffering delays.

Real-time streaming systems traditionally require very fast hardware and specialized infrastructure. The processing logic must be simple and efficient because there is no time for complex calculations or multiple processing steps. These systems are often used in-memory processing and dedicated hardware to achieve the necessary speed. The cost of building and maintaining such systems is typically much higher than that of other approaches due to the specialized hardware and expertise required.

Modern platforms significantly reduce these constraints. Databricks Real-Time Mode, for example, achieves end-to-end latency as low as 5 milliseconds using standard cloud infrastructure. It supports complex operators and UDFs without requiring specialized hardware. This means you can build low-latency streaming pipelines on Azure

Databricks without the traditional cost and complexity of legacy real-time systems. The trade-offs of real-time streaming are significant. While you get the fastest possible response times, the system's complexity increases dramatically. You need to handle failures gracefully, manage resources carefully, and ensure that the system can scale to handle peak loads. The high cost and complexity mean that real-time streaming should only be used when the business value justifies the additional investment.

It is worth noting that Azure Databricks has introduced Real-Time Mode for Structured Streaming, which significantly reduces many of these traditional constraints. Real-Time Mode enables end-to-end latency as low as five milliseconds by running long-running batches that process data as it arrives and scheduling all query stages simultaneously. It supports most Structured Streaming operators, including UDFs, making real-time processing much more accessible than in legacy systems. This means that the cost and complexity gap between real-time and near real-time streaming has narrowed considerably on the Databricks platform.

## Near Real-Time Streaming

Near real-time streaming processes data in seconds to minutes rather than milliseconds. This approach provides a good balance between speed and cost, making it suitable for most business applications that need relatively current data but do not require instant processing.

Social media feeds are a perfect example of near real-time streaming. When you post something on a social media platform, it appears in your followers' feeds within a few seconds or minutes. This delay is acceptable because social media is not a critical business function that requires instant updates. Recommendation engines also use near real-time streaming to suggest products or content based on recent user behavior, updating recommendations as users interact with the system.

Near real-time systems can use more complex processing logic because they have more time to work with the data. This allows for more sophisticated analytics, data transformations, and business logic. The hardware requirements are less demanding than those of real-time systems, and the overall cost is more manageable for most organizations.

The requirements for near real-time streaming are more flexible than those of real-time systems. You can use standard cloud infrastructure and commodity hardware, which significantly reduces costs. The processing can include multiple steps, data validation, and even some batch-like operations as long as the overall latency remains within acceptable limits.

## Choosing the Right Type

The choice between real-time and near real-time streaming depends on several factors that you need to consider carefully. The most important question is how fast you actually need results. Many applications that seem to require real-time processing can actually work perfectly well with near real-time processing.

*Table 5.2* provides a decision framework to help you choose the right streaming approach.

| Factor | Real-Time Streaming | Near Real-Time Streaming |
|---|---|---|
| **Latency** | Milliseconds to seconds | Seconds to minutes |
| **Cost** | Very high | Moderate |
| **Complexity** | Very high | Moderate |
| **Hardware** | Specialized, expensive | Standard cloud infrastructure |
| **Processing** | Simple, fast operations | Complex analytics allowed |
| **Use cases** | Trading, emergency alerts | Analytics, recommendations |
| **Scalability** | Difficult to scale | Easy to scale |
| **Maintenance** | Requires experts | Standard DevOps practices |

*Table 5.2: Decision framework to help you choose the right streaming approach.*

The first question to ask is whether your application truly needs results in milliseconds or if seconds or minutes would be acceptable. Many business applications that appear to require real-time processing can actually work well with near real-time processing. For example, a

dashboard that shows sales data does not need to update every millisecond; updating every few seconds or even every minute might be perfectly adequate for business decision-making.

The second consideration is your budget and resources. Traditional real-time streaming systems are expensive to build and maintain. They need specialized hardware, expert developers, and ongoing monitoring. However, if you are using Azure Databricks, this constraint is much smaller. Databricks Real-Time Mode uses the same Structured Streaming API as near real-time processing — you change one line of code to switch trigger types. There is no new infrastructure to set up and no new skills to learn. The main cost difference comes from compute time, not complexity.

The third factor is the complexity of your processing requirements. Traditional real-time systems must use simple, fast processing logic because there is no time for complex calculations. However, Databricks Real-Time Mode supports most Structured Streaming operators, including UDFs, joins, aggregations, and custom transformations. This means you can run sophisticated analytics on streaming data without sacrificing low latency. The main limitation is that stateful operators before Union operations are not supported in Real-Time Mode. For everything else, you have the full power of Structured Streaming at your disposal.

Another important consideration is how your system will scale over time. Real-time systems are difficult to scale because they require careful tuning and optimization. Near real-time systems can typically scale more easily using standard cloud infrastructure and auto-scaling features.

Finally, think about the maintenance and operational requirements. Real-time systems require constant monitoring and expert attention to keep them running smoothly. Near real-time systems can often be managed using standard DevOps practices and tools, making them easier to maintain over the long term.

The key is to be honest about your actual requirements rather than assuming you need the fastest possible processing. Many organizations start with near real-time streaming and find that it meets their needs perfectly. You can always upgrade to real-time processing later if you discover that the additional speed is truly necessary for your business.

In the next section, we will explore Azure Databricks and Apache Spark capabilities for streaming, which provide powerful tools for both real-time and near real-time processing scenarios.

# Azure Databricks and Spark Streaming Capabilities

Now that we understand the fundamentals of streaming data and the different types of streaming approaches, it is time to explore the specific tools and capabilities that Azure Databricks provides for building streaming applications. Azure Databricks combines the power of Apache Spark Structured Streaming with cloud-native features that make it easier to build, deploy, and manage streaming systems at scale.

## What is Apache Spark Structured Streaming?

Apache Spark Structured Streaming is Spark's unified engine for processing streaming data. The key insight behind Structured Streaming is that it treats streaming data as a continuous table that grows over time, allowing you to use the same familiar DataFrame and SQL APIs as for batch processing. This approach makes streaming much easier to understand and implement because you do not need to learn completely new concepts or APIs.

The reason that Structured Streaming is so powerful is that it works like batch processing, but for streams. Instead of processing large chunks of data at once, Structured Streaming processes small micro-batches of data continuously. This micro-batch approach provides a good balance between latency and throughput, making it suitable for most streaming applications while maintaining the simplicity and reliability of batch processing.

In addition to micro-batch processing, Databricks now offers Real-Time Mode as an alternative trigger type. Real-Time Mode achieves end-to-end latency as low as five milliseconds by processing data as it arrives within long-running batches and using streaming shuffle to pass data between stages immediately. For workloads that require sub-second latency, Real-Time Mode removes the need for specialized infrastructure while still using the familiar Structured Streaming API.

*Figure 5.2* illustrates how Structured Streaming divides incoming data into small micro-batches or tables and processes them using the same DataFrame API as for batch processing.

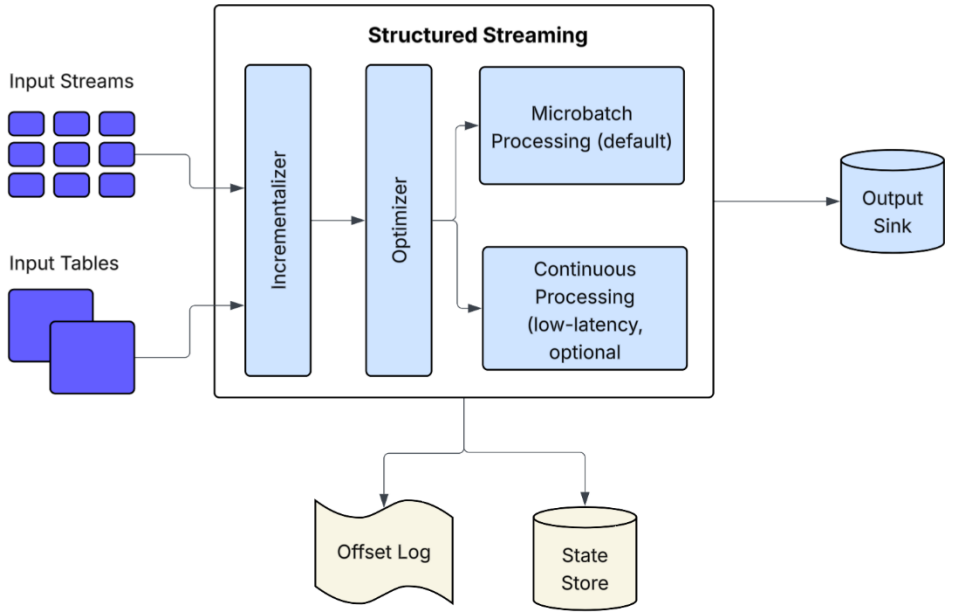



*Figure 5.2: Apache Spark Structured Streaming Architecture.*

One of the most important features of Structured Streaming is its ability to automatically handle late-arriving data. In real-world streaming scenarios, data does not always arrive in the correct order or on time. Network delays, system failures, or time zone differences can cause data to arrive late. Structured Streaming uses watermarks to determine when to stop waiting for late data and proceed with processing, ensuring that your applications continue to work even when some data arrives out of order.

Built-in fault tolerance is another crucial feature that makes Structured Streaming reliable for production use. The system automatically handles failures by using checkpointing to track the progress of streaming queries. If a failure occurs, Structured Streaming can restart from the last successful checkpoint and continue processing without losing data or producing duplicate results. This fault tolerance is essential for production systems that need to run continuously without manual intervention.

The unified API is what makes Structured Streaming so appealing to developers. You can write streaming applications using the same DataFrame operations, SQL queries, and machine learning functions that you use for batch processing. This consistency reduces the learning curve and allows teams to leverage their existing Spark knowledge when building streaming applications. The API also makes it easier to migrate from batch to streaming processing or to build applications that can handle both batch and streaming data.

## Azure Databricks Streaming Features

Azure Databricks enhances Apache Spark Structured Streaming with cloud-native features that make it easier to build, deploy, and manage streaming applications at scale. These enhancements address many of the operational challenges of running streaming systems in production environments.

Auto-scaling is one of the most valuable features that Databricks provides for streaming workloads. Traditional streaming systems require careful capacity planning and manual scaling decisions, which can be difficult to get right. Databricks automatically monitors your streaming workloads and adds or removes compute resources based on the current demand. This means you do not need to worry about provisioning enough capacity for peak loads or paying for unused resources during quiet periods.

Integration with Azure services is another key advantage of using Databricks for streaming. Databricks provides native connectors for Azure Event Hubs, Azure Blob Storage, Azure Data Lake Storage, and many other Azure services. These connectors are optimized for performance and reliability, automatically handling the complexities of authentication, retry logic, and error handling. This integration makes it much easier to build end-to-end streaming pipelines that span multiple Azure services.

Built-in monitoring and observability tools help you track your streaming applications and quickly identify issues as they occur. Databricks provides detailed metrics about streaming query performance, including processing rates, batch durations, and resource utilization. The monitoring dashboard shows you exactly what is happening with your streams in real time, making it easier to optimize performance and troubleshoot issues.

Enterprise-grade security features ensure your streaming data is protected in accordance with your organization's requirements. Databricks supports Azure Entra ID integration, role-based access control, and data encryption at rest and in transit. These security features are essential for organizations that need to comply with regulations or handle sensitive data in their streaming applications.

Lakeflow Spark Declarative Pipelines is the recommended framework for building new Structured Streaming workloads on Databricks. Lakeflow simplifies pipeline development by letting you declare your data transformations and quality expectations, while the platform handles orchestration, monitoring, error recovery, and scaling automatically. For new streaming projects, Databricks documentation recommends adopting Lakeflow Declarative Pipelines as the default approach.

## Common Azure Streaming Patterns

Understanding common streaming patterns helps you design effective solutions and avoid common pitfalls. These patterns represent proven approaches that work well in Azure environments and can serve as starting points for your own streaming applications.

Event Hubs to Databricks is one of the most popular streaming patterns for ingesting events from IoT devices, web applications, and other event sources. Azure Event Hubs acts as a high-throughput event ingestion service that can handle millions of events per second. Databricks can read from Event Hubs using the built-in Kafka connector via the Event Hubs Kafka-compatible endpoint, process events in real time, and write results to various destinations. This pattern is ideal for scenarios where you need to collect data from many sources and process it centrally. The Kafka connector is the recommended approach because it is built into the Databricks Runtime, requires no additional Maven libraries, and is compatible with Lakeflow Declarative Pipelines.

*Figure 5.3* illustrates how data flows from IoT devices and applications through Azure Event Hubs to Databricks for processing, then to various output destinations.

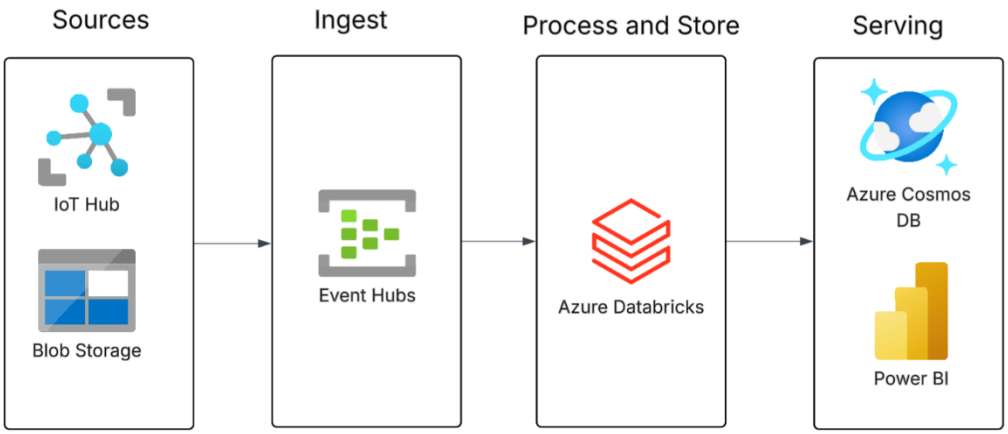



*Figure 5.3: Event Hubs to Databricks Streaming Pattern*

Autoloader for file streaming is another powerful pattern that simplifies file-based data ingestion. Instead of manually monitoring file systems for new files, Autoloader automatically detects new files as they arrive in Azure Blob Storage or Azure Data Lake Storage. This pattern is particularly useful for scenarios where data arrives as files rather than individual events, such as log files, CSV exports, or data dumps from external systems.

The Autoloader pattern handles many of the complexities of file-based streaming automatically, including file discovery, schema evolution, and exactly-once processing. You

simply specify the source location and Autoloader takes care of the rest, making it much easier to build reliable file-based streaming pipelines.

Kafka to Databricks is the pattern of choice for high-volume streaming scenarios where you need maximum throughput and flexibility. Apache Kafka is a distributed streaming platform that can handle millions of messages per second, making it ideal for scenarios with extremely high data volumes. Databricks can read from Kafka topics using the Kafka connector and process the data using Structured Streaming.

This pattern is commonly used in scenarios where you need to process data from multiple Kafka topics or when you need to integrate with existing Kafka-based systems. The pattern provides excellent performance and scalability, but it also requires more operational overhead compared to simpler patterns like Event Hubs.

Databricks to Power BI or other BI tools is a common pattern for delivering streaming results to business users through interactive dashboards. After processing streaming data in Databricks, you can write the results to a database or data warehouse that BI tool can access. BI tools can then create real-time dashboards that update automatically as new data arrives.

*Table 5.3* compares the different Azure streaming patterns.

| Pattern | Best For | Complexity | Performance | Cost |
|---|---|---|---|---|
| **Event Hubs to Databricks** | IoT, web events | Low | High | Medium |
| **Autoloader for Files** | File-based data | Very Low | Medium | Low |
| **Kafka to Databricks** | High-volume streaming | High | Very High | High |
| **Databricks to Power BI and BI tools** | Business dashboards | Medium | Medium | Medium |

*Table 5.3: Different Azure streaming patterns*

The key to successful streaming implementations is choosing the right pattern for your specific use case and combining multiple patterns when necessary. Most real-world streaming systems use a combination of these patterns to handle different types of data and different processing requirements. The flexibility of Databricks and Azure services makes it easy to build complex streaming architectures that can evolve as your requirements change.

In the next section, we will build our first streaming pipeline using these concepts and patterns, providing hands-on experience with the tools and techniques we have discussed.

# Building Your First Streaming Pipeline

Now that we understand the concepts and capabilities of streaming in Azure Databricks, it is time to build our first streaming pipeline. This hands-on section will walk you through creating a complete streaming solution using your existing Databricks workspace. We will start by setting up the necessary Azure resources, then build a streaming job that processes data in real time.

## Setting Up Your Azure Account

Before we can start building streaming pipelines, we need to set up the necessary Azure resources. Since you already have a Databricks workspace named `adb-streaming-wrkspc` in the `streaming-rg` resource group, we will create the additional components needed for our streaming pipeline.

### Prerequisites

To follow along with this tutorial, you will need the following:

- An active Azure subscription with appropriate permissions to create resources
- Access to the existing Databricks workspace `adb-streaming-wrkspc`
- Basic knowledge of Python programming
- A web browser to access the Azure portal and Databricks workspace

## Creating an Azure Event Hub

Our first streaming example will use Azure Event Hubs to ingest real-time data. Event Hubs is ideal for scenarios that require collecting data from multiple sources and processing it in real-time.

### Step 1: Create an Event Hub Namespace

1. Open the Azure portal and navigate to your streaming-rg resource group
2. Click "Create" and search for "Event Hubs"
3. Select "Event Hubs" and click "Create"
4. Fill in the following details:
    - **Namespace name**: `streaming-events-ns`
    - **Pricing tier**: Standard. We chose the Standard tier because it supports multiple consumer groups and message retention beyond 24 hours, which are needed for

production streaming scenarios. The Basic tier limits you to one consumer group and shorter retention.

- **Resource group**: `streaming-rg`
- **Region**: Same as your Databricks workspace

5. Click **Review + create** and then **Create**.

## Step 2: Create an Event Hub

1. Once the namespace is created, navigate to it
2. Click **Event Hubs** on the left menu
3. Click **Create** to add a new Event Hub
4. Fill in the following details:
   - **Name**: `sensor-data`
   - **Partition count**: 4. We use 4 partitions because each partition can be read by one consumer in parallel. More partitions mean higher throughput. For this tutorial, 4 partitions match our 4-worker cluster and are enough to handle the sensor data load.
   - **Retention time (hrs)**: 24. This keeps your data available for 24 hours. If your Databricks pipeline goes down, you can restart it and reprocess data from the last 24 hours without losing any events.
5. Click **Create**.

## Step 3: Create a Shared Access Policy

1. In your Event Hub, go to **Shared access policies**.
2. Click **Add** to create a new policy
3. Fill in the following details:
   - **Policy name**: `databricks-access`
   - **Claims**: Check **Listen** and **Send**
4. Click **Create**.
5. Copy the connection string for later use

**Note**

Azure Event Hubs supports the Kafka protocol natively. Databricks recommends connecting to Event Hubs using the built-in Kafka connector rather than the older Azure Event Hubs Spark connector, which required a Maven library and is no longer supported on current Databricks Runtimes. In the next section, we use the legacy

> format("eventhubs") approach to show the core concepts, but you can also connect using the Kafka API — see the Databricks documentation on Kafka streaming with EventHubs for the recommended production approach.

## Setting Up Azure Blob Storage

For our second example using Autoloader, we need to create a Blob Storage account to store files that will be processed as they arrive.

### Step 1: Create a Storage Account

1. In the Azure portal, navigate to your `streaming-rg` resource group
2. Click **Create** and search for **Storage account**.
3. Select **Storage account** and click **Create**.
4. Fill in the following details:
    - **Storage account name**: `streamingdata123` (must be globally unique)
    - **Performance**: Standard
    - **Replication**: Locally-redundant storage (LRS)
    - **Enable Hierarchical namespace**: Yes (recommended for better performance with Databricks)
    - **Resource group**: `streaming-rg`
    - **Region**: Same as your Databricks workspace
5. Click **Review + create** and then **Create**.

### Step 2: Create a Container

1. Once the storage account is created, navigate to it.
2. Click **Container** on the left menu.
3. Click **Container** to create a new container.
4. Fill in the following details:
    - **Name**: `input-files`
    - **Public access level**: Private
5. Click **Create**.

## Writing Your First Streaming Job

Now that we have our Azure resources set up, let's create our first streaming job. We will build two examples: one using Event Hubs for real-time event processing, and another using Autoloader for file-based streaming.

### Reading Streaming Data from Event Hubs

Our first example will read sensor data from Event Hubs and process it in real-time. This pattern is commonly used in IoT applications where devices continuously send telemetry data.

### Step 1: Create a New Notebook

1. Create and open Databricks workspace with name `adb-streaming-wrkspc`
2. Click **Workspace** on the left menu
3. Right-click and select **Create** > **Notebook**.
4. Name it *Event Hubs Streaming Example*.
5. Set the default language to **Python**.
6. Click **Create**.

The notebook will require us to attach a cluster. Create a new cluster with the following configuration:

To create the cluster, click **Compute** in the left sidebar of your Databricks workspace, then click **Create Compute**. Give your cluster a name like *streaming-cluster*, select the runtime version listed above, choose a worker type with at least 4 cores and 16 GB memory, and set the number of workers to 1 for this tutorial. Click **Create Cluster** and wait for it to start. Once the cluster is running, go back to your notebook and attach it by clicking the cluster dropdown at the top of the notebook.

No additional Maven libraries are needed. The Kafka connector is built into the Databricks Runtime and is the recommended approach for connecting to Event Hubs via its Kafka-compatible endpoint. The older Maven-based `azure-eventhubs-spark` connector is deprecated, incompatible with newer runtimes, and not supported in Lakeflow Declarative Pipelines.

The complete code example will be in the book repository: `01_streaming_pipeline_with_event_hub.py`.

### Step 2: Configure Event Hubs Connection

Add the following code to your notebook to set up the connection to Event Hubs:

```
NAMESPACE = "streaming-events-ns"
EVENTHUB = "sensor-data"
```

```
KEY_NAME = "databricks-access"
KEY_VALUE = "YOUR_KEY"
conn = (
    f"Endpoint=sb://{NAMESPACE}.servicebus.windows.net/;"
    f"SharedAccessKeyName={KEY_NAME};"
    f"SharedAccessKey={KEY_VALUE};"
    f"EntityPath={EVENTHUB}"
)
# for 2.3.15+ we should encrypt the connection string
enc = spark._jvm.org.apache.spark.eventhubs.EventHubsUtils.encrypt(conn)
ehConf = {
    "eventhubs.connectionString": enc,
}
```

## Step 3: Read Streaming Data

Add this code to read data from Event Hubs:

```
# Read streaming data from Event Hubs
df = (
    spark.readStream
    .format("eventhubs")
    .options(**ehConf)
    .load()
)
```

**Note**

The code above uses the format (`"eventhubs"`) connector, which works well for this tutorial. Databricks now recommends a newer approach using the built-in Kafka connector. Azure Event Hubs supports the Kafka protocol natively, so no extra libraries are needed. The Kafka connector is part of the Databricks Runtime and delivers higher throughput. If you are building a new production pipeline, consider using this approach instead.

With the Kafka connector, the read step looks like this:

```
jaas_config = (
    "org.apache.kafka.common.security.plain.PlainLoginModule required "
    f'username="$ConnectionString" '
    f'password="{conn}";'
```

```
)

df = (spark.readStream
    .format("kafka")
    .option("kafka.bootstrap.servers", f"{NAMESPACE}.servicebus.windows.net:9093")
    .option("subscribe", EVENTHUB)
    .option("kafka.sasl.mechanism", "PLAIN")
    .option("kafka.security.protocol", "SASL_SSL")
    .option("kafka.sasl.jaas.config", jaas_config)
    .load())
```

This uses the same `NAMESPACE`, `EVENTHUB`, and connection string variables from Step 2. The key difference is that format("kafka") connects through the Kafka-compatible endpoint of Event Hubs. The SASL/SSL settings handle authentication. Once you `call .load()`, the resulting DataFrame has the same structure, and you can process it with the same Step 4 code.

## Step 4: Process the Streaming Data

Add this code to process the incoming data:

First, add the necessary imports at the top of your notebook:

```
from pyspark.sql.types import StructType, StructField, StringType, DoubleType,
ArrayType
from pyspark.sql.functions import col, from_json, explode, current_timestamp
```

Now add the processing code:

```
# create schema for our data to land
# table will be created by Spark
spark.sql("CREATE SCHEMA IF NOT EXISTS iot")
# schema for ONE event
payload_schema = StructType([
    StructField("deviceId", StringType()),
    StructField("ts", StringType()),
    StructField("temperature", DoubleType()),
    StructField("humidity", DoubleType()),
    StructField("pressure", DoubleType()),
    StructField("battery", DoubleType()),
    StructField("lat", DoubleType()),
    StructField("lon", DoubleType()),
    StructField("status", StringType())
```

```
])
# parse the whole array
parsed_array = df.select(
    col("enqueuedTime").alias("eh_enqueued_ts"),
    col("partition"),
    from_json(col("body").cast("string"), ArrayType(payload_schema)).alias("arr")
)
# explode into individual events
events = parsed_array.selectExpr("eh_enqueued_ts", "partition", "explode(arr) as
event")
# flatten JSON
final = events.select(
    "eh_enqueued_ts", "partition",
    col("event.deviceId"), col("event.ts").alias("local_ts"),
    col("event.temperature"), col("event.humidity"),
    col("event.pressure"), col("event.battery"),
    col("event.lat"), col("event.lon"), col("event.status"),
    current_timestamp().alias("server_ts_utc")
)
q = (
    final.writeStream
    .format("delta")
    .outputMode("append")
    .option("checkpointLocation", "/mnt/streaming/checkpoints/
eventhub_array_split")
    .toTable("iot.eventhub_parsed")
)
```

## Step 5: Generate dummy events

In the GitHub repository for the book, you can find the file `iot_sample.json`. It is a list of 10 events from sensors. We want to send them to EventHub.

1. Go to our EventHub namespace
2. On the left sidebar, click on **Data Explorer**
3. Click on **Send Events**
4. Copy-paste our dummy events and click **Send**

Spark stream will capture events and load them into the table.

### Step 6: Test events

Run the SQL query to see the events:

```
# test data in the table
display(spark.sql("""
SELECT deviceId, local_ts, temperature, humidity, battery, server_ts_utc
FROM iot.eventhub_parsed
ORDER BY server_ts_utc DESC
"""))
```

We should stop our Cluster when we finish to avoid wasting credits.

## Using Autoloader for File Streaming

Our second example will use Autoloader to process files as they arrive in Blob Storage. This pattern is perfect for scenarios where data arrives as files rather than individual events.

### Step 1: Create a New Notebook

1. Create a new notebook called *Autoloader File Streaming Example*.
2. Set the default language to **Python**.

You can find the full code in the GitHub Repository for this chapter - `02_streaming_pipeline_with_autoloader.py`.

### Step 2: Configure Blob Storage Connection

Add this code to set up the connection to Blob Storage:

```
account_name = "streamingdata123"
container_name = "input-files"
# Prefer Databricks Secrets in real projects; plain text here for demo:
account_key = "YOUR_STORAGE_KEY"
spark.conf.set(
    f"fs.azure.account.key.{account_name}.blob.core.windows.net",
    account_key
)
# Input folder in your Blob container where you will drop JSON files
input_path = f"wasbs://{container_name}@{account_name}.blob.core.windows.net/"
```

## Step 3: Set Up Autoloader

Add this code to configure Autoloader:

```
schema_path = "/mnt/streaming/autoloader/schema/iot_incoming_v1"
checkpoint_path = "/mnt/streaming/autoloader/checkpoints/iot_autoloader_parsed_v1"
raw_df = (
    spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.includeExistingFiles", "true")
    .option("cloudFiles.schemaLocation", schema_path)
    .option("cloudFiles.rescuedDataColumn", "_rescued_data")
    .option("multiLine", "true")
    .load(input_path)
)
```

## Step 4: Process Files with Autoloader

Add this code to process the files:

```
parsed_df = (
    raw_df
    .select(
        col("deviceId").cast("string").alias("deviceId"),
        to_timestamp(col("ts")).alias("local_ts"),
        col("temperature").cast("double").alias("temperature"),
        col("humidity").cast("double").alias("humidity"),
        col("pressure").cast("double").alias("pressure"),
        col("battery").cast("double").alias("battery"),
        col("lat").cast("double").alias("lat"),
        col("lon").cast("double").alias("lon"),
        col("status").cast("string").alias("status"),
        current_timestamp().alias("server_ts_utc"),
        col("_rescued_data")  # keep for troubleshooting; drop later if you want
    )
)
q = (
    parsed_df.writeStream
    .format("delta")
    .outputMode("append")
    .option("checkpointLocation", checkpoint_path)
```

```
    .toTable("iot.autoloader_parsed")
)
```

## Autoloader vs. Event Hub

When building streaming pipelines in Azure Databricks, you have two main approaches for ingesting data: **Event Hubs** for real-time event streaming and **Autoloader** for file-based streaming. Understanding the differences between these approaches is crucial for choosing the right solution for your use case.

| Aspect | Event Hubs | Autoloader |
|---|---|---|
| **Data Format** | Individual events/ messages | Files (JSON, CSV, Delta, etc.) |
| **Latency** | Very low (milliseconds) | Low to medium (seconds to minutes) |
| **Throughput** | Very high (millions of events/second) | High (depends on file size and frequency) |
| **Data Source** | Real-time applications, IoT devices | File drops, batch processes, ETL pipelines |
| **Schema Evolution** | Manual handling required | Automatic with `cloudFiles.schemaEvolution` |
| **Cost** | Pay per message + throughput units | Pay per file processed + storage |
| **Complexity** | Medium (requires message parsing) | Low (automatic file discovery) |
| **Reliability** | Built-in message retention and replay | File-level atomicity and checkpointing |

*Table 5.4: Azure Event Hubs vs. Autoloader*

## When to Use Event Hubs

Use Event Hubs when you need:

- **Real-time processing**: Data must be processed as soon as it arrives

- **High-frequency events**: Thousands to millions of events per second
- **IoT and telemetry data**: Sensor readings, user activities, system logs
- **Event-driven architectures**: Microservices communication, real-time analytics
- **Low latency requirements**: Sub-second processing delays

### When to Use Autoloader

Use Autoloader when you have:

- **File-based data sources**: Data arrives as complete files
- **Micro-batch processing needs**: Periodic file drops from external systems
- **Schema evolution requirements**: Data structure changes over time
- **Cost optimization**: Lower cost for large file processing
- **Simpler setup**: Minimal configuration and maintenance

## Monitoring and Debugging

Once your streaming pipeline is running, you need to monitor its performance and troubleshoot any issues that arise. Databricks provides several tools to help you keep track of your streaming applications.

### Using the Databricks UI

The Databricks UI provides several ways to monitor your streaming jobs:

1. View Streaming Queries
    a. In your Databricks workspace, click on **Workflows** in the left menu
    b. Click on **Streaming** to see all active streaming queries
    c. You will see a list of running queries with their status, progress, and metrics
2. Monitor Query Progress
    a. Click on any streaming query to see detailed information
    b. The **Progress** tab shows processing rates, batch durations, and input/output rates
    c. The **Raw Data** tab shows sample data being processed
    d. The **Sources** tab shows information about your data sources
3. View Query Metrics
    a. In the query details, click on Metrics to see performance graphs
    b. You can see processing time, input rate, and output rate over time
    c. These metrics help you identify performance bottlenecks

## Common Problems and Solutions

1. **Problem 1**: Query Fails to Start
   **Symptoms**: The streaming query shows "Failed" status immediately after starting.

   **Solutions**:

   a. Check your connection strings and credentials
   b. Verify that your Event Hub or Blob Storage account is accessible
   c. Check the notebook logs for specific error messages
   d. Ensure you have the necessary permissions to access the data sources

2. **Problem 2**: Slow Processing
   **Symptoms**: The query is running but processing data very slowly.

   **Solutions**:

   a. Increase the cluster size to get more compute power
   b. Optimize your transformations to reduce processing time
   c. Check if you are processing too much data at once

   Use Delta format for all output tables, as it is the default and recommended format in Databricks for streaming workloads

3. **Problem 3**: Memory Issues
   **Symptoms**: The query fails with out-of-memory errors.

   **Solutions**:

   a. Increase the cluster memory allocation
   b. Optimize your data processing logic
   c. Consider using streaming aggregations instead of collecting all data
   d. Check for memory leaks in your code

## Performance Tips

1. Optimize Your Cluster
   - Use clusters with more cores for CPU-intensive operations
   - Increase memory for operations that require large datasets

   Consider using spot instances to reduce costs for non-critical workloads. However, be aware that spot instances can be terminated at any time due to availability constraints, which introduces instability risks for long-running streaming pipelines. For stateful streaming workloads that maintain state across batches, spot instance interruptions

can cause state loss and require expensive recomputation. Use spot instances only for stateless or easily recoverable streaming jobs and always use on-demand instances for critical production streaming pipelines.

2. Tune Your Queries
    - Use appropriate output modes (append vs. complete vs. update)
    - Set reasonable trigger intervals based on your latency requirements
    - Use checkpointing to enable fault tolerance and faster recovery
3. Monitor Resource Usage
    - Keep an eye on CPU and memory utilization
    - Monitor network I/O for data-intensive operations
    - Use the Databricks monitoring tools to identify bottlenecks

By following these monitoring and debugging practices, you can ensure that your streaming pipelines run smoothly and efficiently. The key is to proactively monitor your applications and address issues before they become critical problems.

In the next section, we will explore advanced streaming concepts that will help you build production-ready streaming systems.

# Advanced Streaming Concepts

Now that we have built our first streaming pipeline and learned to monitor and debug it, it is time to explore advanced concepts to help you build production-ready streaming systems. These advanced features address the complex challenges that arise when streaming systems need to handle stateful processing, late-arriving data, and high-performance requirements.

## Managing Stateful Streaming

Stateful streaming is one of the most powerful features of modern streaming systems. Unlike stateless streaming, which processes each record independently, stateful streaming remembers information from previous records and uses it to process new ones. This capability enables sophisticated analytics and real-time decision-making.

### What is State in Streaming?

State in streaming refers to data that persists between processing batches and can influence how new data is processed. Think of the state as the memory of your streaming application. Just as a person remembers previous conversations to have meaningful discussions, a stateful streaming application remembers previous data to make intelligent decisions about new data.

## When to Use Stateful Streaming

Stateful streaming is essential for many common use cases that require maintaining information across multiple records. Counting operations are a perfect example of when you need stateful processing. If you want to count the number of events that have occurred in the last hour, you need to remember the count from previous batches and add new events to that count.

Aggregating data over time windows is another common use case for stateful streaming. For example, calculating the average temperature over the last 10 minutes requires maintaining a running sum and count of temperature readings. Without state, you would need to reprocess all the data every time you want to calculate the average, which would be extremely inefficient.

## Stateful Operations in Spark Structured Streaming

Spark Structured Streaming provides several operations for stateful processing. Databricks recommends using `transformWithState` over the legacy `mapGroupsWithState` and `flatMapGroupsWithState` operators for arbitrary state transformations. The `transformWithState` API, introduced in Apache Spark 4.0 and available in Databricks Runtime 16.2+, provides better separation of concerns with distinct methods for handling input rows and expired timers. It also supports multiple timers, state **time-to-live** (**TTL**) with automatic eviction, and schema evolution without state loss. These operations allow you to define custom logic for maintaining and updating state as new data arrives.

In the GitHub repository, we have code that demonstrates stateful operations in `03_stateful_streaming_pipeline.py`. It is using Event Hubs, and you would need to use Data Explorer to ingest JSON data from the file `iot_sample_with_alerts.json`.

# Handling Late Data and Fault Tolerance

One of the biggest challenges in streaming systems is handling late-arriving data and ensuring that the system can recover from failures without losing data or producing incorrect results.

## Watermarks: Managing Late-Arriving Data

Watermarks are a crucial mechanism for handling late-arriving data in streaming systems. A watermark is a timestamp that indicates the latest time the system expects to receive data. Any data that arrives with a timestamp earlier than the watermark is considered late and may be dropped or processed differently.

We use the watermarks example with the stateful operations example:

```
# Add watermarks for stateful processing
sensor_data_with_watermark = sensor_data.withWatermark("event_timestamp", "5
minutes")
```

## Checkpointing: Ensuring Fault Tolerance

Checkpointing is the mechanism that enables streaming systems to recover from failures without losing data or producing duplicate results. When a streaming query fails, checkpointing allows the system to restart from the last successful processing point rather than starting over from the beginning.

The checkpoint stores the current state of the streaming query, including the progress through the input data and any intermediate state computed. When the query restarts, it reads the checkpoint information and continues processing from where it left off.

We used checkpoints for our streaming examples:

```
.option("checkpointLocation", "/mnt/streaming/checkpoints/eventhub_array_split")
```

## Exactly-Once Processing

Exactly-once processing ensures that each record is processed exactly once, even in the presence of failures. This guarantee is crucial for applications that cannot tolerate duplicate processing or data loss, such as financial transactions or inventory management systems.

It uses the same idea as checkpoints:

```
.option("checkpointLocation", "/mnt/streaming/checkpoints/device_summary")
```

# Performance Optimization

Building high-performance streaming systems requires careful attention to resource utilization, data partitioning, and system configuration. The goal is to maximize throughput while minimizing latency and resource costs.

## Partitioning for Performance

Data partitioning is one of the most important factors in streaming performance. The way you partition your data determines how efficiently it can be processed and how well the system can scale to handle larger volumes.

The key principle is to partition the data so that the processing load is evenly distributed across all available resources. This means avoiding hot spots where some partitions receive much more data than others and ensuring that related data is processed together when possible.

## Caching for Performance

Caching frequently accessed data in memory can significantly improve streaming performance by reducing the time needed to access reference data or lookup tables. However, caching also consumes memory, so it should be used judiciously.

## Resource Tuning for Streaming

Proper resource configuration is essential for achieving optimal streaming performance. The key is to balance CPU, memory, and network resources based on your specific workload characteristics.

## Autoloader Optimization

Autoloader provides several configuration options that can significantly impact performance. The key is to balance the frequency of file discovery with processing efficiency.

The following parameters are the most important ones to tune:

- `cloudFiles.maxFilesPerTrigger`: Controls how many files Autoloader processes in each micro-batch. The default is 1,000. Increase this if your files are small and you want higher throughput. Decrease it if you need lower latency.
- `cloudFiles.maxBytesPerTrigger`: Sets a soft limit on the amount of data processed per micro-batch. Use this instead of `maxFilesPerTrigger` when your file sizes vary significantly.
- `cloudFiles.backfillInterval`: Controls how often Autoloader scans for files that were missed during the initial listing. Set this to a duration like 1 day to catch any gaps without slowing down normal processing.
- `spark.sql.shuffle.partitions`: Controls parallelism when Autoloader processes data. The default is 200, which is too high for small datasets. Set it closer to the number of cores in your cluster.
- `spark.databricks.delta.optimizeWrite.enabled`: When set to true, Databricks automatically coalesces small files before writing to Delta. This reduces the small file problem common in streaming workloads.

### Monitoring and Tuning Performance

Continuous monitoring is essential for maintaining optimal streaming performance. You should track key metrics, including processing rate, latency, resource utilization, and error rates.

Here are some concrete parameters you can tune for better streaming performance: Set `spark.sql.shuffle.partitions` to 2-4 times the number of cores in your cluster for optimal parallelism. Use `spark.sql.streaming.stateStore.providerClass` to choose between the default HDFS-backed state store and the RocksDB state store for large stateful operations. Adjust `spark.databricks.delta.optimizeWrite.enabled` to true to reduce the number of small files written by streaming jobs. Set `maxOffsetsPerTrigger` to control how many records are processed per micro-batch, which helps balance latency and throughput. For Autoloader, tune `cloudFiles.maxFilesPerTrigger` to control how many files are processed per batch, and `cloudFiles.maxBytesPerTrigger` to limit the data volume per batch.

This concludes our exploration of Azure Databricks streaming capabilities. You now have the knowledge and tools to build sophisticated streaming applications capable of handling real-world production requirements.

## Summary

In this chapter, we explored the fundamentals and advanced concepts of building real-time streaming data pipelines using Azure Databricks. We began by understanding what streaming data is and how it differs from traditional batch processing, learning that streaming data arrives continuously in small pieces and requires immediate processing for real-time insights.

We examined different streaming approaches, distinguishing between real-time streaming for applications requiring millisecond response times and near-real-time streaming for scenarios where seconds or minutes of delay are acceptable. This understanding helps in choosing the right approach based on business requirements, budget constraints, and technical complexity.

We then delved into Azure Databricks and Apache Spark Structured Streaming, discovering how Databricks enhances Spark with cloud-native features such as auto-scaling, seamless Azure integration, built-in monitoring, and enterprise-grade security. We explored common streaming patterns, including Event Hubs integration, Autoloader for file-based streaming, Kafka connectivity and Power BI dashboard delivery.

The hands-on section provided practical experience building streaming pipelines using your existing Databricks workspace. We created Azure resources, including Event Hubs and Blob Storage, then built complete streaming jobs that process real-time sensor data and automatically handle file-based data ingestion using Autoloader.

Finally, we covered advanced streaming concepts essential for production systems. We learned about stateful streaming for maintaining information across processing batches, watermark processing for handling late-arriving data, checkpointing for fault tolerance, and guarantees of exactly-once processing. We also explored performance optimization techniques, including data partitioning, caching strategies, resource tuning, and comprehensive monitoring approaches.

In the next chapter, we will dive into Delta Lake, the storage layer that brings ACID transactions and schema evolution capabilities to your data lakehouse. While doing data ingestion and processing, Delta Lake ensures that the data you write is reliable, consistent, and queryable. You'll learn how Delta Lake's features - including time travel, change data capture, and schema enforcement - complement your streaming architectures to build robust, production-grade data platforms that maintain data quality and integrity at scale.

## Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to `packtpub.com/unlock`). Search for this book by name, confirm the edition, and then follow the steps on the page.

UNLOCK NOW

*Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.*

# 6

# Working with Delta Lake: ACID Transactions and Schema Evolution

Cloud object storage is cheap and scalable, but it comes with several problems. Raw files have no concept of transactions, versioning, or schema control. A failed write can leave a table in a broken state. A schema change can silently corrupt downstream systems. And once bad data lands, recovering from it is painful and manual.

Delta Lake solves these problems by adding a transaction log on top of standard cloud storage. Data is stored physically as Parquet files, with the transaction log acting as a control layer on top. This chapter covers what that means in practice. How Delta Lake protects data quality, tracks every change to a table, and makes it possible to capture and propagate changes across a pipeline.

In this chapter, we will cover the following topics:

- Introduction to Delta Lake: Why It Matters
- Delta Lake fundamentals and ACID transactions
- Schema enforcement and schema evolution
- Time travel and data versioning
- Change Data Capture and Change Data Feed

# Technical requirements

This chapter is mostly theoretical. A few sections include SQL and Python examples to illustrate the concepts in practice.

To run the examples, you need access to a Databricks workspace. If you do not have an existing subscription, Databricks offers a free edition that is sufficient for all the code in this chapter.

No additional libraries or external dependencies are required.

# Introduction to delta lake: Why it matters

With the move to cloud-native solutions and the separation of storage and compute, cloud object storage enabled inexpensive, scalable, and durable storage capabilities. Different systems could easily access storage, and it became the main repository for all data.

However, even with these benefits, data operations were still entirely file-based, using formats such as ORC and Parquet. While each format served a specific need, none were designed to provide traditional database capabilities, such as transactional guarantees, versioning, automatic schema enforcement, or data recovery.

Some of the day-to-day problems with working with individual files in cloud object storage include:

- Partial or corrupted writes when clusters fail during job execution
- Readers accessing inconsistent data while a write is still in progress
- Schema mismatches when upstream sources change unexpectedly
- Difficulty recovering or rolling back changes
- Concurrent writers overwriting each other's outputs
- Slow and expensive metadata operations because every query had to list and inspect thousands of individual Parquet files
- Difficult and expensive to modify existing data (updates and deletes), requiring full file or partition rewrites even for minor changes
- Complex manual logic to keep data clean and consistent (custom scripts to fix problems the storage system couldn't handle, e.g., detecting partial files after failures, reconciling mismatched schemas, deduplicating records from concurrent writes, or cleaning up stale/old partition files)

To address the operational challenges of traditional data lakes, Databricks designed Delta Lake.

Delta Lake isa storage layer built on top of cloud object storage, while the underlying data continues to be stored in the familiar Parquet format used by many compute engines. Delta Lake enhances this foundation by providing:

- ACID transactions for reliable concurrent reads and writes
- Schema enforcement and schema evolution to prevent bad data and support controlled change
- Table versioning and time travel for auditing, rollback, and reproducible analytics
- Unified batch and streaming support using the same table abstractions
- Standard DML operations such as `INSERT`, `UPDATE`, `DELETE`, and `MERGE`

Throughout the rest of this chapter, we will explore how these capabilities are implemented.

# Enabling ACID transactions for reliable data processing

In this section, we introduce the Delta Log and explain how it enables ACID guarantees in Delta Lake. We then walk through how each ACID property—atomicity, consistency, isolation, and durability—is implemented in practice, and how these mechanisms work together to provide reliable reads and writes at scale.

## Introduction to delta log

What differentiates Delta Lake from a plain collection of Parquet files is the presence of a write-ahead transaction log, called the Delta Log. This log records every change made to a table and enables ACID-compliant transactions on top of object storage.

Delta Lake's transactional guarantees are enabled by optimistic concurrency control, made possible through the Delta Log. An important design decision was to store the Delta Log alongside the data files in cloud object storage, leveraging the durability and reliability inherent to these storage systems.

Although Delta Lake manages both Parquet data files and the Delta Log in cloud object storage, none of this complexity is exposed to the user. The SQL statements declare the rows to be inserted. Delta Lake automatically handles file creation, versioning, and metadata updates behind the scenes. From the user's perspective, interacting with a Delta table feels no different from working with a traditional relational database.

*Figure 6.1* demonstrates the difference between querying raw Parquet files and querying a Delta Lake table.

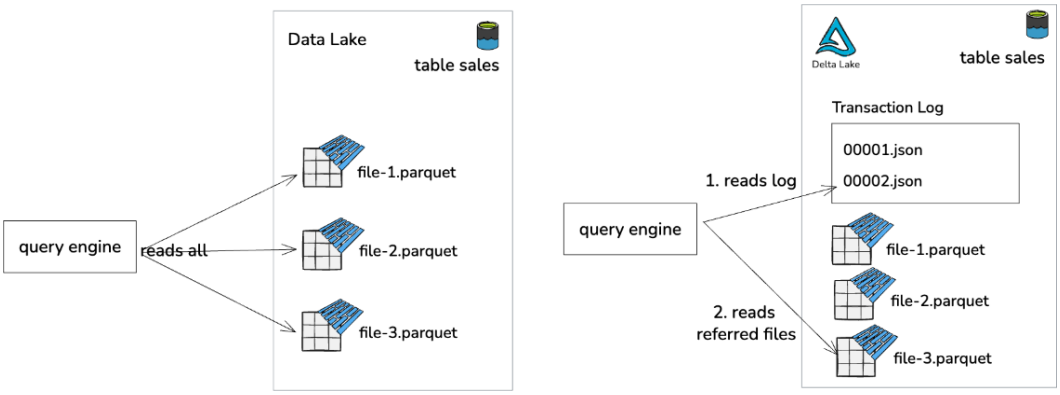



*Figure 6.1: Querying raw Parquet files versus querying a Delta Lake table*

As shown in *Figure 6.1*, the query engine first reads the `_delta_log` to determine which Parquet files belong to the latest committed version of the table. Only after that does it load the required data files from storage.

Beyond tracking table changes, the Delta Log also stores critical metadata used to optimize query execution, including:

- File-level statistics
- Schema definitions
- Partitioning information
- Table properties
- Deletion vectors and other optimization metadata

Query engines use this metadata to determine which Parquet files to read. Instead of scanning data files first and discarding irrelevant ones later, the engine consults the Delta Log upfront. This approach significantly reduces unnecessary I/O and fundamentally improves how compute engines interact with cloud object storage.

*Figure 6.2* below illustrates how a Delta Lake table is stored in cloud object storage. For clarity, some file names have been shortened or adjusted. In practice, both Parquet data files and Delta Log JSON files follow strict naming conventions. The goal here is to convey the core concept rather than the exact file names.

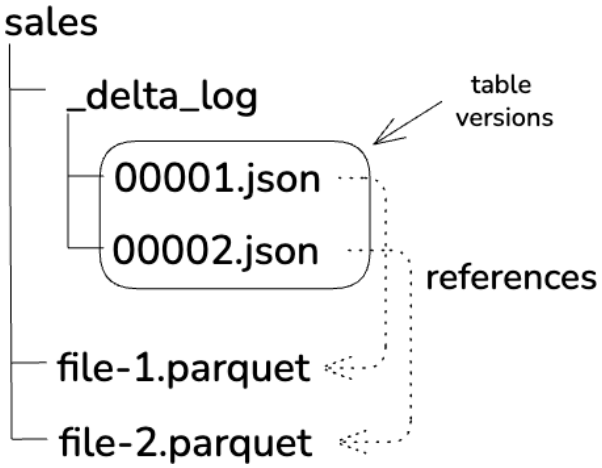



*Figure 6.2: How a Delta Lake table is stored in cloud object storage*

*Figure 6.2* shows a sales table stored as two Parquet data files and two JSON files inside the `_delta_log` directory. As discussed earlier, the Delta Log stores metadata that describes the current state of the table and records every change applied to it.

Now, imagine the following two `INSERT` operations occur sequentially:

```
INSERT INTO sales (sales_id, order_id, product_id, quantity, amount)
VALUES (101, 5001, 200, 3, 19.99);

INSERT INTO sales (sales_id, order_id, product_id, quantity, amount)
VALUES (102, 5001, 200, 3, 19.99);
```

Each statement results in a new commit to the Delta Log, along with other metadata:

- 00001.json records that `file-1.parquet` was added to the table
- 00002.json records the addition of `file-2.parquet`
- Each commit also has an accompanying `.crc` file containing checksums and statistics

Each commit references the newly written data file and represents a new version of the table.

While the example we demonstrated inserts, the same mechanism applies to updates, deletes, and merge operations. In those cases, the Delta Log contains a combination of add and remove actions that precisely describe how the table evolved, allowing Delta Lake to reconstruct the correct table state for any version.

Next, we will explain what transaction guarantees mean and how Delta Lake implements them.

## What do ACID guarantees mean?

Delta Lake provides ACID guarantees for both reads and writes. While many proprietary and open-source systems also support ACID transactions, the underlying implementations differ. In Delta Lake, these guarantees are enabled and enforced through the Delta transaction log.

In the following sections, we will briefly revisit what ACID stands for and then break down the concepts and components that deliver ACID guarantees in Delta Lake.

- **Atomicity**: A transaction must either fully succeed or be completely rolled back. There is no concept of a partial or incomplete state.
- **Consistency**: A transaction must move the database from one valid state to another, while adhering to all defined rules and constraints.
- **Isolation**: Isolation governs the concurrent execution of multiple operations and the potential conflicts that may arise from them. Isolation rules specify how modifications made by concurrent transactions are visible to one another.
- **Durability**: Durability ensures that once a transaction is committed, its results are permanently recorded and will persist even in the event of failures.

So, how does Delta Lake implement atomicity? Let's see in the next section.

## What role does the _delta_log play?

As discussed earlier, the `_delta_log` directory records every change made to a Delta table. Each Delta Log JSON file represents a new version of the table at a specific point in time. This naturally raises several questions:

- When are transactions logged?
- Are failed operations recorded?
- How does rollback work?
- What happens to files written by a failed job?
- How does the query engine know which metadata files to read?

## When are transactions written to the Delta Log?

Delta Lake uses an append-only transaction log.

- Changes are recorded by adding new JSON files to the `_delta_log` directory
- Existing log files are never modified or overwritten
- Every successful table modification is captured as an atomic commit

Each JSON file represents exactly one commit:

- Commits start at 00000000000000000000.json (version 0)
- Each subsequent commit increments the version number by one
- Each numbered file corresponds to a new table version

## Are failed operations recorded?

No - only successful commits appear in the Delta Log.

- If a job fails before committing:
    a. No JSON commit file is written
- Even if Parquet files were physically created in object storage:
    a. They are **not referenced** in the Delta Log
    b. They are **never visible** to query engines
    c. These uncommitted files are eventually removed through the VACUUM process

## How does Delta Lake handle rollbacks?

There is no explicit rollback mechanism for failed transactions.

- Failed operations are never committed
- Without a commit entry, the table state remains unchanged

However, Delta Lake does allow restoring a table to a previously committed version. This feature is known as Time Travel, and it will be covered later in this chapter.

## Why is delta log the single source of truth?

The Delta Log is the state of the Delta table.

- Query engines do not scan object storage directly and first read the Delta Log to determine:
    a. Which Parquet files belong to the table?
    b. Which files are valid for the latest version?

Without the log, the Delta table state cannot be reconstructed.

## How does reading the Delta Log scale?

A common concern is scalability: If a table has thousands of commits, does the engine need to read every JSON file?

Delta Lake addresses this with checkpointing.

- By default, a checkpoint is created every 10 commits
- A checkpoint summarizes the entire table state up to that version

When reading a table:

- The engine loads the latest checkpoint
- Then applies only the JSON commits written after it

If a table has fewer than 10 commits and no checkpoint exists yet, the engine reads the available JSON files.

## How Delta Lake implements consistency

*Figure 6.3* illustrates how a Delta Lake write operation transitions the table from one valid state to the next.

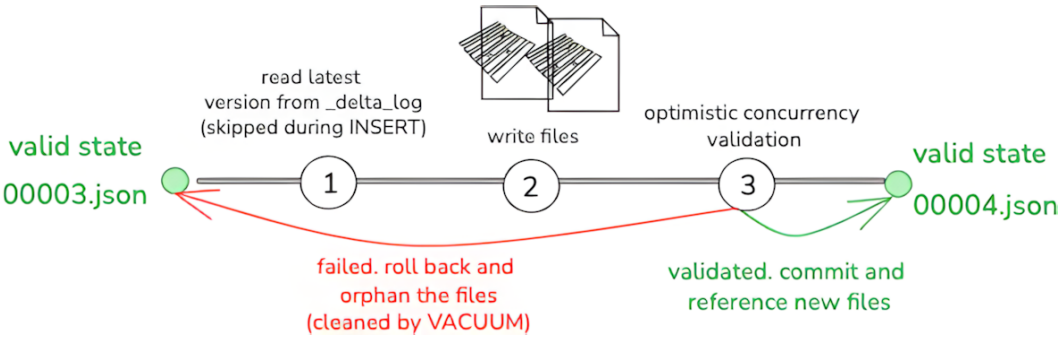



*Figure 6.3: Maintaining table consistency with Delta Lake write transactions*

> **Note**
>
> **Download the color images**
>
> Your purchase includes a color, DRM-free PDF copy of this book, ideal for viewing color images, screenshots, and diagrams. Refer to *Free benefits with your book* section at the end of the *Preface* to unlock your PDF copy.

As shown in *Figure 6.3*:

- The process begins at a valid table state, represented by 00003.json. The write operation reads this latest version from the `_delta_log` (step 1 in the diagram). During INSERT, this step is skipped. Schema validation leverages metadata from the transaction log.

- The engine then writes new Parquet files to cloud object storage (step 2). At this point, the files physically exist, but they are not yet committed and therefore not visible to readers.
- Delta Lake performs validation (step 3) to ensure no conflicting writes have occurred since the last read of 00003.json.
- If the validation fails, the write operation is aborted. No new JSON commit file is created; the table remains at version 00003.json, and the newly written Parquet files become orphan files, later removed by VACUUM (red path in the diagram).
- If the validation succeeds, Delta Lake creates a new commit file, 00004.json, which references the newly written Parquet files. This becomes the next valid state of the table (green path in the diagram).
- At no point are existing JSON files modified; the system advances by adding a new commit file that represents the updated table version.

Write operation validation in Delta Lake uses optimistic concurrency control. In simple terms, before committing any changes, Delta Lake checks whether new conflicting writes were committed to the table after the operation began.

## How Delta Lake implements isolation

By default, isolation in Databricks is achieved through serialization for writes and snapshot isolation for reads.

Snapshot isolation means that a transaction (`tx1`) reads a snapshot of the table based on the latest committed version at the moment the read begins. That snapshot remains unchanged throughout tx1. Even if another transaction (`tx2`) commits new data afterward, those changes do not affect the snapshot seen by `tx1`.

Concurrent write operations are handled through serialization. Although multiple write transactions may run concurrently, Delta Lake ensures they are committed in a strictly ordered, sequential manner. From the table's perspective, write operations appear as if they occurred one at a time.

This serialization uses optimistic concurrency control. Processes on Delta Lake operate without knowing about each other, even when they run concurrently. Unlike systems that rely on pessimistic concurrency control, Delta Lake does not use transaction-level locking, so operations do not block each other. Instead, validation occurs at commit time, when Delta Lake checks whether a write can be safely applied.

As discussed earlier, the Delta Log supports append-only commits; it does not allow existing commits to be overwritten or modified. When multiple write operations occur concurrently, Delta Lake will manage the ordering of changes as if operations were occurring one at a time in

a consistent, ordered sequence. No matter how many transactions happen at the same time, each writer must produce a unique commit file name—two writers can never commit using the same version number.

*Figure 6.4* demonstrates what happens when a concurrent situation occurs:

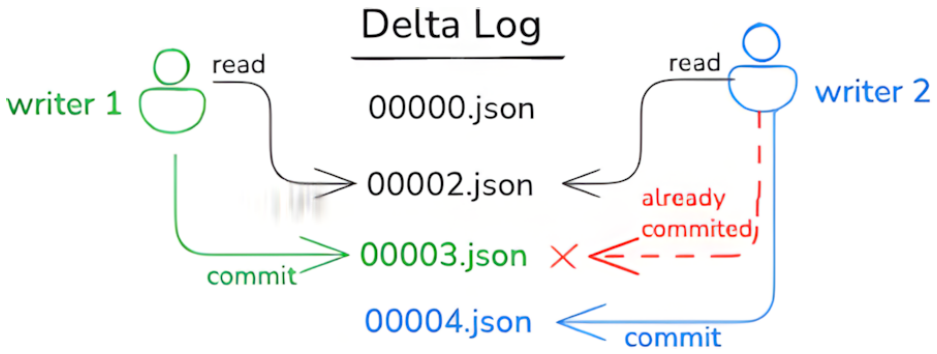



*Figure 6.4: Resolving concurrent write conflicts in Delta Lake*

To better understand this behavior, let's walk through the steps shown in *Figure 6.4*:

1. Writer 1 and Writer 2 both read 00002.json. Each takes a snapshot of the table as it exists at that moment.
2. Both writers process independently based on that same snapshot
3. Writer 1 finishes first and commits 00003.json. The table version advances from 2 to 3.
4. Writer 2 then tries to commit, but still assumes version 2 is current
5. Delta Lake detects that 00003.json already exists and rejects Writer 2's commit
6. If there is no conflict, Writer 2 will commit staged changes as a new version 00004.json
7. If there is a conflict, Writer 2's operation will fail, and the VACUUM process eventually cleans up uncommitted Parquet files
8. This preserves isolation by serializing writes: only one writer can successfully commit each version, and the history remains ordered

Databricks supports two serializable isolation levels: `WriteSerializable` and Serializable.

`WriteSerializable` is the default and guarantees serializability only for write operations.

Serializable isolation requires the table's contents to match the exact commit order recorded in the history. If a write cannot maintain that ordering, the transaction fails. Typical situation when Serializable conflicts occur is when a long-running transaction finishes based on outdated data. If another transaction commits changes that the long-running one never saw,

Serializable isolation rejects the long-running commit because it can no longer match the exact order shown in history.

### How Delta Lake implements durability

As we mentioned, the important design decision was to store the Delta Log alongside the data files in cloud object storage, leveraging the durability and reliability inherent to these storage systems. Therefore, durability is guaranteed by the underlying object store and its redundancy.

Now that we've covered the fundamentals of the Delta Log and Delta Lake's transactional guarantees, we can now explore schema enforcement and schema evolution. These capabilities play a critical role in preventing bad data from entering Delta Lake while still allowing schemas to change in a controlled, intentional way as data requirements evolve.

## Schema enforcement and evolution

In this section, we will explore how Delta Lake enforces schemas at write time, how it handles incompatible or unexpected data, and how schema evolution enables safe, managed change without compromising data quality or downstream stability.

## Why schema management matters

In traditional data lakes built directly on Parquet files, keeping data reliable was always a challenge. Parquet is a great storage format, but it was never meant to behave like a table with a stable, enforced structure. Each file contains its own version of the schema, and nothing in storage guarantees that all files agree with one another. As upstream systems change, those differences often accumulate unnoticed and only surface when downstream jobs or processes fail.

Some of the issues with Parquet-only data lakes include:

- **Silent schema drift**: New or missing columns appear across files without warning
- **Inconsistent types**: The same field shows up as different data types in different partitions
- **Half-written or corrupted files**: No transactions mean readers may pick up incomplete output
- **Messy evolution**: Every consumer must handle every historical schema version manually
- **Failures that appear too late**: Read-time errors instead of write-time validation

All of these challenges stem from a single core difference between working directly with Parquet and with Delta Lake. Parquet has no built-in schema enforcement. The schema is

inferred only when data is read, which is known as schema-on-read. Delta Lake validates the schema at write time, a schema-on-write approach. This means bad data is rejected before it enters the table, and consumers always read a well-defined, consistent structure.

As illustrated in *Figure 6.5*, schema-on-read defers schema interpretation until query time, whereas schema-on-write enforces the schema during data ingestion.

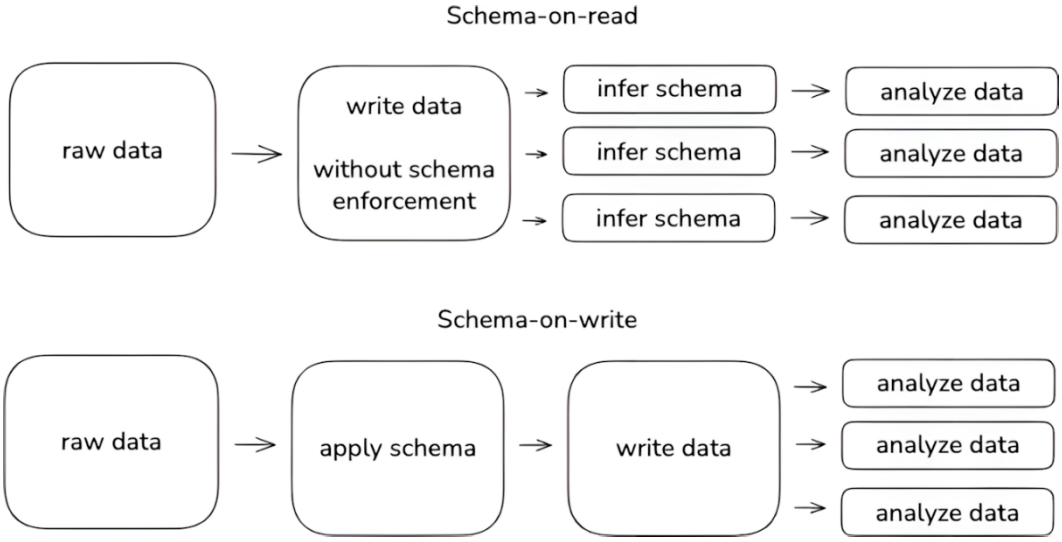



*Figure 6.5: Comparison of schema-on-read and schema-on-write data processing models*

## Delta Lake's approach to schema enforcement

Delta Lake provides the flexibility of data lakes while addressing their reliability gaps. It validates every write operation to ensure new data matches the table's defined structure. This is known as schema enforcement, and it prevents silent drift and inconsistencies.

Schema enforcement examples

- Reject unexpected columns that aren't part of the schema
- Reject incompatible types (e.g., writing a STRING to an INT column)
- Reject nullability violations when a NOT NULL column receives nulls

We will illustrate how schema enforcement works using two simple examples. These examples intentionally write data that does not match the table's schema, allowing us to observe how Delta Lake behaves.

**Note**

You can use your own catalog and schema if you already have one configured. The objects below are created only to keep the example self-contained.

```
CREATE CATALOG IF NOT EXISTS analytics_dev;
CREATE SCHEMA IF NOT EXISTS analytics_dev.operations;
```

The following statement creates a simple sales table with a fixed schema. Each column is explicitly typed and marked as `NOT NULL`, which allows Delta Lake to validate incoming data strictly.

```
-- Base sales table
CREATE TABLE analytics_dev.operations.sales_delta      (
  sale_id   INT NOT NULL,
  sale_date DATE NOT NULL,
  amount    DECIMAL(5,2) NOT NULL
)
USING DELTA;
```

Next, we insert a couple of valid rows that fully conform to the table schema. These inserts succeed without any issues.

```
-- Insert initial rows
INSERT INTO analytics_dev.operations.sales_delta (sale_id, sale_date, amount)
VALUES
  (1001, '2025-01-01', 100.00),
  (1002, '2025-01-02', 75.50);
```

To confirm the table structure, we can inspect the schema directly:

```
-- Check schema
DESCRIBE TABLE sales_delta;
```

**Example 1**: Extra column in incoming data

In this example, we attempt to insert a new record that includes an additional column (discount), which does not exist in the target table.

```
-- New sales record with extra column "discount"
CREATE OR REPLACE TEMP VIEW new_sales_with_discount AS
SELECT
  1003 AS sale_id,
  '2025-01-03' AS sale_date,
  50.00 AS amount,
  5.00 AS discount;  -- NEW COLUMN

INSERT INTO analytics_dev.operations.sales_delta
SELECT sale_id, sale_date, amount
FROM new_sales_with_discount;
```

Because schema enforcement is enabled by default, Delta Lake rejects this operation. The extra column is not silently ignored or added to the table schema.

**Example 2**: Incompatible data type

Next, we attempt to insert a record in which the amount column is provided as a string rather than a numeric value.

```
-- New sales record with incompatible type
CREATE OR REPLACE TEMP VIEW new_sales_with_amount_as_string AS
SELECT
  1004 AS sale_id,
  '2025-01-03' AS sale_date,
  'Fifty' AS amount;   -- AMOUNT AS STRING

INSERT INTO analytics_dev.operations.sales_delta
SELECT *
FROM new_sales_with_amount_as_string;
```

This insert also fails. Delta Lake validates data types at write time and prevents incompatible values from being written to the table.

Schema enforcement helps maintain stability in systems that require consistent schemas. For example, BI and visualization tools often break or produce unreliable results when data arrives with unexpected structures. While many modern tools offer ways to preprocess or reshape incoming data, relying on them introduces instability for teams, creates extra maintenance, and can lead to technical debt over time.

## Delta Lake's approach to schema evolution

At the same time, Delta supports schema evolution, allowing users to intentionally update a table's schema while keeping those changes tracked and governed through the transaction log.

Schema Evolution examples:

- Adding new columns as data sources evolve
- Type widening (e.g., `DECIMAL(5,2) → DECIMAL(7,2)`) when appropriate
- Changing data types
- Renaming columns
- Removing columns
- Changing column order

Below are some examples of schema evolution:

```
-- You now have multiple stores and want to track the sales source
ALTER TABLE analytics_dev.operations.sales_delta
ADD COLUMNS (store_id STRING);
-- You need to accommodate larger individual sales amounts
ALTER TABLE analytics_dev.operations.sales_delta
SET TBLPROPERTIES ('delta.enableTypeWidening' = 'true');
ALTER TABLE analytics_dev.operations.sales_delta
ALTER COLUMN amount TYPE DECIMAL(7,2);
```

To demonstrate column `DROP` and `RENAME` operations, we first enable the column-mapping table property:

```
ALTER TABLE analytics_dev.operations.sales_delta SET TBLPROPERTIES (
   'delta.columnMapping.mode' = 'name'
)
```

Column mapping is a Delta Lake feature that allows columns to be renamed or dropped without rewriting the underlying Parquet files. These operations become metadata-only changes, which makes them fast.

```
-- Your naming standards have changed
ALTER TABLE analytics_dev.operations.sales_delta
RENAME COLUMN sale_date TO transaction_date;
-- You want to track discounts in a different table
```

```
ALTER TABLE analytics_dev.operations.sales_delta
DROP COLUMN store_id;
```

As mentioned earlier, when a column is dropped using column mapping, the physical field still exists in the Parquet files on storage—it is no longer exposed in the table schema; it is soft deleted. If a column must be fully removed from storage for regulatory or compliance reasons, in addition to overwriting, we need to perform `PURGE` and `VACUUM` operations.

**Note**

**Warning**: `VACUUM RETAIN 0 HOURS` permanently deletes all time travel history and cannot be undone. Do not run this in production without fully understanding the consequences.

```
REORG TABLE analytics_dev.operations.sales
APPLY PURGE;
--  in order to set a retention period of zero
SET spark.databricks.delta.retentionDurationCheck.enabled = false;
VACUUM analytics_dev.operations.sales RETAIN 0 HOURS;
```

## Schema evolution: Overwrite schema approach

If column mapping is not enabled, dropping a column requires overwriting the entire table. Even after overwriting, the old data files still exist because overwriting creates a new table version; it does not physically remove previous versions. You must run `VACUUM` to clean up the old files.

The overwrite strategy can also be used for other schema changes, such as modifying column types or renaming columns.

The syntax for overwriting a table with a new schema (assuming `df` is a PySpark DataFrame containing the updated schema) is:

```
df.write \
  .mode("overwrite") \
  .option("overwriteSchema", "true") \
  .saveAsTable("analytics_dev.operations.sales")
```

## Schema evolution: Automated approach

`ALTER TABLE` is the intended mechanism for controlled schema changes in Delta Lake.

Here are some use cases:

- Manual schema updates performed by engineers during development or maintenance
- CI/CD pipeline in a versioned environment: migration scripts executed during deployment
- Automated data processing workflows that perform explicit schema checks and safely update the schema only when allowed

You can enable schema evolution by:

- Adding `.option("mergeSchema", "true")` to DataFrame write operations
- Executing merges with the `MERGE … WITH SCHEMA EVOLUTION` syntax
- Setting `spark.databricks.delta.schema.autoMerge.enabled=true` for the current session

## Schema evolution for auto loader

Autoloader configuration has more options regarding schema evolution. It can be configured with the option `cloudFiles.schemaEvolutionMode` and can have the following modes: `addNewColumns`, `rescue`, `failOnNewColumns`, or none.

`addNewColumns` is the default mode. When a new column is encountered, Autoloader automatically adds it to the schema and restarts the stream. Existing columns are unaffected.

If rescue mode is selected, the data is saved in the column `_rescued_data` by default, or the name can be changed. A record lands in the rescued column for one of three reasons: the value has a type mismatch, the column name has a case mismatch, or the column is missing from the schema entirely.

## Schema evolution best practices

Schema evolution is a powerful capability in Delta Lake, but it should be applied with care. Recommended best practices include:

- Avoid uncontrolled schema evolution to prevent instability in pipelines and downstream systems
- Establish governance with clear team guidelines on when and how schema evolution should be used
- Evaluate downstream impact before applying schema changes, especially in shared datasets
- Collaborate early with other teams when schema updates may affect their processes or reports

Schema enforcement and schema evolution ensure that Delta Lake tables remain correct, consistent, and intentionally structured as data changes over time. By rejecting invalid writes and governing how schemas evolve, Delta Lake prevents bad data from entering the system while still allowing change to occur in a controlled and auditable way.

In the next section, we will be exploring the time travel feature of Delta Lake. It allows us to query, audit, and restore previous versions of a table, including historical schemas and data states, using the same transactional mechanisms we have already discussed.

# Delta versioning and time travel

In this section, we explore why time travel matters, how Delta Lake versions data, how to query historical table states using SQL, and how to safely restore previous versions while preserving a complete and auditable history.

## Why time travel matters

Working with raw Parquet files in a cloud data lake used to mean that once the data was overwritten or deleted, the previous state was gone forever. This made debugging, auditing, and reproducing results difficult. Delta Lake solves this by adding time travel: the ability to query a table exactly as it looked at any earlier point in time. This feature allows teams to recover from mistakes, compare changes over time, and build more reliable pipelines.

Time travel helps teams with the following use cases:

- Recovering from accidental deletes or overwrites without backups
- Debugging pipeline issues by comparing current and previous versions
- Producing historical snapshots of data for audit purposes
- Reproducible analytics and ML experiments using fixed historical states
- Understanding how data changed over time during development or troubleshooting

## How does the Delta Lake version data

Delta Lake versions data using a copy-on-write model: whenever updates or merges occur, Delta writes new Parquet files and records the old ones as removed in the transaction log (`_delta_log`). For `DELETE`, `UPDATE`, and `MERGE` operations, Delta can use deletion vectors, which mark individual rows as modified without rewriting the Parquet file. These vectors are applied using a merge-on-read strategy, meaning the engine filters out deleted rows at query time. Each of these actions produces a new table version. Because both the original files and the associated deletion vectors are preserved, Delta can accurately reconstruct the table as it appeared at any previous point in time, enabling reliable time travel.

## Querying historical data with the SQL API

If you've followed the earlier examples in this chapter, you should already have a Delta table for sales data. If you don't already have the table, create it using the following code.

```
CREATE CATALOG IF NOT EXISTS analytics_dev;
CREATE SCHEMA IF NOT EXISTS analytics_dev.operations;
-- Base sales table
CREATE TABLE IF NOT EXISTS analytics_dev.operations.sales (
  sale_id   INT NOT NULL,
  sale_date DATE NOT NULL,
  amount    DECIMAL(5,2) NOT NULL
)
USING DELTA;
-- Enable column mapping
ALTER TABLE analytics_dev.operations.sales
SET TBLPROPERTIES (
  'delta.columnMapping.mode' = 'name'
);
```

## Creating multiple table versions

To explore time travel, we need a table with several versions in its history. The three statements below create that history by inserting, updating, and deleting rows. Each statement produces a new table version, which we can then query by version number or timestamp.

Creating the table and setting its properties constitute version 0 and version 1 of the table, before any data was written.

Start by inserting two rows into the table. This creates version 2:

```
-- Version 1 → Version 2: initial load
INSERT INTO analytics_dev.operations.sales (sale_id, sale_date, amount) VALUES
  (1, DATE '2025-01-01', 100.00),
  (2, DATE '2025-01-02', 150.00);
```

Next, update the amount for `sale_id 2`. This creates version 3:

```
-- Version 3 → Version 3: update a row
UPDATE analytics_dev.operations.sales
SET amount = 200.00
WHERE sale_id = 2;
```

Then delete the row for sale_id 1. This creates version 4:

```
-- Version 3 → Version 4: delete a row
DELETE FROM analytics_dev.operations.sales
WHERE sale_id = 1;
```

After running all three statements, the table contains only one row. To confirm, run the following query:

```
-- Check the state
SELECT * FROM analytics_dev.operations.sales;

+--------+------------+--------+
| sale_id| sale_date  | amount |
+--------+------------+--------+
|      2 | 2025-01-02 | 200.00 |
+--------+------------+--------+
```

The other row was deleted in version 3, but it still exists in the table's history. To see every version that was created, run the following command:

```
DESCRIBE HISTORY analytics_dev.operations.sales
```

| Version | Timestamp | Operation | UserName |
|---|---|---|---|
| **4** | 2025-01-10T10:06:00.000Z | DELETE | ... |
| **3** | 2025-01-10T10:05:00.000Z | UPDATE | ... |
| **2** | 2025-01-10T10:04:00.000Z | WRITE | ... |
| **1** | 2025-01-10T10:03:00.000Z | SET TBLPROPERTIES | ... |
| **0** | 2025-01-10T10:02:00.000Z | CREATE TABLE | ... |

Each row in this output corresponds to one commit in the Delta Log. We now have three versions to work with. In the next two examples, we will read the table as it appeared in version 1, first by version number, then by timestamp.

## Querying by version number

To read the table as it looked at version 1, right after the initial insert, run the following query. The `VERSION AS OF` clause tells Delta Lake which version to return:

```
-- Read the table as it looked right after the initial insert (version 1)
SELECT *
FROM analytics_dev.operations.sales VERSION AS OF 2;

+--------+------------+--------+
| sale_id| sale_date  | amount |
+--------+------------+--------+
|      1 | 2025-01-01 | 100.00 |
|      2 | 2025-01-02 | 150.00 |
+--------+------------+--------+
```

Both rows are visible because the update and delete had not yet occurred at version 1.

## Querying by timestamp

If you do not know the version number, you can query by timestamp instead. Run the following query. Delta Lake returns the most recent commit at or before the timestamp you provide, which in this case is version 1:

```
SELECT *
FROM analytics_dev.operations.sales
TIMESTAMP AS OF '2025-01-10T10:03:30Z';
```

Delta will pick the most recent commit **at or before** that timestamp (in our example, version

```
+--------+------------+--------+
| sale_id| sale_date  | amount |
+--------+------------+--------+
|      1 | 2025-01-01 | 100.00 |
|      2 | 2025-01-02 | 150.00 |
+--------+------------+--------+
```

The result is the same as the version number query. Both methods give you a read-only view of the table then.

We've just explored several ways to query historical versions of a Delta table. But a natural next question is: can a historical version be promoted back to the current state of the table?

## Restore the table state

A time travel read using VERSION AS OF, or TIMESTAMP AS OF, is always read-only. It allows you to inspect, validate, or compare historical data, but it never changes the table itself.

To actually revert data, Delta Lake provides the RESTORE TABLE command. Rather than modifying or deleting existing files, RESTORE TABLE creates a new table version whose metadata points to the same data files that were active in the chosen historical version.

```
-- Restore to a specific version RESTORE TABLE
analytics_dev.operations.sales TO VERSION AS OF 5;
-- Restore to a specific timestamp RESTORE TABLE
analytics_dev.operations.sales TO TIMESTAMP AS OF '2025-12-21 09:30:00';
```

This approach preserves the complete table history while safely reverting the table's visible state.

A rollback can also be performed manually by overwriting the table with data from a historical snapshot, for example, using INSERT OVERWRITE in combination with a time travel query:

```
INSERT OVERWRITE analytics_dev.operations.sales SELECT * FROM
analytics_dev.operations.sales VERSION AS OF 5;
```

While this method offers more flexibility, it is less explicit and less revealing of intent than `RESTORE TABLE`.

Use `RESTORE TABLE` for straightforward rollbacks. It is atomic and explicit. Use `INSERT OVERWRITE` when you need more control, such as reverting only a subset of rows or applying transformations during the rollback.

The key principle is that Delta Lake never mutates old versions.

All rollbacks, whether performed through `RESTORE TABLE` or manual overwrites, produce a new table version using copy-on-write semantics. This design ensures that rollbacks are safe, auditable, and fully transactional.

## Retention and data lifecycle

Time travel depends on Delta retaining historical files and transaction logs. By default, Delta keeps enough history to support time travel for a limited period. These retention windows are configurable using table properties such as:

- `delta.logRetentionDuration`: Controls how long Delta remembers the table's change history. The default is 30 days.
- `delta.deletedFileRetentionDuration`: Defines how long physical data files that are no longer part of the current table version are kept. The default is 7 days.

Over time, older data files that are no longer referenced by any valid table version can be physically removed using `VACUUM`.

To protect against accidental data loss, Delta enforces a minimum retention period and blocks unsafe vacuum operations unless explicitly overridden.

Delta Lake's versioning and time travel capabilities provide a reliable foundation for working with evolving data. By recording every change as a new table version in the Delta Log, Delta Lake enables historical queries, safe rollbacks, reproducible analytics, and strong auditing guarantees—all without mutating existing data.

In the next section, we explore how Delta Lake participates in CDC pipelines—both as a target for incoming change events from upstream systems and as a source of changes for downstream consumers.

# Change data capture in Delta Lake

The previous sections showed how Delta Lake records every table change as a versioned commit in the transaction log. That same mechanism makes Delta Lake a natural fit for Change Data Capture. CDC captures row-level changes from source system tables, specifically `INSERT`, `UPDATE`, and `DELETE` operations, rather than reprocessing entire datasets on each run. The result is that downstream systems stay in sync with upstream data at a fraction of the cost and data movement.

## Why CDC matters for analytical use cases

- In most architectures, data is initially generated in upstream systems such as transactional databases

- For analytics, machine learning, and AI use cases, this data must be moved into analytical storage systems
- The goal of CDC is to efficiently replicate transactional data into analytics stores without reprocessing entire datasets

*Figure 6.6* illustrates the end-to-end CDC process, from detecting changes in the source system to delivering them to a target system.

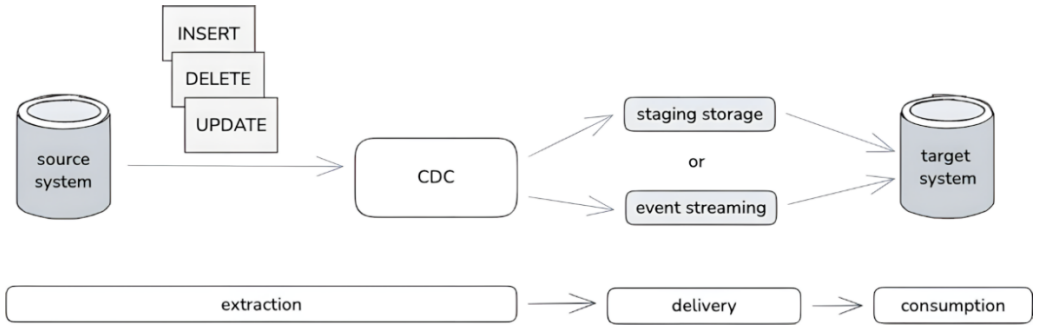



*Figure 6.6: CDC process from source to target systems*

As shown in the diagram, the process consists of three distinct phases, each representing a different responsibility in a CDC pipeline:

- **Extraction**: Change events are detected and extracted from the source systems
- **Delivery**: Once captured, the changes are either written to cloud object storage for batch processing or streamed into message streaming systems for real-time use cases
- **Consumption**: CDC data is applied to recreate or update tables in Delta Lake

Within this broader CDC architecture, Delta Lake can play two distinct roles:

- As a target, where incoming changes are applied
- As a source, where changes are emitted for downstream consumers

## Delta table as a target for CDC

In this role, Delta Lake ingests change events from upstream systems and applies them to target tables using operations such as `MERGE` or `AUTO-CDC`.

The focus of this section is to explain how, after data has been extracted from source systems, these changes can be efficiently and reliably applied to Delta Lake.

## Approach using MERGE INTO

Before the introduction of the AUTO CDC API for Spark Declarative Pipelines (formerly Delta Live Tables), the most common pattern for applying CDC data to Delta Lake on Databricks was to use the `MERGE INTO` statement.

In practice, there were several scenarios where native implementation of `MERGE INTO` in combination with foreachBatch could produce incorrect or incomplete results if not carefully implemented. These situations often require additional logic, and even post-processing steps to ensure that the Delta table accurately reflects the full state of the source system.

When applying CDC manually with `MERGE INTO`, several edge cases require special handling to avoid incorrect table state:

- **Multiple updates to the same key in the same microbatch**
  If multiple change events for the same primary key arrive in a single batch, `MERGE INTO` does not inherently guarantee correct ordering. Additional logic, such as deduplication or windowing, is required.
- **Out-of-order updates**
  CDC streams may deliver changes out of sequence. A late-arriving update or delete can overwrite a newer state unless explicit ordering logic (for example, based on commit timestamps or sequence numbers) is enforced.
- **Out-of-order updates that arrive after deletes**
  This scenario is explicitly called out in the Databricks documentation. A `DELETE` operation is processed first, followed by an out-of-order `UPDATE` arriving in a later microbatch. Although the update occurred earlier in the source system, it reaches the pipeline after the delete. To handle this correctly, the pipeline must retain knowledge of the delete event. A common approach is to perform a soft delete, where the row is marked as deleted rather than removed immediately.

The scenarios above represent only a subset of the challenges that can arise when applying CDC data to target systems. Each case requires additional logic to ensure correctness, and further complexity is introduced when handling CDC Type 2 updates.

## Approach using AUTO CDC

Spark Declarative Pipelines will be covered in detail in a later chapter. For now, we highlight their powerful built-in support for processing CDC data streams. While the API exposes a small number of configuration options, the underlying complexities, such as schema evolution, out-of-order events, and deduplication, are handled automatically for the user.

At a minimum, users are required to specify only:

- A target table to apply changes to
- A source containing the CDC records
- Primary key(s) identifying each record
- An ordering column (any column that can be monotonically sequenced)

All other parameters are optional and depend on the source system's CDC semantics.

The following example highlights only the CDC-related configuration. The full pipeline setup is intentionally omitted for clarity:

```
from pyspark import pipelines as dp
# The pipeline setup is not complete and is meant only
# to illustrate the CDC configuration.
dp.create_streaming_table(name=<target_uc_table>)
dp.create_auto_cdc_flow(
target=<target_uc_table>,
source=<source_dataframe>,
keys=[<primary_keys>],
    sequence_by=col(<sequence_column>),
    apply_as_deletes=expr("operation = 'DELETE'"),
    except_column_list=[<except_column_list>],
    stored_as_scd_type=1
)
```

With this configuration in place, Spark Declarative Pipelines automatically manage all remaining concerns. When you query the target table, you will see the additional metadata and guardrails that enable Delta Lake to apply changes safely and correctly without requiring the user to implement complex `MERGE INTO` logic manually.

## Delta table as a source of CDC

In this role, Delta Lake itself becomes the producer of change events. Downstream systems can consume changes from Delta tables to propagate updates to another Delta Lake table or other external systems.

This feature in Delta Lake is called Change Data Feed. When a Delta table is enabled for this feature, each record includes additional metadata indicating whether it represents an insert, update, or delete operation. In addition to actual data recorded by a job, the table schema will record information in additional columns:

- `_change_type`: Possible values: `insert`, `update_preimage`, `update_postimage`, `delete`

- `_commit_version`: This value references the Delta Log
- `_commit_timestamp`: The timestamp of the commit

## Enabling change data feed

There are several ways to enable change data feed, and it must be done explicitly. By default, a Delta table does not track changes.

### Enable at table creation

```
CREATE TABLE analytics_dev.operations.sales (
  sales_id INT,
  order_id INT,
  product_id INT,
  quantity SMALLINT,
  amount DOUBLE
)
TBLPROPERTIES (delta.enableChangeDataFeed = true);
```

### Enable for existing table

```
ALTER TABLE analytics_dev.operations.sales
SET TBLPROPERTIES (delta.enableChangeDataFeed = true);
```

### Enable for all tables created in the Spark session

```
SET spark.databricks.delta.properties.defaults.enableChangeDataFeed = true;
```

## How to read changes

A change data feed can be consumed by both batch and streaming queries.

### Batch queries

When using change data feed in batch pipelines, it is required to specify the table version (starting and, optionally, ending) or the timestamp.

### Specify starting version

```
of Spark.read \
  .option("readChangeFeed", "true") \
  .option("startingVersion", 0) \
  .table("analytics_dev.operations.sales")
```

### Specify an ending version

```
spark.read \
  .option("readChangeFeed", "true") \
  .option("startingVersion", 0) \
  .option("endingVersion", 5) \
  .table("analytics_dev.operations.sales")
```

### Specify an ending timestamp

```
spark.read \
  .option("readChangeFeed", "true") \
  .option("startingTimestamp", '2025-12-21 09:30:10') \
  .option("endingTimestamp", '2025-12-21 12:00:00') \
  .table("analytics_dev.operations.sales")
```

### Streaming queries

The default behavior of change feed when reading by streaming query is incremental processing. On the first run, the streaming query returns the latest snapshot of a table as inserted records.

```
spark.readStream \
  .option("readChangeFeed", "true") \
  .table("analytics_dev.operations.sales")
```

We can also specify the table versions, same as for the batch queries:

```
spark.readStream \
  .option("readChangeFeed", "true") \
  .option("startingVersion", 0) \
  .table("analytics_dev.operations.sales")
```

## How VACUUM affects change data feed

Change data feed relies on Delta log and data file retention policy. `VACUUM` settings can limit how far back you can read changes from the table. Refer to the time travel section of this chapter, where we discussed retention of the data lifecycle.

Change Data Feed is a powerful Delta Lake feature that enables the propagation of changes from Delta tables to downstream consumers. It simplifies architecture and integration while supporting efficient incremental processing.

This section concludes our exploration of Delta Lake's core capabilities. You now have a solid understanding of how Delta Lake brings transactional guarantees, schema governance, versioned history, and change data capture to cloud data lakes. These features form the foundation for building reliable, scalable, and maintainable data platforms that can support both batch and streaming workloads in production environments.

## Summary

The Delta transaction log is the foundation that connects everything. It is what makes ACID transactions possible on top of plain cloud object storage. It is also what powers schema enforcement, time travel, and Change Data Feed. Every commit, every schema change, and every version is recorded there. Without it, Delta Lake would be no different from a folder of Parquet files.

With that foundation in place, you now understand how Delta Lake prevents bad data from entering a table, how it handles concurrent reads and writes safely, and how every change to a table is preserved and queryable. You can inspect historical states, recover from mistakes, and trace exactly how a table evolved. You also know how to use Delta tables as both a target and a source in CDC pipelines, consuming upstream changes and feeding downstream systems with minimal custom logic.

These are not just individual features. Together, they address the core weaknesses of traditional data lakes and make it practical to build pipelines on Azure Databricks that are reliable, auditable, and maintainable in production.

## Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to `packtpub.com/unlock`). Search for this book by name, confirm the edition, and then follow the steps on the page.

*Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.*

# 7

# Automating Data Systems with Lakeflow Spark Declarative Pipelines

Building and maintaining data pipelines traditionally requires significant manual effort—writing code to manage dependencies, implementing data quality checks, handling errors, and monitoring execution. As data systems grow in complexity, this imperative approach becomes increasingly difficult to maintain and scale. Databricks Lakeflow **Spark Declarative Pipelines** (**SDP**), formerly known as **Delta Live Tables** (**DLT**), takes a fundamentally different approach by allowing you to declare what you want your data to look like rather than specifying every step in producing it.

In this chapter, you will learn how to build automated, self-managing data pipelines using Lakeflow SDP. We will explore the declarative syntax for defining tables and transformations, implement comprehensive data quality expectations, and understand how Lakeflow automatically handles dependency management, incremental processing, and error recovery. By the end of this chapter, you will be able to build production-ready pipelines that are easier to develop, maintain, and operate than traditional approaches.

In this chapter, we will cover the following topics:

- Introduction to Lakeflow Spark Declarative Pipelines
- Building Your First Lakeflow Pipeline
- Managing Data Quality with Expectations
- Optimizing Lakeflow Pipeline Performance

# Technical requirements

To complete the examples in this chapter you will need:

- An active Databricks workspace on Azure with a **Premium** plan (or higher) and Unity Catalog enabled. Lakeflow Spark Declarative Pipelines requires the Premium plan.
- Permission to create schemas, volumes, and tables within a Unity Catalog catalog (we use `cat_dev` throughout this chapter)
- Permission to create and run Lakeflow pipelines in your workspace.
- The sample data files (orders.csv and customers.csv) from the book's GitHub repository.

To access the repository, follow the steps in the "Download the example code files" section of the *Preface*.

# Introduction to LakeFlow Spark Declarative Pipelines

In this section, you'll learn how Lakeflow changes the way data pipelines are designed and operated—shifting from imperative, step-by-step coding to a declarative model where you define what your data should look like, and the system handles the rest. Lakeflow **SDP** represents a paradigm shift in how we build data pipelines. Instead of writing imperative code that specifies every step of data processing, Lakeflow allows you to declare the desired end state of your data, and the system figures out how to achieve it.

# The Challenge with Traditional Pipelines

Traditional data pipeline development, whether using Structured Streaming (covered in *Chapter 5, Building Real-Time Data Pipelines*) or batch notebooks (*Chapter 3, Data Ingestion Strategies for Azure Databricks*), requires you to manage many operational concerns manually:

- **Dependency management**: You must carefully orchestrate the execution order of transformations, ensuring upstream tables are ready before downstream processing begins. As pipelines grow, this dependency graph becomes complex and error-prone.
- **Data quality**: Implementing comprehensive data quality checks requires custom code, error handling logic, and separate systems for tracking and alerting on quality metrics.

- **Operational overhead**: You need to manually handle checkpointing, recovery from failures, incremental processing logic, and monitoring infrastructure.
- **Change management**: When you modify pipeline logic, you must carefully manage schema evolution, reprocess historical data appropriately, and ensure downstream systems aren't disrupted.

Consider a typical imperative pipeline that processes customer orders. It requires you to manage execution order, implement quality checks, handle errors, and coordinate multiple jobs.

To address these challenges, Lakeflow transforms this imperative approach into a declarative one. Instead of writing code that describes how to process data, you declare what your data should look like. With this declarative approach:

- Lakeflow automatically determines dependencies
- Data quality expectations are built into the pipeline definition
- Incremental processing and checkpointing are handled automatically
- The pipeline is self-documenting - Lakeflow renders the dependency graph as a visual DAG in the pipeline UI, making data flow and table relationships immediately visible to anyone on the team

## Key benefits of Lakeflow SDP

- **Automatic dependency management**: Lakeflow analyzes your pipeline code to build a **directed acyclic graph** (**DAG**) of dependencies. When you reference one table from another, Lakeflow automatically knows the execution order and handles orchestration.
- **Built-in data quality**: Expectations provide first-class support for data quality rules. You can specify which rules should drop invalid records, which should only log violations, and which should fail the entire pipeline. All quality metrics are automatically tracked and available in the pipeline UI.
- **Simplified operations**: Lakeflow handles checkpointing, incremental processing, schema evolution, and error recovery automatically. This reduces operational complexity and makes pipelines more reliable.
- **Automatic optimization**: Lakeflow applies performance optimizations automatically, including Z-ordering (covered in *Chapter 10, Optimizing Query Performance and Cost Management*), automatic scaling, and efficient incremental processing strategies.
- **Lineage and observability**: Every pipeline execution produces detailed lineage information showing exactly how data flowed through your pipeline. Quality metrics, processing statistics, and error information are automatically captured and visualized.

## When to use Lakeflow SD

Lakeflow is ideal for:

- **Multi-stage transformations** where you need to manage dependencies between multiple tables
- **Data quality-critical pipelines** where you need comprehensive validation and monitoring
- **Production pipelines** that require high reliability and operational simplicity
- **Team collaboration** where declarative code improves maintainability and understanding

You might prefer traditional Structured Streaming (*Chapter 5, Building Real-Time Data Pipelines*) when:

- You need very low latency processing (sub-second)
- Your pipeline has complex custom logic that doesn't fit the declarative model
- You need fine-grained control over exactly how streaming is executed

## Core concepts

Before we dive into building pipelines, let's understand the fundamental concepts in Lakeflow:

- **Tables, materialized views, and views**: Lakeflow supports three dataset types. **Streaming** tables process data incrementally using `readStream` and are ideal for large or continuously arriving datasets. Materialized views (defined with `@dp.materialized_view`) are batch-computed result sets that Lakeflow refreshes completely on each pipeline run; they are useful when you need exact aggregations without streaming constraints. Views (defined with `@dp.view`) are temporary, named transformations that exist only during pipeline execution — they are not persisted to storage and are useful for intermediate logic you don't need to materialize as a table.
- **The `@dp.table` decorator**: The `@dp.table` decorator (from `from pyspark import pipelines as dp`) tells Lakeflow to create and manage a table. The function name becomes the table name (e.g., `def sdp_orders_bronze()` creates a table named `sdp_orders_bronze`). You can add comments, table properties, and expectations using decorator parameters.
- **Reading from other pipeline tables**: Within your pipeline, you use `spark.read.table("table_name")` or `spark.readStream.table("table_name")` to reference other tables in the same pipeline. For example, `spark.readStream.table("sdp_orders_bronze")` reads from the Bronze orders table

and automatically creates a dependency relationship. Lakeflow uses these references to build the execution graph and determine the correct processing order.

- **Expectations**: Expectations are Lakeflow's way of expressing data quality rules. They can drop invalid records (`@dp.expect_or_drop`), log violations for monitoring (`@dlt.expect`), or fail the pipeline if violations occur (`@dp.expect_or_fail`).
- **Pipelines**: A Lakeflow pipeline is a collection of table definitions (in one or more pipeline source files — `.py` or `.sql` files organized within a folder in the Lakeflow Pipelines Editor) configured with a target catalog/schema, cluster resources, and scheduling options. The pipeline manages the lifecycle of all tables it defines.

**Note**

If you're using SQL for your Lakeflow pipeline instead of Python, you would use the LIVE keyword to reference tables (e.g., FROM LIVE.sdp_orders_bronze). Our examples use Python with the module, which provides more flexibility for complex transformations.

Now that we understand the motivation and core concepts, let's build a complete Lakeflow pipeline.

# Building your first Lakeflow pipeline

In this section, we'll build a complete end-to-end pipeline that demonstrates the key features of Lakeflow. Our example will process customer and order data through a medallion architecture pattern, applying data quality rules and producing analytics. By the end of this section, you will have created, deployed, and run a fully functional Lakeflow pipeline covering Bronze, Silver, and Gold layers, complete with data quality expectations and streaming aggregations.

## The example scenario

We have two CSV files stored in a Unity Catalog Volume:

- `Orders.csv`: Contains 500 customer order transactions with the following columns: `order_id`, `customer_id`, `order_date` (in `yyyy-MM-dd` format), `order_total` (numeric), and `order_status`. The status field contains four values: `Delivered`, `Shipped`, `Pending`,

and `Cancelled`. During the Silver layer, we will filter out `Cancelled` orders as a data quality rule, which removes 52 records.

- `Customers.csv`: Contains 100 customer records with columns: `customer_id`, name, `email`, `city`, `state`, `country`, and `signup_date` (in `yyyy-MM-dd` format). Of these, 11 customers have a `signup_date` before `2023-01-01`, which we will flag as a monitoring expectation in the Silver layer.

Knowing these details upfront will help you understand the data quality violations you'll see when the pipeline runs, rather than encountering them unexpectedly during execution.

Our goal is to:

1. Ingest the raw data (Bronze layer)
2. Clean and validate the data (Silver layer)
3. Create geographic sales analytics (Gold layer)

## Setup: Preparing your data

Before we start, make sure you are logged into your Databricks workspace and have a notebook open. The setup code in this section should be run in a standard interactive notebook (not a Lakeflow pipeline notebook). You can create one by navigating to Workspace in the left sidebar, clicking Create, and selecting Notebook. Attach the notebook to any available shared cluster. You will need permission to create volumes and upload files in the Unity Catalog catalog you are using (we use `cat_dev` throughout this chapter). Volumes (introduced in Chapter 2) provide a governed, file-based storage location that integrates with Unity Catalog.

First, create a volume and upload your data files. Run this setup code once in a notebook:

```
# Create the schema if it doesn't exist
spark.sql("""
    CREATE SCHEMA IF NOT EXISTS cat_dev.sandbox
    COMMENT 'Schema for Chapter 7 Lakeflow examples'
""")
# Create a volume for Chapter 7 data
spark.sql("""
    CREATE VOLUME IF NOT EXISTS cat_dev.sandbox.vol_chapter7
    COMMENT 'Volume for Chapter 7 Lakeflow examples'
""")
# Verify the volume exists
volume_path = "/Volumes/cat_dev/sandbox/vol_chapter7/sdp/"
print(f"Volume path: {volume_path}")
```

After the volume is created, upload the `orders.csv` and `customers.csv` files to the path `/Volumes/cat_dev/sandbox/vol_chapter7/sdp/`. You can do this through the Databricks UI by navigating to Catalog in the left sidebar, browsing to the `vol_chapter7` volume, and using the Upload button. Alternatively, you can copy the files programmatically from your local machine using the Databricks CLI or the Files API.

```
# Verify the files are present
display(dbutils.fs.ls(volume_path))
# Preview the data to understand structure
orders_preview = spark.read.format("csv").option("header",
"true").load(f"{volume_path}orders.csv")
print("Orders schema:")
orders_preview.printSchema()
orders_preview.show(5)
customers_preview = spark.read.format("csv").option("header",
"true").load(f"{volume_path}customers.csv")
print("Customers schema:")
customers_preview.printSchema()
customers_preview.show(5)
```

Now that our data is ready, let's build the pipeline.

## The complete pipeline code

Create a new Python source file for your Lakeflow pipeline. This code cannot be run interactively in a standard notebook — it must be executed within a Lakeflow pipeline. The `dp` decorators and `spark.readStream.table()` references to pipeline-managed tables only resolve when the Lakeflow engine orchestrates the pipeline. We will walk through the steps to create and configure the pipeline in the *Creating and Running the Pipeline* section later in this chapter. This notebook will not be run interactively—it will be executed by the Lakeflow engine when you configure and start a pipeline (we'll cover that shortly). The complete pipeline code is available in the book's GitHub repository at Chapter07/code/01_spark_declarative_pipeline.py. We'll build the pipeline progressively, explaining each layer as we go. Let's start with the imports and configuration that every Lakeflow pipeline needs.

## Pipeline setup: Imports and configuration

Every Lakeflow pipeline begins with importing the `dp` module, which provides the decorators and functions for defining pipeline tables. We also import PySpark functions for our transformations and define the volume path where our source data resides:

```
# ============================================
# Chapter 7: Lakeflow Pipeline - Complete Example
# ============================================
# This pipeline demonstrates Lakeflow Spark Declarative Pipelines with:
# - Bronze layer: Raw data ingestion
# - Silver layer: Data cleaning and quality checks
# - Gold layer: Business analytics
# ============================================
from pyspark import pipelines as dp
from pyspark.sql.functions import *
# Configuration
VOLUME_PATH = "/Volumes/cat_dev/sandbox/vol_chapter7/sdp/"
```

The `dp` module is automatically available in Lakeflow pipeline environments—you do not need to install it. The `VOLUME_PATH` variable points to the Unity Catalog Volume where we uploaded our CSV files in the setup step.

## Understanding the bronze layer

The Bronze layer is responsible for raw data ingestion with minimal transformation. Let's examine the key elements:

```
# ============================================
# SECTION 1: Bronze Layer - Raw Ingestion
# ============================================

@dp.table(
    comment="Raw orders data from CSV files - Bronze layer",
    table_properties={
        "quality": "bronze",
    }
)
def sdp_orders_bronze():
    """
    Bronze layer: Ingest raw orders data with minimal transformation.
    Uses Auto Loader (cloudFiles) for incremental processing.
```

```
    Schema is automatically inferred from the CSV file.
    """
    return (
        spark.readStream
        .format("cloudFiles")
        .option("cloudFiles.format", "csv")
        .option("header", "true")
        .option("cloudFiles.inferColumnTypes", "true")
        .load(f"{VOLUME_PATH}orders.csv")
    )
@dp.table(
    comment="Raw customers data from CSV files - Bronze layer",
    table_properties={
        "quality": "bronze"
    }
)
def sdp_customers_bronze():
    """
    Bronze layer: Ingest raw customer data.
    Schema is automatically inferred from the CSV file.
    """
    return (
        spark.readStream
        .format("cloudFiles")
        .option("cloudFiles.format", "csv")
        .option("header", "true")
        .option("cloudFiles.inferColumnTypes", "true")
        .load(f"{VOLUME_PATH}customers.csv")
    )
```

- **The `@dp.table` Decorator**: This decorator tells Lakeflow to create and manage a table. The function name (`sdp_orders_bronze`) becomes the table name. The comment parameter provides documentation that appears in the Unity Catalog, and `table_properties` allows you to set Delta table properties like quality level tags.
- **Auto Loader for Incremental Processing**: We use `cloudFiles` format (Auto Loader, covered in Chapter 5) rather than a simple `spark.read`. This enables incremental processing—when you add new files to the volume, Lakeflow automatically detects and processes only the new data.

- **Schema Inference**: The option `cloudFiles.inferColumnTypes` tells Auto Loader to automatically determine column data types from the CSV file. Within Lakeflow pipelines, schema tracking is managed automatically — you do not need to specify a `schemaLocation`.
- **Streaming for Bronze**: Using `readStream` creates a streaming table that processes data incrementally. This is more efficient than full refreshes, especially as your data grows.

The `sdp_customers_bronze` table follows the same pattern, ingesting customer data from a separate CSV file. Note that each table defines its own `schemaLocation` to track schema evolution independently. Together, these two functions define the entire Bronze layer—Lakeflow will automatically create and manage both tables.

## Understanding the silver layer

The Silver layer cleans and validates data, applying business rules and data quality expectations:

```
# ============================================
# SECTION 2: Silver Layer - Data Quality
# ============================================

@dp.table(
    comment="Cleaned orders with data quality checks - Silver layer",
    table_properties={
        "quality": "silver"
    }
)
@dp.expect_all_or_drop({
   "valid_order_id": "order_id IS NOT NULL",
   "valid_customer_id": "customer_id IS NOT NULL",
   "valid_order_status": "order_status IN ('Delivered', 'Shipped', 'Pending')"
})
@dp.expect("valid_order_total", "order_total > 0")
def sdp_orders_silver():
    """
    Silver layer: Apply business rules and data quality checks.

    Quality rules:
    - Drop rows with invalid/missing critical fields (order_id, customer_id)
    - Drop rows with unexpected order status (only Delivered, Shipped, Pending are
valid)
```

```
    - Log violations for order_total but keep the rows for investigation
    - Standardize date formats
    - Calculate derived fields

    Expected violations:
    - 52 orders with status 'Cancelled' will be dropped (not in allowed statuses)
    """
    return (
        spark.readStream.table("sdp_orders_bronze")
        # Remove duplicates based on order_id
        .dropDuplicates(["order_id"])
        # Convert to TIMESTAMP (not DATE) for watermarking support in downstream
tables
        .withColumn("order_date", to_timestamp("order_date", "yyyy-MM-dd"))
        # Extract date components for partitioning/analytics
        .withColumn("order_year", year("order_date"))
        .withColumn("order_month", month("order_date"))
        .withColumn("order_quarter", quarter("order_date"))
        .withColumn("order_day_of_week", dayofweek("order_date"))
        # Categorize order value
        .withColumn("order_value_category",
                    when(col("order_total") < 500, "Low")
                    .when(col("order_total") < 1500, "Medium")
                    .when(col("order_total") < 2500, "High")
                    .otherwise("Premium"))
        # Clean order status (standardize case)
        .withColumn("order_status", initcap(trim(col("order_status"))))
    )
```

- **Data Quality Expectations**: The expectations define data quality rules with different enforcement levels:
- **Critical Validations (`expect_all_or_drop`)**: This decorator accepts a dictionary of rule names to SQL expressions. Any record that violates *any* of these rules is dropped:
    a. `valid_order_id`: Orders must have an ID
    b. `valid_customer_id`: Orders must be associated with a customer
    c. `valid_order_status`: Only orders with status `'Delivered'`, `'Shipped'`, or `'Pending'` are kept. Orders with `'Cancelled'` status are dropped.

- **Monitoring Validations (expect)**: These rules log violations but keep the records:
    a. `valid_order_total`: Monitors orders with zero or negative amounts (data quality issues)
- **Reading from Bronze**: The `spark.readStream.table("sdp_orders_bronze")` call reads from the Bronze table we defined earlier. Lakeflow automatically:
    a. Understands this creates a dependency (Silver depends on Bronze)
    b. Ensures Bronze is processed before Silver
    c. Handles incremental processing (only reads new records from Bronze)
- **Standard Data Cleaning**: The transformations apply typical data cleaning operations:
    a. Removing duplicates
    b. Parsing dates into proper timestamp types
    c. Standardizing text fields (trimming whitespace, normalizing case)
    d. Creating derived fields (date components, categorizations)

The Silver layer for customers follows a similar structure but with expectations appropriate to customer data. Here is the complete customer Silver table:

```
@dp.table(
    comment="Cleaned customers with validation - Silver layer",
    table_properties={
        "quality": "silver"
    }
)
@dp.expect_or_drop("valid_customer_id", "customer_id IS NOT NULL")
@dp.expect_or_drop("valid_name", "name IS NOT NULL AND length(name) > 0")
@dp.expect("valid_country", "country IS NOT NULL")
@dp.expect("recent_signup", "signup_date >= '2023-01-01'")
def sdp_customers_silver():
    """
    Silver layer: Clean and validate customer data.

    Quality rules:
    - Drop rows with missing customer_id or name
    - Log violations for missing country (but keep the rows for manual review)
    - Log violations for customers who signed up before 2023 (historical
customers)
    - Standardize country and state names
    - Parse signup dates and calculate tenure
```

```
    Expected violations:
    - 11 customers signed up before 2023-01-01 (logged as warnings)
    """
    return (
        spark.readStream.table("sdp_customers_bronze")
        .dropDuplicates(["customer_id"])
        # Standardize date format
        .withColumn("signup_date", to_date("signup_date", "yyyy-MM-dd"))
        # Calculate customer tenure
        .withColumn("customer_tenure_days",
                    datediff(current_date(), col("signup_date")))
        .withColumn("customer_tenure_years",
                    round(col("customer_tenure_days") / 365, 1))
        # Standardize text fields (trim whitespace, title case)
        .withColumn("name", trim(col("name")))
        .withColumn("city", initcap(trim(col("city"))))
        .withColumn("state", upper(trim(col("state"))))
        .withColumn("country", initcap(trim(col("country"))))
        .withColumn("email", lower(trim(col("email"))))
        # Categorize customer by tenure
        .withColumn("customer_tenure_segment",
                    when(col("customer_tenure_days") < 180, "New")
                    .when(col("customer_tenure_days") < 365, "Growing")
                    .when(col("customer_tenure_days") < 730, "Established")
                    .otherwise("Loyal"))
    )
```

In production, dropDuplicates() on a streaming table requires a watermark to bound state size, because without one, Spark must track every record it has ever seen. For our CSV-based example the data is finite, but in a production pipeline with continuous data arrival you should add a watermark before deduplication (e.g., `.withWatermark("order_date", "1day").dropDuplicates(["order_id"])`).

This function applies two critical expectations (`valid_customer_id` and `valid_name`) that drop records, plus two monitoring expectations (`valid_country` and `recent_signup`) that log violations while retaining the records. The transformations calculate customer tenure in both days and years and segment customers into groups based on their registration duration.

## Understanding the gold layer

The Gold layer produces business-ready analytics by joining and aggregating Silver tables. In streaming contexts, this requires special handling:

```
# ============================================
# SECTION 3: Gold Layer - Business Analytics
# ============================================

@dp.table(
    comment="Geographic sales distribution - Gold layer",
    table_properties={
        "quality": "gold"
    }
)
def sdp_geographic_sales():
    """
    Gold layer: Sales metrics by geographic location.

    This table uses streaming aggregation with watermarks and time windows.
    The window function is required for streaming aggregations with watermarks -
    it groups data into time-based buckets (30-day periods) and allows the
    aggregation to complete once the watermark passes, ensuring correct results
    even with late-arriving data.

    Note: We use approx_count_distinct instead of countDistinct because
    exact distinct counts are not supported in streaming aggregations.

    Provides:
    - Revenue and customer distribution by state/country over 30-day windows
    - Regional performance metrics with time periods
    """
    from pyspark.sql.functions import approx_count_distinct

    # Read orders as stream with watermark
    # Watermark allows data up to 1 day late before finalizing aggregations
    orders_stream = (
        spark.readStream.table("sdp_orders_silver")
        .withWatermark("order_date", "1 day")
    )
```

```
    # Read customers as batch (not streaming) for join
    # This creates a stream-to-static join which is more efficient
    customers_batch = spark.read.table("sdp_customers_silver")

    # Join orders with customer location data
    orders_with_location = orders_stream.join(
        customers_batch.select("customer_id", "city", "state", "country"),
        "customer_id",
        "inner"
    )

    # Aggregate by geography with 30-day windows
    # Window aggregation is REQUIRED for streaming aggregations with watermarks
    return (
        orders_with_location
        .groupBy(
            window("order_date", "30 days"),  # 30-day time windows
            "country",
            "state"
        )
        .agg(
            count("order_id").alias("total_orders"),
            sum("order_total").alias("total_revenue"),
            avg("order_total").alias("avg_order_value"),
            approx_count_distinct("customer_id").alias("unique_customers"),
            max("order_total").alias("max_order_value"),
            # Status metrics
            sum(when(col("order_status") == "Delivered",
col("order_total")).otherwise(0)).alias("delivered_revenue")
        )
        .withColumn("revenue_per_customer", col("total_revenue") /
col("unique_customers"))
        # Extract window start and end times as separate columns
        .select(
            col("window.start").alias("period_start"),
            col("window.end").alias("period_end"),
            "country",
            "state",
            "total_orders",
            "total_revenue",
            "avg_order_value",
```

```
            "unique_customers",
            "max_order_value",
            "delivered_revenue",
            "revenue_per_customer"
        )
    )
```

The Gold layer transforms Silver data into business-ready metrics, with specific design choices to handle streaming efficiently and accurately. Key considerations include:

- **Streaming Join Strategy**: The code uses a **stream-to-static join** by reading orders as a stream and customers as a batch table. This is more efficient than a stream-to-stream join because customer data is relatively static (doesn't change frequently). Spark automatically optimizes this join pattern.
- **Why Window Aggregations Are Required**: Streaming aggregations with watermarks must use time windows. Without windows, Spark cannot determine when to finalize aggregation results. The window groups data into 30-day buckets—once the watermark passes the end of a window period, that window's results are finalized and output. This ensures correct results even when data arrives late.
- **Watermark for Late Data**: The `.withWatermark("order_date", "1 day")` tells Lakeflow to wait up to 1 day for late-arriving data before considering a time window complete. For example, orders for January 15th might arrive on January 16th and still be included in the correct window.
- **Approximate vs Exact Counts**: We use `approx_count_distinct()` instead of `countDistinct()` because streaming aggregations don't support exact distinct counts. The approximation uses `HyperLogLog` algorithm and is typically 97%+ accurate while being much more efficient.
- **Reading Silver Tables**: The Gold layer reads from both `sdp_customers_silver` and `sdp_orders_silver`. Lakeflow automatically:
    a. Understands this table depends on both Silver tables
    b. Ensures both Silver tables are ready before processing Gold
    c. Handles the execution order correctly
- **Output Schema**: The `final .select()` extracts the window boundaries as separate columns (period_start and period_end), making it easy to understand which time period each row represents when querying the results

## Tables vs. materialized views in LakeFlow

Understanding when Lakeflow creates tables versus materialized views is crucial for controlling your pipeline's behavior and ensuring you have the right permissions.

The **decorator** you use determines the dataset type. `@dp.table` declares a streaming table — a regular Delta table that processes data incrementally and requires standard `CREATE TABLE` permissions. `@dp.materialized_view` declares a materialized view in Unity Catalog, which refreshes completely on each pipeline run and requires `CREATE MATERIALIZED VIEW` permissions. The read method you use inside the function must be consistent with the decorator: `spark.readStream.table()` for streaming tables, and `spark.read.table()` for materialized views.

In our Gold layer, we chose `spark.readStream.table()` for three practical reasons. First, streaming tables only require `CREATE TABLE` permissions, making them accessible to more users. Second, windowed streaming aggregation is more efficient for large datasets - as new orders arrive, only the affected time windows are updated rather than recalculating everything. Third, stream-to-static joins (streaming orders with batch customers) work reliably with watermarks and windows.

Streaming aggregations do come with some restrictions: you must include a time window when using watermarks, exact distinct counts are not supported (use `approx_count_distinct()` instead), and the `.orderBy()` clause cannot be used. These constraints are why our Gold layer uses 30-day time windows and approximate counts.

As an alternative, if you have `CREATE MATERIALIZED VIEW` permissions and your dataset is small enough for full reprocessing, you can use `spark.read.table()` for both tables in the Gold layer. This removes the need for windows and watermarks and allows exact distinct counts, but reprocesses all data on every pipeline run:

```
# Batch/materialized view approach (alternative - requires permissions)
@dp.materialized_table
def sdp_geographic_sales():
    customers = spark.read.table("sdp_customers_silver")  # Batch read
    orders = spark.read.table("sdp_orders_silver")        # Batch read

    # No window needed, results in materialized view (UC) or batch table (Hive)
    # Full refresh on each run, supports exact distinct counts
    return (
        orders.join(customers, "customer_id")
        .groupBy("state")
```

```
        .agg(count("*").alias("total_orders"))
    )
```

For production pipelines in Unity Catalog without materialized view permissions, we recommend using streaming tables with time windows throughout your Gold layer. This approach provides incremental processing, broad compatibility with permissions, and natural time-based analytics.

## Creating and running the pipeline

Now that we have our pipeline code, let's create and run the pipeline in Databricks:

### Step 1: Create the pipeline

In the Databricks workspace, navigate to **Jobs and Pipelines** in the left sidebar.

1. Click **ETL Pipeline**
2. Click **Create Pipeline**

### Step 2: Configure the pipeline

Fill in the pipeline configuration:

1. **Pipeline name**: `chapter7_orders_pipeline`
2. **Product edition**: Select **Advanced** (provides full features, including expectations)
3. **Notebook libraries**: Click **Add source code** and select the Python file (`.py`) containing your pipeline code. The Lakeflow Pipelines Editor uses workspace source files, not standard notebooks, as the primary authoring format.
4. **Storage location**: Select **Unity Catalog**
5. **Catalog**: `cat_dev` (your development catalog)
6. **Target schema**: `sandbox` (or your preferred schema)
7. **Cluster mode**: Select **Fixed size with 1 worker** (for development/testing)
8. **Cluster policy**: Leave as default or select an appropriate policy for your organization

If you click **Dry run**, the UI will create a lineage graph in *Figure 7.1*.

*Figure 7.1.: Spark Declarative Pipeline lineage graph*

## Step 3: Start the pipeline

1. Click **Create** to save the pipeline configuration
2. Click **Start** to begin pipeline execution
3. The pipeline UI will show the execution progress

## Step 4: Monitor execution

As the pipeline runs, you'll see:

- **Lineage Graph**: A visual representation showing Bronze → Silver → Gold flow with arrows indicating dependencies
- **Processing Status**: Each table shows its processing state (pending, running, completed)
- **Data Quality Metrics**: Click on any Silver table to see expectation violation counts
- **Row Counts**: See how many records are in each table
- Expected Data Quality Results: When you run this pipeline, you'll see the following in the data quality dashboard

**Results for `sdp_orders_silver`:**

- **52 records dropped** due to `valid_order_status` violation (orders with 'Cancelled' status)
- These dropped records are not included in the Silver table, enforcing business rules that we only process non-cancelled orders

**Results For `sdp_customers_silver`:**

- **11 expectation warnings** for `recent_signup` (customers who signed up before 2023-01-01)
- These are historical customers, and the records are kept in the table for analysis but flagged for visibility

*Figure 7.2: Spark Declarative Pipeline lineage graph with data quality results and outputs*

These violations demonstrate how Lakeflow's expectations work in practice:

- Critical rules (`expect_or_drop`) actively filter your data, removing records that don't meet business requirements
- Monitoring rules (`expect`) provide visibility into data patterns without blocking the pipeline

The first run will process all data. Subsequent runs will only process new data added to the source files.

The pipeline execution UI (*Figure 7.3*) displays all tables in the data pipeline, including their processing status, duration, and output row counts. The interface highlights data quality metrics, showing which expectations were met or violated, including the number of records that were dropped due to quality rules and those that triggered warnings but were retained for review.

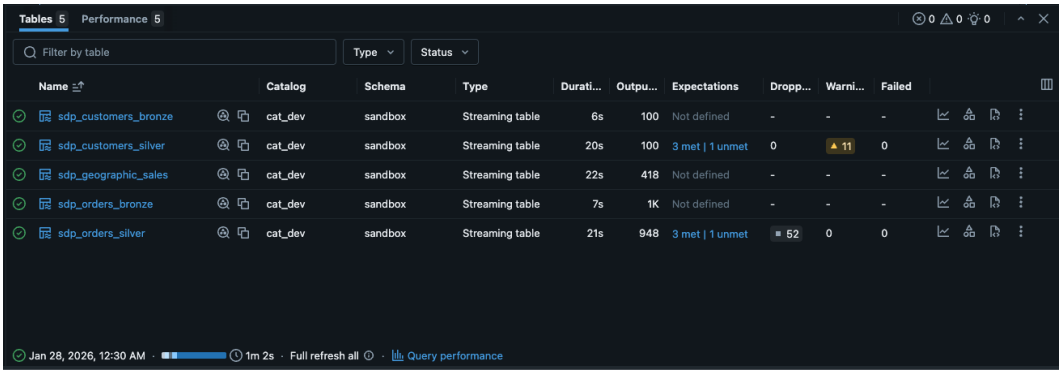



*Figure 7.3: Spark Declarative Pipeline run results*

## Verifying pipeline results

After the pipeline is completed successfully, you can query the tables using SQL:

```
-- Check row counts across all layers
SELECT 'sdp_orders_bronze' as table_name, COUNT(*) as row_count
FROM cat_dev.sandbox.sdp_orders_bronze
UNION ALL
SELECT 'sdp_customers_bronze', COUNT(*) FROM cat_dev.sandbox.sdp_customers_bronze
UNION ALL
SELECT 'sdp_orders_silver', COUNT(*) FROM cat_dev.sandbox.sdp_orders_silver
UNION ALL
SELECT 'sdp_customers_silver', COUNT(*) FROM cat_dev.sandbox.sdp_customers_silver
UNION ALL
SELECT 'sdp_geographic_sales', COUNT(*) FROM cat_dev.sandbox.sdp_geographic_sales;
```

## Viewing data quality metrics in the pipeline UI

After the pipeline runs, click on the `sdp_orders_silver` or `sdp_customers_silver` tables in the pipeline lineage graph. You'll see:

For `sdp_orders_silver`:

- **52 records dropped** due to `valid_order_status` expectation
- This shows in the UI as "Records dropped: 52" with the expectation name highlighted
- The Silver table contains only orders with 'Delivered', 'Shipped', or 'Pending' status

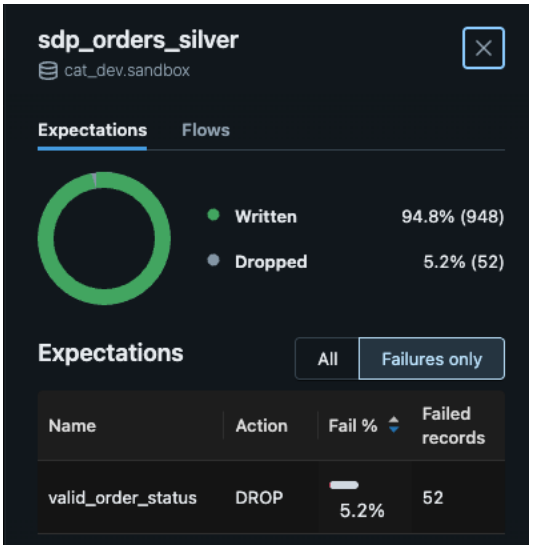



*Figure 7.4: Detailed description of failed expectation*

For `sdp_customers_silver`:

- **11 expectation warnings** for `recent_signup`
- The UI shows "`recent_signup: 11 violations (kept)`" with a yellow warning indicator
- All 11 customers are retained in the table but flagged for review

These metrics demonstrate the difference between `@dp.expect_or_drop` (actively filters data) and `@dp.expect` (monitors but keeps data). This visibility helps you assess your data quality without writing quality-check queries manually.

You can also view the tables in the Unity Catalog UI:

- Navigate to **Catalog** in the left sidebar
- Browse to `cat_dev` → `sandbox`
- Click on any table to see its schema, sample data, and lineage information

With the pipeline built, deployed, and verified, you now have a working end-to-end Lakeflow pipeline processing data through Bronze, Silver, and Gold layers. You've seen how expectations automatically enforced data quality rules—dropping 52 canceled orders and flagging 11 historical customer signups. In the next section, we'll explore Lakeflow's expectation system in greater depth, looking at additional patterns for managing data quality across production pipelines.

# Managing data quality with expectations

Data quality is critical for reliable analytics and decision-making. Lakeflow provides built-in support for defining, enforcing, and monitoring data quality through expectations. In this section, we'll explore the different types of expectations, how to implement them effectively, and how to monitor quality metrics.

## Types of expectations

Lakeflow provides three types of expectations, each with different enforcement behaviors:

1. `@dp.expect_or_drop`: Drops records that violate the expectation. Use this for critical data quality rules where invalid records would cause downstream problems.

   ```
   @dp.expect_or_drop("valid_order_id", "order_id IS NOT NULL")
   @dp.expect_or_drop("valid_order_total", "order_total > 0")
   ```

   When a record violates these expectations, it's dropped from the table entirely. The violation is logged and counted in the pipeline metrics, but the invalid record doesn't appear in the output.

2. `@Dp.expect`: Logs violations but keeps the records. Use this for data quality rules where you want to monitor issues but still need to keep all records for manual review or investigation.

   ```
   @dp     .expect("valid_email", "email LIKE '%@%'")
   @dp     .expect("valid_order_status", "order_status IN ('Delivered',
   'Shipped', 'Pending', 'Cancelled')")
   ```

   Records that violate these expectations remain in the table, but the violations are tracked. You can review violation counts in the pipeline UI and query the event log to identify specific problematic records.

3. `@.expect_or_fail`: Fails the entire pipeline if any record violates the expectation. Use this for critical business rules where any violation indicates a serious problem that requires immediate attention.

   ```
   @dp     .expect_or_fail("critical_revenue", "total_revenue > 0")
   ```

If any record violates this expectation, the pipeline stops immediately with an error. This is useful for catching data corruption or critical upstream failures early.

## Implementing effective quality rules

When designing quality expectations, consider these principles:

1. **Layer-Appropriate Validation**: Apply different types of expectations at different layers:
    - **Bronze Layer**: Minimal or no expectations—preserve raw data as-is for auditability
    - **Silver Layer**: Comprehensive expectations—enforce data types, business rules, referential integrity
    - **Gold Layer**: Business logic expectations—ensure aggregations and calculations produce valid results
2. **Fail Fast vs Fail Later**: Use `expect_or_drop` for issues you can handle automatically (missing values, format problems). Use `expect_or_fail` for issues that indicate serious upstream problems requiring human intervention.
3. **Composite Rules**: Expectations can use complex SQL expressions:

```
@dp.expect_or_drop(
    "valid_date_range",
    "order_date >= '2020-01-01' AND order_date <= current_date()"
)

@dp.expect_or_drop(
    "consistent_totals",
    "ABS(order_total - (unit_price * quantity)) < 0.01"
)
```

4. **Quarantine Patterns**: For critical pipelines, you might want to preserve invalid records for investigation rather than dropping them. You can implement a quarantine pattern:

```
# Main silver table - drops invalid records
@dp.table
@dp.expect_or_drop("valid_order", "order_total > 0 AND customer_id IS NOT
NULL")
def sdp_orders_silver():
    return spark.readStream.table("sdp_orders_bronze").filter(...)
# Quarantine table - captures only invalid records
@dp.table(
    comment="Quarantined orders that failed validation"
```

```
)
def df_orders_quarantine():
    return (
        spark.readStream.table("sdp_orders_bronze")
        .filter("order_total <= 0 OR customer_id IS NULL")
        .withColumn(
            "quarantine_reason",
            when(col("order_total") <= 0, "Invalid order total")
            .when(col("customer_id").isNull(), "Missing customer ID")
            .otherwise("Unknown")
        )
        .withColumn("quarantine_timestamp", current_timestamp())
    )
```

This pattern allows you to analyze problematic records separately without polluting your clean Silver table.

## Monitoring quality metrics

Lakeflow automatically tracks expectation violations and provides several ways to monitor quality:

1. **Pipeline UI Metrics**: The pipeline execution view shows:
    - Total records processed by each table
    - Expectation violation counts for each rule
    - Visual indicators (green checkmarks for passing, yellow/red for violations)
2. **Event Log Queries**: You can query detailed metrics programmatically using the event log. The event log is a Delta table that records all pipeline execution details:

```
# Query expectation metrics from the event log
from pyspark.sql.functions import explode, col
# Read event log for your pipeline
events = spark.read.format("delta").load(
    "/pipelines/<pipeline-id>/system/events"
)
# Extract expectation metrics
expectation_metrics = (
    events
    .filter("event_type = 'flow_progress'")
    .select(
```

```
        col("timestamp"),
        col("origin.flow_name").alias("table_name"),

explode(col("details.flow_progress.metrics.data_quality.expectations")).alias("expectation")
    )
    .select(
        "timestamp",
        "table_name",
        col("expectation.name").alias("expectation_name"),
        col("expectation.dataset").alias("dataset"),
        col("expectation.passed_records").alias("passed_records"),
        col("expectation.failed_records").alias("failed_records")
    )
    .filter("failed_records > 0")
    .orderBy(desc("timestamp"))
)
display(expectation_metrics)
```

You can also check expectations directly on tables:

```
-- View data quality metadata
DESCRIBE DETAIL cat_dev.sandbox.sdp_orders_silver;
-- Check Delta table properties which include quality metrics
SHOW TBLPROPERTIES cat_dev.sandbox.sdp_orders_silver;
```

3. **Alerting on Quality Issues**: For production pipelines, you'll want to set up alerts when quality thresholds are exceeded. You can:
    - Create a monitoring notebook that queries the event log for expectation failures
    - Schedule this notebook to run after pipeline completion
    - Send alerts (via email, Slack, PagerDuty) when violations exceed thresholds

Example monitoring logic:

```
def send_alert(message):
    """Placeholder: replace with your alerting integration.
    Examples: email via SMTP, Slack webhook, PagerDuty API, or
    Databricks SQL Alert.
    """
    print(f"ALERT: {message}")
    # In production, integrate with your preferred alerting system:
    # - Slack: requests.post(webhook_url, json={"text": message})
```

```
    # - Email: use smtplib or a managed email service
    # - PagerDuty: use their Events API


def check_quality_thresholds(table_name, max_failure_rate=0.01):
    """Check if expectation failure rate exceeds threshold"""

    # Query latest pipeline run
    latest_metrics = (
        expectation_metrics
        .filter(f"table_name = '{table_name}'")
        .orderBy(desc("timestamp"))
        .limit(1)
    )

    for row in latest_metrics.collect():
        total_records = row.passed_records + row.failed_records
        failure_rate = row.failed_records / total_records if total_records
> 0 else 0

        if failure_rate > max_failure_rate:
            # Send alert
            alert_message = f"""
            Quality Alert: {table_name}
            Expectation: {row.expectation_name}
            Failed records: {row.failed_records} ({failure_rate*100:.2f}%)
            Threshold: {max_failure_rate*100}%
            """
            send_alert(alert_message)

check_quality_thresholds("sdp_orders_silver", max_failure_rate=0.01)
```

# Best practices for data quality

Based on production experience, follow these guidelines:

- **Start Simple**: Begin with basic expectations (NOT NULL, value ranges) and add complexity as you understand your data better. Over-engineering expectations upfront can slow pipeline development.
- **Monitor Trends**: Track expectation violations over time. A sudden spike often indicates upstream data issues that need investigation.

- **Document Expectations**: Use clear, descriptive names for expectations and include comments explaining the business rule:

```
@dp.expect_or_drop(
    "valid_customer_age",
    "customer_age >= 18 AND customer_age <= 120"
)
# Business rule: Only adult customers (18+) can place orders
# Upper bound of 120 catches data entry errors
```

- Regular Review: Schedule periodic reviews of quarantine tables and expectation violations to identify patterns and improve upstream data sources

# Optimizing Lakeflow pipeline performance

While Lakeflow handles many optimizations automatically, understanding performance considerations helps you build efficient pipelines. This section covers key optimization techniques specific to Lakeflow.

## Automatic optimizations

Lakeflow applies several optimizations automatically:

1. **Auto Scaling**: Lakeflow automatically scales cluster resources based on data volume. You specify the cluster size in the pipeline configuration, and Lakeflow scales up during peak processing and scales down during idle periods.
2. **Incremental Processing**: When you use streaming tables (`readStream` and `read_stream`), Lakeflow automatically:
    - Tracks which data has been processed using checkpoints
    - Only processes new or changed data on subsequent runs
    - Handles failures gracefully by resuming from the last successful checkpoint
3. **Predictive Optimization:** For Unity Catalog-managed tables, Databricks Predictive Optimization is the platform default — it automatically runs OPTIMIZE and VACUUM operations based on table usage patterns. The legacy property `pipelines.autoOptimize.managed` is no longer required and can be omitted from your table definitions. For more details on optimization operations, see Chapter 10, Optimizing Query Performance and Cost Management.
4. **Smart Shuffles:** Lakeflow optimizes data shuffles in aggregations and joins by analyzing your pipeline DAG and reusing shuffle results when possible

## Pipeline configuration

The pipeline configuration significantly impacts performance:

- **Cluster Sizing**: For development, a single-node or small cluster is sufficient. For production:
- **High-throughput ingestion**: Use clusters with more cores (8–16 cores per worker).
- **Complex transformations**: Use memory-optimized instances.
- **Large joins**: Ensure adequate memory (32GB+ per worker).
- **Enhanced Autoscaling**: Enable this option for production pipelines with variable workloads. Lakeflow will scale the cluster between minimum and maximum sizes based on demand.
- **Channel Management**: Lakeflow processes tables in parallel when possible. The UI shows channels representing parallel execution paths. If you see many tables waiting, you might benefit from a larger cluster to increase parallelism.

## Clustering strategies

While Lakeflow handles many optimizations, you still control partitioning for your tables:

```
@dp.table(
    cluster_by=["order_year", "order_month"],
    comment="Orders partitioned by date for efficient querying"
)
def sdp_orders_silver():
    return (
        spark.readStream.table("sdp_orders_bronze")
        .withColumn("order_year", year("order_date"))
        .withColumn("order_month", month("order_date"))
        # ... more transformations
    )
```

Databricks best practices recommend **liquid clustering** over traditional partitioning for Lakeflow SDP tables. Liquid clustering (specified via the `cluster_by` parameter) automatically organizes data for efficient querying without the pitfalls of traditional partitioning—such as too many small files from over-partitioning or skewed partition sizes. Liquid clustering adapts to your query patterns and can be reconfigured without rewriting the underlying data. For detailed clustering and optimization guidance, see Chapter 10, Optimizing Query Performance and Cost Management.

## Optimizing joins and aggregations

- **Broadcast Joins**: When joining a large table with a small table (< 10MB), Lakeflow automatically uses broadcast joins for better performance. You can explicitly hint this:

```
from pyspark.sql.functions import broadcast

def df_orders_with_customer_info():
    customers = spark.readStream.table("sdp_customers_silver")
    orders = spark.readStream.table("sdp_orders_silver")

    # Hint that customers table should be broadcast
    return orders.join(broadcast(customers), "customer_id")
```

- **Pre-Aggregation**: For Gold tables with complex aggregations, consider creating intermediate tables with partial aggregations:

```
# Intermediate table - daily metrics
@dp.table
def df_daily_order_metrics():
    return (
        spark.read.table("sdp_orders_silver")
        .groupBy(to_date("order_date").alias("order_date"), "customer_id")
        .agg(
            count("order_id").alias("daily_orders"),
            sum("order_total").alias("daily_total")
        )
    )
# Final table - monthly metrics from daily
@dp.table
def df_monthly_customer_metrics():
    return (
        spark.read.table("df_daily_order_metrics")
        .withColumn("order_month", trunc("order_date", "month"))
        .groupBy("order_month", "customer_id")
        .agg(
            sum("daily_orders").alias("monthly_orders"),
            sum("daily_total").alias("monthly_total")
        )
    )
```

This approach reuses daily aggregations for monthly calculations, improving efficiency.

## Monitoring Pipeline Performance

Use these techniques to identify performance bottlenecks:

- **Pipeline Run History**: The Lakeflow UI shows execution time for each table. Look for tables that take disproportionately long—these are candidates for optimization.
- **Spark UI**: Click on a running pipeline in the Lakeflow UI, then click **View Spark UI** to see detailed execution plans, shuffle metrics, and task timings.
- **Metrics Queries**: Query the event log to analyze performance trends:

```
# Analyze processing times by table
events = spark.read.format("delta").load(
    "/pipelines/<pipeline-id>/system/events"
)
processing_times = (
    events
    .filter("event_type = 'flow_progress'")
    .filter("details.flow_progress.status = 'COMPLETED'")
    .select(
        col("origin.flow_name").alias("table_name"),

col("details.flow_progress.metrics.num_output_rows").alias("rows_processed"),
        (col("details.flow_progress.metrics.time_range.end_time") -

col("details.flow_progress.metrics.time_range.start_time")).alias("processing_time_ms")
    )
    .groupBy("table_name")
    .agg(
        avg("processing_time_ms").alias("avg_processing_time_ms"),
        sum("rows_processed").alias("total_rows")
    )
)
display(processing_times.orderBy(desc("avg_processing_time_ms")))
```

For comprehensive performance optimization techniques applicable to all Spark workloads, including Z-ordering, caching, and advanced tuning, refer to *Chapter 10, Optimizing Query Performance and Cost Management*.

# Summary

In this chapter, you learned how to build automated, production-ready data pipelines using Lakeflow Spark Declarative Pipelines. We covered:

- **Declarative Pipeline Development**: Rather than writing imperative code that specifies how to process data, Lakeflow allows you to declare what your data should look like. The system automatically handles dependency management, execution ordering, and operational concerns.
- **Medallion Architecture**: We implemented a complete Bronze → Silver → Gold pipeline, demonstrating how to structure data transformations from raw ingestion through cleaned data to business-ready analytics.
- **Data Quality with Expectations**: Lakeflow's built-in expectations provide first-class support for data quality rules. You learned to use `expect_or_drop` for critical validations, `expect` for monitoring, and `expect_or_fail` for critical business rules. Quality metrics are automatically tracked and visualized.
- **Tables vs Materialized Views**: You learned how the choice between `and` determines whether Lakeflow creates streaming tables or materialized views, and how this affects performance, permissions, and incremental processing behavior.
- **Pipeline Operations**: We walked through creating, configuring, and running pipelines in the Databricks UI, and explored how to monitor execution through lineage graphs and quality metrics.
- **Performance Optimization**: While Lakeflow handles many optimizations automatically, you learned how to configure pipelines effectively, optimize joins and aggregations, and monitor performance through the pipeline UI and event logs.

Lakeflow represents a significant improvement over traditional pipeline development approaches. By handling dependency management, data quality tracking, and operational concerns automatically, it allows you to focus on the business logic of your transformations rather than the mechanics of pipeline orchestration. For production data engineering, Lakeflow provides the reliability, observability, and maintainability required for mission-critical data systems.

In the next chapter, we'll explore advanced Delta Lake features including time travel, schema evolution, and change data feed capabilities that provide additional flexibility for managing your data over time.

## Get this book's PDF copy, code bundle, and more.

Scan the QR code (or go to `packtpub.com/unlock`). Search for this book by name, confirm the edition, and then follow the steps on the page.

UNLOCK NOW

*Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.*

# 8

# Orchestrating Data Workflows: From Notebooks to Production

Getting notebooks into production is where most data work hits reality. Notebooks work great for trying things out, but running stuff reliably at scale needs more than just executing notebooks on a schedule. You're moving from experimenting to building workflows that won't break when real data hits them.

Lakeflow Jobs is the native orchestrator within Databricks, designed to handle everything from simple scheduled notebook runs to complex, multi-task dependency graphs. Databricks recommends using Lakeflow Jobs as your primary orchestrator for all task dependencies within the platform. While you can integrate these encapsulated workflows into external orchestrators like Azure Data Factory or Apache Airflow for cross-system scenarios, Lakeflow Jobs now offers the advanced capabilities needed for most enterprise data workflows natively. You still develop in notebooks, but now you can layer on the monitoring and controls that production requires. Knowing how to orchestrate well is what separates projects that work on your laptop from systems that run every day.

Real workflows are complicated. Jobs depend on each other; things fail, resources need tuning, and everything needs watching. Your pipelines must handle larger-than-expected data, adapt when the business changes direction, and meet deadlines without blowing your budget. You need to understand orchestration patterns, write modular notebooks, and monitor things before they break.

This chapter covers workflow orchestration in Databricks from development through deployment. You'll use Lakeflow Jobs to integrate enterprise tools, write better notebooks, and set up monitoring to catch issues early. The examples are practical and based on what actually happens in production.

This matters if you're trying to move past experiments and ship real data products. You'll learn to build workflows that grow with your company while keeping the flexibility that makes Databricks useful.

This chapter covers the following topics:

- **Using Lakeflow Jobs for Task Scheduling**: We will explore the architecture of Lakeflow Jobs and provide a hands-on guide to creating, configuring, and optimizing them.
- **Integrating Databricks with Azure Data Factory and Airflow**: This section details how to connect Databricks with powerful external orchestrators, including architecture patterns and development guides for both ADF and Airflow.
- **Orchestration Tool Comparison and Selection**: We will compare Lakeflow Jobs, ADF, and Airflow across several dimensions like performance, complexity, and cost to help you choose the right tool for your needs.
- **Best Practices for Modularizing Notebooks**: Learn how to transform monolithic exploratory notebooks into modular, reusable, and production-ready code using software engineering principles and **Declarative Automation Bundles (DABs)**.
- **Comprehensive Monitoring and Debugging**: This section covers setting up robust monitoring, logging, and alerting, along with practical techniques for debugging production workflows.

By the end of this chapter, you will have learned how to:

- Design, implement, and manage production-grade data workflows using Lakeflow Jobs.
- Integrate Databricks with enterprise orchestration tools like Azure Data Factory and Apache Airflow.
- Apply best practices for writing modular, testable, and maintainable notebook code.
- Establish comprehensive monitoring and debugging practices to ensure workflow reliability and performance.

# Technical Requirements

To follow the examples and apply the concepts in this chapter, you will need the following:

- **Azure Databricks Workspace**: A Premium tier workspace is recommended to access all features, including advanced cluster policies and monitoring.

- **Databricks Runtime**: Version 13.0 or higher is required to use features like Liquid Clustering. Photon-enabled runtimes are highly recommended for optimal performance.
- **Cloud Subscription**: An active Azure subscription with permissions to create and manage Databricks clusters and access storage.

# Using Lakeflow Jobs for Task Scheduling

Lakeflow Jobs are the cornerstone of production orchestration on the platform, providing a robust mechanism to run non-interactive code on a schedule or in response to events. This section will delve into the architecture of Lakeflow Jobs, walk through the process of creating and configuring them, and explore advanced patterns for optimization and cost management.

## Understanding Lakeflow Jobs Architecture

Lakeflow Jobs are how you run production workflows without babysitting them. Unlike interactive notebooks, where you click "run" yourself, jobs execute automatically with dependency handling, error recovery, and resource management built in. The system scales compute up and down based on what's actually running while keeping costs under control through smart cluster management.

Jobs have a few moving parts working together. The scheduler kicks off jobs based on time, file arrivals, or API calls. The cluster manager spins up compute when needed and shuts it down when done, using reactive auto-scaling to match resources to the workload. The execution engine runs your notebook or JAR code, spreads tasks across nodes, and retries failures automatically. Lakeflow Jobs uses a DAG-based (Directed Acyclic Graph) task dependency model, meaning you can define complex workflows where tasks run sequentially, in parallel, or conditionally based on the success or failure of previous tasks.

You've got options for how jobs use resources. Job clusters spin up fresh for each run and terminate when done - maximum isolation, easiest cost control. All-purpose clusters stick around and handle both interactive work and scheduled jobs, which works better if you're constantly using Databricks anyway, though it's important to note that all-purpose clusters have a higher billing rate. Picking between them depends on how often jobs run, what resources they need, and whether your org wants strict separation or shared infrastructure.

## Hands-on Job Creation and Configuration

Building production jobs means knowing both the UI and the API - you'll need both depending on the situation. The Databricks UI works well when you're setting up and testing jobs initially. However, to programmatically manage jobs as part of CI/CD, Databricks recommends using **Declarative Automation Bundles** (**DABs**) or the Databricks Terraform provider. The API

is what you need for managing infrastructure as code. Learn both, and you can handle jobs however your organization works.

Start by defining the basics: which notebook or JAR to run, which cluster to use (or the specs for a new one), how to pass parameters, and what happens when things fail or time out. These choices affect whether your job runs reliably, how fast it goes, and what it costs. Think about how much data you're processing, how complex the work is, and how critical the job is to the business.

Here is an example of an essential job configuration. To execute this, you would typically save it as a JSON file (e.g., `job_config.json`) and submit it using the Databricks CLI: jobs create `--json @job_config.json`.

```
{
  "name": "daily_etl_pipeline",
  "job_clusters": [
    {
      "job_cluster_key": "etl_cluster",
      "new_cluster": {
        "spark_version": "16.4.x-scala2.12",
        "node_type_id": "Standard_DS3_v2",
        "autoscale": {
          "min_workers": 1,
          "max_workers": 8
        }
      }
    }
  ],
  "tasks": [
    {
      "task_key": "etl_notebook",
      "job_cluster_key": "etl_cluster",
      "notebook_task": {
        "notebook_path": "/Workspace/Users/data-team/production/etl_pipeline",
        "base_parameters": {
          "environment": "production",
          "date": "{{job.start_time.iso_date}}"
        }
      },
      "timeout_seconds": 3600,
      "max_retry_interval_millis": 60000
```

```
    }
  ],
  "email_notifications": {
    "on_failure": ["data-team@company.com"],
    "on_success": ["stakeholders@company.com"]
  }
}
```

Parameters keep jobs from being hardcoded messes. Bake in defaults when you create the job, swap them out at runtime, or grab them from notebook widgets. Use all three, and you won't rewrite code every time requirements shift.

Multi-task jobs handle pipelines that do more than one thing. Fire off ingestion tasks simultaneously, string transformations together, and branch when data looks wrong. Tell tasks which other tasks to wait for, give heavy steps bigger clusters, and throw in conditions where it makes sense. Complex stuff stays trackable, and you can figure out what broke when things go sideways.

## Advanced Job Patterns and Optimization

Advanced job patterns let you build workflows that handle real-world messiness. Conditional execution means jobs can check data quality, verify that upstream systems are ready, or verify that files have arrived before proceeding. Your pipeline adapts instead of crashing when data's late or something upstream is broken.

Dynamic parameters pull settings from databases, configuration files, or APIs at runtime rather than hardcoding everything. Jobs can process different sources, apply different business logic, or interact with external systems whose requirements change. You're not constantly updating job configs when things shift around.

Job pools are pre-warmed clusters shared across jobs. No waiting for clusters to boot every time, better resource usage, lower costs. Makes sense when you run jobs frequently or have tight SLAs that can't absorb five - minute startup delays.

Advanced job pool configuration:

```
job_pool_config = {
     "instance_pool_name": "production_etl_pool",
     "min_idle_instances": 2,
     "max_capacity": 20,
     "node_type_id": "Standard_DS3_v2",
     "idle_instance_autotermination_minutes": 15,
     "preloaded_spark_versions": ["11.3.x - scala2.12"],
```

```
        "azure_attributes": {
            "availability": "SPOT_WITH_FALLBACK_AZURE",
            "spot_bid_max_price": 0.50
        }
    }
```

Cutting costs on Lakeflow Jobs means playing with cluster size, instance types, auto-scaling, and spot instances. You need to know your workloads and what Azure charges for different options. Smart shops run critical jobs in reliable instances and use cheaper spot instances for stuff that can handle interruptions or retries.

Monitoring shouldn't just tell you if jobs passed or failed. Hook into Azure Monitor for centralized logs and metrics, set up alerts based on execution patterns, automate escalations when critical stuff breaks. Good monitoring keeps you ahead of SLA violations and helps you fix problems before users notice.

Connecting jobs to version control and CI/CD pipelines is how you avoid chaos in production. The current Databricks-recommended approach for CI/CD is DABs. Teams can review code, run tests, deploy in stages - basically treat data workflows like actual software instead of random scripts. This matters if you want consistency and the ability to ship improvements without breaking things.

# Integrating Databricks with Azure Data Factory and Airflow

Azure Data Factory and Apache Airflow represent two distinct approaches to enterprise orchestration, each with unique strengths for coordinating Databricks workflows across complex data ecosystems. This section explores both integration patterns, providing practical guidance on architecture, implementation, and when to choose each tool. We will begin with Azure Data Factory, Microsoft's cloud-native orchestration service that integrates deeply with the Azure ecosystem.

# Azure Data Factory Integration Architecture

Azure Data Factory layers enterprise orchestration on top of Databricks. ADF moves data around, schedules work, and connects Azure services, while Databricks handles the actual analytics and ML-heavy lifting. Use both when your pipelines span multiple systems, or you need one place to control everything.

ADF talks to Databricks through a Linked Service that handles authentication. Microsoft's current guidance is that Managed Identity is the preferred authentication method. The

Databricks activities in ADF run notebooks, JARs, or Python files, pass in data, and grab results. Conditionals and error handling come standard.

You can point ADF at clusters that are already running or create new ones per job. Running clusters is faster, but it wastes money sitting there. Job clusters are cheaper but take time to start. Choose based on pipeline frequency, resources needed, and who's paying for what.

To configure the ADF-Databricks integration, you must follow these steps:

1. Create an Azure Data Factory instance.
2. Configure Managed Identity in ADF.
3. Register the ADF Managed Identity as a Service Principal in Databricks.
4. Grant workspace permissions to the Service Principal.
5. Assign the Contributor role in Azure IAM.
6. Create the Linked Service in ADF.

## Hands-on ADF Pipeline Development

Building ADF pipelines for Databricks means knowing both the visual designer and the JSON underneath. The drag-and-drop interface works fine for getting started and testing. JSON editing gives you more control and lets you deploy pipelines programmatically. Learn both so you can pick whichever fits the task.

Common patterns show up repeatedly. ETL pipelines use ADF to move data around and Databricks to do the hard transformations and analysis. Event-driven setups trigger Lakeflow jobs when files arrive or external systems send notifications. Batch processing coordinates large workflows across multiple Lakeflow jobs that depend on one another and require careful resource planning.

ADF Pipeline with Databricks integration. Note that while you can define cluster configurations in the pipeline JSON, the pipeline will execute successfully but ignore the intended cluster configuration entirely, using the Linked Service settings instead.

```
{
  "name": "databricks_etl_pipeline",
  "properties": {
    "activities": [
      {
        "name": "data_ingestion",
        "type": "DatabricksNotebook",
        "linkedServiceName": {
          "referenceName": "AzureDatabricks_LinkedService",
```

```
          "type": "LinkedServiceReference"
        },
        "typeProperties": {
          "notebookPath": "/Shared/production/data_ingestion",
          "baseParameters": {
            "source_path": "@pipeline().parameters.source_path",
            "target_table": "@pipeline().parameters.target_table",
            "execution_date": "@formatDateTime(utcnow(), 'yyyy-MM-dd')"
          }
        }
      }
    ],
    "parameters": {
      "source_path": {
        "type": "string",
        "defaultValue": "/Volumes/analytics_dev/sales_raw/daily_data"
      },
      "target_table": {
        "type": "string",
        "defaultValue": "processed.daily_metrics"
      }
    }
  }
}
```

Passing parameters between ADF and Databricks gets complicated fast. ADF lets you use pipeline parameters, system variables, and outputs from earlier steps. Learning these options means building pipelines that handle different data sources and changing rules without constantly editing the pipeline itself.

Error handling needs to cover failures at both layers: ADF problems and Databricks issues, such as clusters that won't start, notebooks that crash, or running out of resources. Good error handling means knowing what can break, setting smart retry rules, and having backups when things fail. Use circuit breakers to prevent bad requests from piling up, dead-letter queues for items that keep failing, and alerts so your team knows when manual fixes are needed.

## Apache Airflow Integration Patterns

Airflow gives you more flexibility and control than ADF, especially if your team likes defining pipelines in code or you've got really complex workflows. While Databricks has explicitly positioned Lakeflow Jobs as the modern successor to Airflow for Databricks-centric workloads,

Airflow remains useful when you need detailed control over execution logic or connections to systems outside Azure.

Airflow communicates with Databricks via a couple of operators. `DatabricksSubmitRunOperator` runs notebooks or JARs directly, passing parameters and grabbing results. `DatabricksRunNowOperator` triggers jobs you've already set up in Databricks, letting you use existing job configs while Airflow handles scheduling and dependencies.

Setting up connections between Airflow and Databricks takes some care. Store your auth credentials and endpoints in Airflow connections, and then operators use those to talk to the Databricks API and monitor job execution. Get this config right, or you'll end up with security gaps or jobs that fail at random.

Advanced Airflow DAG for Databricks:

```
from airflow import DAG
from airflow.providers.databricks.operators.databricks import
DatabricksSubmitRunOperator
from datetime import datetime, timedelta

# Essential Airflow DAG configuration
default_args = {
    'owner': 'data-engineering',
    'depends_on_past': False,
    'start_date': datetime(2024, 1, 1),
    'email_on_failure': True,
    'email_on_retry': False,
    'retries': 2,
    'retry_delay': timedelta(minutes=5)
}

dag = DAG(
    'databricks_etl_workflow',
    default_args=default_args,
    description='Daily ETL workflow using Databricks',
    schedule_interval='0 2 * * *',  # Daily at 2 AM
    catchup=False,
    max_active_runs=1
)

# Databricks job configuration
databricks_task = DatabricksSubmitRunOperator(
```

```
    task_id='run_etl_notebook',
    databricks_conn_id='databricks_default',
    new_cluster={
        'spark_version': '11.3.x-scala2.12',
        'node_type_id': 'Standard_DS3_v2',
        'num_workers': 2,
        'autoscale': {'min_workers': 1, 'max_workers': 8}
    },
    notebook_task={
        'notebook_path': '/Shared/production/daily_etl',
        'base_parameters': {
            'execution_date': '{{ ds }}',
            'environment': 'production'
        }
    },
    dag=dag
)
```

Airflow DAGs for Databricks use several common patterns. Fan-out/fan-in splits work across multiple sources or partitions in parallel, then bring everything back together for aggregation. Conditional branching changes the workflow path based on data quality, characteristics, or external system behavior. Sensor-based triggering watches for files, database changes, or API events before kicking off Lakeflow jobs.

Running Airflow at scale on Azure means picking the right infrastructure. Host it on Azure Container Instances or Kubernetes, use Azure Database for PostgreSQL as the metadata store. Hook into Azure Monitor and Log Analytics for centralized logging and monitoring across your Airflow and Databricks workflows - you need this visibility when things are running in production.

## Orchestration Tool Comparison and Selection

Picking between Lakeflow Jobs, Azure Data Factory, and Airflow isn't straightforward - you need to weigh organizational needs, team capabilities, costs, and how you prefer to operate. Each tool has strengths and weaknesses that make it better suited for certain workflows and company contexts. Understanding these tradeoffs helps you choose an orchestration architecture that works now and scales later.

Lakeflow Jobs shine when your workflows live mostly within Databricks, with minimal external system coordination. The tight platform integration delivers strong performance and simpler management, and the built-in scheduling and monitoring cut operational overhead.

The limitations show up when you need extensive external integrations, intricate dependency graphs, or advanced workflow patterns that complex enterprise pipelines often demand.

Azure Data Factory excels at workflows requiring heavy data movement, connections to multiple Azure services, or unified governance across different processing systems. The visual designer and broad connector ecosystem make ADF attractive for organizations juggling complex data integration across varied sources. The downside is added complexity and cost when your workflows are mainly Databricks-centric, with little need for external orchestration capabilities.

| Orchestration Tool | Best Use Cases | Advantages | Limitations | Cost Considerations |
|---|---|---|---|---|
| **Lakeflow Jobs** | Databricks-centric workflows, ML pipelines | Deep platform integration, native DAG support, event triggers | Limited to Databricks execution environment | Included in DBU costs |
| **Azure Data Factory** | Enterprise data integration, hybrid cloud | Visual designer, massive connector library, Azure native | Higher learning curve for JSON, complex debugging | Pay-per-execution model |
| **Apache Airflow** | Complex cross-system workflows, code-first | Maximum flexibility, Python-based DAGs, open source | Operational overhead, infrastructure management | Variable cost based on hosting approach |

*Table 8.1: A comparison of Lakeflow Jobs, Azure Data Factory, and Apache Airflow for workflow orchestration.*

Performance factors matter when choosing orchestration tools - look at job startup times, how efficiently resources get used, and how things scale under different loads. Lakeflow Jobs usually start fastest for native Databricks work, while ADF and Airflow add latency due to their orchestration layers. That overhead might be worth it if you need the extra capabilities and flexibility these platforms offer.

Integration complexity depends on how many external systems you're connecting to, how tangled your data dependencies are, and what error handling and recovery you need. Simple workflows with few outside dependencies work fine with Lakeflow Jobs. Complex enterprise pipelines touching multiple systems benefit from ADF or Airflow's advanced features. Factor in your team's technical skills and how much long-term maintenance each approach demands.

Cost optimization varies across these tools and requires careful analysis beyond just the sticker price. Lakeflow Jobs are usually cheapest for straightforward workflows. ADF and Airflow can be more cost-effective for complex scenarios that would otherwise require multiple Lakeflow Jobs or custom glue code. Calculate total cost of ownership - development time, operational overhead, infrastructure expenses - to make decisions that fit your budget and performance needs.

# Best Practices for Modularizing Notebooks

While orchestration tools provide the framework for running workflows reliably, the quality and maintainability of the underlying code are equally critical to production success. This section focuses on transforming exploratory notebooks into modular, production-ready code by applying software engineering principles. This is also the ideal time to introduce DABs, the recommended tool for adopting software engineering best practices in Databricks. We will cover core modularization principles, hands-on refactoring techniques, and production patterns that ensure your code is testable, maintainable, and ready to scale.

## Notebook Modularization Principles

Moving from exploratory notebooks to production code means fundamentally restructuring how you build and organize things. Modularization is the foundation - it lets you create reusable, testable, maintainable code that grows with your needs. Good modularization isn't just tidying up code. It's applying software engineering principles such as separation of concerns, dependency management, and interface design to make notebooks production-ready.

Separation of concerns means clearly dividing functionality: data access in one place, business logic in another, and transformations separate from output generation. This lets you develop, test, and maintain each piece independently while keeping individual notebooks simpler. It

also means pulling configuration out of your business logic so notebooks adapt to different environments and data sources without code changes.

Reusable function libraries are crucial for sharing code across workflows without duplication. Design them with clean interfaces, solid documentation, and proper error handling so they work safely in different contexts. Think through parameter validation, standardize return values, and handle exceptions consistently so these libraries integrate reliably wherever they're used.

Configuration management needs to be flexible but not chaotic. External configuration files, environment variables, and parameter widgets each solve the problem differently. Declarative Automation Bundle targets (dev, staging, prod) are a great fit here. Getting this right means your workflows deploy smoothly across environments without touching code each time.

## Hands-on Notebook Refactoring Techniques

Turning monolithic notebooks into modular production code takes systematic refactoring that keeps things working while making them better to maintain and reuse. Start by identifying distinct chunks of functionality in your existing notebooks and pulling them out into separate modules or functions. You need to carefully track data dependencies, shared variables, and execution order so the refactored version behaves exactly like the original.

Function extraction is the basic refactoring move: find code blocks that do specific jobs and turn them into reusable functions with clean interfaces. Design parameters thoughtfully, specify return values clearly, and handle errors properly so these functions work safely wherever you use them. Also hunt down hidden dependencies, like global variables or assumptions about data structures, that'll break when the function runs in a different environment.

Notebook refactoring: Before and After:

```
# Before: Monolithic notebook cell
df = spark.read.format("delta").load("/Volumes/analytics_dev/sales_raw/staging/
sales_data")
df_filtered = df.filter(col("date") >= "2024-01-01")
df_aggregated = df_filtered.groupBy("region", "product").agg(
    sum("revenue").alias("total_revenue"),
    count("*").alias("transaction_count")
)
df_aggregated.write.format("delta").mode("overwrite").save(
    "/Volumes/analytics_dev/sales_processed/summary/sales_summary"
)
# After: Modularized approach
```

```
def load_sales_data(path: str, start_date: str = None) -> DataFrame:
    """Load and optionally filter sales data by date."""
    df = spark.read.format("delta").load(path)
    if start_date:
        df = df.filter(col("date") >= start_date)
    return df
def aggregate_sales_by_region_product(df: DataFrame) -> DataFrame:
    """Aggregate sales data by region and product."""
    return df.groupBy("region", "product").agg(
        sum("revenue").alias("total_revenue"),
        count("*").alias("transaction_count")
    )
def save_processed_data(df: DataFrame, path: str, mode: str = "overwrite") ->
None:
    """Save DataFrame to Delta format."""
    df.write.format("delta").mode(mode).save(path)
# Usage in notebook
sales_df = load_sales_data(
    "/Volumes/analytics_dev/sales_raw/staging/sales_data",
    "2024-01-01"
)
summary_df = aggregate_sales_by_region_product(sales_df)
save_processed_data(
    summary_df,
    "/Volumes/analytics_dev/sales_processed/summary/sales_summary"
)
```

Building shared utility libraries means setting clear patterns for organization, documentation, and versioning so teams can safely share code across projects. Organize libraries by functional domain - data access, data quality, business logic - with consistent naming and well-defined interfaces. Include thorough testing, generate docs, and use version control so the library stays reliable and you can update it without breaking everyone's code.

Parameter - driven notebooks adapt to different data sources, processing needs, and business rules without code changes. Determine what should be configurable and expose it via runtime parameters. Design parameters carefully - validate inputs, set sensible defaults, handle errors - so notebooks behave predictably no matter what parameter combinations people throw at them.

Version control for notebooks is trickier than regular code because of notebook file formats. You're dealing with output clearing, metadata quirks, and merge conflicts that play out

differently than in standard source files. Get version control practices right, and you maintain code quality while enabling actual collaboration on production notebooks, rather than everyone working in silos.

## Production Notebook Patterns and Standards

Production notebooks need solid error handling and logging to run reliably and enable troubleshooting when things break. Error handling has to cover expected problems, like poor data quality, as well as unexpected ones, like running out of resources or external systems going down. Build robust error handling by understanding different exception types, designing recovery strategies that make sense, and capturing error details in formats that actually help you debug issues.

Logging in production notebooks means finding the balance between capturing enough information and not tanking performance or drowning in log noise. Use structured logging formats that you can analyze programmatically, set log levels to capture what matters without overwhelming your log system, and hook into centralized logging to correlate events across multiple notebook runs. Also watch for sensitive data - make sure you're not accidentally dumping confidential information into logs where it shouldn't be.

```
import logging
from typing import Optional, Dict, Any
import json
# Production logging configuration
def setup_production_logging(log_level: str = "INFO") -> logging.Logger:
    """Configure structured logging for production notebooks."""
    logger = logging.getLogger(__name__)
    logger.setLevel(getattr(logging, log_level.upper()))

    # Create structured formatter
    formatter = logging.Formatter(
        '{"timestamp": "%(asctime)s", "level": "%(levelname)s",'
        '"module": "%(name)s", "message": "%(message)s"}'
    )

    # Configure handler
    handler = logging.StreamHandler()
    handler.setFormatter(formatter)
    logger.addHandler(handler)

    return logger
# Error handling with context
```

```
def safe_data_processing(func, *args, **kwargs) -> Dict[str, Any]:
    """Execute data processing function with comprehensive error handling."""
    logger = logging.getLogger(__name__)

    try:
        logger.info(f"Starting {func.__name__} with args: {args}, kwargs:
{kwargs}")
        result = func(*args, **kwargs)
        logger.info(f"Successfully completed {func.__name__}")
        return {"status": "success", "result": result}

    except Exception as e:
        logger.error(f"Error in {func.__name__}: {str(e)}")
        return {"status": "error", "message": str(e)}
```

Performance monitoring helps you spot slowdowns and optimization opportunities before they become problems. Track execution time, monitor resource usage, and analyze data volumes to identify trends and unusual spikes in performance. You can introduce system tables to achieve this. For example, `system.lakeflow.job_run_timeline` and `system.billing.usage` allow SQL analysis of job execution times, task durations, DBU consumption, and cost trends across all runs. Hook this into centralized monitoring systems and set up alerts that notify your ops team when performance starts degrading - before users complain.

Documentation standards keep production notebooks maintainable even when the original developer has moved on. Document what the notebook does, what goes in and out, parameter definitions, and operational gotchas. Add troubleshooting guides and runbooks so people can actually support and maintain these workflows without hunting down tribal knowledge.

Testing notebook code means adapting traditional testing to how notebooks work. Unit test your extracted functions, integration test full workflows, and quality test the output data. Balance test coverage against the speed you need to ship, while ensuring critical stuff gets validated before hitting production.

Quality assurance should include code reviews, automated testing in your pipeline, and deployment validation to keep standards consistent across all production notebooks. Integrate this with CI/CD so quality checks run automatically and broken notebooks can't make it to production. Find the balance between being thorough and moving fast while maintaining high quality.

## Comprehensive Monitoring Infrastructure

Monitoring production workflows in Azure Databricks needs multiple layers - platform metrics, application monitoring, and business process tracking - to give you full visibility into how things are running. Your monitoring infrastructure should capture everything from individual task times to complete workflow rates, and allow you to drill into specifics when problems arise. It also needs to integrate with the monitoring tools your organization already uses to provide centralized visibility and enable metric correlation across systems.

Databricks recommends analyzing system tables with Databricks-native tools such as Lakehouse Monitoring and AI/BI Dashboards for the best integration, performance, and cost optimization. These native tools should be the starting point for any Databricks-centric monitoring strategy. Azure Monitor is available as an option for customers who need to unify Databricks data with their broader Azure observability stack.

Custom metrics and alerts need to track what matters to the business without drowning your team in notifications. Good alerting means distinguishing symptoms from root causes, aggregating and correlating related alerts, and escalating appropriately so that critical issues get attention. Wherever possible, automate recovery to reduce the load on support teams while keeping systems running.

Log aggregation pulls logs from Databricks clusters, orchestration systems, and external dependencies so you can troubleshoot and analyze performance across the entire stack. Your logging infrastructure should handle structured logs for automated analysis while letting you run ad-hoc queries when debugging specific problems. Data Quality Monitoring (part of Lakehouse Monitoring) includes automated anomaly detection for Delta tables — tracking changes in data distribution, schema evolution, and metric drift over time to spot emerging issues before they impact the business.

## Hands-on Debugging Techniques and Tools

Debugging production workflows means finding root causes fast while keeping things running. Common failures include resource constraints, bad data quality, external system problems, and config errors - each needs its own debugging approach and tools. Knowing these patterns and what they look like helps you troubleshoot faster and get things fixed quicker.

Resource-constrained debugging involves examining cluster utilization, memory usage, and execution times to identify bottlenecks and optimization opportunities. You need to understand how different workloads use cluster resources and how constraints manifest across different failure types. Good resource debugging also means understanding the

relationships among your data characteristics, processing algorithms, and the resources you actually need, so you can plan capacity and optimize proactively.

```
# Production monitoring and debugging utilities
class WorkflowMonitor:
    def __init__(self, spark, workflow_name: str):
        self.spark = spark
        self.workflow_name = workflow_name
        self.start_time = time.time()

    def track_stage(self, stage_name: str, func, *args):
        """Track execution time for workflow stages."""
        start = time.time()
        try:
            result = func(*args)
            duration = time.time() - start
            print(f"Stage {stage_name}: {duration:.2f}s - SUCCESS")
            return result
        except Exception as e:
            print(f"Stage {stage_name}: FAILED - {str(e)}")
            raise
# Usage
monitor = WorkflowMonitor(spark, "daily_etl_pipeline")
raw_data = monitor.track_stage("data_loading", load_raw_data, "/Volumes/
analytics_dev/sales_raw/staging/data")
processed = monitor.track_stage("transformation", transform_data, raw_data)
```

Data Quality Monitoring (Lakehouse Monitoring) provides automated baseline tracking, anomaly detection, and schema drift monitoring for Delta tables. Profile your data comprehensively, establish quality baselines, and automate detection of common issues. Good debugging also means understanding your upstream data sources and their quirks so you can trace problems to their root causes and prevent them from recurring.

External system dependency debugging focuses on integration points with databases, APIs, and file systems to identify connectivity issues, performance problems, or missing data. You need to know how external systems behave under load and set appropriate timeouts and retries. Implement circuit breakers and fallback mechanisms, so workflows stay reliable even when external systems are struggling.

Root cause analysis for production failures needs systematic approaches that quickly identify underlying causes while documenting what you find for future prevention. Collect

comprehensive diagnostics, analyze logs and metrics, and correlate events across systems to trace causal relationships. Include recommendations for preventive measures and improvements to reduce the likelihood of similar failures.

## Operational Excellence and Continuous Improvement

**Service Level Agreement (SLA)** management for production workflows means setting clear performance targets, monitoring compliance, and having escalation procedures when things slip. Understand the business impact of different failures and set response times based on criticality. Lakeflow Jobs supports configuring duration warnings and alerts directly in the job definition — when a job run exceeds a defined threshold, notifications are sent automatically to track SLA trends and spot potential problems before they become violations.

Incident response needs to balance fixing things quickly with thorough documentation and learning from what happened. Set clear roles and responsibilities, keep stakeholders updated, and run post-incident reviews to identify improvements. Automate escalations and integrate with on-call systems so critical issues get attention at 3 am or on weekends.

Capacity planning requires understanding growth patterns, resource trends, and how performance changes under different loads. Analyze historical data, model future scenarios, and implement scaling strategies that accommodate growth without hurting performance. Consider cost optimization and the tradeoffs between performance and expense when planning how to scale.

Continuous improvement should include regular performance reviews, hunting for optimization opportunities, and systematically implementing changes based on operational experience and evolving requirements. Get dev and ops teams collaborating to ensure improvements tackle real problems without compromising reliability. Share knowledge across teams so everyone learns from each other's experiences instead of repeating mistakes.

Performance optimization encompasses algorithmic improvements, resource tuning, and infrastructure upgrades that boost efficiency and reduce costs. Understand how different workflow components perform and find the bottlenecks limiting overall throughput. Consider tradeoffs between optimization approaches and their impact on complexity and maintainability.

Orchestrating data workflows from notebooks to production is critical for organizations trying to operationalize analytics at scale. This chapter covered the tools, techniques, and practices you need to implement production - grade orchestration using Azure Databricks and complementary platforms. Moving from exploratory notebooks to robust production requires fundamental shifts in approach - systematic scheduling, enterprise integration, modular code, and comprehensive monitoring.

Lakeflow Jobs form the foundation for workflow orchestration in the Azure Databricks ecosystem, offering native scheduling optimized for Databricks workloads, with cost-effective resource management and reliable execution. Lakeflow Jobs now supports DAG-based task dependencies, conditional branching, for-each loops, event-driven triggers, and native Unity Catalog governance — capabilities that satisfy most sophisticated dependency management requirements natively. Integration with Azure Data Factory and Apache Airflow is an option for specific cross-system scenarios, not a general capability extension. Knowing when and how to use each orchestration approach lets you pick the right solution for your needs while staying flexible for future growth.

Modularizing code is crucial for building maintainable, testable, reusable data processing components. Apply software engineering principles to notebook development, and you get the reliability and maintainability needed for production while keeping the flexibility and rapid development that make notebooks valuable for data science work. Comprehensive monitoring and debugging practices ensure production workflows can be maintained and optimized over time, delivering the operational excellence that business-critical systems require.

The skills from this chapter help data engineering teams successfully move from experimental notebooks to production-ready workflows. Master these orchestration capabilities, and you unlock the full potential of Azure Databricks while maintaining the agility and innovation that drives competitive advantage. The combination of technical expertise and operational best practices provides the foundation for scalable, reliable, cost-effective data processing systems that evolve with changing business needs and technological advances.

While this chapter focused on orchestrating and monitoring production workflows, the next chapter tackles another critical piece of the production puzzle: automating deployment itself. Chapter 9 explores CI/CD and DevOps practices for Azure Databricks, where you'll learn to set up Git integration, build automated deployment pipelines with Azure DevOps, manage infrastructure as code with Terraform, and implement automated testing. These practices transform how you ship changes to production - moving from manual, error - prone deployments to automated, repeatable processes that let you deploy confidently and frequently while maintaining system stability.

# Summary

This chapter provided a comprehensive guide to orchestrating data workflows in a production environment using Azure Databricks. We began by establishing a foundation with Lakeflow Jobs, the native enterprise orchestrator for managing complex data pipelines and task dependencies. We then expanded our toolkit by integrating with powerful external orchestrators, including Azure Data Factory and Apache Airflow, to learn how to manage cross-system integrations. A key theme was the importance of elevating code development through

modularization and DABs, turning exploratory code into robust, reusable, and testable assets. Finally, we covered the critical practices of comprehensive monitoring, logging, and debugging using Databricks-native tools to ensure the reliability and performance of our production workflows.

The concepts covered in this chapter are the bedrock of operationalizing data engineering projects. By mastering workflow orchestration, you can ensure that your data pipelines run reliably, efficiently, and at scale. These skills form the prerequisite for the next stage in production maturity: automation. In the next chapter, we will build directly on these concepts, exploring how to create a full CI/CD pipeline to automate the testing and deployment of the orchestrated workflows we learned to build here.

## Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to `packtpub.com/unlock`). Search for this book by name, confirm the edition, and then follow the steps on the page.

*Note: Have your invoice handy. Purchases made directly from the Packt website don't require*

# 9

# CI/CD and DevOps for Azure Databricks

In the previous chapters, we focused on techniques and approaches for building data pipelines. You ingested data from source systems, built transformations, materialized the results as Delta Lake tables, and applied initial performance optimizations - the pipeline works.

The next question is how to run this pipeline in production. How do you safely deploy it and evolve it over time? What happens when you need to fix bugs, improve performance, or introduce new features? And how do you manage the development and maintenance of multiple data-engineering projects, with multiple developers working in parallel?

These questions move us beyond pipeline implementation and into production readiness, collaborative development, and the operational lifecycle of data pipelines.

That's where DevOps practices for data engineering come in - such as version control, CI/CD, environment separation, testing, release management, and operational monitoring.

In this chapter, you will build an end-to-end CI/CD workflow for Azure Databricks, supported by Git-based collaboration, Terraform for infrastructure provisioning, and Declarative Automation Bundles for deploying pipelines and jobs across environments.

## Technical Requirements

This chapter offers three practical walk-throughs: Terraform for infrastructure provisioning, Automation Bundles for data assets deployment and CI/CD pipelines. Each topic stands on its own. You can work through them in any order, skip one, or do all three. Each walk-through will have its own set of technical requirements.

All sections use the project files from the Chapter 9 folder. To access the repository, follow the steps in the *"Download the example code files"* section in the *Preface*.

# DevOps Practices in Databricks

The following practices form the foundation of a mature Databricks DevOps workflow:

1. **Version Control**: Every artifact - SQL scripts, Python notebooks, pipeline definitions, configuration files - lives in Git.
2. **Environment Isolation**: Maintaining distinct development, testing, and production environments ensures that developers can experiment without impacting production data or users.
3. **Infrastructure as Code**: Ensures that Infrastructure is version-controlled alongside application code, environments can be reproduced reliably, and changes are auditable and reviewable
4. **Declarative Automation Bundles**: Automation Bundles provide a native way to package and deploy Databricks resources and integrate seamlessly with CI/CD pipelines.
5. **Continuous Integration and Continuous Deployment (CI/CD)**: CI/CD is the automation of the process, which starts from code commit to production deployment.
6. **Automated Testing**: Testing validates that code works as intended before it reaches production.
7. **Monitoring and Observability**: Production pipelines must be observable so teams can detect and diagnose issues quickly. Common approaches: job monitoring, data quality monitoring, performance monitoring, and cost monitoring.

This chapter covers all the core DevOps practices for Data Engineering, supplemented with hands-on walkthroughs for Terraform, Automation Bundles and CI/CD. We start with the foundation on which everything else depends: version control and Git integration.

# Setting Up Git Integration for Databricks

Databricks connects to Git via a feature called Git Folders (formerly Repos), which lets you clone repositories directly into the workspace and work with version-controlled notebooks and files natively.

Follow these steps to set up git integration:

1. Connect Your Git Provider. In the Databricks workspace, navigate to **User Settings → Linked Accounts → Git Integration**. Select your Git provider and enter a **Personal Access Token (PAT)** with repo read/write permissions. This links your Databricks account to your Git identity.
   Generate a PAT from your Git provider's settings page:
   - Github: https://github.com/settings/personal-access-tokens
   - **Azure DevOps**: `https://dev.azure.com/<your_organization_name>/_usersSettings/tokens`

   **Note**

   For this walkthrough, we recommend connecting via GitHub using a Personal Access Token. It requires fewer prerequisites than other options. Note that for production use, Databricks recommends the Databricks GitHub App over PATs. Azure DevOps users can alternatively authenticate automatically via Microsoft Entra ID if your Databricks workspace and DevOps org share the same tenant.

2. **Create a Git Folder**: In the left sidebar, go to **Workspace** > **Repos** > **User** > **Create** > **Git folder**. Paste your repository URL, confirm the provider, and click **Create**. Databricks will clone the repository into your workspace.
3. **Work Inside the Git Folder**: Open the cloned folder. All notebooks and files here are Git-tracked. You can create or edit notebooks as normal.
4. **Commit and Push Changes**: At the top of any notebook inside a Git Folder, you'll see a **Git branch indicator**. Click it to open the Git panel, where you can view changed files, write a commit message and commit and push to your remote branch

Version control, powered by Git, provides several critical capabilities that make collaborative development and reliable deployments possible:

- **Complete history**: Every change is tracked with information about who made it, when, and why. This audit trail is invaluable for understanding how your codebase evolved and diagnosed when issues were introduced.
- **Code review and collaboration**: Changes can be reviewed and discussed before being merged, catching issues early and spreading knowledge across the team.

- **Rollback capability**: When issues arise, you can quickly revert to a previous well-good state rather than scrambling to "fix forward" under pressure.
- **Foundation for automation**: Version control enables CI/CD pipelines that automatically test and deploy code, transforming manual deployment processes into reliable, repeatable workflows.

While these benefits apply universally, how teams organize their version-control workflows can vary significantly. Let's examine the most common approaches.

## Branching Strategy

Two predominant branching strategies have emerged in software development, each with different philosophies about how code should flow from development to production:

- **GitFlow** is a branching model that uses multiple long-lived branches to represent different stages of the development lifecycle - typically main for production code, develop for integration, plus feature branches, release branches, and hotfix branches. GitFlow provides clear structure and explicit control over what code goes where, but this structure comes with complexity as teams must manage multiple long-lived branches and coordinate merges between them.
- **Trunk-Based Development** takes a simpler approach with a single long-lived branch (main) representing the current state of the codebase. Developers create short-lived feature branches, merge them to main after review and testing, and deploy directly from main. This approach emphasizes simplicity, rapid integration, and keeping the main branch always deployable.

## Our Approach: Trunk-based Development with Environment Separation

For this book, we'll use a trunk-based approach combined with environment-based deployment control. Rather than using branches to represent environments, we use a single main branch for code and rely on separate physical environments - distinct Databricks workspaces to provide isolation and control. Further in the chapter, we will review more isolation strategies. The proposed workflow, which we will build for this chapter project, is shown in *Figure 9.1*.

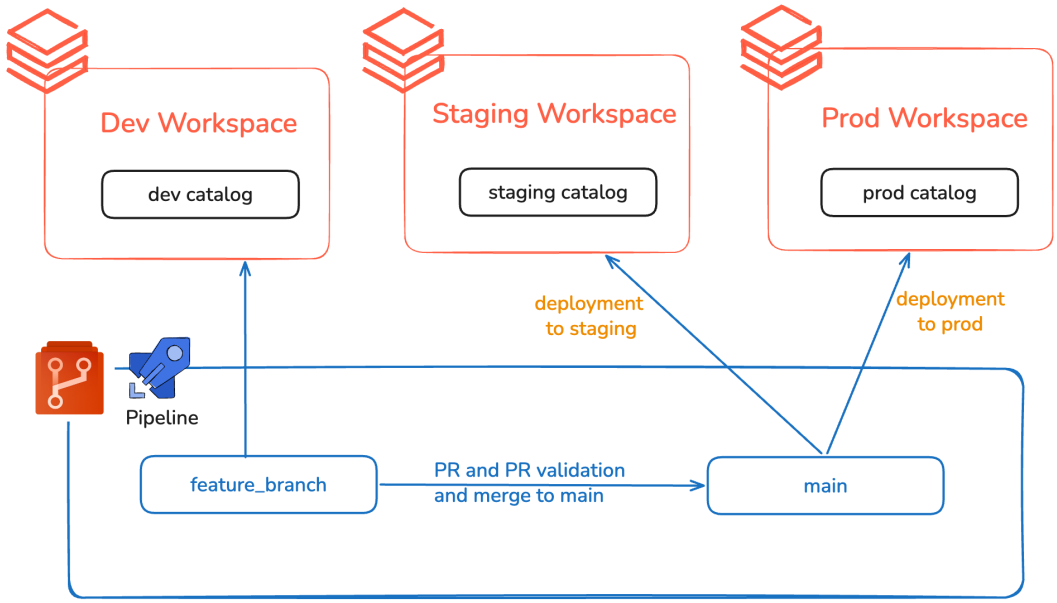



*Figure 9.1. CI/CD Workflow for Databricks Environments*

Based on the diagram in *Figure 9.1*, let us break down the workflow from development to production.

1. **Feature development**: Engineers create feature branches from main to develop new capabilities or fix issues
2. **Pull request opened**: When work is complete, engineers open a pull request, which triggers unit tests to verify individual functions and transformations. Before opening a PR, developers can test locally by deploying to the dev workspace, where each developer's resources are prefixed with their username to avoid conflicts. Unit tests must pass before the PR can proceed. If tests fail, developers fix issues and push updates to the feature branch, re-triggering unit tests.
3. **Optional**: If the team needs additional validation before staging, resources can be deployed to a pre-staging environment and integration tests performed.
4. **Code review and approval**: Once all tests pass, the PR is sent for code review and approval
5. **Merge to main**: After review approval, the feature branch is merged to main.

6. **Automatic deployment to staging**: The merge to main triggers automatic deployment to a staging environment that triggers integration tests to run to validate that components work together correctly.
7. **Promotion to production**: After validation in staging, the code is promoted to production

The key principle is that the same code artifact flows through all environments, with increasing levels of validation at each stage. Code must be validated in dev before it can be merged to main, and validated in test before it can reach production.

This approach works well when you want to release to test frequently, ideally after each pull request represents a complete, testable feature. However, if your workflow requires batching multiple features together before promoting to test, you might want to consider a variation of GitFlow with two or more long-lived branches that let you control exactly when accumulated changes are moved to test and production environments.

Depending on where you are in your CI/CD journey, you might not agree with every aspect of this approach - and that's perfectly fine. Organizations at different maturity levels may need to adjust testing frequency, deployment automation, or environment promotion processes based on their current capabilities and constraints. However, this workflow provides a solid foundation to start from and can be adapted as your practices evolve.

When choosing a branching strategy for your organization, consider factors such as team size, deployment frequency, and how you'll handle rollbacks and hotfixes. These operational concerns can significantly influence which approach works best for your context, though detailed discussion of these scenarios is beyond the scope of this chapter.

With version control establishing a foundation for collaboration and reliable deployments, the next critical practice is ensuring that development work doesn't disrupt production systems.

## Environment Separation

Maintaining distinct development, testing, and production environments is essential for safe, reliable deployments. Environment separation ensures that developers can experiment and test changes without impacting production data or users, while providing controlled validation gates before code reaches production.

In the context of Databricks, there are three primary approaches to isolating environments:

- **Workspace Isolation**: Create separate Databricks workspaces for each environment - one for dev, one for staging/test, and one for production. Each workspace operates independently with its own assets and resources.

- **Catalog Isolation**: Use separate Unity Catalog catalogs within the same workspace for each environment (e.g., `analytics_dev`, `analytics_staging`, `analytics_prod`). Each catalog serves as a logical boundary for data and metadata. It provides clear data separation while simplifying workspace management, enables fine-grained access control through Unity Catalog permissions, and makes it easy to promote code across environments (same workspace, different catalog targets)
- **Storage Isolation**: Configure separate Azure Data Lake Storage storage accounts for each environment's data. Each environment catalogs points to its own dedicated storage location. This ensures complete physical separation of data at the cloud storage layer

### Our Approach

While each isolation method can be used independently, for this book, we will all isolations methods together:

- **Three workspaces**: `dbw-analytics-dev`, `dbw-analytics-staging`, `dbw-analytics-prod`
- **Environment-specific catalogs**: `analytics_dev`, `analytics_staging`, `analytics_prod` (one in each workspace)
- **Dedicated storage**: Separate storage containers (or accounts) backing each environment's catalogs

This setup, which builds on the Unity Catalog foundation established in Chapter 2, provides robust isolation and is manageable for most organizations.

Now we talked about environment separation; however, what happens when you need to deploy your pipelines, jobs, and other assets to a test environment? Or hand it off to another developer? Or promote it to production? You would need to recreate the pipeline definition by hand. You would set up the schemas, configure the volume, wire up the alerts, and deploy the dashboard, all by clicking through the Databricks UI. And you would repeat this every time something changes. Declarative Automation Bundles solve this problem.

To fully appreciate what Declarative Automation Bundles offer, we will first take a step back and look at Terraform. DABs adopt the same declarative, code-first philosophy, and seeing that foundation will make the DABs concepts click faster when we return to them

## Infrastructure as Code with Terraform

In this section, you will provision Unity Catalog infrastructure for multiple Databricks environments using Terraform.

A data platform needs more than pipelines and dashboards. Before any application code runs, the platform infrastructure must be in place: Unity Catalog metastores, storage accounts, access connectors, and catalogs. Terraform handles this layer.

You describe the target state of your infrastructure in configuration files. Terraform compares the state to the existing state and creates, updates, or removes resources to match. Teams get a repeatable way to provision and change environments without clicking through the Azure portal.

This section walks through a Terraform project that provisions a complete Unity Catalog environment on Azure Databricks. The project is parametrized by environment. You run the same code for dev, staging, and prod, and each environment gets its own isolated set of resources. Once the infrastructure is in place, the next section covers Declarative Automation Bundles, which deploy the application layer (pipelines, jobs, dashboards, and alerts) on top of it.

## Prerequisites

The Terraform provisions Unity Catalog infrastructure (metastore, storage accounts, access connectors, and catalogs) using code rather than manual setup. This requires Premium tier workspaces. Azure offers a 14-day Premium trial for new workspaces.

To complete this walkthrough, you will need:

- Basic understanding of Terraform.
- Three Azure Databricks workspaces on the Premium tier (one per environment: dev, staging, prod).
- Terraform and the Azure CLI are installed on your machine.
- An Azure identity with the Databricks account admin role (see chapter 2 for the explanation of how to obtain the Databricks account admin role).
- No existing Unity Catalog metastore in your target region. If one exists, delete it from the Databricks account console (open the account console directly via `https://accounts.azuredatabricks.net`).
- The Databricks account ID (open account console directly via `https://accounts.azuredatabricks.net`, click on the user icon in the top right corner, copy account ID).
- The workspace resource ID for each environment (in the Azure portal, go to your Databricks workspace page, copy it from JSON View under the workspace Overview). The workspace resource ID looks like this: `/subscriptions/xxxx/resourceGroups/my-rg/providers/Microsoft.Databricks/workspaces/my-ws`

## What the Terraform Project Creates

The project provisions these resources per environment:

- An Azure Managed Identity access connector. The catalog uses this identity to read and write data in storage.
- An ADLS Gen2 storage account with hierarchical namespace turned on.
- Storage containers for catalog storage.
- A role assignment granting the managed identity Storage Blob Data Contributor access on the storage account.
- A storage credential and external location that point to the catalog container.
- Unity catalog for each environment.

### Project Structure

The project splits resources across files by concern. All files sit in the same directory and share variables, locals, and resource references directly.

```
main.tf              required providers
locals.tf            data sources and computed values
providers.tf         azurerm and databricks provider configs
variables.tf         input variables
storage.tf           storage account and catalog containers
connectors.tf        access connector, role assignment, storage credential
external_locations.tf external locations for catalog storage
metastore.tf         metastore creation, lookup, and workspace attachment
catalog.tf           environment catalogs
terraform.tfvars     account, region, workspace IDs, and catalog list
```

### Configuration

Create `terraform.tfvars` based on `terraform.tfvars.example` and fill in your values. The file holds the Databricks account ID, the region for all resources, a map of workspace resource IDs, and the list of catalogs to create per environment. Follow the prerequisites to get the values for these variables:

```
account_id = "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx"
region     = "eastus"
workspaces = {
  dev     = "/subscriptions/<sub-id>/resourceGroups/<rg>/providers/
Microsoft.Databricks/workspaces/<dev-workspace>"
  staging = "/subscriptions/<sub-id>/resourceGroups/<rg>/providers/
```

```
Microsoft.Databricks/workspaces/<staging-workspace>"
  prod    = "/subscriptions/<sub-id>/resourceGroups/<rg>/providers/
Microsoft.Databricks/workspaces/<prod-workspace>"
}
catalogs = {
  dev     = ["analytics_dev"]
  staging = ["analytics_staging"]
  prod    = ["analytics_prod"]
}
```

Note, the region value controls the location of every resource. Set it to the Azure region where your workspaces live. Adjust region value as needed.

## Deploying

Before running any Terraform commands, authenticate with your personal identity using the Azure CLI: `az login`. No additional credentials or configuration required.

Our project uses Terraform workspaces to keep state files separate per environment. Each workspace stores its own state, so dev, staging, and prod never interfere with each other.

Initialize the project once:

```
terraform init
```

Deploy dev first. On the very first run, set `create_metastore=true` so Terraform creates the shared metastore:

```
terraform workspace select -or-create dev
terraform plan -var="environment=dev" -var="create_metastore=true"
terraform apply -var="environment=dev" -var="create_metastore=true"
```

The apply takes a few minutes. When it finishes, your dev workspace has a running metastore, storage account, access connector, external location, and catalog.

**Note**

The metastore is a shared resource. Azure Databricks allows one metastore per region per account, so the project creates it once during the first environment deployment. The second and third environments reuse the same metastore. All workspaces attach to it, and catalogs provide the isolation between environments. The metastore itself has no

dedicated storage. Each catalog stores its data in its own container through its external location.

Deploy staging next. The metastore already exists, and `create_metastore` defaults to false, so you do not need to pass it:

```
terraform workspace select -or-create staging
terraform plan -var="environment=staging"
terraform apply -var="environment=staging"
```

And lastly deploying prod:

```
terraform workspace select -or-create prod
terraform plan -var="environment=prod"
terraform apply -var="environment=prod"
```

For all subsequent runs of any environment, the default works. The metastore only needs to be created once:

```
terraform workspace select <env>
terraform apply -var="environment=<env"
```

To tear down an environment without destroying the shared metastore:

```
terraform workspace select prod
terraform destroy -var="environment=prod"
terraform workspace select staging
terraform destroy -var="environment=staging"
```

To tear down the last environment and the metastore with it:

```
terraform workspace select dev
terraform destroy -var="environment=dev" -var="create_metastore=true"
```

With the infrastructure layer in place, the next section moves up the stack to the application layer, using Automation Bundles to deploy pipelines, jobs, dashboards, and alerts.

# Declarative Automation Bundles

In this section, you will define a Declarative Automation Bundle for a sample project and use it to deploy pipelines, jobs, dashboards, and alerts across environments.

Declarative Asset Bundle provides a structured way to package your source code, resource definitions, and configuration into a single deployable unit. Instead of creating Databricks resources through the UI by hand, you describe them in YAML files. The Databricks CLI then deploys everything for you.

Think of it as infrastructure-as-code for your Databricks workspace. Your pipeline definitions, schema configurations, dashboard layouts, and alert rules all live in version-controlled files. You can review, test, and deploy them the same way across environments.

## Prerequisites

- **One Databricks workspace**: A single dev workspace is sufficient. The free edition works.
- **A catalog in the dev workspace**: If you completed the Terraform walkthrough, this was created automatically. Otherwise, run the following in the SQL editor: `CREATE CATALOG analytics_dev`
- **Git Folders configured** in the dev workspace.
- **Chapter project files**: Copy the `Chapter09/automation_bundle_and_cicd_project` folder from the GitHub repository.

## Databricks CLI commands

The Databricks CLI provides commands to work with bundles:

- `databricks bundle validate` checks your configuration for errors before you deploy
- `databricks bundle deploy` creates or updates all resources in your target workspace
- `databricks bundle run` triggers execution of a specific resource (a pipeline, a job)
- `databricks bundle destroy` removes all resources managed by the bundle

## Project Structure

We will be working on the sales orders project Automation Bundle. You can find it in Chapter 9. 2 folder in the book Github repository. Let's walk through the project structure to understand

assets we will be working with. If you are new to Automation Bundles, focus first on `databricks.yml` and the `resources/` and `src/` directory, as these define what gets deployed.

```
sales_orders/
├── databricks.yml                # Main bundle configuration
├── resources/                    # Resource definitions (YAML)
│   ├── schemas.yml               # Unity Catalog schemas
│   ├── volumes.yml               # Storage volumes
│   ├── sales_orders_pipeline.yml # Pipeline definition
│   ├── sales_orders_job.yml      # Job definition
│   ├── sales_dashboard.yml       # AI/BI dashboard
│   └── alerts.yml                # Monitoring alerts
├── src/                          # Source code
│   ├── upload_raw_files.py
│   ├── dashboards/
│   │   └── sales_analytics_dashboard.lvdash.json
│   ├── pipelines/
│   │   ├── load_bronze_to_gold.py  # Pipeline logic
│   │   └── utils/                  # Helper modules
│   │       ├── schemas.py
│   │       ├── bronze.py
│   │       ├── silver.py
│   │       └── features_orders.py
│   └── monitoring/
│       └── monitoring.sql
├── test/                         # Tests
│   ├── unit_test/
│   │   ├── conftest.py           # Test fixtures
│   │   ├── test_features_orders.py
│   │   └── test_silver.py
│   └── integration_test/
│       ├── conftest.py           # Integration test fixtures
│       └── test_pipeline_output.py
├── .cicd/                        # CI/CD pipeline definitions
│   ├── azure-pipelines-dev.yml
│   └── azure-pipelines-staging-prod.yml
└── pytest.ini                    # Test configuration
```

The project has three main areas. `databricks.yml` sits at the root and holds bundle-wide configuration. `resources/` declares what Databricks resources to create. `src/` contains the actual code that runs inside those resources. Let's walk through each one.

## The Bundle Configuration: databricks.yml

The `databricks.yml` file is the entry point for your bundle. It defines the bundle name, includes resource files, declares variables, and sets up deployment targets.

The key components of `databricks.yml` that you will interact with in this walkthrough are:

- **Variables and substitution**: Variables use the `${var.variable_name}` syntax. They can reference other variables or built-in values like `${bundle.target}` (the target name: "dev" or "prod") and `${workspace.current_user.userName}` (the deploying user's email). This keeps the bundle independent of any single environment. The catalog name `analytics_${bundle.target}` resolves to `analytics_dev` when you deploy to dev, `analytics_staging` when you deploy to staging and `analytics_prod` when you deploy to production.
- **The `lookup` type**: The `warehouse_id` variable uses a special lookup type instead of a hardcoded default. This tells the CLI to find the warehouse by name in the target workspace and resolve it to the actual ID. Resource IDs differ between workspaces, so lookups are the right approach.
- **Targets**: The targets section defines where and how the bundle gets deployed. Each target can override variables and set a deployment mode. The `mode: development` setting is useful: it automatically prefixes resource names with your username (like `dev_<user_name>_sales_bronze`). This way, multiple developers can deploy to the same workspace without their resources colliding.

*Table 9.1* summarizes how each target resolves the catalog name.

| Target | Catalog | Schema prefix | Mode |
|---|---|---|---|
| `dev` | `analytics_dev` | `dev_<username>_` | development |
| `staging` | `analytics_staging` | none | none |
| `prod` | `analytics_prod` | none | production |

*Table 9.1: Automation Bundle targets and catalogs*

## Resource Definitions

The `resources/` directory contains YAML files that declare what Databricks resources the bundle should manage. Each file maps directly to a Databricks API resource type. Let's look at each one.

- **Schemas** (`schemas.yml`): This file creates four Unity Catalog schemas, one for each layer of our medallion architecture:

```
resources:
  schemas:
    bronze:
      name: ${var.bronze_schema}
      catalog_name: ${var.catalog}
      comment: "Bronze layer for raw ${var.department} data ingestion"
    silver:
      name: ${var.silver_schema}
      catalog_name: ${var.catalog}
      comment: "Silver layer for cleansed and validated ${var.department}
data"
    gold:
      name: ${var.gold_schema}
      catalog_name: ${var.catalog}
      comment: "Gold layer with aggregated metrics for AI/BI dashboards"
    ml_features:
      name: ${var.ml_schema}
      catalog_name: ${var.catalog}
      comment: "Feature store for ML models"
```

  Every value uses variables instead of hardcoded names. When deployed to dev, the bronze schema becomes `dev_<user_name>_sales_bronze`. When deployed to prod, it becomes `sales_bronze`. Same code, different environments.

- **Volumes (**`volumes.yml`**)**: The volume provides a storage location inside Unity Catalog where we place the raw CSV files for ingestion:

```
resources:
  volumes:
    raw_data:
      name: "raw"
      catalog_name: ${var.catalog}
```

```
      schema_name: ${var.bronze_schema}
      volume_type: "MANAGED"
      comment: "Raw CSV files for ingestion"
```

- **Pipeline** (`sales_orders_pipeline.yml`): This is where we connect our Python pipeline code to a Databricks pipeline resource:

```
resources:
  pipelines:
    sales_orders_pipeline:
      name: "${var.department}_orders_pipeline"
      catalog: ${var.catalog}
      target: "${var.department}_pipeline"
      development: true
      channel: "PREVIEW"
      serverless: true
      libraries:
        - notebook:
            path: ../src/pipelines/load_bronze_to_gold.py
      configuration:
        environment: "${bundle.target}"
        catalog: ${var.catalog}
        department: ${var.department}
        bronze_schema: "${var.schema_prefix}${var.bronze_schema}"
        silver_schema: "${var.schema_prefix}${var.silver_schema}"
        gold_schema: "${var.schema_prefix}${var.gold_schema}"
        ml_schema: "${var.schema_prefix}${var.ml_schema}"
        raw_volume_path: ${var.raw_volume_path}
```

The `libraries` section points to our pipeline notebook in `src/`. The configuration section passes variables as Spark configuration properties. The Python code reads them at runtime with `spark.conf.get()`. This is the bridge between your YAML configuration and your Python code. The pipeline code does not hardcode any catalog names, schema names, or paths. It reads them from the configuration that the bundle provides.

- **Dashboard** (`sales_dashboard.yml`). The dashboard resource points to a JSON layout file and sets the data context:

```
resources:
   dashboards:
     sales_analytics_dashboard:
       display_name: "Sales Analytics Dashboard"
       warehouse_id: ${var.warehouse_id}
       file_path: ../src/dashboards/sales_analytics_dashboard.lvdash.json
       dataset_catalog: ${var.catalog}
       dataset_schema: "${var.schema_prefix}${var.gold_schema}"
```

  The `dataset_catalog` and `dataset_schema` properties specify which catalog and schema to query from. The dashboard JSON itself uses simple, unqualified table names like `SELECT * FROM sales_daily_metrics`. This keeps the dashboard layout independent of any specific environment.

- **Alerts** (`alerts.yml`): Alerts define monitoring rules that fire when data quality drops:

```
resources:
  alerts:
    daily_revenue_drop:
      display_name: "Daily Revenue Drop Alert"
      warehouse_id: ${var.warehouse_id}
      query_text: |
        WITH daily AS (
          SELECT date, total_revenue,
            LAG(total_revenue) OVER (ORDER BY date) as prev_revenue
          FROM ${var.catalog}.${var.gold_schema}.sales_daily_metrics
        )
        SELECT date, total_revenue, prev_revenue,
          ROUND((total_revenue - prev_revenue) / prev_revenue * 100, 2)
            as revenue_change_pct
        FROM daily
        WHERE date = (SELECT MAX(date)
          FROM ${var.catalog}.${var.gold_schema}.sales_daily_metrics)
      evaluation:
        source:
          name: "revenue_change_pct"
        comparison_operator: "LESS_THAN"
        threshold:
```

```
          value:
            double_value: ${var.revenue_drop_threshold}
```

The alert threshold itself is a variable (`${var.revenue_drop_threshold}`). You can set different thresholds for dev, staging and prod by overriding the variable per target.

## Source Code

The `src/` directory contains the actual code that runs inside the Databricks resources. The key point is how the pipeline code connects to the bundle configuration.

Look at `src/pipelines/load_bronze_to_gold.py`. It reads all of its configuration from Spark properties with `spark.conf.get()`: catalog names, schema names, volume paths. It does not hardcode any of these values. They come from the configuration block in the pipeline resource YAML, which in turn references the bundle variables.

This separation between configuration and code is the foundation of the design. The Python code does not know whether it is running in dev, staging, or prod. It reads schema names from the bundle configuration and uses them. When you deploy to dev, `bronze_schema` resolves to `dev_<user_name>_sales_bronze`. When you deploy to staging or prod, it resolves to `sales_bronze`. The code stays the same in both cases.

The helper functions in `src/pipelines/utils/` handle specific transformation logic. `src/pipelines/utils/bronze.py` and `src/pipelines/utils/silver.py` add audit columns in the bronze and silver layers. `src/pipelines/utils/schemas.py` defines table schemas. `src/pipelines/utils/features_orders.py` computes ML features. These are pure functions that take a DataFrame and return a DataFrame. This makes them easy to test on their own.

## Tests

The `test/` directory contains unit tests that validate the transformation logic. Tests use Databricks Connect with serverless compute. You run PySpark code locally while the Spark engine runs in the cloud.

The test setup lives in `test/unit_test/conftest.py`. It creates a DatabricksSession using serverless compute and provides reusable test fixtures: small DataFrames with known values that the tests use as inputs.

The actual tests live in files like `test/unit_test/test_features_orders.py` and `test/unit_test/test_silver.py`. They import the utility functions from `src/pipelines/utils/`, apply them to the test fixtures, and assert that the results match expectations. For example, the feature order tests verify that window functions such as "last order amount" and "days since

previous order" compute the correct values per customer. If someone changes the logic later, these tests will catch it.

The test configuration in `pytest.ini` points `pytest` to the test directory and sets useful defaults. You run the tests locally with `pytest`, and they execute against serverless compute in your dev workspace. The same tests are run in CI before the code is merged. We will cover that workflow in the CI/CD section.

The project also includes integration tests in `test/integration_test/`. These validate the deployed pipeline's output. They run on Databricks serverless compute after the staging pipeline completes, using a test runner notebook (`src/run_integration_tests.py`). The integration tests check that expected tables exist in the target catalog, contain data, and pass basic quality checks. For example, they confirm that silver tables are non-empty, gold metrics have no null dates, and the feature store covers all customers from the orders table. This catches issues that unit tests cannot: broken pipeline wiring, schema mismatches, or missing data in a deployed environment. If teams want to catch integration issues before merging, integration testing can also be done as part of pre-staging validation.

## Deploying

With everything defined in code, deployment becomes a single command:

```
databricks bundle deploy -t dev
```

This command does several things:

- Validates your YAML configuration
- Resolves all variable substitutions for the target environment
- Uploads source files to the workspace
- Creates or updates all declared resources (schemas, volumes, pipeline, dashboard, alerts)
- Tracks the deployment state so subsequent deploys only update what changed

Optionally if staging and prod targets configured, when you are ready to deploy to staging and prod targets, you run the same command with a different target:

```
databricks bundle deploy -t staging
databricks bundle deploy -t prod
```

The same code, the same resource definitions, the same pipeline logic. All deployed to a different environment with different catalog names, schema names, and resource configurations. No manual steps. No clicking through the UI. No forgetting to update a schema name somewhere.

To sum it up, Automation Bundles bring several important capabilities to your data engineering workflow: *environment agnosticism*, *declarative resource management*. You describe what resources should exist. The bundle CLI figures out how to create, update, or remove them. No more manual UI configuration.

By the end of this section we defined our project as a Declarative Automation Bundle. We showed how `databricks bundle deploy` moves code between environments. But running that command by hand after every merge does not scale; that is where CICD pipelines come into play.

## Implementing CI/CD Pipelines with Azure DevOps

In this section, we will connect our Git repository to Azure DevOps. We will configure the service to run our YAML pipeline definitions and walk through the full flow from PR to production.

We will build an automated pipeline that performs validation and runs tests on every pull request, when code merges to main, deploys to staging with a manual approval gate, and promotes production with manual approval as well.

### Prerequisites

- **Three Databricks workspaces**: Dev, staging, and prod. The free edition works for all.
- **Catalogs in each workspace**: If you completed the Terraform walkthrough, these were created automatically. Otherwise, run the following in the SQL editor of each workspace:
    a. **Dev**: `CREATE CATALOG analytics_dev`
    b. **Staging**: `CREATE CATALOG analytics_staging`
    c. **Prod**: `CREATE CATALOG analytics_prod`
- **Azure DevOps project and repository**: Create one at `https://dev.azure.com` if you do not have one.
- **Free parallelism for Azure DevOps pipelines** (more details below).
- **Git Folders configured** in dev workspace.

### Requesting Free Parallel Jobs

The free quota for running CI/CD pipelines is not assigned automatically to an Azure DevOps Project. Microsoft requires you to request free parallelism before you can run CI/CD pipelines in Azure DevOps. Without it, your pipelines will sit in a queue and never run.

Go to `https://aka.ms/azpipelines-parallelism-request` and fill out the form. Select your Azure DevOps organization and request a free grant of parallel jobs. If you want to use the free quota, submit the request early; you cannot run pipelines until this is approved.

Now, let's move on to the creation of automated DevOps pipelines.

## CI/CD pipeline creation guide

The following 7 steps will guide you through CICD pipelines creation.

### Step 1: Copy project files from the book repo

You need to copy the project files to Azure DevOps Repos. The quickest way is to use a code editor on your local machine, copy the project files from the book repo, and push them to the main branch as starter code. We will lock **main** branch during CICD pipelines are set up; therefore, further enhancements should go through **feature** branches.

### Step 2: Generate Databricks Personal Access Tokens

Each environment (dev, staging and prod) needs its own access token. In your Databricks workspace, go to **Settings** > **Developer** > **Access tokens** (link `https://<workspace-link>/settings/user/developer/access-tokens`) and generate a token. Give it a descriptive name like `azure-devops-dev` and save the token value. You will not be able to see it again.

If your dev, staging, and prod environments are in separate Databricks workspaces, generate a token for each. Otherwise, you can use the same token.

### Step 3: Create Variable Groups in Azure DevOps

Variable groups store the Databricks credentials that pipelines read at runtime. You need three groups, one per environment.

In Azure DevOps, go to **Pipelines** > **Library** and **click + Variable** group as in *Figure 9.2*.

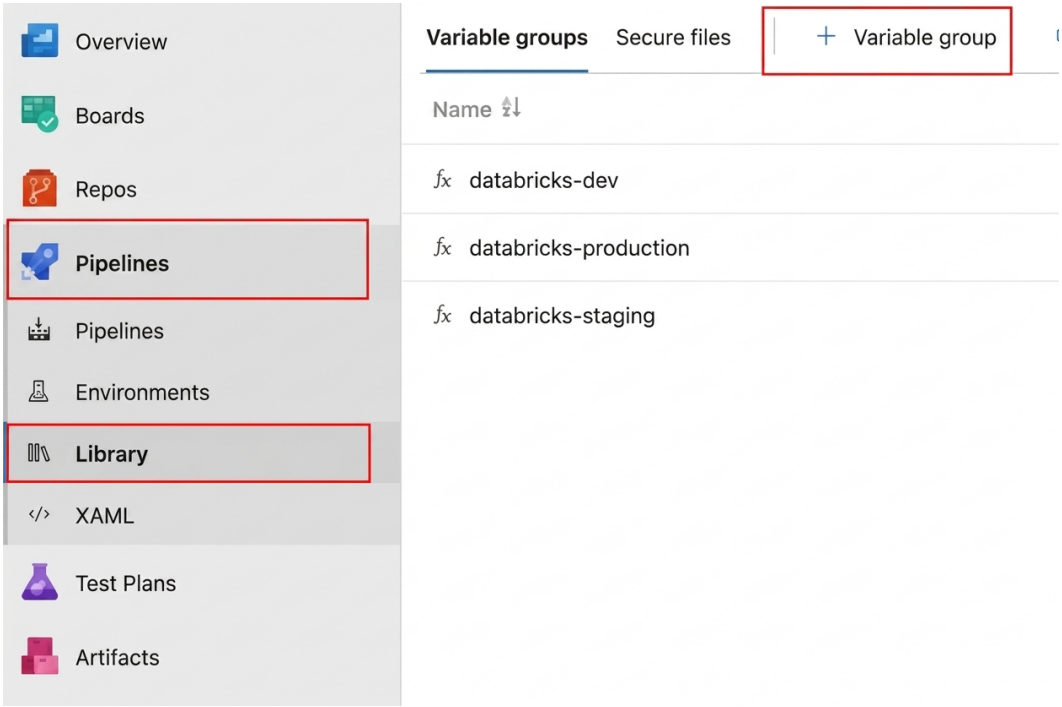



*Figure 9.2: Variable group in Azure DevOps*

As shown in *Figure 9.2*, you need to create 2 variable groups: `databricks-dev` and `databricks-staging`. In each variable group create 2 variables: `DATABRICKS_HOST` (example `https://<your-workspace>.azuredatabricks.net`` or ) and `DATABRICKS_TOKEN`.

Mark the `DATABRICKS_TOKEN` variables as secret by clicking the lock icon. This masks the token value in pipeline logs and prevents anyone from reading it back through the UI.

## Step 4: Create the Dev Pipeline

Go to **Pipelines** > **Pipelines** and click **New** pipeline in the top-right corner. Select Azure Repos Git as your repository source and point it to your repository. When prompted to configure, choose **Existing Azure Pipelines YAML** file and select `/.cicd/azure-pipelines-dev.yml` from the **main** branch.

Click **Save and run** to create the pipeline. You can rename it to something like **"Dev - PR Validation" in** the pipeline settings.

Here is what the dev pipeline does:

- The pipeline triggers when a pull request is opened or updated against main. It does not trigger on direct pushes. That is the job of the staging and production pipeline.
- The following steps are executed: install Python, install test dependencies, install the Databricks CLI, validate the bundle configuration, and run unit tests. No deployment happens at this stage.
- The unit tests use Databricks Connect to run Spark operations on serverless compute.
- If the tests fail, the pipeline fails, and the PR cannot be merged (assuming you have set up branch policies, which we configure shortly).
- In the dev workspace, each developer deploys their own set of resources, prefixed with their username. This prevents conflicts when multiple developers work on the same project simultaneously. If the team needs additional pre-staging validation, resources can be deployed to a dedicated pre-staging environment before merging to main, and integration tests performed to validate the changes.

## Step 5: Create the Staging and Production Pipeline

Create a second pipeline the same way. Click **New pipeline**, select your repo, choose **Existing Azure Pipelines YAML file**, and this time select `/.cicd/azure-pipelines-staging-prod.yml`. Click **Save and run**, then rename the pipeline to **Staging & Production**.

This pipeline triggers on pushes to main and has 3 stages.

- **Stage 1: Staging Approval**: Before anything is deployed to staging, the pipeline pauses and waits for manual approval through the staging Azure DevOps environment. This gives the team a chance to review what is about to be deployed.
- **Stage 2: Staging Deploy & Test**: After approval, this stage installs dependencies, runs unit tests as a safety net, deploys the bundle to staging, runs the data pipeline, and then runs integration tests. If any step fails (unit tests, deployment, pipeline run, or integration tests), a rollback job kicks in automatically. It checks out the previous commit and redeploys that version to staging. The environment is not left broken while you work on a fix. For reference on rollback see project file `Chapter09/automation_bundle_and_cicd_project/.cicd/azure-pipelines-staging-prod.yml`, where it specifies **job: Rollback**
- **Stage 3: Production**: This stage depends on staging succeeding. It requires a second manual approval through the production environment. It deploys the bundle to production. Both approval gates are configured through Azure DevOps environments, which we set up next.

## Step 6: Create Environments with Approval Gates

The staging and prod pipeline uses Azure DevOps environments to gate deployments. Both the staging and production stages require manual approval before proceeding. You need to create two environments.

Go to **Pipelines** > **Environments** and click **New environment**. Name it staging and set the resource type to **None**. Once the environment is created, click into it and go to **Approvals and checks** tab. Click the + button and add an **Approvals** check. Add yourself as an approver for now.

Repeat the same process to create a second environment named **production** with its own approval check.

The staging environment should look like in *Figure 9.3*.

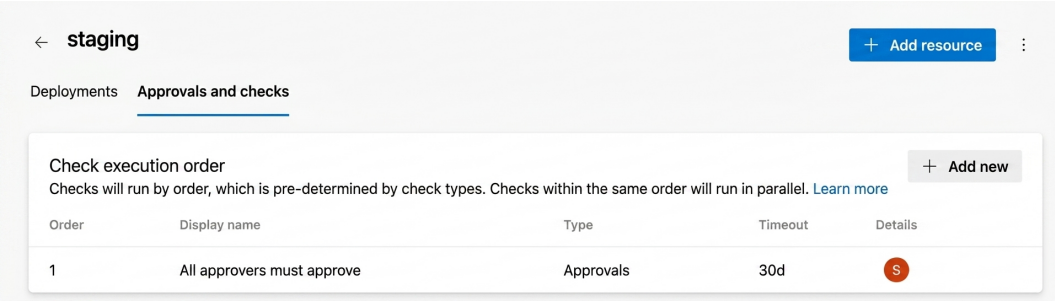



*Figure 9.3: Staging environment approval gate setting*

Now, when a merge to **main** triggers the **Staging & Production** pipeline, it will first pause at the staging approval gate as per setup in *Figure 9.3*. Once approved, it is deployed and tested on staging. If staging succeeds, the pipeline pauses again at the production approval gate and waits for approval before being deployed to production.

## Step 7: Set Up Branch Policies

You need to configure a branch policy on main so the dev pipeline must pass before code can be merged.

Go to **Repos** > **Branches**, find the main branch, and click the three dots menu > **Branch policies** as shown in *Figure 9.4*.

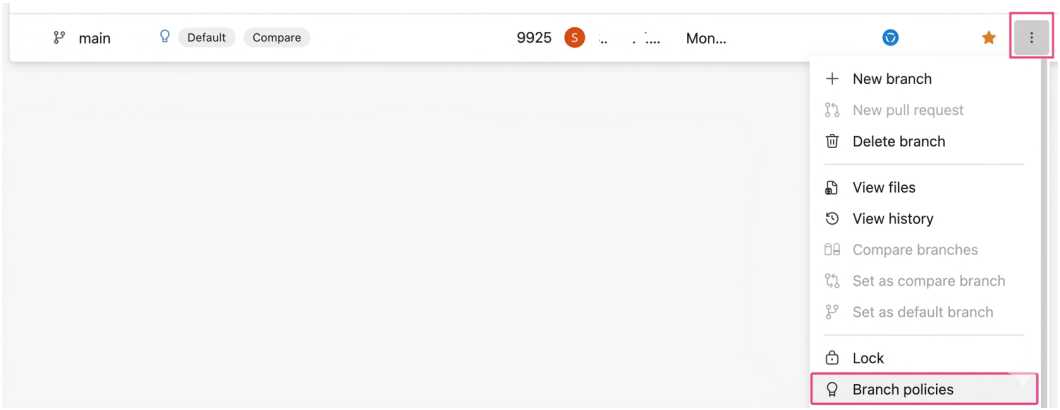



*Figure 9.4: How to find branch policies*

Under **Build validation**, add a build policy:

- **Build pipeline**: Select your dev pipeline (Dev - PR Validation)
- **Trigger**: Automatic
- **Policy requirement**: Required
- **Build expiration**: Immediately when `main` is updated

  This means every PR must pass the dev pipeline (bundle validation, unit tests) before the merge button becomes available.
- Once set, build validation will be enabled as shown in *Figure 9.5*.

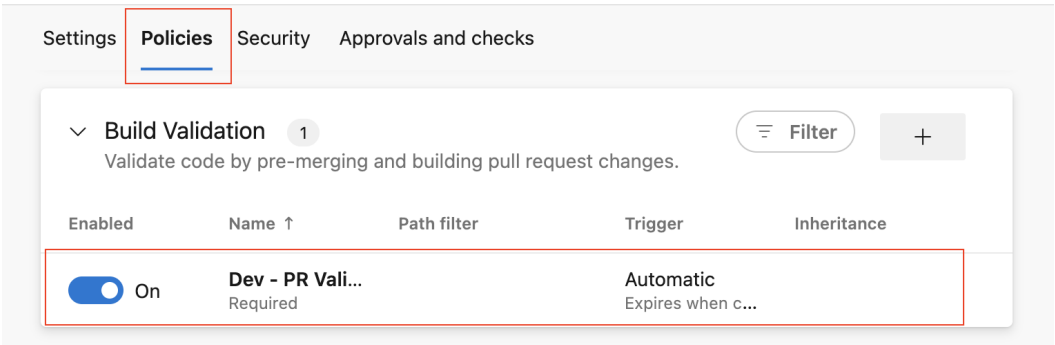



*Figure 9.5: Build validation enabled*

You can also turn on other policies here, like requiring at least one reviewer or requiring linked work items.

With the pipelines configured, Azure DevOps enforces this flow automatically through PR validation, staged deployment, and approval gates.

1. A developer creates a feature branch and opens a **pull request** against `main`.
2. Azure DevOps triggers the **Dev pipeline**, which validates the bundle and runs unit tests. No deployment happens at this stage. A successful run will have a green check on the dev pipeline, as shown in *Figure 9.6*.

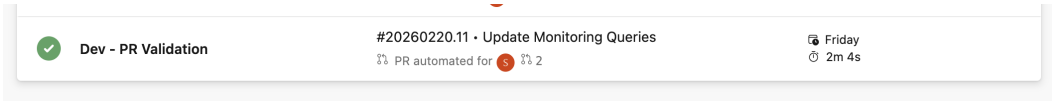



*Figure 9.6: Successful run of dev pipeline*

3. The PR shows a green check if the dev pipelines passes as shown in *Figure 9.7* (or a red X if it fails). Merging is blocked until the dev pipelines succeed.

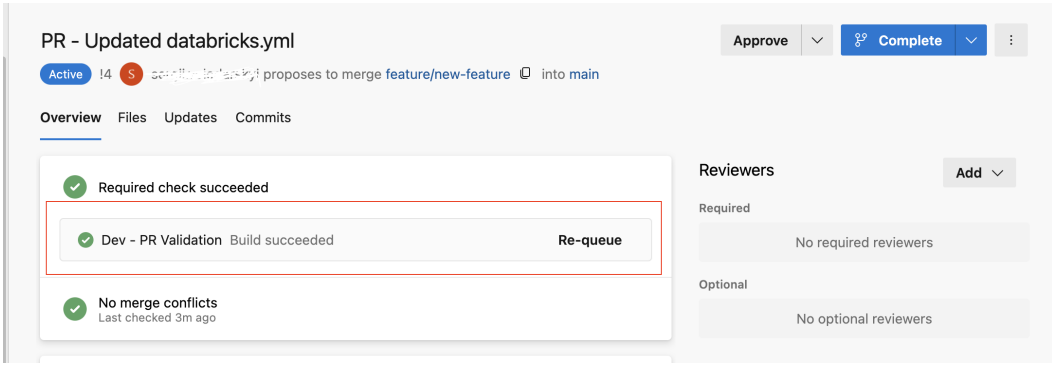



*Figure 9.7: Successful run of dev pipeline – PR perspective*

4. A reviewer approves the PR, and the developer merges the PR to main.
5. The merge triggers the staging and production pipeline, which first requests staging approval.
6. An approver approves the staging deployment
7. The staging stage runs:
    a. Unit tests again as a safety net
    b. Deploys the bundle to the staging workspace
    c. Runs the data pipeline end-to-end on staging
    d. Runs integration tests on Databricks serverless to verify pipeline output

e. If anything fails, it automatically rolls back staging to the previous commit

f. If successful, executed staging and production pipeline to staging will have all green ticks as shown in *Figure 9.8*.

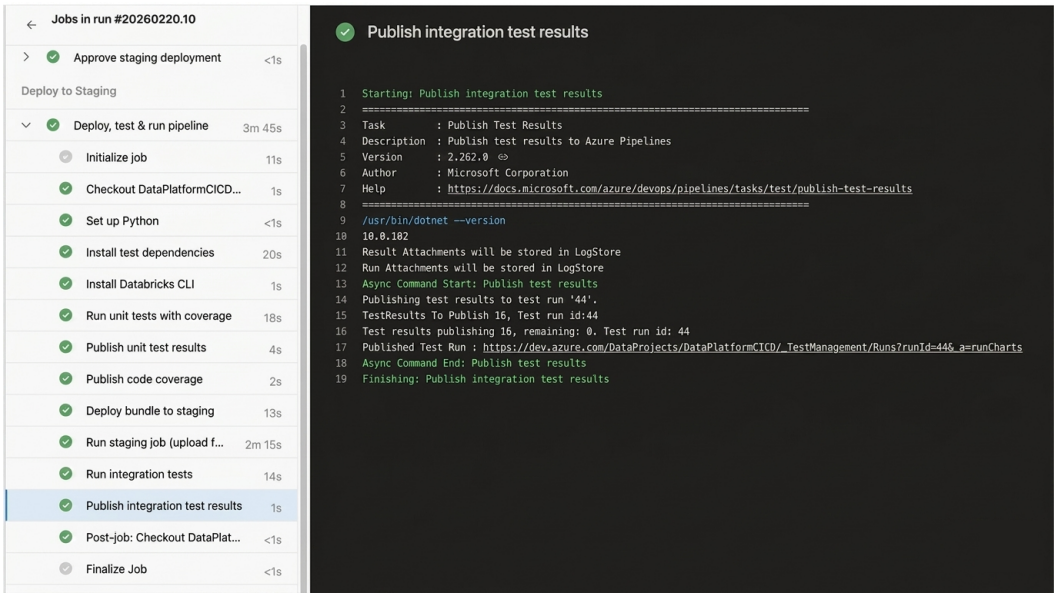



*Figure 9.8: Successful deployment automation bundle to Staging environment*

8. If staging succeeds, the production stage requests production approval
9. An approver reviews the staging results and approves
10. The bundle is deployed to production

The CICD flow described above means no code reaches production without passing automated tests, running successfully in a staging environment, and receiving human approval. The two YAML files in your repository drive the entire process. If you need to change the pipeline behavior, you change the YAML, and it goes through the same review process as any other code change.

Once your pipelines and jobs run in production, you need visibility into their behavior.

## Observability

After deploying assets to production, the natural questions become: Are jobs succeeding? Is data quality holding? Are costs under control? Databricks provides system tables that

automatically capture operational data. Combined with the bundle-managed alerts we defined earlier, these tables give us three monitoring layers.

- **Job Monitoring**: The file `src/monitoring/monitoring.sql` in the automation bundle queries Databricks system tables to track job execution. The first query joins `system.lakeflow.job_run_timeline` with `system.lakeflow.jobs` to show each job run from the past seven days with its name, duration, and result state. A second query aggregates run outcomes by day over 30 days and computes a daily success rate. Together, these two queries answer: what failed recently, and is the overall trend healthy?
- **Data Quality Monitoring**: The alerts defined in `resources/alerts.yml` in the automation bundle run on a daily schedule and validate gold-layer metrics, firing notifications when revenue drops or order counts fall below configured thresholds.
- **Cost Monitoring**: The third query in the file `src/monitoring/monitoring.sql` joins `system.billing.usage` with `system.billing.list_prices` to compute daily DBU consumption and estimated cost in USD, broken down by compute SKU. This provides a rolling seven-day view of spending across different compute types.

Because both the monitoring file and the alerts are declared as bundle resources, they deploy alongside everything else through Databricks bundle deploy. There is no separate setup and no manual configuration in the Databricks UI. The same monitoring runs consistently in every environment.

# Summary

In this chapter, you built a production-ready DevOps workflow for a Databricks data project. You provisioned Unity Catalog infrastructure across environments using Terraform, keeping infrastructure changes version-controlled, repeatable, and auditable. You organized your pipeline code, resource definitions, and configuration into a Declarative Automation Bundle.

You created two Azure DevOps pipelines. The dev pipeline validates the bundle and runs unit tests on every pull request. The staging and production pipeline deploys to staging after manual approval, runs the full pipeline and integration tests, and rolls back automatically if anything fails. You set up branch policies so no code reaches main without passing the dev pipeline.

You added three layers of observability: job monitoring, Data quality alerts, and cost monitoring.

You can now version-control every Databricks artifact, deploy across environments with a single command, and catch failures before they reach production.

With pipelines running reliably in production, the next question is whether they run efficiently. The next chapter covers query performance tuning and cost management.

## Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to `packtpub.com/unlock`). Search for this book by name, confirm the edition, and then follow the steps on the page.

*Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.*

# 10

# Optimizing Query Performance and Cost Management

Having established a robust CI/CD pipeline in, your data workflows are now automated, reliable, and consistently deployed. This is a massive achievement, but the journey to production excellence doesn't end there. Once workflows are running in production, two new critical questions emerge: Are they running as fast as they can? And are they running as cheaply as they can? In the world of big data, performance is cost, and inefficient queries or over-provisioned clusters can quickly erode your budget and negate the benefits of your carefully crafted data platform.

With your CI/CD pipelines now humming along, you can deploy changes with confidence and speed. But automation is only half the battle. As your data volume and complexity grow, you will inevitably face performance bottlenecks and rising cloud costs. This is where optimization becomes the next critical stage in your data engineering journey. This chapter dives deep into the art and science of optimization, covering the techniques you need to make your Databricks jobs run faster and more efficiently. We will explore how to tune Spark and Delta Lake, leverage powerful features such as Adaptive Query Execution, and implement cost-management strategies to keep your cloud bills in check.

This chapter covers the following topics:

- **Performance Tuning for Spark and Delta Lake**: We will explore fundamental and advanced techniques for optimizing your queries, from file layout and data skipping in Delta Lake to join strategies and partitioning in Spark.
- **Adaptive Query Execution (AQE) and Caching Techniques**: Learn how to leverage Spark's intelligent runtime optimizer and Databricks' multi-layered caching to accelerate performance without manual intervention.

- **Managing Cluster Costs and Autoscaling Strategies**: This section provides a practical guide to controlling your Databricks bill, covering cluster right-sizing, effective autoscaling, and the strategic use of spot instances.
- **Monitoring and Debugging Performance Bottlenecks**: Discover how to use the Spark UI and other monitoring tools to diagnose performance issues, identify root causes, and validate your optimization efforts.

# Technical Requirements

To follow the examples and apply the concepts in this chapter, you will need the following:

- **Azure Databricks Workspace**: A Premium tier workspace is recommended to access all features, including advanced cluster policies and monitoring.
- **Databricks Runtime**: Version 15.2 or higher is required to use features like Liquid Clustering. Photon-enabled runtimes are highly recommended for optimal performance.
- **Cloud Subscription**: An active Azure subscription with permissions to create and manage Databricks clusters and access storage.

# Performance Tuning for Spark and Delta Lake

Performance tuning is a continuous process of identifying and eliminating bottlenecks to make your data workloads run faster and more efficiently. In the Databricks ecosystem, this involves optimizing at two primary levels: the storage layer, represented by Delta Lake, and the processing layer, powered by Apache Spark.

This initial section of the chapter will focus exclusively on these foundational tuning techniques. We will cover the essentials of structuring your data on disk with Delta Lake (using compaction, Liquid Clustering, Z-Ordering, and partitioning) and influencing Spark's processing behavior (through join strategies, predicate pushdown, column pruning, and shuffle management). Later sections will build on this foundation by exploring automatic optimization features like AQE and cost management strategies. By the end of this section, you will have a strong command of the manual tuning levers that give you granular control over your data platform's performance.

## The Philosophy of Optimization

Before diving into specific commands and configurations, it's crucial to adopt the right mindset for optimization. Performance tuning is not about blindly applying a checklist of best practices. It is an iterative, scientific process:

1. **Measure**: Establish a baseline. Before making any changes, measure the performance and cost of your current workload. Use Spark UI, logs, and cost monitoring tools to understand where time and money are being spent.
2. **Identify**: Based on your measurements, form a hypothesis about the primary bottleneck. Is it I/O bound? Is there data skew? Is the cluster under-provisioned?
3. **Change**: Make one, and only one, change based on your hypothesis. This could be adding a Z-ORDER clause, changing a join strategy, or adjusting a cluster configuration.
4. **Measure Again**: Rerun the workload and measure its performance and cost again. Compare the results to your baseline.
5. **Analyze**: Did the change have the intended effect? Did it improve performance, reduce cost, or both? Sometimes, a change that improves performance can increase costs, or vice versa. Understanding these trade-offs is key.

By following this iterative loop, you can systematically improve your workloads based on data-driven evidence rather than guesswork.

## Advanced Storage Layer Optimization with Delta Lake

Optimizing the storage layer is about arranging data on disk to minimize the amount of data that Spark needs to read. Less data read means less I/O, less network traffic, and faster queries.

Delta Lake's transactional format provides a powerful foundation for performance, but its true potential is unlocked through deliberate optimization of the underlying data layout. How you organize the data files on disk directly impacts how quickly Spark can find and read the data it needs. This section covers the key techniques for tuning your Delta tables.

## File Compaction and the Small File Problem

Streaming ingestion or frequent small updates can lead to a common big-data ailment: the "small file problem." When a table is composed of thousands of small files, the overhead of opening, reading, and closing each file overwhelms the query engine, crippling read performance. The `OPTIMIZE` command solves this by compacting small files into larger, more optimally sized ones. While 1 GB is the general default target for `OPTIMIZE`, Unity Catalog-managed tables use automatic file size tuning to determine the optimal file size based on table operations intelligently.

You can run the `OPTIMIZE` command on a table using the following SQL syntax:

```
-- Compact small files for the entire table

OPTIMIZE events;

-- Compact files only for a specific partition

OPTIMIZE events WHERE date = '2024-02-15';
```

Running `OPTIMIZE` periodically is a critical maintenance task for any Delta table that is frequently updated. For many workloads, Databricks can handle this automatically by compacting files during writes, preventing the small-file problem from developing in the first place. You can enable automatic file size tuning using `autoOptimize, autoCompact,` and `autoOptimize.optimizeWrite` table properties. `autoCompact` runs a lightweight background compaction job after a write operation is complete, while `optimizeWrite` dynamically optimizes partition sizes during the write operation itself.

For Unity Catalog-managed tables, Predictive Optimization is now the recommended approach. Predictive optimization is enabled by default and intelligently schedules maintenance operations such as `OPTIMIZE` and `VACUUM` based on data access patterns and table statistics, ensuring your tables remain performant without manual intervention.

If you are working with external tables where Predictive Optimization is not available, you will sometimes need to run the `OPTIMIZE` command manually. The best times to run this command are:

- **After large batch loads**: If you ingest a massive amount of data in a single batch, running `OPTIMIZE` immediately afterward ensures subsequent reads are fast.
- **On a regular schedule**: For tables with continuous streaming ingest or frequent small updates, schedule an `OPTIMIZE` job to run daily or weekly during off-peak hours.
- **Before critical queries**: If you have a complex, long-running reporting query, running `OPTIMIZE` beforehand can significantly reduce its execution time.

## Data Skipping

Delta Lake automatically collects statistics (min/max values) for the first 32 columns in a table. When you query the table, Spark uses these statistics to skip files that do not contain data matching your `WHERE` clause. For example, if a file's maximum timestamp is from yesterday, and your query filters for today's data, Spark ignores that file entirely. You can customize this behavior using the `delta.dataSkippingStatsColumns` or

`delta.dataSkippingNumIndexedCols` table properties to change which columns have statistics collected.

## Z-Ordering

Z-Ordering is a technique for co-locating related information within a single set of files. This significantly enhances the effectiveness of data skipping. If you frequently filter by a specific column (e.g., `customerId`), Z-Ordering by that column ensures that data for a given customer is concentrated in a few files, rather than scattered across the entire table.

It's important to understand the difference between Z-Ordering and partitioning. Partitioning physically separates data into different directories based on one or more columns. Z-Ordering, on the other hand, arranges data within files. They are not mutually exclusive and can be used together. A common pattern is to partition by a low-cardinality column (like date) and then Z-Order by higher-cardinality columns (like `userId` or `productId`).

Here is an example of how to apply Z-Ordering alongside partitioning during an optimization run:

```
-- Partition by date and Z-Order by userId

OPTIMIZE events WHERE date = '2024-02-15'

ZORDER BY (userId);
```

## Partitioning Strategies

Just as `OPTIMIZE` organizes data within files, partitioning organizes files within directories. Partitioning your Delta table by a low-cardinality column (like date or country) that is almost always used as a filter can provide a massive performance boost. This is because Spark can perform partition pruning, ignoring entire directories of data that are not relevant to the query.

The following code demonstrates how to create a partitioned table using SQL:

```
-- Create a partitioned table

CREATE TABLE events (
    -- columns
)
PARTITIONED BY (date);
```

However, be cautious. Partitioning by a high-cardinality column (like userId) can create millions of tiny partitions, reintroduce the small-file problem, and degrade performance.

Databricks recommendations are based on the table's size instead of the number of distinct values. The documentation guidance is more conservative: you should not partition tables below 1 TB, and each partition should contain at least 1 GB of data.

## Generated Columns for Partitioning

Sometimes, you don't have a perfect low-cardinality column to partition by. In these legacy cases, you could use generated columns. For example, if you want to partition by timestamp but the raw timestamp has too high cardinality, you can generate a date column from the timestamp and partition by that. It is important to note that Liquid clustering handles high cardinality natively. Generated columns were a clever workaround specifically designed to make high-cardinality columns usable for Hive-style partitioning.

Here is an example of creating a generated column for partitioning:

```
-- Create a table with a generated column for partitioning
CREATE TABLE events (
    eventId STRING,
    eventTimestamp TIMESTAMP,
    -- other columns
    eventDate DATE GENERATED ALWAYS AS (CAST(eventTimestamp AS DATE))
)
PARTITIONED BY (eventDate);
```

# Liquid Clustering: The Modern Approach

For newer Databricks runtimes, **Liquid Clustering** is replacing traditional partitioning and Z-Ordering as the recommended best practice. Liquid Clustering is a more flexible and adaptive way to organize data. Instead of being constrained by static partition boundaries, it can dynamically cluster data based on the columns you choose, even if those columns have high cardinality. This simplifies data layout management and provides better performance across a wider range of query patterns.

```
-- Create a table with liquid clustering
CREATE TABLE events (...) CLUSTER BY (eventType, sourceIP);

-- Automatic liquid clustering allows Databricks to intelligently choose
clustering keys

CREATE TABLE events (...) CLUSTER BY AUTO;

-- OPTIMIZE automatically reclusters the data
```

```
OPTIMIZE events;
```

You can enable liquid clustering on existing tables and change clustering keys at any time by running an `ALTER TABLE` command:

```
-- Enable or change clustering keys on an existing table
ALTER TABLE events CLUSTER BY (newColumn);
```

# Advanced Processing Layer Tuning with Spark

While Delta Lake optimizes data access, Spark tuning focuses on how the data is processed in memory across the cluster. This involves influencing Spark's query plan to ensure that operations such as joins and aggregations are executed as efficiently as possible, leveraging techniques such as predicate pushdown and column pruning.

## Join and Shuffle Optimization

Shuffling—the process of redistributing data across the cluster for operations like joins and aggregations—is one of the most expensive operations in Spark. A major goal of performance tuning is to minimize or avoid shuffles. Spark employs several join strategies, including sort-merge joins (which sort data on both sides before merging) and shuffle hash joins (which build a hash table on the smaller side and stream the larger side).

When joining a large table to a small table, you can often avoid a shuffle entirely by using a broadcast join. This sends a copy of the small table to every node in the cluster, allowing the join to happen locally without data movement. Spark's automatic broadcast decision is governed by the `spark.sql.autoBroadcastJoinThreshold` configuration, which has a default value of 10 MB.

Spark often figures this out on its own, but providing an explicit hint can be beneficial. Here is how you can explicitly hint to Spark to use a broadcast join in PySpark:

```
from pyspark.sql.functions import broadcast
# Hint to Spark to broadcast the smaller DataFrame
large_df.join(broadcast(small_df), "join_key")
```

Spark often figures this out on its own, but providing an explicit hint can be beneficial. This is a key area where Adaptive Query Execution, which we discuss next, provides significant automatic improvements.

### Tuning Shuffle Partitions

The number of partitions used during a shuffle is controlled by the `spark.sql.shuffle.partitions` configuration. The default is 200, which might not be optimal for all workloads. If you have very large datasets, you may need to increase this value to provide more parallelism. If you have smaller datasets, you might decrease them to reduce overhead. This is a parameter that often requires experimentation to find the sweet spot for your specific workload.

You can adjust this configuration for your current session using the following code:

```
# Set the number of shuffle partitions for the current session
spark.conf.set("spark.sql.shuffle.partitions", 400)
```

## The Perils of User-Defined Functions (UDFs)

UDFs allow you to execute custom Python or Scala code on your data. While they offer great flexibility, they can be a major performance bottleneck. Spark treats UDFs as black boxes and cannot apply its own optimizations to the code inside them. Whenever possible, you should use Spark's built-in functions, which are highly optimized. If you must use a UDF, consider using a **Pandas UDF** (also known as a vectorized UDF), which operates on Arrow record batches and can be significantly faster than row-at-a-time Python UDFs.

Mastering these manual tuning techniques provides a strong foundation for optimizing your workloads. Now, let's explore how Databricks can automate much of this work for us with features like Adaptive Query Execution and intelligent caching.

# Adaptive Query Execution and Caching Techniques

Modern Spark includes powerful features that can automatically optimize queries at runtime and accelerate repeated reads without manual intervention. **Adaptive Query Execution** (**AQE**) re-optimizes query plans on the fly using live statistics, while Databricks' multi-layered caching system intelligently stores hot data in faster storage tiers. This section will cover the benefits of the Photon engine, the specific runtime optimizations provided by AQE, a detailed comparison between Disk Cache and Spark Cache, storage levels, and guidance on when caching should be avoided. Understanding and enabling these features is one of the easiest ways to get a significant performance lift.

## The Power of Photon

Before diving into AQE, it is essential to mention Photon, Databricks' high-performance, C++-based vectorized query engine. Photon is designed to be a faster, more efficient replacement for the legacy Spark engine. It is fully compatible with Spark APIs, so enabling it requires no code

changes. For many workloads, simply enabling Photon can provide a 2-3x performance improvement out of the box. Photon and AQE work together, with Photon providing a faster execution engine and AQE providing dynamic, runtime optimizations.

## Dynamic Re-optimization with Adaptive Query Execution (AQE)

Traditionally, Spark would create a single, static query plan and execute it. The problem is that this plan is based on estimates. If those estimates are wrong (e.g., due to stale statistics or complex data skew), the plan can be highly inefficient. AQE, enabled by default in modern Databricks runtimes, solves this by re-optimizing the query plan at execution.

AQE introduces several key runtime optimizations:

- **Dynamically Coalescing Shuffle Partitions**: AQE can merge small shuffle partitions, reducing the number of tasks and scheduling overhead for downstream stages.
- **Dynamically Switching Join Strategies**: AQE can convert a sort-merge join into a more efficient broadcast join at runtime if it detects that one side of the join is small enough.
- **Dynamically Optimizing Skew Joins**: If AQE detects severe data skew in a join, it can split the skewed partitions into smaller sub-partitions and join them independently, preventing a few tasks from bottlenecking the entire stage.

Because AQE is on by default, you are likely already benefiting from it. You can see its effects in the Spark UI, where you will see `AQEShuffleRead` and other adaptive nodes in the query plan graph.

## A Deeper Look at AQE Features

- **Dynamically Coalescing Shuffle Partitions**: After a shuffle, you might end up with many small partitions. AQE can examine the sizes of these partitions and automatically merge adjacent small ones into larger, more optimally sized partitions. This reduces the number of tasks in the next stage, thereby reducing scheduling overhead.
- **Dynamically Switching Join Strategies**: Spark might initially plan a sort-merge join based on table statistics. However, after one side of the join is materialized during the shuffle, AQE might realize it's small enough to fit in memory. It can then dynamically change the plan to a broadcast hash join, which is significantly faster as it avoids the expensive sort and merge operations on the larger table.
- **Dynamically Optimizing Skew Joins**: This is one of AQE's most powerful features. When it detects a partition in a join that is much larger than the others (a skewed partition), it can split that large partition into smaller pieces. It then joins each of these

smaller pieces with the corresponding partition from the other side of the join. This turns a single, long-running task into multiple, smaller, parallel tasks, dramatically reducing the overall stage time.

## Accelerating Reads with Caching

Databricks provides a multi-layered caching system to speed up data access. When you read the same data repeatedly, caching can provide orders-of-magnitude performance improvements by serving it from a faster tier rather than re-reading it from remote object storage.

Understanding the difference between the automatic Disk Cache and the manual Spark Cache is critical for making the right optimization choice. The following table provides a side-by-side comparison of these two caching mechanisms:

| Feature | Disk Cache | Apache Spark Cache |
|---|---|---|
| **Stored as** | Local files on a worker node | In-memory blocks, but it depends on the storage level |
| **Applied to** | Any Parquet table stored on ABFS and other file systems | Any DataFrame or RDD |
| **Triggered** | Automatically, first read (if cache is enabled) | Manually requires code changes |
| **Evaluated** | Lazily | Lazily |
| **Availability** | Can be enabled or disabled with configuration flags; enabled by default on certain node types | Always available |

| Feature | Disk Cache | Apache Spark Cache |
|---|---|---|
| **Evicted** | Automatically in LRU fashion, or on any file change, or manually when restarting a cluster | Automatically in LRU fashion, manually with **unpersist** |

*Table 10.1: A comparison of Databricks Disk Cache and Apache Spark Cache.*

The table above provides a high-level comparison. Now, let's dive into the details of each caching mechanism and when to use them:

- **Disk Cache (formerly Delta Cache)**: This is the most important caching layer. It automatically creates copies of remote data files on the local SSDs of the cluster nodes. Subsequent reads of the same data can then be served from the fast local disk instead of the slower cloud storage. Disk cache is enabled by default on instances with local SSDs (such as the L-series on Azure) and works transparently without any code changes.
- **Spark Cache**: This is the traditional `df.cache()` or `spark.catalog.cacheTable()` mechanism. It allows you to explicitly cache a DataFrame in memory, on disk, or both. While useful, it is more manual. Crucially, for Delta Lake tables, you should almost always prefer the automatic Disk Cache over the manual Spark Cache. Databricks does not recommend using Spark caching because you lose any data that can be skipped by additional filters applied on top of the cached DataFrame. The Disk Cache is more efficient and does not interfere with Delta's time travel and other features.

As a general rule of thumb, prefer the automatic Disk Cache for Delta tables, and reserve manual Spark caching for specific reuse scenarios, which we explore next.

## Spark Cache Storage Levels

When you do need to use manual Spark caching, it's important to understand the different storage levels available to optimize memory usage and performance.

When you use `df.cache()`, you can specify different storage levels:

- `MEMORY_ONLY`: Stores the data in memory as deserialized Java objects. This is the most CPU-efficient option, but if the data doesn't fit in memory, it won't be cached.

- `MEMORY_AND_DISK`: Stores the data in memory. If it doesn't fit, the overflow partitions are stored on disk. This is a good general-purpose option.
- `DISK_ONLY`: Stores the data only on disk. This is useful for very large DataFrames that you want to reuse without consuming memory.

### When Not to Cache

Caching is not a silver bullet. Because caching is often overused as a default optimization, it's equally important to recognize when it is unnecessary or counterproductive. Here are some scenarios where you should avoid caching:

- **You only read the data once**: Caching only provides a benefit on the second and subsequent reads. If you only read a DataFrame once, caching it adds unnecessary overhead.
- **The underlying data changes frequently**: If the source data for your cached DataFrame changes, the cache will become stale and you will need to invalidate and refresh it manually.
- **Your transformation is simple**: If the DataFrame is the result of a very simple, fast transformation, it might be faster to recompute it each time rather than paying the cost of writing it to and reading it from the cache.

By keeping these trade-offs in mind, you can use caching as a powerful performance tuning tool without falling into common traps. Having explored how to optimize data access with caching, let's now turn our attention to the other side of the optimization coin: managing the cost of the underlying compute resources.

## Managing Cluster Costs and Autoscaling Strategies

Performance optimization is intrinsically linked to cost management. A faster query often means a cheaper query because it consumes fewer cluster resources over a shorter period. However, direct cost control focuses on ensuring you use the right amount of infrastructure for your workload and avoid paying for idle resources. This involves making smart choices about cluster configuration, leveraging autoscaling, and taking advantage of cloud pricing models. This section establishes the decision framework for the practical guidance that follows, helping you understand how cost and performance trade off in real-world workloads. Rarely the cheapest, and the cheapest configuration is rarely the fastest. The goal of cost optimization is not to achieve the absolute lowest cost, but to find the sweet spot on the cost-performance curve that meets your business requirements. For a nightly batch job with a 6-hour window to complete, there is no business value in spending extra money to run it in 30 minutes. Conversely, for an interactive query that a business analyst is waiting on, a few extra cents to make it run in seconds rather than minutes is likely a worthwhile investment.

## Cluster Configuration and Sizing

Databricks offers two primary types of clusters, and choosing the right one is the first step in cost management:

- **Serverless Compute:** This is now the recommended option for most workloads. Serverless compute removes the need to manage infrastructure entirely, instantly scaling resources to meet demand and charging only for the compute used during execution.
- **All-Purpose Clusters**: These are persistent clusters that can be used for interactive analysis by multiple users. They are great for development and exploration, but can become costly if left running idle.
- **Job Clusters**: These clusters are created on the fly for a specific automated job and terminate automatically when the job is complete. For production workflows, **always use job clusters**. You only pay for the resources while the job is actively running, eliminating the cost of idle time.

Another effective cost optimization strategy is the use of **Cluster Pools**. Cluster pools maintain a set of idle, ready-to-use instances. When a cluster needs to scale up or start, it can draw from the pool rather than wait for the cloud provider to provision new VMs. This significantly reduces cluster start times and improves autoscaling responsiveness, while still allowing you to control costs by setting maximum pool sizes and idle termination times.

**Right-sizing** your cluster by choosing the appropriate VM instance types and worker count is also critical. Don't over-provision with massive instances if your workload isn't memory- or compute-intensive. Start with a reasonable size and use monitoring to determine if you need to scale up or down.

Choosing the right instance family is a key part of right-sizing:

### Instance Families

- **General Purpose**: A good starting point for many workloads. Example: D-series (Dv3, Dv4, Dv5).
- **Compute Optimized**: Ideal for CPU-intensive tasks like complex transformations and feature engineering. Example: F-series (Fsv2).
- **Memory Optimized**: Best for workloads that require a large amount of memory, such as those with large shuffles or caches. Example: E-series (Ev3, Ev4, Ev5) or M-series for very large memory.
- **Storage Optimized**: These instances come with fast, local NVMe SSDs, which are perfect for workloads that benefit from the Disk Cache. Example: L-series (Lsv2, Lsv3).

## Enhanced Autoscaling

Instead of guessing the exact number of workers, you can use autoscaling. Databricks' **Enhanced Autoscaling** automatically adds and removes workers from your cluster based on the workload. You set a minimum and maximum number of workers.

You can configure this in the cluster settings UI or directly in the cluster's JSON definition, which is useful for infrastructure-as-code workflows. The following JSON snippet shows how to configure a cluster to autoscale between 2 and 20 workers:

```
{
  "autoscale": {
    "min_workers": 2,
    "max_workers": 20
  }
}
```

This feature is highly effective at matching resources to demand, scaling up for heavy stages and scaling down during lulls to save money. It is a must-have for any production cluster running variable workloads.

While autoscaling is powerful, you can achieve even greater cost savings by using spot instances.

## Autoscaling for Streaming

For Structured Streaming workloads, Databricks offers a specialized autoscaling mode that is optimized for the continuous nature of streaming data. It is more conservative about scaling down to avoid impacting the stream processing latency.

## Using Spot Instances for Cost Savings

Cloud providers offer unused compute capacity at a significant discount (often up to 90%) as **spot instances** (or preemptible VMs). The catch is that the provider can reclaim this capacity at any time with little notice. While this makes them unsuitable for highly critical, time-sensitive workloads, they are an excellent choice for many fault-tolerant data engineering tasks that can handle retries.

You can configure your Databricks cluster to use spot instances for all workers or use a mix with a reliable on-demand driver and spot workers. This is one of the single most effective ways to reduce your Databricks compute costs dramatically.

To use spot instances effectively, follow these best practices:

## Spot Instance Best Practices

- **Use for fault-tolerant workloads**: Spot instances are best for jobs that can be retried if an instance is reclaimed.
- **Don't use for the driver**: The driver node is critical for the stability of the cluster. Always use an on-demand instance for the driver.
- **Enable autoscaling**: Combining spot instances with autoscaling allows the cluster to replace reclaimed instances and maintain the desired level of parallelism.

## Cost Control with Cluster Policies and Tagging

**Cluster policies** are a powerful governance tool for controlling costs. Administrators can define templates that restrict the types and sizes of clusters users can create. For example, a policy could enforce:

- A maximum number of workers.
- The use of autoscaling with a specific min/max range.
- A mandatory auto-termination timeout (e.g., 60 minutes).
- The use of spot instances for workers.
- Restrictions on which instance families are allowed.

While cluster policies provide guardrails, tagging provides visibility. By applying tags to your clusters, you can attribute costs to specific teams, projects, or environments. This is essential for chargeback and showback models, and for identifying areas of high spend. Since earlier sections discussed Predictive Optimization, it's worth noting here that its costs are trackable. Predictive Optimization costs appear in the `system.storage.predictive_optimization_operations_history` system table.

```
{
  "custom_tags": {
    "Project": "Data-Platform-Migration",
    "Team": "Data-Engineering",
    "Environment": "Production"
  }
}
```

By consistently applying tags, you can use Databricks' cost analysis tools or your cloud provider's billing console to get a detailed breakdown of your spend. This visibility is the foundation of effective cost management. However, even with a perfectly sized cluster, a poorly performing query can still be slow and expensive. To close the loop, we need to know

how to diagnose why a job is underperforming, which requires diving into the monitoring and debugging tools we will cover in the next section.

# Monitoring and Debugging Performance Bottlenecks

Optimization is not a one-time task; it is an iterative cycle of measuring, analyzing, and improving. To effectively tune your workloads, you need to know how to identify performance bottlenecks. The Spark UI is the most powerful tool for this, providing a detailed, behind-the-scenes view of how your query is executed.

## The Spark UI: A Deeper Dive

The Spark UI is your primary window into the performance of your jobs. Understanding how to navigate it is the most important skill for performance tuning.

## Navigating the Spark UI

The Spark UI is accessible from the Databricks cluster page. It is organized into several tabs, but for performance tuning, the most important ones are:

- **Jobs Tab**: Shows a high-level overview of the jobs triggered by your query. A job is created for each action (e.g., `collect()`, `write()`).
- **Stages Tab**: Breaks down each job into stages. A new stage is typically created at each shuffle boundary. Look for stages that take a long time to complete, as these are your primary bottlenecks.
- **SQL / DataFrame Tab**: This is often the best place to start. It shows the query plan for your DataFrame or SQL operations. You can view the logical plan, the physical plan, and the executed plan (which includes AQE modifications). Clicking a query takes you to a detailed graphical representation of the executed plan.
- **Executors Tab**: Provides a summary of the resources used by each executor, including memory, disk usage, and task activity. This is useful for identifying resource contention or overloaded executors.
- **Environment Tab**: Shows all the Spark configuration properties for the current session. This is useful for verifying that your configuration changes have been applied.

## Identifying Common Bottlenecks

When analyzing the query plan in the Spark UI, look for these common signs of trouble:

- **Long-Running Stages**: A stage that takes significantly longer than others is a clear bottleneck. Drill into that stage to see what operations it is performing.

- **Large Data Shuffles**: Look at the "shuffle read" and "shuffle write" metrics for each stage. A large amount of shuffled data indicates an expensive operation that could be a candidate for optimization (e.g., with a broadcast join).
- **Data Skew**: In the stage detail view, look at the summary statistics for task duration. If the "max" duration is much higher than the "median" or "mean," it's a strong indicator of data skew, where one or two tasks are doing far more work than the others.
- **Spill to Disk**: When memory is insufficient, Spark will spill data to disk during operations like shuffles and sorts. This severely impacts performance and can be identified in the Spark UI stage details.

By using the Spark UI to pinpoint these issues, you can move from guessing to data-driven optimization, applying the tuning techniques discussed in this chapter precisely where they are needed most.

## Other Monitoring Tools

While the Spark UI is the most important tool, Databricks provides other monitoring capabilities:

- **Databricks System Tables:** System tables (like `system.lakeflow.job_run_timeline` and `system.billing.usage`) provide queryable logs of operational data, making it easy to monitor job performance, cluster usage, and costs over time.
- **Ganglia Metrics**: For clusters using older runtimes, Ganglia provides detailed, real-time metrics on cluster resource utilization, such as CPU, memory, and network I/O.
- **Cluster Event Logs**: These logs provide a high-level overview of cluster activity, including autoscaling events, spot instance reclamations, and driver/executor lifecycle events.
- **Driver and Executor Logs**: For deep debugging, you can access the raw log files (`stdout` and `stderr`) for the driver and each executor. This is where you will find detailed error messages and stack traces.

## Real-World Example: Debugging a Skewed Join

Imagine you have a job that is running much slower than expected. You open the Spark UI and go to the **Stages** tab. You notice that one stage takes hours, while all the others take minutes. You click on that stage and see the following task duration summary:

| Metric | Value |
|---|---|
| **Duration** | 2.5 h |
| **Tasks: Succeeded/Total** | 200/200 |
| **Shuffle Read** | 500.0 GB |
| **Task Time (median)** | 2.1 min |
| **Task Time (max)** | 2.1 h |

*Table 10.2: Spark UI task summary showing evidence of data skew*

That massive difference between the median and max task time is a classic sign of data skew. One task takes over two hours, while the median task takes only two minutes! This single task is bottlenecking the entire stage. You then look at the query plan in the **SQL / DataFrame** tab and confirm that this stage corresponds to a join operation. With AQE enabled, you might see that Spark has already attempted to handle the skew, but in some extreme cases, manual intervention is still needed. You could then apply a technique such as salting the join key to more evenly distribute the skewed data, thereby resolving the bottleneck.

## Example: Salting a Skewed Join Key

If you have a join where one key is overwhelmingly dominant (e.g., a `null` or `unknown` key), you can isolate that key and handle it separately. A more general approach is salting, where you add a random number to the join key to break up the skewed value into multiple, smaller keys.

```
from pyspark.sql.functions import lit, col, rand, concat
import pyspark.sql.functions as F
# Assume 'users' is skewed on 'country_code' = 'US'
salt_factor = 10

# Add a salt column to the skewed DataFrame
salted_users = users.withColumn("salt", (rand() * salt_factor).cast("int")) \
    .withColumn("salted_country_code", concat(col("country_code"), lit("_"),
col("salt")))
# Explode the corresponding dimension table to match the salt
salted_countries = countries.withColumn("salt_array", F.array([F.lit(i) for i in
range(salt_factor)])) \
    .withColumn("salt", F.explode(col("salt_array"))) \
    .withColumn("salted_country_code", concat(col("country_code"), lit("_"),
```

```
col("salt")))
# Join on the salted key
result = salted_users.join(salted_countries, "salted_country_code")
```

This is an advanced technique, but it illustrates the level of manual control you can exert when automatic optimization is not enough. By understanding how to interpret Spark UI metrics and applying targeted solutions like salting, you can overcome even the most stubborn performance bottlenecks.

# Summary

In this chapter, we transitioned from simply running data workflows to running them with peak efficiency and cost-effectiveness. You learned how to analyze and optimize Delta Lake tables using tools like `OPTIMIZE`, `Z-ORDER`, and the modern Liquid Clustering approach to ensure your data is stored efficiently. We explored how to diagnose and resolve query bottlenecks by interpreting Spark UI metrics and identifying issues like data skew, allowing you to implement advanced performance techniques such as join optimization and salting for skewed joins.

We also saw how to leverage modern Spark features like Adaptive Query Execution (AQE) and Photon to automatically accelerate your workloads, and how to make informed caching decisions by choosing between the automatic Disk Cache and manual Spark caching based on your workload patterns. Finally, we covered how to configure and manage cluster costs effectively by right-sizing clusters, implementing autoscaling, using spot instances, and setting up cluster policies, empowering you to apply a systematic optimization philosophy to continuously measure, identify, and improve the performance and cost-efficiency of your data platform.

We explored the dual pillars of performance tuning in Databricks: optimizing the Delta Lake storage layer and tuning the Spark processing layer. We saw how modern features like Adaptive Query Execution and the Disk Cache provide powerful, automatic performance boosts. Furthermore, we established a framework for rigorous cost management, emphasizing the use of job clusters, effective autoscaling, and the strategic deployment of spot instances to control cloud spend.

By mastering these optimization and cost-management strategies, you ensure that your data platform is not only technically sound but also financially sustainable. The ability to deliver results quickly and economically is a hallmark of mature data engineering practice. This foundation of efficiency is crucial as we move into the final and most critical aspect of managing an enterprise data platform: security. In the next chapter, we will explore how to

secure your data, manage compliance, and implement robust data governance to ensure your valuable data assets are protected and used responsibly.

## Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to `packtpub.com/unlock`). Search for this book by name, confirm the edition, and then follow the steps on the page.

*Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.*

# 11

# Security, Compliance, and Data Governance

In the preceding chapters, we built a complete data engineering platform on Azure Databricks: ingesting data, transforming it through medallion layers, automating pipelines, and optimizing performance. But a well-architected data platform is only as valuable as the trust stakeholders place in it. That trust depends on securing data against unauthorized access, meeting regulatory obligations, and maintaining transparent governance over every data asset.

In *Chapter 2, Setting Up an End-to-End Azure Databricks Environment*, we established the foundational elements of Unity Catalog: configuring the metastore, designing the three-level namespace, creating managed and external objects, and setting up role-based access control with account-level groups and privilege inheritance. This chapter builds on that foundation by addressing the security, compliance, and governance capabilities that turn a functional Databricks environment into an enterprise-ready data platform.

In this chapter, we will cover the following topics:

- Securing the Databricks environment with network isolation, encryption, and workspace-level controls
- Implementing fine-grained access controls using column masks, row filters, and dynamic views
- Auditing and monitoring data access through Unity Catalog system tables and Azure diagnostic settings
- Ensuring compliance with regulatory standards such as GDPR and HIPAA
- Tracking data lineage and performing impact analysis with Unity Catalog
- Sharing data securely across organizations with Delta Sharing

## Technical requirements

To follow the examples in this chapter, you will need:

1. **Azure Resources**:
    - An Azure Databricks workspace on the Premium pricing tier (required for Unity Catalog and advanced security features)
    - Unity Catalog enabled and configured with a metastore (as set up in *Chapter 2, Setting Up an End-to-End Azure Databricks Environment*)
    - An Azure Key Vault instance (for customer-managed key examples)
    - (Optional) An Azure Log Analytics workspace for diagnostic logging examples
2. **Databricks Configuration**:
    - Account admin access for configuring system tables and workspace settings (note: account admins can query system tables by default, but other users require explicit `USE` and `SELECT` grants on the system schemas)
    - A cluster running Databricks Runtime 13.3 LTS or later with Unity Catalog enabled
    - At least two account-level groups for testing access control scenarios (such as `pii_viewers` and `analysts`)
3. **Permissions**:
    - Unity Catalog `CREATE FUNCTION` privilege on a schema (for masking function examples)
    - `ALTER` privilege on tables (for applying column masks and row filters)
    - Account admin or workspace admin role (for workspace security settings and system tables)
4. **Code Files:**
   All code examples in this chapter are available in the book's GitHub repository. To access the repository, follow the steps in the "Download the example code files" section of the *Preface*. Throughout this chapter, replace placeholder values like `cat_dev` with your actual catalog name.

# Securing the Databricks environment

While Unity Catalog governs access to data objects, securing a Databricks environment also requires attention to the infrastructure layer. Azure Databricks operates within a shared responsibility model: Microsoft manages the underlying cloud infrastructure, while you are

responsible for configuring network boundaries, encryption policies, and workspace-level security settings. This section covers the infrastructure security controls that protect your Databricks deployment from network-level threats and unauthorized access.

## Network security with VNet injection

By default, Azure Databricks deploys clusters in a Databricks-managed virtual network. While this simplifies setup, it provides limited control over network traffic. For production environments, VNet injection deploys Databricks clusters into your own Azure Virtual Network, giving you full control over inbound and outbound network traffic.

When you deploy a Databricks workspace with VNet injection, Databricks creates resources in two dedicated subnets within your VNet: a host subnet (for the driver and worker nodes of the Spark cluster) and a container subnet (for infrastructure-level containers). You must create these subnets before deploying the workspace and delegating them to `Microsoft.Databricks/workspaces` service.

The following ARM template snippet shows the subnet configuration for a VNet-injected workspace:

```
{
  "type": "Microsoft.Network/virtualNetworks/subnets",
  "name": "[concat(parameters('vnetName'), '/databricks-host')]",
  "properties": {
    "addressPrefix": "10.0.1.0/24",
    "delegations": [
      {
        "name": "databricks-delegation",
        "properties": {
          "serviceName": "Microsoft.Databricks/workspaces"
        }
      }
    ],
    "networkSecurityGroup": {
      "id": "[resourceId('Microsoft.Network/networkSecurityGroups', 'databricks-
nsg')]"
    }
  }
}
```

With VNet injection in place, you can attach **Network Security Groups** (**NSGs**) to the Databricks subnets. Databricks requires certain NSG rules to function (for cluster

communication and connections to the control plane), but you can add additional rules to restrict outbound traffic. For example, you might allow outbound connections only to your Azure Data Lake Storage accounts and block all other internet-bound traffic. This is particularly important in regulated industries where data exfiltration is a concern.

**Azure Private Link** takes network isolation further by ensuring that traffic between your corporate network and the Databricks workspace never traverses the public internet. There are 3 types of Private Link connections relevant to Databricks:

- The **front-end Private Link** (also called the web application private endpoint) provides private connectivity from user browsers and API clients to the Databricks workspace UI and REST API. Instead of accessing `https://adb-xxxx.azuredatabricks.net` over the public internet, users connect through a private endpoint in your VNet, and DNS resolves the workspace URL to a private IP address.
- The **back-end Private Link** (also called the connectivity private endpoint) provides private connectivity from the Databricks data plane (where your clusters run) to the control plane (the Databricks-managed infrastructure). This ensures that even the internal communication between your clusters and Databricks management services stays within private network paths.
- The **browser authentication Private Link** (also called the web authentication private endpoint) provides private connectivity for the SSO login callback. During authentication, the user's browser is redirected to the Databricks login endpoint; without this private endpoint, that redirect traverses the public internet even if the front-end Private Link is configured. In deployments that require strictly no public internet traffic, all three private endpoints — front-end, back-end, and browser authentication — must be configured together.

| | **Front-End Private Link** | **Back-End Private Link** | **Browser Authentication Private Link** |
|---|---|---|---|
| **What it protects** | User-to-workspace traffic (UI, REST API) | Cluster-to-control-plane traffic | SSO login callback traffic |
| **Who connects** | Users, API clients, BI tools | Databricks data plane (clusters) | User browsers during authentication |

| | Front-End Private Link | Back-End Private Link | Browser Authentication Private Link |
|---|---|---|---|
| **Destination** | Databricks workspace URL | Databricks control plane services | Databricks login endpoint |
| **When to use** | When users must not access the workspace over the public internet | When cluster communication must stay on private network paths | When the SSO callback must not traverse the public internet |
| **Requires VPN/ ExpressRoute** | Yes, users need private network access to reach the endpoint | No, applies to internal Databricks communications | Yes, user browsers must reach the endpoint via a private network |
| **Complexity** | Moderate, requires DNS configuration for workspace URL resolution | Higher requires additional subnet and endpoint configuration | Moderate, requires an additional private endpoint and DNS entry |
| **Recommended for** | All production deployments handling sensitive data | Regulated environments requiring fully private connectivity | Any deployment using front-end Private Link with strict no-public-internet requirements |

*Table 11.1: Comparison of Private Link connections*

All three Private Link types share common characteristics: traffic stays within the Microsoft Azure backbone network and never traverses the public internet, they require an Azure Private Endpoint resource in your VNet, and they can be combined for maximum network isolation.

To deploy a workspace with both Private Link connections and no public network access, configure the workspace with `publicNetworkAccess` set to `Disabled` and `requiredNsgRules` set to `NoAzureDatabricksRules`. This is the most secure configuration, but it requires that all users access the workspace through a private network connection (VPN or ExpressRoute).

**Tip**

For most production deployments, start with VNet injection and front-end Private Link. Add browser authentication Private Link alongside the front-end endpoint to keep SSO callbacks private. Add back-end Private Link when your compliance requirements mandate fully private connectivity. The configuration complexity increases with each layer, so adopt these controls incrementally based on your threat model.

## Encryption: At rest and in transit

Azure Databricks encrypts data both at rest and in transit by default. Understanding these encryption layers helps you satisfy auditor questions and identify where additional controls are available.

**Encryption at rest** applies to all data stored in the Databricks-managed storage account (used for DBFS root storage, system data, and notebook results), as well as any Azure Data Lake Storage accounts you connect through Unity Catalog. By default, Azure encrypts this data using **Microsoft-managed keys (MMKs) with AES-256**. For organizations that require control over encryption keys, Azure Databricks supports **customer-managed keys (CMK)** through Azure Key Vault for three distinct services: managed services (encrypts the notebook and secret data stored in the Databricks control plane), managed disks (encrypts the temporary storage on cluster VMs), and the DBFS root storage account.

To configure customer-managed keys for managed services, you create an Azure Key Vault, generate or import an RSA key, and associate it with your Databricks workspace during creation or update. The following Azure CLI command updates an existing workspace to use CMK for managed services:

```
az databricks workspace update \
  --name my-databricks-workspace \
  --resource-group my-resource-group \
  --key-source Microsoft.Keyvault \
  --key-name my-cmk-key \
  --key-vault https://my-keyvault.vault.azure.net/ \
  --key-version <key-version>
```

**Encryption in transit** is enforced by default for external connections. All connections to the Databricks workspace UI and API require TLS 1.2 or higher, and connections from clusters to Azure Storage services use HTTPS. However, communication between worker nodes within a cluster is not encrypted by default. To enable inter-node encryption, you must configure wire-level encryption in the workspace settings (see the Azure Databricks documentation on encrypting traffic between cluster worker nodes).

**Double encryption** provides an additional layer of protection by encrypting data with two separate encryption keys—one at the infrastructure level and one at the service level. Azure Databricks supports infrastructure encryption for DBFS root storage, which means data passes through two independent encryption layers before being written to disk.

> **Note**
>
> Customer-managed keys must be configured at the time of workspace creation for some encryption targets. Plan your encryption strategy before deploying production workspaces, as retroactive changes may require workspace recreation.

## Workspace-Level security settings

Beyond network and encryption controls, several workspace-level settings harden the environment against common security risks. These settings are configured in the Azure Databricks admin console or through the workspace API.

**Token management** controls how users authenticate to the Databricks REST API. By default, users can generate **personal access tokens** (**PATs**) with any expiration period. In production, restrict token lifetime to a maximum duration (such as 90 days) and enable token management to require admin approval for new tokens. You can also disable PATs entirely and require users to authenticate exclusively through Azure Active Directory (now Microsoft Entra ID) tokens, which integrate with your organization's conditional access policies and multi-factor authentication requirements.

**IP access lists** restrict which IP addresses can connect to your Databricks workspace, independent of network-level controls. You can configure allow lists (only specified IPs can connect) or block lists (specified IPs are denied). This is particularly useful when you cannot use Private Link but still want to limit access to known corporate IP ranges.

**Cluster policies** define constraints on cluster configuration that users must follow. From a security perspective, cluster policies ensure that users cannot create clusters that bypass security controls. For example, a policy can enforce that all clusters use Unity Catalog-enabled runtimes, run in Dedicated access mode (formerly single-user) or Standard access mode (formerly Shared) (preventing legacy credential passthrough), are limited to specific node

types to control cost and data residency, and use specific Spark configuration parameters required by your security team.

The following cluster policy ensures Unity Catalog compliance and restricts cluster size:

```
{
  "spark_version": {
    "type": "regex",
    "pattern": "1[3-9]\\.[0-9]+\\.x-scala.*",
    "defaultValue": "15.4.x-scala2.12"
  },
  "data_security_mode": {
    "type": "fixed",
    "value": "DATA_SECURITY_MODE_STANDARD"
  },
  "num_workers": {
    "type": "range",
    "maxValue": 10,
    "defaultValue": 2
  },
  "autotermination_minutes": {
    "type": "range",
    "minValue": 10,
    "maxValue": 120,
    "defaultValue": 30
  }
}
```

This policy forces user isolation (the secure mode for shared clusters with Unity Catalog), limits clusters to a maximum of 10 workers, and requires auto-termination within two hours. Data engineers can create clusters within these constraints without requiring admin intervention, while the security team retains control over the boundaries.

# Fine-Grained access control with Unity Catalog

In *Chapter 2, Setting Up an End-to-End Azure Databricks Environment*, we configured Unity Catalog with account-level groups and role-based permissions at the catalog, schema, and table levels. That approach works well for broad access patterns - granting the analytics team read access to the Gold layer, for example, or giving data engineers write access to Silver schemas. But many real-world scenarios require finer granularity: hiding sensitive columns from certain users, filtering rows based on a user's department, or masking data values to

show only partial information. Unity Catalog provides three mechanisms for these requirements: column masks, row filters, and dynamic views.

## Column masking

Column masking allows you to apply a function to a column's values at query time, transforming the output based on who is running the query. The underlying data remains unchanged; the mask applies only to the query results. This is particularly useful for columns containing PII such as email addresses, phone numbers, and national identification numbers.

To implement column masking, you first create a SQL function that defines the masking logic, then apply it to a column using `ALTER TABLE`. The function receives the column value and, optionally, the invoking user's identity, and returns the masked value.

The following example creates a masking function that shows the full email address to members of the `pii_viewers` group and returns a masked version for everyone else:

```
-- Create a masking function for email addresses
CREATE OR REPLACE FUNCTION cat_dev.sandbox.mask_email(email_value STRING)
RETURNS STRING
RETURN CASE
  WHEN IS_ACCOUNT_GROUP_MEMBER('pii_viewers') THEN email_value
  ELSE CONCAT(LEFT(email_value, 2), '****@',
       SPLIT(email_value, '@')[1])
END;
```

Now apply the mask to the email column on a table:

```
-- Apply the mask to the customers table
ALTER TABLE cat_dev.sandbox.customers_gold
ALTER COLUMN email
SET MASK cat_dev.sandbox.mask_email;
```

Once the mask is applied, a member of the `pii_viewers` group querying the table sees `jane.doe@company.com`, while other users see `ja****@company.com`. The masking is transparent, users query the table normally with `SELECT * FROM customers`, and the mask is applied automatically. No application code changes are required.

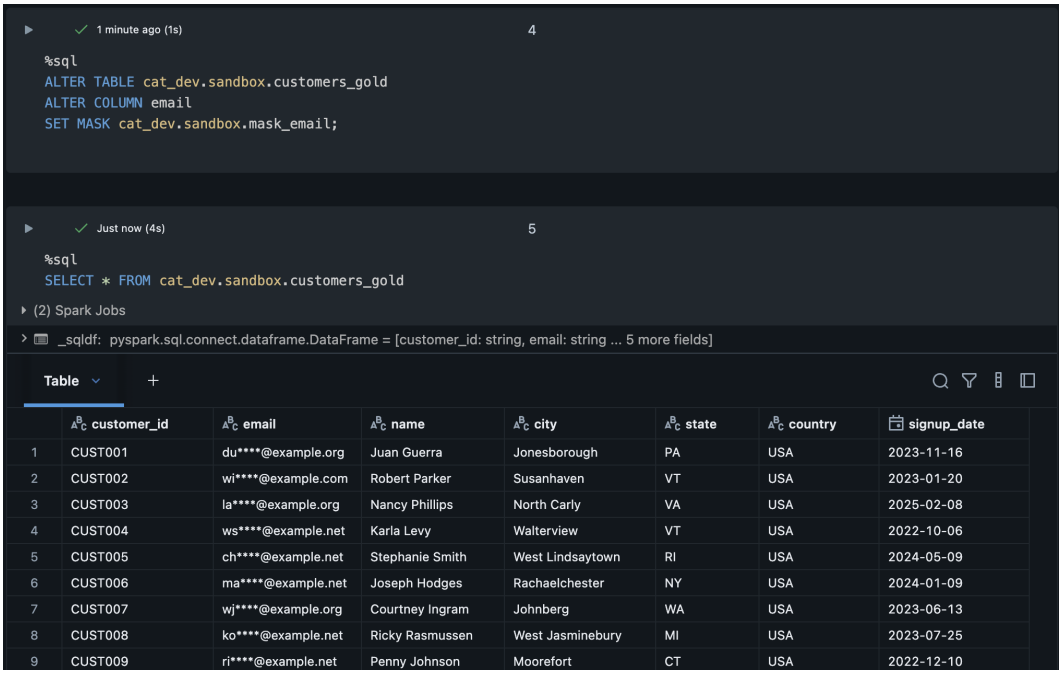



*Figure 11.1: Apply masking function to the customers table*

You can create different masking functions for different sensitivity levels. A phone number mask might show the last four digits; a salary mask might return a range instead of the exact value; a credit card mask might show only the last four digits. The masking function has access to the current user context via functions such as `IS_ACCOUNT_GROUP_MEMBER()` and `SESSION_USER()`, enabling flexible authorization logic.

**Tip**

When writing column mask or row filter functions that check the caller's identity, use `SESSION_USER()` rather than `CURRENT_USER()`. The `SESSION_USER()` function returns the identity of the user who initiated the session, which is the recommended approach in Databricks Runtime 14.1 and above. The `CURRENT_USER()` function is technically valid but is not referenced in the official column mask documentation.

To remove a mask when it is no longer needed:

```
ALTER TABLE cat_dev.sandbox.customers_gold
ALTER COLUMN email
DROP MASK;
```

**Note**

**Important**: Column masks apply to direct table queries. If a user creates a materialized view or writes query results to another table, the masked values are what get stored. Design your data pipeline so that sensitive columns flow through governed tables with masks applied, rather than being copied to ungoverned locations.

## Row-Level security with row filters

Row filters restrict which rows a user can see when querying a table, based on the user's identity or group membership. Like column masks, row filters are implemented as SQL functions and applied declaratively—the underlying data is not partitioned or duplicated.

Consider a scenario where sales data should be filtered by region: users on the EMEA team should see only European records, while the North America team should see only US and Canadian records. A row filter function implements this logic:

```
-- Create a row filter function based on regional access
CREATE OR REPLACE FUNCTION cat_dev.sandbox.region_filter(region_value STRING)
RETURNS BOOLEAN
RETURN CASE
  WHEN IS_ACCOUNT_GROUP_MEMBER('admins') THEN TRUE
  WHEN IS_ACCOUNT_GROUP_MEMBER('emea_team') AND region_value IN ('EU', 'UK',
'EMEA') THEN TRUE
  WHEN IS_ACCOUNT_GROUP_MEMBER('na_team') AND region_value IN ('US', 'CA', 'NA')
THEN TRUE
  ELSE FALSE
END;
```

Apply the filter to the table:

```
-- Apply row filter to the orders table
ALTER TABLE cat_dev.sandbox.orders_gold
SET ROW FILTER cat_dev.sandbox.region_filter ON (region);
```

**Note**

Calling `IS_ACCOUNT_GROUP_MEMBER()` directly inside a row filter or column mask UDF is identified as a performance anti-pattern in the official Azure Databricks documentation. Each invocation triggers a group membership lookup, which can significantly degrade query performance on large tables. For production deployments with frequent queries, consider caching group membership results or restructuring the access control logic using mapping tables. See the Databricks documentation on UDF best practices for ABAC policies for recommended patterns.

Now, when a member of the `EMEA_team` group queries the orders table, they see only rows where the region is EU, UK, or EMEA. Members of the `admins` group see all rows. The filter is enforced at the query engine level, so it applies regardless of which tool (notebooks, SQL editor, BI tools) accesses the table.

Row filters and column masks can be combined on the same table. A table might have a row filter that restricts records by department while simultaneously masking the salary column for non-HR users. This layered approach provides comprehensive protection without requiring separate copies of the data for each audience.

To remove a row filter:

```
ALTER TABLE cat_dev.sandbox.orders_gold
DROP ROW FILTER;
```

## Dynamic views for complex access patterns

Dynamic views offer an alternative approach to fine-grained access control that predates column masks and row filters. A dynamic view is a standard SQL view that uses built-in functions like `IS_ACCOUNT_GROUP_MEMBER()` and `SESSION_USER()` to conditionally transform or filter data based on the querying user's identity.

```
CREATE OR REPLACE VIEW cat_dev.sandbox.v_customer_360 AS
SELECT
  customer_id,
  first_name,
  last_name,
  -- Mask email for non-PII users
  CASE
    WHEN IS_ACCOUNT_GROUP_MEMBER('pii_viewers') THEN email
```

```
    ELSE REGEXP_REPLACE(email, '(^[^@]{2})[^@]*', '$1****')
  END AS email,
  -- Mask phone for non-PII users
  CASE
    WHEN IS_ACCOUNT_GROUP_MEMBER('pii_viewers') THEN phone
    ELSE CONCAT('***-***-', RIGHT(phone, 4))
  END AS phone,
  -- Show exact CLV only to finance team
  CASE
    WHEN IS_ACCOUNT_GROUP_MEMBER('finance_team') THEN customer_lifetime_value
    WHEN customer_lifetime_value > 10000 THEN 'High'
    WHEN customer_lifetime_value > 1000 THEN 'Medium'
    ELSE 'Low'
  END AS customer_lifetime_value,
  region,
  segment
FROM cat_dev.sandbox.customers_gold
-- Row-level filtering within the view
WHERE
  IS_ACCOUNT_GROUP_MEMBER('admins')
  OR region = (
    SELECT region FROM cat_dev.sandbox.user_region_mapping
    WHERE user_name = SESSION_USER()
  );
```

**Note**

This `CASE` expression intentionally changes the column's data type. When the caller belongs to finance_team, the original `DECIMAL` value is returned; otherwise, a categorical string label ('High', 'Medium', or 'Low') is returned. Databricks resolves this mixed-type `CASE` to `STRING`, so all callers see a string column. This is a common dynamic view pattern where structural changes to the output provide an additional layer of data abstraction beyond simple masking.

Dynamic views are particularly useful when the access control logic involves joins to external mapping tables, complex conditional expressions that go beyond what a single function can express, or when you need to transform the output structure (renaming or combining columns) based on the audience.

The trade-off is that dynamic views require you to grant users access to the view while revoking direct access to the underlying table. With column masks and row filters, users access the table directly, and the protections are applied transparently. Choose the approach that best fits your organizational workflow: column masks and row filters for straightforward PII protection, and dynamic views for complex multi-table access logic.

**Choosing between masks, filters, and dynamic views**: Column masks and row filters were introduced as generally available features in Unity Catalog to provide a simpler, more declarative approach than dynamic views. For new implementations, prefer column masks and row filters for their simplicity and transparency. Use dynamic views when the access logic requires joins, complex expressions, or structural changes to the output that cannot be expressed in a single function.

## Working example: Securing a customer analytics table

Let us bring these concepts together with a complete example that applies multiple layers of fine-grained access control to a customer analytics table. This example uses customer data from our earlier chapters to demonstrate how to configure security controls that serve different user communities simultaneously.

First, set up the security functions in a dedicated schema:

```
-- Create a security schema for all masking and filter functions
CREATE SCHEMA IF NOT EXISTS cat_dev.sandbox;

-- Masking function: email addresses
CREATE OR REPLACE FUNCTION cat_dev.sandbox.mask_email(email_value STRING)
RETURNS STRING
RETURN CASE
  WHEN IS_ACCOUNT_GROUP_MEMBER('pii_viewers') THEN email_value
  ELSE CONCAT(LEFT(email_value, 2), '****@', SPLIT(email_value, '@')[1])
END;

-- Masking function: phone numbers (show last 4 digits)
CREATE OR REPLACE FUNCTION cat_dev.sandbox.mask_phone(phone_value STRING)
RETURNS STRING
RETURN CASE
  WHEN IS_ACCOUNT_GROUP_MEMBER('pii_viewers') THEN phone_value
  ELSE CONCAT('***-***-', RIGHT(REGEXP_REPLACE(phone_value, '[^0-9]', ''), 4))
END;

-- Masking function: monetary values (NULL for non-finance)
```

```
CREATE OR REPLACE FUNCTION cat_dev.sandbox.mask_revenue(revenue DECIMAL(10,2))
RETURNS DECIMAL(10,2)
RETURN CASE
  WHEN IS_ACCOUNT_GROUP_MEMBER('finance_team') THEN revenue
  ELSE NULL
END;

-- Row filter: regional access control
CREATE OR REPLACE FUNCTION cat_dev.sandbox.filter_by_region(region_value STRING)
RETURNS BOOLEAN
RETURN CASE
  WHEN IS_ACCOUNT_GROUP_MEMBER('global_admins') THEN TRUE
  WHEN IS_ACCOUNT_GROUP_MEMBER('emea_team')
    AND region_value IN ('EU', 'UK', 'EMEA') THEN TRUE
  WHEN IS_ACCOUNT_GROUP_MEMBER('na_team')
    AND region_value IN ('US', 'CA') THEN TRUE
  WHEN IS_ACCOUNT_GROUP_MEMBER('apac_team')
    AND region_value IN ('APAC', 'JP', 'AU') THEN TRUE
  ELSE FALSE
END;
```

Now apply these controls to the table:

```
-- Apply column masks
ALTER TABLE cat_dev.sandbox.customer_analytics_gold
ALTER COLUMN email SET MASK cat_dev.sandbox.mask_email;

ALTER TABLE cat_dev.sandbox.customer_analytics_gold
ALTER COLUMN phone SET MASK cat_dev.sandbox.mask_phone;

ALTER TABLE cat_dev.sandbox.customer_analytics_gold
ALTER COLUMN total_revenue SET MASK cat_dev.sandbox.mask_revenue;

-- Apply row filter
ALTER TABLE cat_dev.sandbox.customer_analytics_gold
SET ROW FILTER cat_dev.sandbox.filter_by_region ON (region);

-- Tag columns for data discovery
ALTER TABLE cat_dev.sandbox.customer_analytics_gold
  ALTER COLUMN email SET TAGS ('pii' = 'true', 'sensitivity' = 'high');
ALTER TABLE cat_dev.sandbox.customer_analytics_gold
```

```
  ALTER COLUMN phone SET TAGS ('pii' = 'true', 'sensitivity' = 'high');
ALTER TABLE cat_dev.sandbox.customer_analytics_gold
  SET TAGS ('data_classification' = 'confidential', 'contains_pii' = 'true');
```

The return type of the masking function must match (or be castable to) the data type of the masked column. Because revenue is `DECIMAL(10,2)`, the mask returns `NULL` for unauthorized users rather than a descriptive string. If you prefer categorical labels like 'High' or 'Medium', use a dynamic view (as shown earlier in this chapter) rather than a column mask, since views can freely change the output data type.

With these controls in place, the same table serves different audiences without any changes to their queries. An EMEA analyst running `SELECT * FROM cat_dev.sandbox.customer_analytics` sees only European customers, with emails showing as `ja****@company.com`, phone numbers as `***-***-1234`, and revenue as `NULL`. A finance team member in the `pii_viewers` and `finance_team` groups sees the same records, but with full email, phone, and exact revenue values. A global admin sees all regions with full data.

# Auditing and monitoring data access

Securing data and implementing access controls are only effective if you can verify that they work as intended and detect when something goes wrong. Databricks provides comprehensive auditing through system tables—Delta Lake tables within Unity Catalog that record every action taken in your workspace and across your account. These tables are your primary tool for answering the questions auditors ask: who accessed what data, when, and what they did with it.

## Unity catalog system tables

System tables are located in the `system` catalog, which is available in every Unity Catalog-enabled workspace. The most important tables for security and compliance auditing are found in two schemas: `system.access` (for audit logs) and `system.information_schema` (for catalog metadata).

Before querying system tables, an account admin must enable them. Navigate to the admin console, select **System Tables**, and enable the schemas you need. Once enabled, system tables begin recording new events. The retention period for most system tables is 365 days, meaning data is kept for up to 1 year after it is recorded. However, this does not mean that enabling system tables gives you access to historical data from before enabling. To maximize your audit history, enable system tables as early as possible in your workspace lifecycle.

The `system.access.audit` table is the centerpiece of Databricks auditing. Every API call, data access, cluster operation, and administrative action generates an audit event that is recorded in

this table. Each row includes the timestamp, the identity of the user or service principal, the action performed, the workspace where it occurred, and a JSON payload with action-specific details.

The following query retrieves recent data access events for a specific table:

```
-- Find all access events for a specific table in the last 7 days
SELECT
  event_time,
  user_identity.email AS user_email,
  action_name,
  request_params.full_name_arg AS table_name,
  source_ip_address,
  user_agent
FROM system.access.audit
WHERE action_name IN ('getTable', 'commandSubmit',
'generateTemporaryTableCredential')
  AND request_params.full_name_arg = 'cat_dev.sandbox.customers_gold'
  AND event_date >= CURRENT_DATE() - INTERVAL 7 DAYS
ORDER BY event_time DESC;
```

The `generateTemporaryTableCredential` action is particularly important; it fires when a user's query actually reads from or writes to a table's underlying storage. This is the audit event that proves data was accessed, not merely that metadata was queried.

## Building an audit dashboard

Raw audit logs are useful for investigation, but proactive monitoring requires summarized views and alerts. The following queries demonstrate common audit patterns that organizations build into monitoring dashboards.

`Failed access attempts` may indicate unauthorized access attempts or misconfigured permissions:

```
-- Failed access attempts in the past 24 hours
SELECT
  event_time,
  user_identity.email AS user_email,
  action_name,
  request_params,
  response.status_code,
  response.error_message
```

```
FROM system.access.audit
WHERE response.status_code >= 400
  AND event_date >= CURRENT_DATE() - INTERVAL 1 DAY
ORDER BY event_time DESC;
```

`Permission changes` should be tracked to maintain an audit trail of who granted or revoked access:

```
-- Track all permission changes (grants and revokes)
SELECT
  event_time,
  user_identity.email AS changed_by,
  action_name,
  request_params.securable_type AS object_type,
  request_params.securable_full_name AS object_name,
  request_params.changes AS permission_changes
FROM system.access.audit
WHERE action_name IN ('updatePermissions', 'setPermissions')
  AND event_date >= CURRENT_DATE() - INTERVAL 30 DAYS
ORDER BY event_time DESC;
```

**Data access frequency by user** helps identify unusual patterns that might warrant investigation:

```
-- Data access patterns: who accessed which tables, how often
SELECT
  user_identity.email AS user_email,
  request_params.full_name_arg AS table_name,
  COUNT(*) AS access_count,
  MIN(event_time) AS first_access,
  MAX(event_time) AS last_access
FROM system.access.audit
WHERE action_name = 'generateTemporaryTableCredential'
  AND event_date >= CURRENT_DATE() - INTERVAL 30 DAYS
GROUP BY user_identity.email, request_params.full_name_arg
ORDER BY access_count DESC;
```

**Tip**

System tables are themselves Delta Lake tables. You can query them with any tool that connects to Databricks SQL, including BI platforms like Power BI. Build your audit

dashboards in Databricks SQL or export the data to your organization's SIEM (Security Information and Event Management) system for correlation with other security events.

## Monitoring with Azure diagnostic settings

In addition to Unity Catalog system tables, Azure Databricks integrates with Azure Monitor through diagnostic settings. This allows you to stream workspace operational logs and metrics to a Log Analytics workspace, Azure Storage account, or Event Hubs for integration with enterprise monitoring tools.

To configure diagnostic settings, navigate to your Databricks workspace in the Azure portal, select **Diagnostic settings**, and create a new setting. You can enable the following log categories: `dbfs` (file system operations), `clusters` (cluster lifecycle events), `accounts` (user and group management), jobs (job run events), `notebook` (notebook operations), `ssh` (SSH access to clusters), `workspace` (workspace-level operations), `secrets` (secret access events), `sqlPermissions` (SQL-level permission changes), and `unityCatalog` (Unity Catalog audit events).

The following KQL query in Log Analytics identifies users who accessed sensitive tables outside business hours:

```
DatabricksUnityCatalog
| where ActionName == "generateTemporaryTableCredential"
| where RequestParams has "sandbox.customers_gold"
| extend HourOfDay = datetime_part("hour", TimeGenerated)
| where HourOfDay< 7 or HourOfDay> 19
| project TimeGenerated, Identity, ActionName, RequestParams, SourceIPAddress
| order by TimeGenerated desc
```

Combining system tables (for detailed data access auditing) with Azure Monitor (for infrastructure and operational monitoring) provides comprehensive observability across your Databricks environment.

## Setting up alerts

Monitoring is only useful if anomalies trigger timely notifications. Databricks SQL Alerts allow you to define threshold-based alerts on any query, including queries against system tables. For example, you can create an alert that fires when the number of failed access attempts in the past hour exceeds a threshold, when a specific sensitive table is accessed outside of scheduled pipeline windows, or when a new user is granted admin privileges.

To create an alert in Databricks SQL, first save a query that returns the metric you want to monitor. Then create an alert on that query, specifying the column to monitor, the threshold condition, and the notification destination (email, Slack webhook, or PagerDuty). Set the alert to refresh on a schedule that matches your security requirements—every 5 minutes for critical alerts, hourly for informational ones.

For enterprise-scale alerting, stream diagnostic logs to Azure Monitor and use Azure Monitor alert rules, which support more sophisticated conditions, action groups, and integration with incident management platforms.

# Ensuring compliance with regulatory standards

Data governance is not only about controlling access - it also encompasses meeting regulatory obligations that dictate how personal and sensitive data must be handled. Two of the most widely applicable regulations are the **General Data Protection Regulation** (**GDPR**), which governs personal data of EU residents, and the **Health Insurance Portability and Accountability Act** (**HIPAA**), which protects healthcare information in the United States. While the specifics differ, both require similar technical capabilities: knowing where sensitive data lives, controlling who can access it, retaining or deleting data in accordance with defined policies, and demonstrating compliance through auditable records.

This section focuses on practical implementation patterns in Databricks. It does not constitute legal advice—always work with your legal and compliance teams to interpret regulatory requirements for your specific context.

## GDPR: Right to erasure with Delta Lake

The GDPR grants individuals the right to request the deletion of their personal data (Article 17, commonly known as the "right to be forgotten"). In traditional data lakes built on immutable files, fulfilling deletion requests was extremely difficult—you would need to rewrite entire Parquet files to remove specific records. Delta Lake's support for row-level operations makes this tractable.

When a data subject exercises their right to erasure, you need to delete their personal data from every table where it appears. Delta Lake's `DELETE` command handles this efficiently:

```
-- Delete a specific customer's data across all relevant tables
-- Step 1: Delete from Gold layer
DELETE FROM cat_dev.sandbox.customers_gold
WHERE customer_id = 'CUST-12345';

-- Step 2: Delete from Silver layer
DELETE FROM cat_dev.sandbox.customers_silver
```

```
WHERE customer_id = 'CUST-12345';

-- Step 3: Delete from Bronze layer
DELETE FROM cat_dev.sandbox.customers_bronze
WHERE customer_id = 'CUST-12345';
```

However, `DELETE` in Delta Lake creates a new version of the table—the old data still exists in previous versions (time travel) until you explicitly remove it with `VACUUM`. To fully erase personal data, you must also vacuum the table to remove old data files:

```
-- Remove old versions that contain the deleted records
-- Set retention to 0 hours for immediate erasure (use with caution)
SET spark.databricks.delta.retentionDurationCheck.enabled = false;

VACUUM cat_dev.sandbox.customers_gold RETAIN 0 HOURS;
VACUUM cat_dev.sandbox.customers_silver RETAIN 0 HOURS;
VACUUM cat_dev.sandbox.customers_bronze RETAIN 0 HOURS;

SET spark.databricks.delta.retentionDurationCheck.enabled = true;
```

**Note**

**Warning**: Setting the retention period to 0 hours for `VACUUM` removes all historical data immediately. This makes time travel unavailable for those tables. In practice, coordinate erasure requests into a batch process that runs periodically (such as weekly), then vacuum with a short retention window. This balances erasure obligations with operational needs for time travel during incident response.

For organizations processing many erasure requests, consider creating a centralized erasure workflow. Maintain an erasure request log table that tracks incoming requests, and implement a scheduled job that processes pending requests across all relevant tables:

```
# Erasure workflow pattern
erasure_requests = spark.sql("""
    SELECT request_id, customer_id, request_date
    FROM cat_dev.governance.erasure_requests
    WHERE status = 'pending'
""")

# List of tables containing personal data
```

```
pii_tables = [
    "cat_dev.sandbox.customers_gold",
    "cat_dev.sandbox.customers_silver",
    "cat_dev.sandbox.customers_bronze",
    "cat_dev.sandbox.orders_gold"  -- contains customer_id FK
]

for request in erasure_requests.collect():
    customer_id = request.customer_id
    for table in pii_tables:
        spark.sql(f"DELETE FROM {table} WHERE customer_id = '{customer_id}'")

    # Mark request as completed
    spark.sql(f"""
        UPDATE cat_dev.governance.erasure_requests
        SET status = 'completed', completed_date = CURRENT_TIMESTAMP()
        WHERE request_id = '{request.request_id}'
    """)
```

Fulfilling erasure requests requires knowing where personal data resides. Unity Catalog's tagging capability helps classify tables and columns that contain personal data.

```
-- Tag columns containing PII
ALTER TABLE cat_dev.sandbox.customers_gold
  ALTER COLUMN email SET TAGS ('pii' = 'true', 'gdpr_category' = 'contact_info');

ALTER TABLE cat_dev.sandbox.customers_gold
  ALTER COLUMN phone SET TAGS ('pii' = 'true', 'gdpr_category' = 'contact_info');

ALTER TABLE cat_dev.sandbox.customers_gold
  ALTER COLUMN date_of_birth SET TAGS ('pii' = 'true', 'gdpr_category' =
'demographic');

-- Tag the table itself with its data classification level
ALTER TABLE cat_dev.sandbox.customers_gold
  SET TAGS ('data_classification' = 'confidential', 'contains_pii' = 'true');
```

Once tags are applied, you can query the information schema to discover all PII-tagged assets across your entire catalog:

```
-- Discover all columns tagged as PII
SELECT
  catalog_name,
  schema_name,
  table_name,
  column_name,
  tag_name,
  tag_value
FROM system.information_schema.column_tags
WHERE tag_name = 'pii' AND tag_value = 'true'
ORDER BY catalog_name, schema_name, table_name;
```

This query serves as the foundation of your data inventory, a regulatory requirement under both the GDPR (Article 30, Records of Processing Activities) and many other privacy frameworks. Keeping tags up to date ensures that, when a new table containing personal data is created, it is immediately discoverable and subject to the appropriate access controls.

## HIPAA: Protecting health information

HIPAA requires covered entities and their business associates to implement safeguards for **Protected Health Information** (**PHI**). The technical safeguards map directly to the capabilities covered in this chapter: access controls (implemented through Unity Catalog RBAC, column masks, and row filters), audit controls (provided by system tables and Azure diagnostic logs), integrity controls (ensured by Delta Lake's ACID transactions), and transmission security (enforced by TLS encryption on all connections).

To use Azure Databricks for HIPAA-regulated workloads, you must ensure that your Databricks deployment meets the architectural requirements. Microsoft offers a **Business Associate Agreement** (**BAA**) for Azure Databricks, which is required to process PHI. You must request this through your Microsoft account team or the Azure portal's compliance settings.

Beyond the BAA, HIPAA workloads in Databricks should follow a defense-in-depth approach that combines the controls discussed throughout this chapter. Deploy the workspace with VNet injection and Private Link to ensure PHI never traverses the public internet. Enable customer-managed keys for at-rest encryption so your organization controls the encryption lifecycle. Apply column masks to PHI fields (diagnosis codes, treatment information, patient identifiers) so that only authorized clinical or compliance roles can see unmasked values. Implement row filters to enforce the minimum necessary standard—a key HIPAA principle requiring access to only the PHI needed for a specific task. Enable comprehensive audit logging

through both system tables and Azure diagnostic settings, and retain audit logs for at least six years (the HIPAA retention requirement for security-related documentation).

The following example demonstrates a HIPAA-aligned table configuration:

```
-- PHI masking function for patient identifiers
CREATE OR REPLACE FUNCTION cat_dev.sandbox.mask_patient_id(patient_id STRING)
RETURNS STRING
RETURN CASE
  WHEN IS_ACCOUNT_GROUP_MEMBER('clinical_staff')
    OR IS_ACCOUNT_GROUP_MEMBER('compliance_officers') THEN patient_id
  ELSE CONCAT('***-', RIGHT(patient_id, 4))
END;

-- PHI masking function for diagnosis codes
CREATE OR REPLACE FUNCTION cat_dev.sandbox.mask_diagnosis(diagnosis STRING)
RETURNS STRING
RETURN CASE
  WHEN IS_ACCOUNT_GROUP_MEMBER('clinical_staff') THEN diagnosis
  ELSE 'REDACTED'
END;

-- Apply masks to the clinical encounters table
ALTER TABLE cat_dev.sandbox.clinical_encounters
ALTER COLUMN patient_id SET MASK cat_dev.sandbox.mask_patient_id;

ALTER TABLE cat_dev.sandbox.clinical_encounters
ALTER COLUMN primary_diagnosis SET MASK cat_dev.sandbox.mask_diagnosis;

-- Tag columns as PHI for discovery
ALTER TABLE cat_dev.sandbox.clinical_encounters
  ALTER COLUMN patient_id SET TAGS ('phi' = 'true', 'hipaa_category' =
'patient_identifier');
ALTER TABLE cat_dev.sandbox.clinical_encounters
  ALTER COLUMN primary_diagnosis SET TAGS ('phi' = 'true', 'hipaa_category' =
'clinical_data');
```

## Building a compliance framework

Whether you are addressing GDPR, HIPAA, SOC 2, PCI DSS, or industry-specific regulations, the implementation pattern follows a common structure. First, *classify* your data using Unity Catalog tags to identify what data is sensitive and what regulations apply. Second, *protect*

sensitive data using column masks, row filters, and RBAC to enforce the principle of least privilege. Third, *audit* all access through system tables and diagnostic logs, retaining records for the period required by your regulatory framework. Fourth, *automate* compliance workflows —erasure processing, access reviews, and policy enforcement—so they are repeatable and auditable rather than dependent on manual effort.

*Table 11.2* maps common regulatory requirements to the Databricks capabilities that address them:

| Requirement | GDPR | HIPAA | Databricks Capability |
|---|---|---|---|
| **Access control** | Art. 32 | §164.312(a) | Unity Catalog RBAC, column masks, row filters |
| **Audit logging** | Art. 30 | §164.312(b) | System tables, Azure diagnostic logs |
| **Encryption at rest** | Art. 32 | §164.312(a)(2)(iv) | Azure-managed or customer-managed keys |
| **Encryption in transit** | Art. 32 | §164.312(e) | TLS 1.2+ (default) |
| **Data minimization** | Art. 5(1)(c) | Minimum necessary | Row filters, dynamic views |
| **Right to erasure** | Art. 17 | N/A | Delta Lake DELETE + VACUUM |
| **Data inventory** | Art. 30 | §164.310(d) | Unity Catalog tags, information schema |
| **Breach notification** | Art. 33-34 | §164.404-408 | Audit alerts, Azure Monitor integration |

*Table 11.2: Mapping regulatory requirements to Databricks capabilities*

**Note**

This table is a simplified mapping for reference. Your compliance team should conduct a thorough gap analysis for your specific regulatory context and data processing activities.

# Data lineage and impact analysis

Understanding how data flows through your platform, from raw ingestion to final consumption, is essential for both governance and operational efficiency. When a source system changes its schema, you need to know which downstream tables and reports will be affected. When an analyst questions a metric, trace it back to its source. Unity Catalog provides automated data lineage tracking that captures these relationships without requiring you to maintain documentation manually.

## How the Unity Catalog captures lineage

Unity Catalog automatically tracks lineage whenever data is read from or written to Unity Catalog-managed tables. The lineage graph records table-level and column-level relationships: which tables were read as inputs (upstream lineage) and which tables were produced as outputs (downstream lineage). This happens transparently as part of query execution; no additional configuration or instrumentation is required.

Lineage is captured for operations executed through Databricks SQL, notebook commands, Lakeflow Spark Declarative Pipelines (covered in *Chapter 7: Automating Data Pipelines with Lakeflow Spark Declarative Pipelines*), and Databricks Jobs. It covers both direct lineage (Table A is used to create Table B) and transitive lineage (Table A feeds Table B, which feeds Table C).

To view lineage in the Databricks UI, navigate to any table in the Catalog Explorer and click the **Lineage** tab. The UI displays an interactive graph showing upstream and downstream dependencies. You can expand the graph to trace data flows multiple levels deep and click on individual connections to see column-level mappings.

## Querying lineage programmatically

For automated governance workflows, you can query lineage data through the Unity Catalog REST API or through system tables. The `system.access.table_lineage` and `system.access.column_lineage` tables expose lineage relationships as queryable data.

The following query identifies all downstream consumers of a specific table:

```
-- Find all tables that depend on a source table
SELECT DISTINCT
  target_table_catalog,
  target_table_schema,
  target_table_name,
  target_type,
  event_time
FROM system.access.table_lineage
```

```
WHERE source_table_catalog = 'cat_dev'
  AND source_table_schema = 'sandbox'
  AND source_table_name = 'customers_silver'
  AND event_date >= CURRENT_DATE() - INTERVAL 30 DAYS
ORDER BY target_table_catalog, target_table_schema, target_table_name;
```

Column-level lineage is available in the `system.access.column_lineage` table:

```
-- Trace which source columns feed into a specific Gold table column
SELECT
  source_table_full_name,
  source_column_name,
  target_column_name,
  event_time
FROM system.access.column_lineage
WHERE target_table_catalog = 'cat_dev'
  AND target_table_schema = 'sandbox'
  AND target_table_name = 'customer_360'
  AND target_column_name = 'customer_lifetime_value'
  AND event_date >= CURRENT_DATE() - INTERVAL 30 DAYS;
```

This query answers the question "where does the `customer_lifetime_value` metric come from?"—a common question during data quality investigations and audit reviews.

## Impact analysis of schema changes

One of the most practical applications of lineage is impact analysis. Before changing a table's schema (renaming a column, changing a data type, or dropping a table), you should understand which downstream processes and consumers will be affected.

The following query supports a pre-change impact assessment:

```
-- Impact analysis: what breaks if we modify the silver.orders table?
SELECT
  target_table_catalog || '.' || target_table_schema || '.' || target_table_name
AS affected_table,
  COUNT(DISTINCT target_column_name) AS affected_columns,
  MAX(event_time) AS last_data_flow
FROM system.access.column_lineage
WHERE source_table_catalog = 'cat_dev'
  AND source_table_schema = 'sandbox'
  AND source_table_name = 'orders_silver'
```

```
  AND event_date >= CURRENT_DATE() - INTERVAL 30 DAYS
GROUP BY target_table_catalog, target_table_schema, target_table_name
ORDER BY last_data_flow DESC;
```

This query returns all tables that have consumed data from `sandbox.orders_silver` in the past 30 days, along with the number of columns affected and the most recent data flow event. Armed with this information, you can notify the owners of affected tables, coordinate the timing of schema migrations, and prevent unexpected pipeline failures.

## Lineage for compliance

Data lineage also serves a compliance function. When regulators ask, "Where does this piece of personal data flow through your systems?", you can answer that question with a lineage query rather than manually tracing code. Combined with the PII tags discussed earlier, you can find every table that derives from a PII-tagged source:

```
-- Find all downstream tables that receive data from PII-tagged columns
SELECT DISTINCT
  cl.target_table_catalog || '.' || cl.target_table_schema || '.' ||
cl.target_table_name AS downstream_table,
  cl.source_column_name AS pii_source_column,
  cl.target_column_name AS target_column
FROM system.access.column_lineage cl
JOIN system.information_schema.column_tags ct
  ON cl.source_table_catalog = ct.catalog_name
  AND cl.source_table_schema = ct.schema_name
  AND cl.source_table_name = ct.table_name
  AND cl.source_column_name = ct.column_name
WHERE ct.tag_name = 'pii'
  AND ct.tag_value = 'true'
  AND cl.event_date >= CURRENT_DATE() - INTERVAL 30 DAYS
ORDER BY downstream_table;
```

This query is the technical foundation of a **Data Protection Impact Assessment (DPIA)** - a GDPR requirement for high-risk processing activities. By combining tagging, lineage, and access controls, you create a governance framework where sensitive data is classified, its flow is tracked, and its access is controlled and audited at every point.

## Secure data sharing with delta sharing

As data platforms mature, the need to share data securely across organizational boundaries becomes critical. Whether you are sharing data with external partners, clients, or other

business units that operate in separate Databricks workspaces, you need a mechanism that maintains governance controls while enabling collaboration. Unity Catalog provides two approaches to data sharing: internal sharing across workspaces and external sharing through the open Delta Sharing protocol.

## Internal sharing across workspaces

Within the same Unity Catalog metastore, sharing data across workspaces is straightforward—all workspaces attached to the same metastore can access the same catalogs, schemas, and tables, subject to Unity Catalog permissions. This is the simplest sharing model and is appropriate when different teams operate in separate workspaces within the same organization.

For cases where you need more control over cross-workspace access, Unity Catalog supports share-named objects that define a read-only collection of tables, views, and other data assets that can be shared with specific recipients. A share acts as a curated window into your data: the provider decides which assets to include, and the recipient sees only the shared assets.

```
-- Create a share containing curated Gold-layer tables
CREATE SHARE IF NOT EXISTS customer_insights_share;

-- Add specific tables to the share
ALTER SHARE customer_insights_share
ADD TABLE cat_dev.sandbox.customer_segments;

ALTER SHARE customer_insights_share
ADD TABLE cat_dev.sandbox.regional_sales_summary;

-- Grant access to a recipient (another workspace or external party)
GRANT SELECT ON SHARE customer_insights_share
TO RECIPIENT partner_analytics_team;
```

Note that tables with row filters or column masks applied cannot be included in a Delta Sharing share. If you need to share a filtered or masked view of your data, create a separate table or materialized view that applies the transformations, and share that derived table instead. This is an important constraint to consider when designing your data-sharing strategy alongside your access-control policies.

## External sharing with delta sharing

Delta Sharing is an open protocol developed by Databricks for secure data sharing across organizations, regardless of the recipient's compute platform. Recipients can consume shared

data using Databricks, Apache Spark, pandas, Power BI, or any tool that supports the Delta Sharing protocol—they do not need a Databricks workspace.

From a security perspective, Delta Sharing provides several important protections. The data provider retains full control: shared data remains in the provider's storage, and the provider can revoke access at any time. Recipients receive time-limited credentials (tokens) that grant read-only access to the specific assets in the share. All access is logged in the provider's audit trail, providing full visibility into who accessed shared data and when. No data is copied to the recipient's environment—they query the data directly from the provider's storage, which means the provider maintains a single source of truth and can enforce updates or corrections instantly.

To set up external sharing, create a recipient object with authentication credentials:

```
-- Create an external recipient
CREATE RECIPIENT IF NOT EXISTS external_partner
COMMENT 'Analytics team at partner organization';

-- The above command generates an activation link
-- Share this link securely with the recipient
-- They use it to download their credentials file
```

After creating the recipient, add them to your share and they can begin querying the data using their preferred tool. Monitor their access through the system tables:

```
-- Monitor external sharing access
SELECT
  event_time,
  user_identity.email AS recipient_identity,
  action_name,
  request_params
FROM system.access.audit
WHERE action_name LIKE '%deltaSharing%'
  AND event_date >= CURRENT_DATE() - INTERVAL 30 DAYS
ORDER BY event_time DESC;
```

**Tip**

When sharing data externally, always apply column masks and row filters to the shared tables before adding them to a share. Even though the recipient is trusted, the principle of data minimization dictates sharing only the data attributes that are necessary for the recipient's use case. Additionally, review shared assets periodically and remove tables from shares when they are no longer needed.

## Summary

In this chapter, we explored the security, compliance, and governance capabilities that transform a functional Databricks environment into an enterprise-grade data platform. We secured the infrastructure layer with VNet injection, Private Link, and customer-managed encryption keys, and hardened workspace settings through token management, IP access lists, and cluster policies. We implemented fine-grained access controls using column masks, row filters, and dynamic views to protect sensitive data without duplicating tables or modifying application code and demonstrated these concepts with a working example that serves multiple user communities from a single governed table.

We built audit and monitoring capabilities on top of Unity Catalog system tables and Azure diagnostic settings, enabling both reactive investigation and proactive alerting. We translated regulatory requirements from GDPR and HIPAA into concrete implementation patterns—data classification with tags, erasure workflows with Delta Lake, and PHI protection with layered access controls. We used Unity Catalog's automated lineage tracking to understand data flows, assess the impact of changes, and demonstrate compliance through transparent data provenance. Finally, we explored secure data sharing through Unity Catalog shares and the Delta Sharing protocol, extending governance controls to cross-organizational collaboration.

The key takeaway is that security and governance are not bolted onto a completed platform; they are architectural decisions that should be made early and enforced consistently. The controls discussed in this chapter work best when they are part of your standard development workflow: every new table is classified with tags, every sensitive column is masked, every access pattern is logged and monitored, and every sharing arrangement is governed through Unity Catalog.

In the next chapter, we will explore how Azure Databricks supports machine learning, AutoML, and generative AI workflows, bringing together the full spectrum of data engineering and data science capabilities on a single, governed platform.

## Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to `packtpub.com/unlock`). Search for this book by name, confirm the edition, and then follow the steps on the page.

*Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.*

# 12

# Machine Learning and AI on Databricks

Databricks is not just a data platform. It is also a complete environment for machine learning and AI. In this chapter, you will learn how to track experiments, build models, and deploy them to production without leaving the platform.

We will start with the traditional machine learning tools: MLflow for experiment tracking, Feature Store for reusable features, and AutoML for building models with minimal code. Then we will look at Genie, which lets business users ask questions in plain English.

In the second half, we will explore generative AI. You will learn how to search documents by meaning with Vector Search, build a RAG chatbot that answers from your company's data, and control LLM costs with AI Gateway. By the end, you will understand when to use each tool and how they fit together.

In this chapter, we will cover the following topics:

- Introduction to Databricks AI/ML innovation and capabilities
- AutoML for rapid model development
- Building AI applications with Databricks

## Technical Requirements

To follow the examples in this chapter, you need an Azure Databricks workspace with access to Unity Catalog. The workspace should be running Databricks Runtime 15.4 LTS ML or later, which comes with MLflow, PySpark, pandas, NumPy, and scikit-learn pre-installed.

For the generative AI sections, you also need access to Foundation Model APIs and a Vector Search endpoint. These are available on the Databricks Premium tier.

Some examples use the Databricks SDK for Python (`databricks-sdk`) and the Feature Engineering client (databricks-feature-engineering). Both are pre-installed on ML clusters.

This chapter is mostly conceptual. It covers the AI and ML tools available on Databricks, which are complementary areas for data engineers to understand. Each section explains what a tool does and when to use it. Find the solution in the *Chapter 12* folder in this book's GitHub repo. To access the repository, follow the steps in the *"Download the example code files"* section of the *Preface*.

# Introduction to Databricks AI/ML innovation and capabilities

Building machine learning systems has always been hard, not because the algorithms are difficult, but because the tools are disconnected. Data lives in one system, model training happens in another, and deployment requires yet another platform. As a result, data scientists spend more time moving data than building models, compliance and reproducibility become difficult to achieve, and the path from experiment to production takes months.

These challenges are common across organizations of all sizes, as the following diagram illustrates.

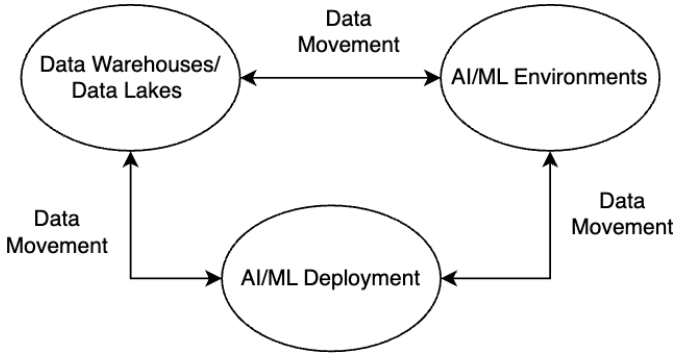



*Figure 12.1: The typical challenges with traditional AI/ML platforms.*

Databricks takes a different approach. It brings together data preparation, feature engineering, model training, and deployment on a single platform. Machine learning runs where the data already lives. The platform is built on widely used open-source tools such as MLflow, Delta Lake, and Apache Spark, and scales from notebook experiments to production systems without code changes.

## The Databricks AI/ML Stack

The Databricks AI/ML platform has two main paths. The first is traditional machine learning: training models on structured data to predict churn or forecast sales. The second is generative AI: building applications powered by large language models. Both paths share the same foundation and can work together.

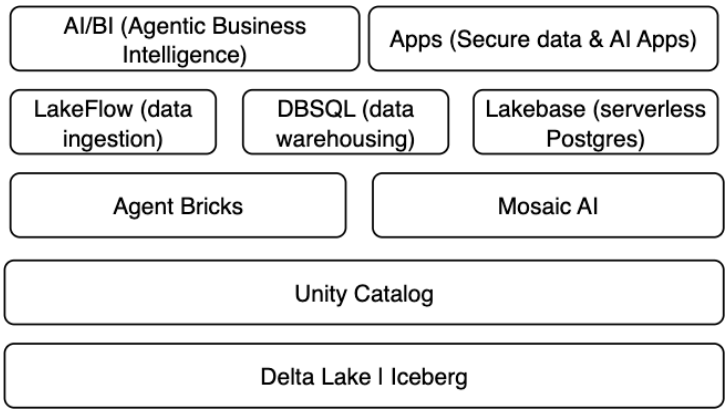



*Figure 12.2: The complete Databricks AI/ML stack.*

Both paths serve different roles at different skill levels. Business users interact with data through Genie, asking questions in plain English. Analysts use AutoML to build models with minimal code. Data scientists and AI engineers get full control with MLflow and the Mosaic AI Agent Framework. ML engineers deploy everything to production using Model Serving, with AI Gateway controlling costs for generative AI workloads.

Both paths share the same infrastructure underneath. Unity Catalog governs everything: data, features, models, vectors, prompts, and agents. MLflow tracks experiments for both traditional ML and generative AI. Model Serving deploys both ML models and AI agents. This shared foundation means you do not need to learn a completely new set of tools when you move from traditional ML to generative AI.

# ML and AI Capabilities Overview

Before we dive into each capability in detail, let us take a quick tour of what the platform offers. This will help you see the big picture and understand how the pieces connect.

## Traditional Machine Learning

Databricks provides a complete toolkit for the ML lifecycle. **MLflow** tracks every experiment and versions your models, giving you a clear audit trail from training to production. Before you start training, **Feature Store** lets you define reusable features that stay consistent across

environments, eliminating training-serving skew. If you are unsure which algorithm to use, **AutoML** will benchmark multiple approaches on your data and surface the best one, complete with a generated notebook you can inspect and extend. Once in production, **Lakehouse Monitoring** watches for model drift so you can catch degradation before it affects your users.

## Generative AI Platform: Mosaic AI

On the generative AI side, **Genie** lets business users query data in plain English, while **Agent Bricks** provides a low-code visual interface for prototyping chatbots. When you need full control, the **Mosaic AI Agent Framework** supports production RAG applications and multi-agent systems. Behind the scenes, **Vector Search** finds documents by meaning rather than keywords, and **Foundation Model APIs** give you access to models like Llama 3, Claude, and Mistral through pre-configured endpoints. **AI Gateway** acts as the control point for all LLM usage, tracking costs, and logging every interaction. **MLflow** extends to generative AI as well, providing tracing, LLM-based evaluation, and prompt management. Finally, **Model Serving** deploys both ML models and AI agents as REST APIs with automatic scaling.

## Integration and Governance

What makes this platform powerful is not a single tool. It is how they all work together. Unity Catalog is the layer that connects everything.

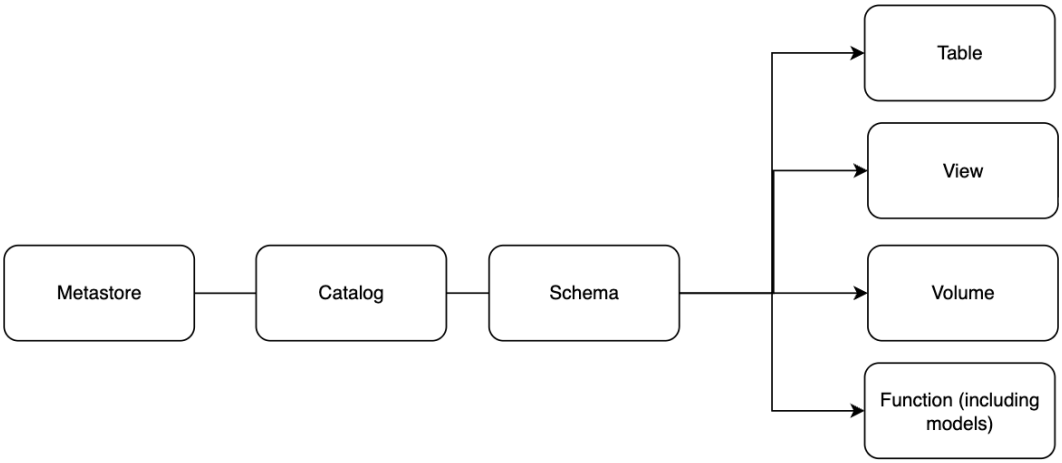



*Figure 12.3: The Unity Catalog governance structure.*

**Unity Catalog** manages many types of assets in one place. It governs data tables with your raw data, feature tables with engineered features, ML models, vector indexes for document search, prompt templates, and AI agents. All of these follow the same permission and access control rules.

Because everything lives in Unity Catalog, governance is automatic. Permissions control who can access each asset. Lineage tracking shows how data flows from raw tables through features, models, and predictions. Audit logs record every action for compliance.

### Lineage Flow Example

Here is how lineage works for a typical ML workflow. Raw customer data sits in a Delta table. A pipeline transforms it into features, which are stored in the Feature Store. A data scientist trains a churn model using those features and logs it in MLflow. The best version gets deployed to Model Serving. Unity Catalog tracks every step. Generative AI workflows follow the same pattern: documents are stored in a Delta table, Vector Search creates embeddings, a RAG agent uses those embeddings to answer questions, and Unity Catalog tracks the full chain.

### Key Integration Points

A few key connections tie everything together. MLflow tracks experiments for both traditional ML and generative AI. Model Serving deploys both ML models and AI agents through the same infrastructure. AI Gateway governs all LLM calls, whether they come from agents, fine-tuning jobs, or evaluation runs. Unity Catalog governs all assets across both paths. And monitoring works for both: drift detection for ML models, LLM judges for generative AI applications.

## MLflow: ML Experiment Tracking and Model Management

MLflow is the most widely used open source platform for managing the machine learning lifecycle. In this section, you will learn how it tracks experiments, version models, and helps you deploy them to production.

### What is MLflow?

Every data scientist faces the same problem. Over the course of a project, you train dozens or even hundreds of models. Each uses different algorithms, settings, and features. Without proper tracking, you quickly lose track of what worked and why.

MLflow solves this by automatically tracking everything. It records the parameters, metrics, and model files for every experiment you run. It stores models in a registry with version history. And it helps you deploy the best model to production.

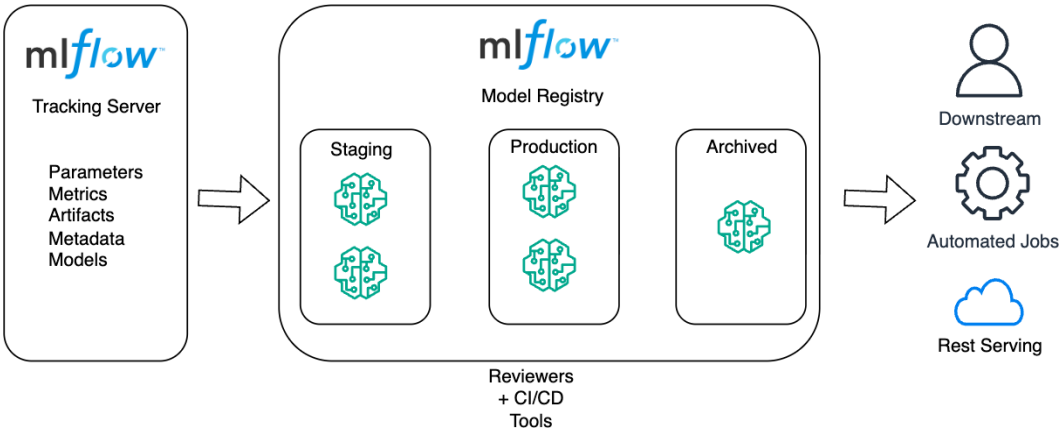



*Figure 12.4: How MLflow fits into the machine learning workflow.*

MLflow is open source and has become the industry standard for ML experiment tracking, with over 200 million annual downloads. It works across different ML frameworks and cloud platforms. On Databricks, MLflow is pre-installed and deeply integrated with Unity Catalog for governance.

## Core Concepts

MLflow is built around two main ideas: experiment tracking and the model registry. Let us look at each one.

### Experiment Tracking

An experiment is a collection of training runs that share a common goal. For example, you might create an experiment called `customer_churn_models`. Within that experiment, each run represents one attempt to build a model. Run 1 might use a Random Forest with 100 trees. Run 2 might use XGBoost with a depth of 5. Each run records what you tried and how well it worked. To make this concrete, *Table 12.1* shows what an experiment with multiple runs looks like side by side.

| Run | Algorithm | Parameters | Accuracy | Status |
|---|---|---|---|---|
| **Run 1** | RandomForest | n_trees=100 | 0.85 | Completed |
| **Run 2** | RandomForest | n_trees=200 | 0.87 | Completed |
| **Run 3** | XGBoost | depth=5 | 0.89 | Best |

| Run | Algorithm | Parameters | Accuracy | Status |
|---|---|---|---|---|
| **Run 4** | Logistic Regression | default | 0.82 | Completed |

*Table 12.1: Example of an experiment with multiple training runs and their outcomes*

Each run automatically captures the parameters, metrics, and artifacts. You will see how this works in the code example that follows.

## Model Registry

Once you find a model that works well, you want to keep track of it. The model registry in Unity Catalog gives you version control for models. You register a model, and each new version gets a number. You can also assign aliases like "Champion" for the production model and "Challenger" for the model being tested.

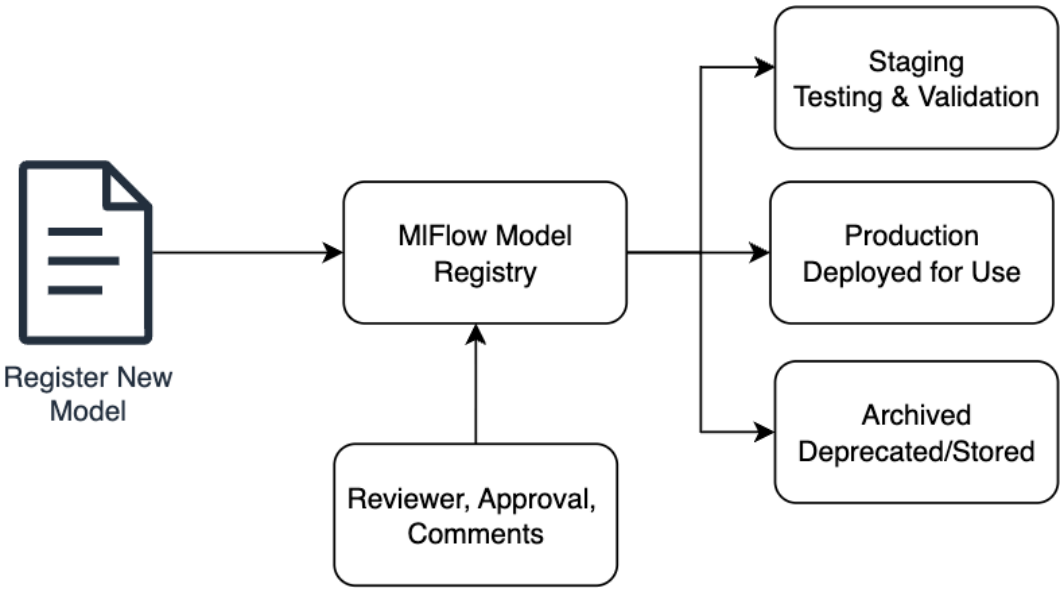



*Figure 12.5: The model registry structure*

The alias system makes deployment simple. Your application always calls the "Champion" model. When you want to promote a new version, you move the alias. No code changes needed. The application automatically picks up the new model.

## Try It: Tracking an Experiment

Here is a simple example. We train a model to predict customer churn and use MLflow to track everything.

```
import mlflow
from sklearn.ensemble import RandomForestClassifier
```

Here's how to import MLflow:

```
from sklearn.model_selection import train_test_split
# Load your data
df = spark.table("main.customers.features").toPandas()
X = df.drop("churned", axis=1)
y = df["churned"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
# Start tracking with MLflow
with mlflow.start_run(run_name="churn_rf_v1"):
    # Log parameters - what settings did you use?
    mlflow.log_param("n_estimators", 100)
    mlflow.log_param("max_depth", 10)
    mlflow.log_param("algorithm", "RandomForest")
    # Train the model
    model = RandomForestClassifier(n_estimators=100, max_depth=10)
    model.fit(X_train, y_train)
    # Log metrics - how well did it perform?
    accuracy = model.score(X_test, y_test)
    mlflow.log_metric("accuracy", accuracy)
    mlflow.log_metric("test_samples", len(X_test))
    # Save the model as an artifact
    mlflow.sklearn.log_model(model, "model")
    print(f"Model trained with accuracy: {accuracy:.4f}")
# MLflow automatically tracks everything
# View in Databricks UI: Experiments → churn_rf_v1
```

Let us walk through what this code does. First, it loads customer data and splits it into training and test sets. Then it starts an MLflow run and logs the settings it will use (algorithm, number of trees, depth). It trains the model and logs its performance on the test set. Finally, it saves the trained model as an artifact. Everything is now recorded. You can find this run in the Databricks UI under Experiments.

The best way to learn MLflow is to try it yourself. The official Databricks tutorial walks you through creating an experiment, logging parameters and metrics, and comparing runs in the UI. You can follow it step by step in your own workspace: `https://docs.databricks.com/en/mlflow/tracking.html`.

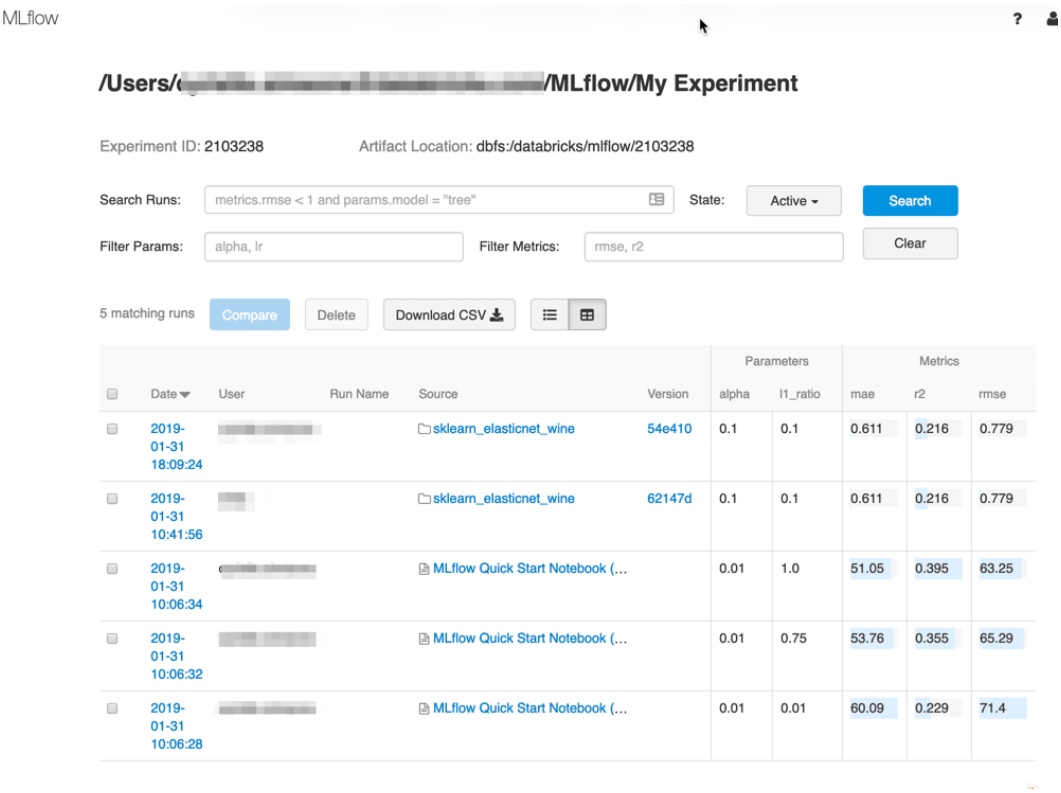



*Figure 12.6: The MLFlow experiment tracking UI.*

## Model Registry and Deployment

Once you have a model that performs well, the next step is to register it and make it available for production use.

### Registering a Model

```
import mlflow
# Register the best model to Unity Catalog
model_uri = "runs:/abc123def456/model"  # From your experiment run
model_name = "main.ml_models.customer_churn"
mlflow.register_model(
```

```
    model_uri=model_uri,
    name=model_name
)
# This creates version 1 of the model in Unity Catalog
# The model is now discoverable and governed
```

For a step-by-step guide to registering models, managing versions with aliases, and deploying to Model Serving, see the official documentation: `https://docs.databricks.com/en/mlflow/model-registry.html`.

Registering a model takes just two lines. You point to the run that produced it and give it a name in Unity Catalog.

## Managing Model Versions with Aliases

Aliases let you manage which model version is in production without changing any application code. Instead of pointing to version 2 or version 3, your application points to the "Champion" alias. When a new model is ready, you move the alias to the new version.

```
from mlflow import MlflowClient
client = MlflowClient()
# Set the Champion alias to the production model
client.set_registered_model_alias(
    name="main.ml_models.customer_churn",
    alias="Champion",
    version=2  # Version 2 is now production
)
# Set a Challenger alias for A/B testing
client.set_registered_model_alias(
    name="main.ml_models.customer_churn",
    alias="Challenger",
    version=3  # Version 3 is being tested
)
# To promote: move the Champion alias to the new version
client.set_registered_model_alias(
    name="main.ml_models.customer_churn",
    alias="Champion",
    version=3  # Now version 3 serves production traffic
)
```

## Deploying to Model Serving

For a complete walkthrough of deploying models as REST APIs with automatic scaling, see: `https://docs.databricks.com/en/machine-learning/model-serving/index.html`

Model Serving turns your registered model into a REST API. Any application can call this API to get predictions. Here is how to create a serving endpoint.

```
from databricks.sdk import WorkspaceClient
from databricks.sdk.service.serving import EndpointCoreConfigInput,
ServedEntityInput
w = WorkspaceClient()
# Create a serving endpoint
w.serving_endpoints.create(
    name="churn_prediction_api",
    config=EndpointCoreConfigInput(
        served_entities=[
            ServedEntityInput(
                entity_name="main.ml_models.customer_churn",
                entity_version="2",
                workload_size="Small",
                scale_to_zero_enabled=True  # Save costs when idle
            )
        ]
    )
)
# The model is now available as a REST API
# POST /serving-endpoints/churn_prediction_api/invocations
```

Once the endpoint is running, any application can send data and receive predictions. Here is a simple example of calling the endpoint.

```
import requests
# Get predictions from the deployed model
response = requests.post(
    "https://<workspace>/serving-endpoints/churn_prediction_api/invocations",
    headers={"Authorization": f"Bearer {token}"},
    json={
        "dataframe_records": [
            {"total_spend": 1500, "months_active": 24, "support_tickets": 3}
        ]
    }
```

```
)
prediction = response.json()
# Returns: {"predictions": [0]}  # 0 = not churning
```

# Business Use Case Ideas

These patterns apply across industries. Consider financial services as an example. Banks face strict rules for credit scoring models. Regulators want a complete history of how each model was built, what data it used, and how it was tested. With MLFlow, every experiment is recorded automatically. When an auditor asks about a model, you can pull up the full history in minutes. The same tracking and governance workflow applies to healthcare model validation, retail A/B testing, and manufacturing predictive maintenance.

Now that you have seen how MLFlow works in practice, *Table 12.2* summarizes the key capabilities and when to use them.

| Capability | What It Does | When to Use |
|---|---|---|
| **Experiment Tracking** | Logs parameters, metrics, and artifacts | Every training run |
| **Model Registry** | Versions and governs models | Production models |
| **Aliases** | Labels for model lifecycle | Managing Champion/ Challenger |
| **Model Serving** | REST API deployment | Production inference |

*Table 12.2: Key MLflow capabilities and their typical use cases*

In the next section, we will explore Feature Store and how to build reusable features that ensure consistency across all your models.

# Feature Store: Centralized Feature Management

Features are the processed inputs to ML models, like "total spend in last 30 days" or "days since last purchase." In many organizations, different teams calculate the same features in different ways. This wastes time and creates inconsistency. When training and production use different feature definitions, a gap called train-serve skew can silently break your predictions.

Feature Store solves both problems. One team creates a feature, and all teams reuse it. The same features used in training are automatically served at inference time. On Databricks, any Delta table with a primary key can be a feature table. Unity Catalog governs access, and any scheduled job can automatically keep features up to date.

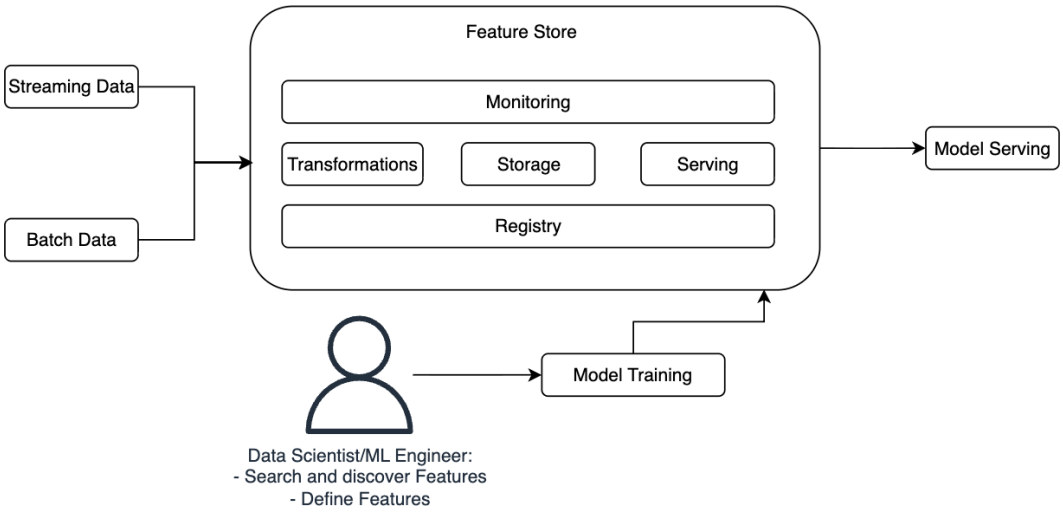



*Figure 12.7: The Feature Store diagram*

## Feature Store Core Concepts

A feature table is a Delta table where each row represents an entity (like a customer) and each column is a computed feature (like total spend or transaction count). The table has a primary key so features can be looked up by entity ID.

The typical flow looks like this. Raw data goes into a pipeline, which can be a notebook job, a DLT pipeline, or a streaming job. The pipeline computes the features and writes them to the feature table. When the raw data updates, the pipeline runs again, and the features stay fresh.

| customer_id | total_spend_30d | transaction_count_30d | days_since_last_purchase |
|---|---|---|---|
| **C001** | 1,250.00 | 15 | 3 |
| **C002** | 450.00 | 4 | 12 |

*Table 12.3: Example structure*

## Try It: Create and Use Features

Here is the basic pattern. You create features in a pipeline, then use the Feature Engineering Client to build training sets and make predictions. The code is simple because Feature Store handles the joins and lookups for you.

```
# 1. Create features (e.g., in a notebook job, DLT pipeline, or streaming job)
# Table: main.features.customer_features with customer_id, total_spend_30d, etc.
# 2. Training: join labels with features
from databricks.feature_engineering import FeatureEngineeringClient, FeatureLookup
fe = FeatureEngineeringClient()
labels_df = spark.table("main.customers.churn_labels")
training_set = fe.create_training_set(
    df=labels_df,
    feature_lookups=[FeatureLookup(table_name="main.features.customer_features",
lookup_key="customer_id")],
    label="churned"
)
training_df = training_set.load_df()
# Train the model
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier(n_estimators=100, max_depth=10)
training_pdf = training_df.toPandas()
X = training_pdf.drop("churned", axis=1)
y = training_pdf["churned"]
model.fit(X, y)
# Log the model with the training set so Feature Store knows the feature lookups
with mlflow.start_run():
    fe.log_model(
        model=model,
        artifact_path="model",
        flavor=mlflow.sklearn,
        training_set=training_set,
        registered_model_name="main.ml_models.churn_model"
    )
# 3. Inference: provide only primary key; features are looked up automatically
new_customers = spark.createDataFrame([("C005",), ("C006",)], ["customer_id"])
predictions = fe.score_batch(model_uri="models:/main.ml_models.churn_model/1",
df=new_customers)
```

Let us walk through what happens here. In step 1, a pipeline creates a feature table with columns such as `total_spend_30d` and `transaction_count_30d`. In step 2, you join your labels (which customers churned) with the features using just the `customer_id`. Feature Store handles the join automatically. In step 3, you score new customers by passing only their IDs. The Feature Store looks up the latest features for each customer and feeds them to the model.

To try Feature Store hands-on, the official tutorial walks you through creating feature tables, building training sets, and serving features for inference: `https://docs.databricks.com/en/machine-learning/feature-store/index.html`.

| Aspect | Manual | Feature Store |
|---|---|---|
| **Creation** | Each team builds independently | Create once, share |
| **Training/Serving** | Different code paths | Same features automatically |
| **Updates** | Manual ETL | Automated with any scheduled job |
| **Governance** | Manual docs | Unity Catalog |

*Table 12.4: Comparison of manual feature engineering versus using a Feature Store*

The three main benefits are reusability (create once, share across teams), consistency (no train-serve skew), and automation (Spark Declarative Pipelines) keeps features fresh when source data changes).

## Example Use Case

Consider a bank with five teams: loans, cards, insurance, investments, and fraud detection. All five teams need similar customer financial features, such as income, account balance, and transaction patterns. Without Feature Store, each team builds these features from scratch. That means five separate pipelines doing the same work, each with slightly different results. With a shared Feature Store, the data engineering team creates the features once. All five teams reuse them. A new model that used to take three months to build now takes one week because the features already exist.

## AutoML for rapid model development: What is AutoML?

Building a good model requires many decisions: which algorithm, what settings, and how to preprocess the data. AutoML automates all of this. You provide the data and tell it what to predict. It tries multiple algorithms, tunes each one, and returns the best model it finds.

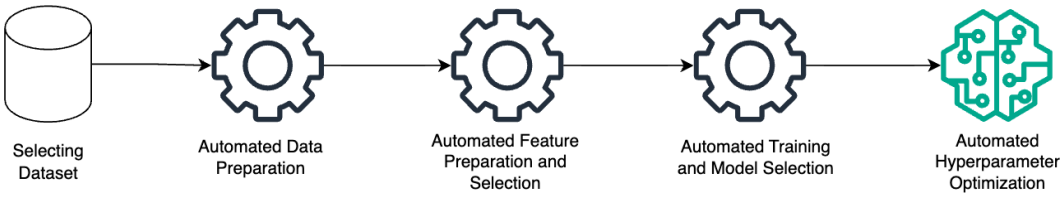



*Figure 12.8: The AutoML process*

What makes Databricks AutoML different from other AutoML tools is transparency. You do not get a black box. You get the trained model, a complete notebook showing all the code, a comparison of every trial it ran, and evaluation metrics with visualizations. If you want to customize anything, open the generated notebook and make changes.

## Core Concepts

Let us look at what AutoML can do and how it works.

### Supported Tasks

AutoML supports three task types: classification (predicting categories such as churn or fraud), regression (predicting numeric values such as house prices or weekly sales), and forecasting (predicting values over time, such as daily sales for 500 stores in a single run).

### The AutoML Process

When you start an AutoML run, it follows a clear process.

First, AutoML analyzes your data to understand its structure, data types, and potential issues. Then it generates a search space of algorithms and settings to try. It runs multiple trials, each with a different combination. After each trial, it evaluates the model on a holdout set that was not used for training. At the end, it ranks all trials by your chosen metric and returns the best one.

| Trial | Algorithm | Key Parameters | Metric (F1) | Duration |
|---|---|---|---|---|
| **1** | LightGBM | learning_rate=0.1, depth=6 | 0.847 | 45s |

| Trial | Algorithm | Key Parameters | Metric (F1) | Duration |
|---|---|---|---|---|
| **2** | XGBoost | learning_rate=0.05, depth=8 | 0.862 | 52s |
| **3** | Random Forest | n_estimators=200, depth=10 | 0.831 | 38s |
| **4** | LightGBM | learning_rate=0.05, depth=8 | 0.871 | 48s |
| **5** | XGBoost | learning_rate=0.03, depth=10 | 0.879 | 67s |

*Table 12.5: Example AutoML trial results*

## Try It: Forecasting with AutoML

Forecasting is one of the most common AutoML use cases. In this example, we forecast daily sales for multiple stores. The entire run takes just a few lines of code.

```
from databricks import automl
df = spark.table("main.sales.daily_store_sales")

summary = automl.forecast(
    dataset=df,
    target_col="sales",
    time_col="date",
    identity_col="store_id",
    horizon=30,
    timeout_minutes=30
)
# Access the best model
best_model_uri = summary.best_trial.model_path
print(f"Best model: {summary.best_trial.model_description}")
print(f"SMAPE: {summary.best_trial.metrics['smape']:.4f}")
print(f"Notebook: {summary.best_trial.notebook_path}")
```

Let us walk through what this code does. You load a table with columns for date, store ID, and sales. You call `automl.forecast` and tell it which column to predict, which column has the dates, and which column identifies each store. The horizon parameter specifies how many days

ahead forecast. AutoML tries different algorithms such as Prophet, ARIMA, and LightGBM and returns the best one.

The other task types follow the same pattern. For classification, call `automl.classify` with your data and the target column. For regression, call `automl.regress`. The interface is the same, only the algorithms and metrics change.

To try AutoML hands-on, the official tutorial shows you how to run classification, regression, and forecasting experiments with just a few lines of code: `https://docs.databricks.com/en/machine-learning/automl/index.html`.

The other task types follow the same pattern. For classification, call `automl.classify` with your data and the target column. For regression, call `automl.regress`. The interface is the same, only the algorithms and metrics change.

## Using AutoML Results

Once AutoML finishes, you can either deploy the best model directly to a Model Serving endpoint or open the generated notebook to further customize it. The notebook shows every step, from data preprocessing to evaluation, so you can add features, adjust the search space, or change the train/test split. AutoML also generates SHAP plots that show which features have the most impact on predictions.

### When to Use AutoML

AutoML is not the right tool for every situation. It works best when you need a model quickly, want to compare many approaches at once, or have a team with limited machine learning expertise.

| Scenario | AutoML Benefit |
|---|---|
| Limited ML expertise | Automates algorithm selection |
| Many similar problems | Scale to hundreds of models |
| Rapid prototyping | Quick baseline in minutes |
| Feature exploration | Automatic feature importance |
| Reproducibility | Generated notebook documents process |

*Table 12.6: Summary of when AutoML helps and when other approaches might be better.*

Here is a real-world example. A retail chain with 500 stores needs daily demand forecasts. Building a separate model for each store by hand would take weeks. With AutoML, they pass in the sales data with `store_id` as the identity column. AutoML builds forecasts for all 500 stores in 30 to 60 minutes. The team schedules the run weekly, so forecasts update automatically as new sales data arrives.

In the next section, we will explore Genie and how it enables business users to query data using natural language.

# Genie: Conversational Analytics for Business Users

Not everyone in your organization knows SQL. But almost everyone needs data. Sales managers want to see quarterly numbers. Executives need answers during meetings. Operations teams track daily metrics. Today, they all wait for an analyst to write a query.

## What is Genie?

Genie changes this. It lets business users ask questions in plain English. Behind the scenes, a large language model converts the question into SQL, runs the query against your Unity Catalog tables, and returns results with charts. Users get answers in seconds without writing a single line of code.

Genie works with any table or dashboard in Unity Catalog. It respects permissions, so users only see data they are allowed to access. Over time, Genie learns your organization's language. If someone asks about "cash cow customers" and Genie does not know what that means, it asks. Once you explain that it means customers spending over $5,000 per month, Genie remembers it for next time.

## How It Works

When a user types a question, Genie breaks it down into parts. It identifies the entities (customers, products), the time periods (last month, Q4), and the type of answer needed (a total, a list, a comparison). Then it looks at the table schemas to understand the data structure. It generates SQL, runs the query, and formats the results with a suggested chart.

## Example and Use Cases

Here is what a typical conversation with Genie looks like.

- User: What was our total revenue last month?
- Genie: Total revenue for December 2024 was $2.5 million. It also shows a line chart of daily revenue for the month.
- The user then asks a follow-up question.
- User: What about in the West region?

Genie remembers the context from the previous question. It knows the user is still asking about last month's revenue, but now filtered to the West region. There is no need to repeat the full question.

There are some limitations to keep in mind. Genie works best for analytical queries like totals, counts, and comparisons. It is not designed for data modification. It also works better when your table schemas are well documented with clear column names and descriptions.

The most common setup is a Genie space for a specific team. A sales team gets a Genie space connected to their revenue and pipeline data. An executive team gets connected to company-wide dashboards. Instead of waiting for an analyst, they ask Genie directly and get answers in seconds.

Building AI applications with DatabricksGenerative AI has changed what we can build with software. Large language models (LLMs) can write text, answer questions, summarize documents, and help with complex reasoning tasks. But using them in real business applications is harder than it looks.

You need more than just API access to an LLM. You need to connect the model to your private data. You need to control costs and track who uses what. You need to measure the quality of answers. And you need to deploy and monitor everything in production.

## Mosaic AI Platform Overview

Mosaic AI is the Databricks platform for enterprise generative AI. It gives you all the tools you need to build, test, and run AI applications in production.

The platform starts with Foundation Model APIs, which give you access to popular LLMs like Llama 3, Claude, and Mistral with no infrastructure to manage. Vector Search converts your documents into embeddings so you can find content by meaning, not just keywords. This is the key building block for RAG applications. The Agent Framework takes things further by letting you build AI agents that use tools, remember conversations, and follow multi-step workflows. AI Gateway routes all LLM calls through one central place to track costs, set usage limits, and log every interaction. To measure quality, Evaluation tools use LLM judges to score your application's responses. And **Model Serving** puts everything into production with auto-scaling and monitoring.

## Common Use Cases

Teams most often use Mosaic AI to build RAG chatbots that answer questions from company documents and document search tools that find similar contracts, research papers, or code by meaning. Other common applications include code assistants that generate code based on internal standards, automated support for routine customer questions, and **content generation** for summaries, reports, and marketing copy.

Now, let us start building. The first thing you need is a way to search your documents by meaning.

# What is Vector Search?

Traditional keyword search matches exact words. If a customer asks about "return policy" and your document says "refund process," keyword search will miss it. Vector Search works differently. It converts text into numbers called embeddings that capture meaning. When a user asks a question, it finds documents whose embeddings are closest. You will see exactly how this works when we set up an index in the next section.

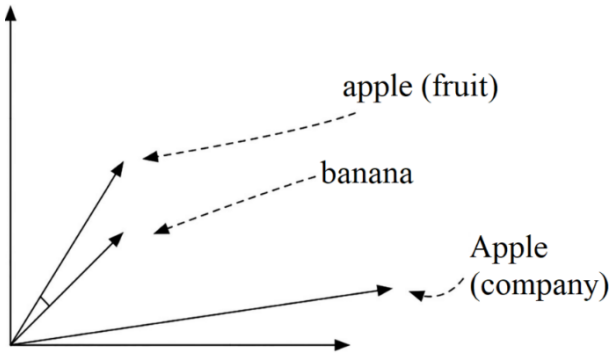



*Figure 12.9: This idea visually*

The process has two phases:

1. **Indexing**: You convert all your documents into embeddings using a model such as BGE or OpenAI embeddings. These embeddings are stored in a vector index for fast search.
2. **Searching**: When a user asks a question, the system converts the question into an embedding. Then it finds the documents with the closest embeddings and returns them ranked by similarity score.

## Creating a Vector Index

Here is how to set them up:

```
# Install the Vector Search SDK (not pre-installed on ML clusters)
%pip install databricks-vectorsearch
dbutils.library.restartPython()
from databricks.vector_search.client import VectorSearchClient
# Create the client
client = VectorSearchClient()
# Step 1: Create a search endpoint (one-time setup)
```

```
client.create_endpoint(name="doc_search_endpoint", endpoint_type="STANDARD")
# Step 2: Create an index on your documents table
# The source table must have a primary key and a text column
index = client.create_delta_sync_index(
    endpoint_name="doc_search_endpoint",
    source_table_name="main.docs.knowledge_base",
    index_name="main.docs.knowledge_index",
    primary_key="doc_id",
    embedding_source_column="content",
    embedding_model_endpoint_name="databricks-bge-large-en"
)
# The index syncs automatically when the source table changes
```

The `create_delta_sync_index` method does the heavy lifting. It reads every row from your source table, converts the `content` column into embeddings, and stores them in the index. When you add or update rows in the source table, the index updates automatically.

For a step-by-step guide to creating endpoints, building vector indexes, and running similarity searches, see the official documentation: `https://docs.databricks.com/en/generative-ai/vector-search/index.html`.

To use Vector Search, you need two things: an endpoint and an index. The endpoint is the compute that runs the search. The index holds your document embeddings.

## Searching the Index

Once the index is ready, you can search it with a question:

```
# Get the index
index = client.get_index("doc_search_endpoint", "main.docs.knowledge_index")
# Search by meaning
results = index.similarity_search(
    query_text="How do I return a product?",
    columns=["content", "title"],
    num_results=5
)
# Print results with similarity scores
for doc in results.get("data_array", []):
    print(f"Score: {doc[3]:.4f} - {doc[1]}")
```

The `similarity_search` method converts your question into an embedding, finds the 5 closest documents, and returns them along with their similarity scores. A higher score means a better match.

### When to Use Vector Search

Vector Search shines when keyword search falls short. In legal document search, it finds similar clauses across thousands of contracts even when lawyers used different words. For customer support, it matches questions to the right help article by meaning, not exact keywords. Research discovery benefits too, since you can find related papers based on what they are about, not just their titles. Even **code search** works, finding functions that do similar things across a codebase, regardless of variable names or coding style.

Now that you can search documents by meaning, let us use this to build something more powerful: a RAG application.

## What is RAG?

RAG stands for Retrieval-Augmented Generation. Without RAG, an LLM can only answer from its training data. With RAG, you first search for relevant documents, then include them in the prompt so the LLM answers based on your data. The answers are accurate, up to date, and specific to your organization.

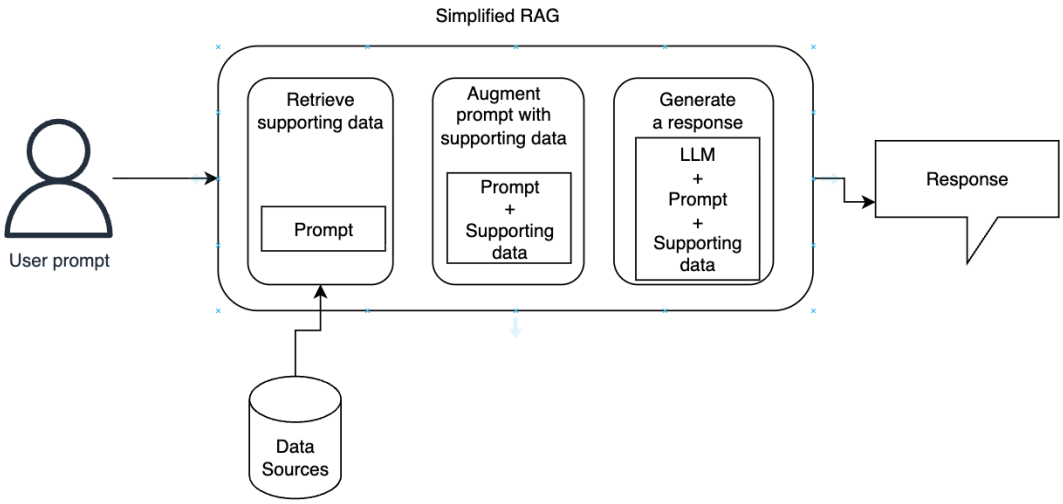



*Figure 12.10: How the RAG pattern works.*

## RAG vs Fine-Tuning: Which Should You Use?

Use RAG when you need the LLM to answer from your documents. It is easier to set up and updates instantly when documents change. Use fine-tuning when you need the model to write in a specific style or learn domain language. For most teams, RAG is the right starting point.

## Building a Simple RAG Function

For a complete RAG tutorial with working code, including document retrieval, prompt building, evaluation, and conversation memory, see the Databricks GenAI Cookbook: `https://docs.databricks.com/en/generative-ai/tutorials/ai-cookbook/index.html`.

Here is a basic RAG function that searches for documents and asks the LLM to answer based on them:

```
import mlflow.deployments
llm = mlflow.deployments.get_deploy_client("databricks")
def rag_answer(question: str) -> str:
    # Step 1: Search for relevant documents
    search_results = index.similarity_search(
        question,
        columns=["content", "title"],
        num_results=3
    )
    # Step 2: Combine the documents into context
    context = "\n\n".join([
        f"{doc[1]}\n{doc[0]}"
        for doc in search_results.get("data_array", [])
    ])
    # Step 3: Build a prompt with the documents
    prompt = f"""Answer based only on these documents.
    If the answer is not in the documents, say "I
don't                                                have that information."
    Documents:
    {context}
    Question: {question}
    Answer:"""
    # Step 4: Send to the LLM
    response = llm.predict(
        endpoint="databricks-llama-3-70b-instruct",
        inputs={
            "messages": [{"role": "user", "content": prompt}],
```

```
            "max_tokens": 500
        }
    )
    return response["choices"][0]["message"]["content"]
```

Let us walk through what happens when a user asks, "How do I return a product?":

## Adding Conversation Memory

A real chatbot needs to remember what was said earlier. If a user asks, "What is the return policy?" and then follows up with "Does that apply to electronics?", the bot needs to know what "that" refers to.

To handle this, keep a history of recent messages. When building the prompt for each new question, include the last few exchanges along with the retrieved documents. This way, the LLM has both the relevant documents and the conversation context.

Here is the general approach:

1. Store each user message and assistant response in a `history` list.
2. For each new message, search for relevant documents as before.
3. Build the prompt with a system message, the retrieved documents, the last 5 conversation turns from history, and the current user question.
4. The LLM can now answer follow-up questions correctly because it sees the full conversation.

In production, add error handling and citation tracking so users can see where each answer came from.

## Evaluating RAG Quality

Before you deploy an RAG application, you need to know if it gives good answers. MLflow provides evaluation tools that use LLM judges to score your responses.

An LLM judge is another language model that reads the question, the expected answer, and your RAG response. Then it scores the response on several dimensions:

- **Relevance**: Does the answer address the question?
- **Correctness**: Is the answer factually accurate?
- **Groundedness**: Is the answer supported by the retrieved documents?

Here is how to run an evaluation:

```
import pandas as pd
import mlflow
```

```
# Create test data with questions and expected answers
eval_data = pd.DataFrame({
    "question": [
        "What is the return policy?",
        "How do I reset my password?"
    ],
    "expected_answer": [
        "30-day returns with receipt for full refund",
        "Click 'Forgot Password' on login page"
    ]
})
# Run evaluation with LLM judges
results = mlflow.evaluate(
    model=rag_answer,
    data=eval_data,
    evaluators="default",
    evaluator_config={
        "col_mapping": {
            "inputs": "question",
            "ground_truth": "expected_answer"
        }
    }
)
# View the scores
print(results.metrics)  # relevance, correctness, groundedness
```

Run this evaluation regularly. It helps you catch problems early, before your users do.

## AI Gateway: Governance and Cost Control

Without central control, LLM costs become invisible, there is no audit trail for compliance, and teams use the wrong models for the wrong tasks. AI Gateway solves this by routing all LLM calls through a single point.

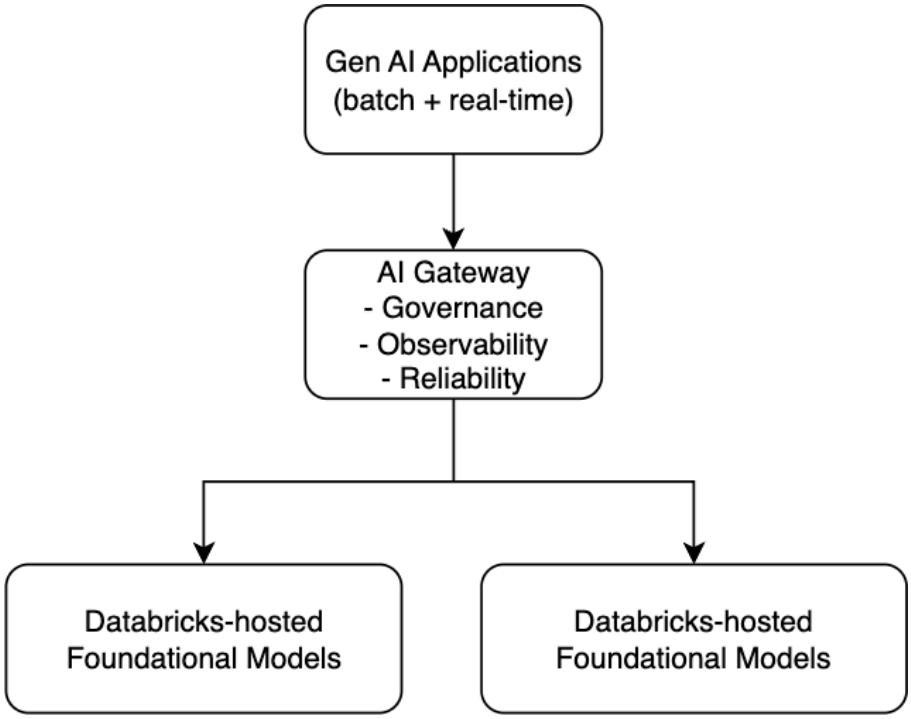



*Figure 12.11: How AI Gateway fits into the architecture*

## What AI Gateway Does

AI Gateway provides four key capabilities. Cost tracking logs every LLM call with token counts and cost, so you can see reports by user, team, or application and set alerts when budgets are exceeded. Rate limiting sets a maximum number of calls per user or application, preventing runaway usage and accidental loops. Audit logging records every prompt and response for compliance and debugging. **Model routing** lets you route simple questions to a cheaper model and complex ones to a more capable one.

## Setting Up AI Gateway

Here is an example:

```
from databricks.sdk import WorkspaceClient
from databricks.sdk.service.serving import (
    EndpointCoreConfigInput,
    ServedEntityInput,
    RateLimits
)
```

```
w = WorkspaceClient()
# Create a gateway endpoint that wraps OpenAI GPT-4
w.serving_endpoints.create(
    name="company_ai_gateway",
    config=EndpointCoreConfigInput(
        served_entities=[
            ServedEntityInput(
                external_model={
                    "name": "gpt-4",
                    "provider": "openai",
                    "openai_config": {
                        "openai_api_key": "{{secrets/ai/openai_key}}"
                    }
                }
            )
        ]
    ),
    # Limit each user to 1,000 calls per day
    rate_limits=[
        RateLimits(calls=1000, renewal_period="DAY", key="user")
    ]
)
```

For a complete guide to creating gateway endpoints, setting rate limits, and querying usage logs, see the official documentation: `https://docs.databricks.com/en/ai-gateway/index.html`.

To set up AI Gateway, you create a serving endpoint that wraps an external model. You store the API key in Databricks secrets and reference it in the endpoint configuration. You can also attach rate limits.

## Using the Gateway

Once the gateway is set up, applications call the gateway endpoint instead of the LLM provider directly. The gateway handles rate limits, logging, and cost tracking behind the scenes.

```
import mlflow.deployments
client = mlflow.deployments.get_deploy_client("databricks")
response = client.predict(
    endpoint="company_ai_gateway",
    inputs={
        "messages": [{"role": "user", "content": "Summarize this report..."}],
```

```
        "max_tokens": 200
    }
)
```

To see how your teams are using the gateway, query the logs:

```
SELECT
    requester,
    DATE(event_time) AS request_date,
    COUNT(*) AS calls,
    SUM(total_tokens) AS total_tokens
FROM system.ai_gateway.usage
WHERE event_time >= current_date() - INTERVAL 30 DAYS
GROUP BY requester, DATE(event_time)
ORDER BY total_tokens DESC
```

This query shows each user's LLM usage over the last 30 days. You can use this data to find heavy users, optimize model choices, and control costs.

# Deployment and Monitoring

You have built an RAG application and tested it with an evaluation. Now it is time to put it into production.

## Deploying AI Agents

For a step-by-step guide to building and deploying AI agents with MLflow, including wrapping your application as a model and creating serving endpoints, see: `https://docs.databricks.com/en/generative-ai/agent-framework/create-agent.html`.

The serving endpoint gives you a REST API. It auto-scales based on traffic. You can also enable scale-to-zero so you do not pay when there are no requests.

Once the endpoint is running, two things keep it healthy. MLflow Tracing lets you add spans around each step of the RAG flow (search, prompt building, LLM call) so you can see where problems or slowdowns occur. For ongoing quality, enable inference logging on the endpoint. This writes every input and output to a Delta table. A quality monitor uses LLM judges to score responses automatically and alerts you when quality drops below a threshold.

To learn more about tracing and observability for GenAI applications, see: `https://docs.databricks.com/en/mlflow/mlflow-tracing.html`.

To deploy your RAG application, wrap it in an MLflow model. This means you create a Python class that loads your resources (vector index, LLM client) and runs the RAG flow when called.

```
class SupportChatbot(mlflow.pyfunc.PythonModel):
    def load_context(self, context):
        # Load resources once when the model starts
        self.index = vsc.get_index(
            "doc_search_endpoint",
            "main.docs.knowledge_index"
        )
        self.llm = mlflow.deployments.get_deploy_client("databricks")
    def predict(self, context, model_input):
        # Run the RAG flow for each request
        question = model_input["question"].iloc[0]
        docs = self.index.similarity_search(question, num_results=3)
        # Build prompt with documents, call LLM
        # ... (same RAG logic as before)
        return {"answer": response}
# Log and register the model
with mlflow.start_run():
    mlflow.pyfunc.log_model(
        artifact_path="chatbot",
        python_model=SupportChatbot(),
        registered_model_name="main.agents.support_chatbot"
    )
# Create a serving endpoint
# w.serving_endpoints.create(name="support_chatbot_api", config={...})
```

# Summary

Databricks brings data, ML, and AI together on one platform. You now understand how to track experiments with MLflow, share features across teams with Feature Store, build models quickly with AutoML, and give business users self-service analytics with Genie. On the generative AI side, you learned how to search documents by meaning with Vector Search, build RAG applications that answer from your own data, and control LLM costs with AI Gateway.

The key takeaway is that these tools share a common foundation. Unity Catalog governs all assets. MLflow tracks experiments for both traditional ML and generative AI. Model Serving deploys everything through the same infrastructure. This means you can start with one capability and expand to others without learning a new set of tools.

## Next Steps

Where you go from here depends on what you want to build.

If your focus is traditional machine learning, a good first project is an end-to-end pipeline that uses MLflow for tracking and Feature Store for shared features. Try running AutoML on a real forecasting or classification problem from your organization. Once a model is in production, set up monitoring to catch drift before it affects your results.

If you are excited about generative AI, the best way to start is by building a RAG chatbot for your team's internal documentation. It is a practical project that delivers value quickly. As more people in your organization start using LLMs, set up AI Gateway early so costs stay visible and under control.

If you work closely with business users, focus on creating Genie spaces for the departments that ask the most questions. Write down your domain terms and business definitions so Genie understands your language from the start. A small investment in setup saves hours of back-and-forth later.

The AI and ML landscape moves fast. Databricks adds new capabilities regularly. The concepts and patterns you learned in this chapter give you a solid foundation to build on. Check the official documentation for the latest features as you start building.

## Get this book's PDF copy, code bundle, and more

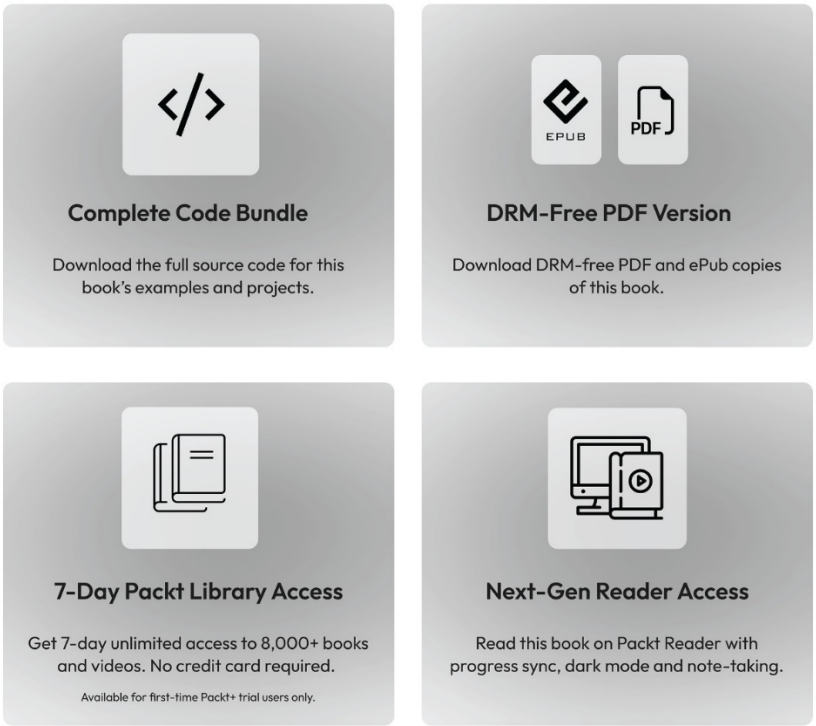
Scan the QR code (or go to `packtpub.com/unlock`). Search for this book by name, confirm the edition, and then follow the steps on the page.

*Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.*

# 13

# Unlock Access to the Code Bundle and the PDF Version

Your copy of this book includes the following exclusive benefits:




Follow this guide to unlock them. The process takes only a few minutes and only needs to be completed once.

# Unlock this book's free benefits in three easy steps

## Step 1

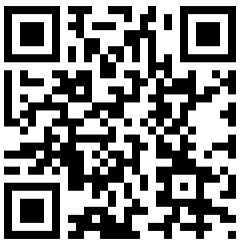
Have your purchase invoice ready for *step 3*. If you have a physical copy, scan it using your phone and save it as a PDF, JPG, or PNG.

For more help on finding your invoice, visit `https://www.packtpub.com/unlock-benefits/help`.

> **Note**
>
> If you bought this book directly from the Packt website, no invoice is required. After *step 2*, you can access your exclusive content right away.

## Step 2

Scan the QR code or go to `packtpub.com/unlock`.



On the page that opens (similar to *Figure 13.1* on desktop), search for this book by name and select the correct edition.

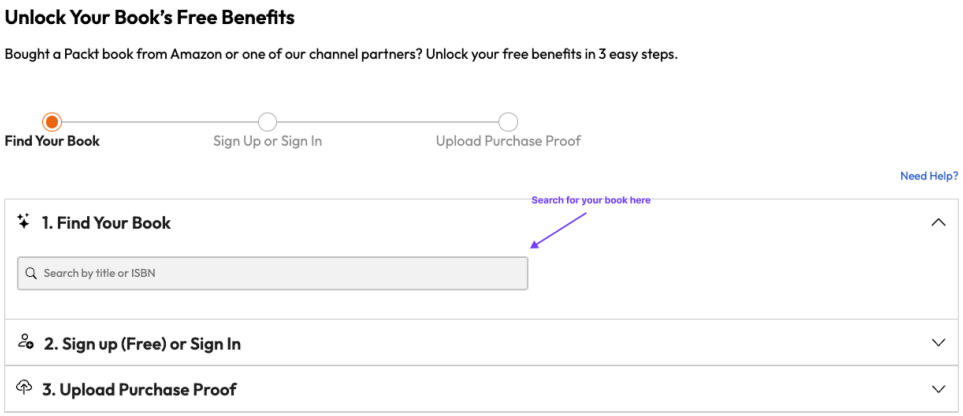



*Figure 13.1: Packt unlock landing page on desktop*

## Step 3

After selecting the book, sign in to your Packt account or create one for free. Then, upload your invoice (PDF, PNG, or JPG, up to 10 MB). Follow the on-screen instructions to finish the process.

## Need help?

If you get stuck and need help, visit `https://www.packtpub.com/unlock-benefits/help` for a detailed FAQ on how to find your invoice and more. This QR code will take you to the help page.

**Note**

If you are still facing issues, reach out to `customercare@packt.com`.

packtpub.com

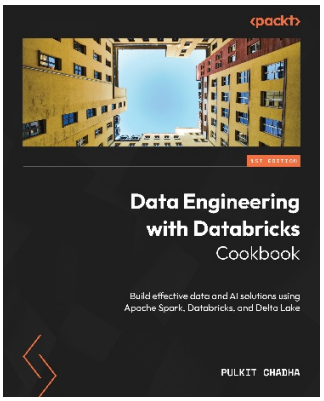
Subscribe to our online digital library for full access to over 7,000 books and videos, as well as industry leading tools to help you plan your personal development and advance your career. For more information, please visit our website.

## Why subscribe?

- Spend less time learning and more time coding with practical eBooks and Videos from over 4,000 industry professionals
- Improve your learning with Skill Plans built especially for you
- Get a free eBook or video every month
- Fully searchable for easy access to vital information
- Copy and paste, print, and bookmark content

At packtpub.com, you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on Packt books and eBooks.

# Other Books You May Enjoy

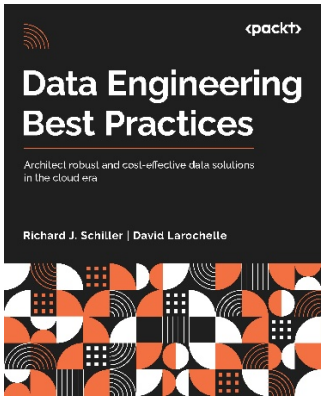



**Data Engineering with Databricks Cookbook**

Pulkit Chadha

ISBN: 978-1-83763-335-7

- Perform data loading, ingestion, and processing with Apache Spark
- Discover data transformation techniques and custom user-defined functions (UDFs) in Apache Spark
- Manage and optimize Delta tables with Apache Spark and Delta Lake APIs
- Use Spark Structured Streaming for real-time data processing
- Optimize Apache Spark application and Delta table query performance
- Implement DataOps and DevOps practices on Databricks
- Orchestrate data pipelines with Delta Live Tables and Databricks Workflows
- Implement data governance policies with Unity Catalog

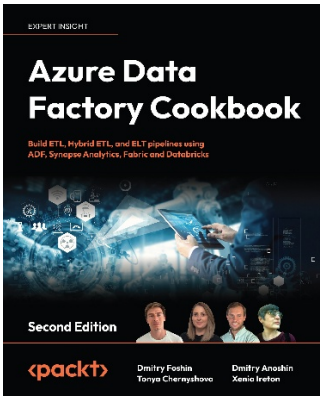



**Data Engineering Best Practices**

Richard J. Schiller, David Larochelle

ISBN: 978-1-80324-736-6

- Architect scalable data solutions within a well-architected framework
- Implement agile software development processes tailored to your organization's needs
- Design cloud-based data pipelines for analytics, machine learning, and AI-ready data products
- Optimize data engineering capabilities to ensure performance and long-term business value
- Apply best practices for data security, privacy, and compliance
- Harness serverless computing and microservices to build resilient, scalable, and trustworthy data pipelines




**Azure Data Factory Cookbook**

Dmitry Foshin, Tonya Chernyshova, Dmitry Anoshin, Xenia Ireton

ISBN: 978-1-80324-182-1

- Build and Manage data pipelines with ease using the latest version of ADF
- Configure, load data, and operate data flows with Azure Synapse
- Get up and running with Fabric Data Factory
- Working with Azure Data Factory and Azure Purview
- Create big data pipelines using Databricks and Delta tables
- Integrate ADF with commonly used Azure services such as Azure ML, Azure Logic Apps, and Azure Functions
- Learn industry-grade best practices for using Azure Data Factory

## Packt is searching for authors like you

If you're interested in becoming an author for Packt, please visit `authors.packt.com` and apply today. We have worked with thousands of developers and tech professionals, just like you, to help them share their insight with the global tech community. You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

## Share your thoughts

Now you've finished *Data Engineering with Azure Databricks*, we'd love to hear your thoughts! Scan the QR code below to go straight to the Amazon review page for this book and share your feedback or leave a review on the site that you purchased it from.

*https://packt.link/r/180610637X*

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

# Index

## Symbols


**_delta_log directory**
role 168


## A


**ACID guarantees, in Delta Lake 168**
_delta_log directory, role 168
concepts and components 168
Delta Log 165 – 167
delta log scale, reading 169, 170
delta log, consideration 169
failed operations recorded 169
handle rollbacks 169
transactions, writing to delta log 168

**ACID transactions**
enabling, for reliable data processing 165

**ADLS batch ingestion 72, 73**

**AI Agents**
deploying 359, 360

**AI Gateway 334, 356**
deployment and monitoring 359
key capabilities 357
setting up 357, 358
using 358, 359

**AUTO CDC**
approach, using 187, 188

**Access Control Lists (ACLs) 51**

**Adaptive Query Execution (AQE)**
Dynamic Re-optimization 287
features 287

**Agent Bricks 334**

**Airflow**
Databricks, integrating with 232

**Amazon Redshift 13**

**Amazon Web Services (AWS) 5**

**Apache Airflow Integration pattern 234 – 236**

**Apache Spark 7, 8, 100**
cluster architecture 100
core components 7, 8, 100
execution memory 107
execution model 103, 104
job scheduling 105
memory management 107
storage memory 107
unified memory management system 108
used, for resolving issues 7

**Apache Spark Structured Streaming 139, 140**

**AutoML 334, 346**
need for 348
using 348

**Autoloader**
Blob Storage connection, configuring 151
files, processing 152
need for 154
notebook, creating 151
setting up 152
using, for file streaming 151
versus Event Hubs 153

**Autoloader optimization 159**
parameters 159
performance, monitoring 160
performance, tuning 160

**Azure Blob Storage 65**
account, creating 146
container, creating 146
setting up 146

**Azure Cosmos DB 94**

**Azure Data Factory (ADF) 75**
capabilities 75
Copy activity pattern 76, 77
Databricks, integrating with 232
to Databricks patterns 77, 78

**Azure Data Factory integration architecture 232**
Apache Airflow Integration pattern 234 – 236
orchestration tool comparison and selection 236 – 238
pipeline development 233, 234

**Azure Data Lake Storage (ADLS)** **29**
**Azure Data Lake Storage (ADLS) Gen2** **65**
**Azure Databricks**
runtime enhancements 109, 110
**Azure Databricks Streaming**
features 141
**Azure DevOps**
CI/CD pipelines, implementing 268
reference link 251
**Azure Event Hub**
creating 144, 145
namespace, creating 144
shared access policy, creating 145
**Azure Private Link** **302**
**Azure SQL**
full table load example 81
sample data, setting up 80
**Azure Synapse Analytics** **86, 87**
**Azure account**
Autoloader, using for file streaming 151
Azure Blob Storage, setting up 146
Azure Event Hub, creating 144
prerequisites 144
setting up 144
streaming job, writing 147
**Azure diagnostic settings**
alerts, setting up 317, 318
configuring 317
**Azure streaming patterns** **142, 143**
**adaptive query execution (AQE)** **105, 110, 117, 127, 286**
**advanced optimization techniques** **116, 117**
**advanced streaming concepts** **156**
late data and fault tolerance, handling 157
stateful streaming, managing 156
**analytical workloads** **113**
**audit dashboard**
building 315, 316
**authentication mechanisms** **87**
**auto-scaling** **125**
configuration 127
**automated deployment pipelines** **128**
**automatic cluster management** **110**
**autoscaling**
strategy 290
structured streaming 292

# B

**BigQuery** **13**
**Bronze, Silver, and Gold tables**
building 42 – 44
data quality 45
**Business Associate Agreement (BAA)** **321**
**back-end Private Link** **302**
**batch ingestion** **63**
characteristics 64
scenarios 64
**batch processing** **122**
versus data streaming 132 – 134
**branching strategy** **252**
approach 252 – 254
GitFlow 252
Trunk-Based Development 252
**broadcast joins** **116, 119**
**bronze layer** **200 – 202**
**browser authentication Private Link** **302**
**bucketing** **117**

# C

**CI/CD pipelines**
creating 269 – 275
implementing, with Azure DevOps 268
**CI/CD pipelines, with Azure DevOps**
free parallel jobs, requesting 269
prerequisites 268
**CPU optimization** **126**
**Catalyst Optimizer** **106**
**Cluster Costs**
configuration and sizing 291
managing 290
**Cluster Manager** **102**
**Comma-Separated Values (CSV)** **67**
advantages 67
best practices 68
**Common Table Expressions (CTEs)** **120**
**Composer** **13**
**Cost-Based Optimization (CBO)** **119**
**caching** **113 – 115, 159**
**caching techniques** **286**
Photon 286

reads, accelerating 288, 289
scenarios 290
**change data capture (CDC)**
in Delta Lake 185
significance, for analytical use cases 185, 186
**change data capture (CDC), in Delta Lake**
change data feed, enabling 189
Delta table, as target for 186 – 188
queries, streaming 190
**change data feed**
enabling 189
**classic compute**
versus serverless compute 56
**classic compute access mode 56**
**cloud storage**
access connector, creating for Azure Databricks 33, 34
account, creating 31
catalog design, considerations 37 – 39
catalog design, creating 37 – 39
connecting 29
credentials, creating with managed identity 36
external location, creating 36, 37
external volume, creating for staging 41, 42
storage containers, creating 32
Unity Catalog storage integration, overview 29 – 31
volume, creating 39, 40
**cluster management 122 – 125**
**cluster policies 293, 305**
cost control 293
**cluster pools 291**
**code optimization 122**
**column masking 307, 308**
**column pruning 118**
**columnar storage formats 127**
**complex access patterns**
customer analytics table, securing 312 – 314
dynamic views 310 – 312
**compliance framework**
building 322, 323
**comprehensive monitoring 127**
**compute option 55**
**custom partitioning strategies 127**
**customer analytics table**
securing 312
**customer-managed keys (CMK) 304**

# D

**Data Protection Impact Assessment (DPIA) 326**
**DataFrame API**
best practices 118, 119
**DataFrames 102**
**Databricks 9 – 13**
Azure-Native Setup 10
Delta Lake & Lakehouse Foundation 11
features 9, 10
Hybrid Control Plane & Compute Plane 10
integrating, with Airflow 232
integrating, with Azure Data Factory (ADF) 232
Interactive Collaboration 10
ML & AI Integration 11
reference architectures 13 – 15
SQL Warehousing 11
use cases 15
**Databricks AI/ML innovation and capabilities 332**
**Databricks AI/ML stack 333**
**Databricks Account Admin 25**
**Databricks Asset Bundles (DABs) 228**
**Databricks CLI commands 260**
**Databricks DevOps**
best practices 250
**Databricks GenAI Cookbook**
reference link 354
**Databricks Job activity 78**
**Databricks UI**
using 154
**Databricks accounts and workspace**
provisioning 22, 23
setting up 26, 27
user admin, creating 25, 26
**Databricks environment**
double encryption 305
encryption, at rest 304
encryption, in transit 305
network security, with VNet injection 301 – 303
securing 300
workspace-Level security settings 305, 306

**Databricks features**
additional requirements 27, 28
use cases 27 – 29
**Dataflow 13**
**Dataproc 13**
**Declarative Automation Bundle (DABs)**
CI/CD pipelines, creating 269
CI/CD pipelines, implementing with Azure DevOps 268
defining 260
observability 275
prerequisites 260
resources/ directory, defining 263 – 266
src/ directory 266
test/ directory 266, 267
test/ directory, deploying 267, 268
**Declarative Automation Bundle (DABs), prerequisites**
Databricks CLI commands 260
databricks.yml file 262
project structure 260
**Declarative Automation Bundles (DABs) 229**
**Delta Lake**
approach, to schema evolution 177, 178
approach, to schema enforcement 174 – 176
change data capture (CDC) 185
implements consistency 170, 171
implements durability 173
implements isolation 171, 172
retention and data lifecycle 185
time travel, significance 180
version data 180
**Delta Lake integration 109**
**Delta Lake, version data**
historical data, querying with SQL API 181
multiple table versions, creating 181, 182
querying, by timestamp 183
querying, by version number 183
table state, restoring 184
**Delta Live Tables (DLT) 141, 193**
**Delta Log 165 – 167**
considerations 169
failed operations recorded 169
handle rollbacks 169
scale, reading 169, 170
transactions, writing to 168
**Disk cache 110, 289**
**data access**
audit dashboard, building 315, 316
auditing 314
monitoring, with Azure diagnostic settings 317
Unity catalog system tables 314, 315
**data engineering**
evolution 2 – 6
**data lineage 324**
**data partitioning 158**
strategies 111, 112
**data quality 215**
best practices 219
implementing 216
managing, with expectations 215
types 215
**data sharing**
external sharing, with delta sharing 327 – 329
internal sharing, across workspaces 327
securing, with delta sharing 326
**data skew 126**
**data skipping 282**
**data streaming 132**
use cases 134, 135
versus batch processing 132 – 134
**databricks.yml file 262**
key components 262
**decorator 209**
**dedicated (Single-User) access mode 56**
**delta lake**
significance 164, 165
**directed acyclic graph (DAG) 101, 195**
**double encryption 305**
**dynamic partition pruning 117**

# E

**Elastic MapReduce 13**
**Event Hubs**
need for 153
versus Autoloader 153
**Event Hubs connection, configuring**
dummy events, generating 150
streaming data, processing 149
streaming data, reading 148, 149
test events 151
**early filtering 118**

**enhanced autoscaling** **292**
**environment separation** **254**
approach 255
**error handling** **89**
**executors** **102**

## F

**Feature Store** **333, 342, 343**
AutoML, using 348
**Feature Store, core concepts** **343**
AutoML forecasting 347, 348
AutoML process 346
supported tasks 346
try it feature 344, 345
**Feature Store, try it feature**
AutoML for rapid model development 346
use case, example 345
**Foundation Model APIs** **334**
**file formats**
Comma-Separated Values (CSV) 67, 68
comparing 69
JavaScript Object Notation (JSON) 68
Parquet 69
reading 66
**file-based partitioning** **112**
**filter ordering** **119**
**fine-grained access control, with Unity Catalog** **306**
column masking 307 – 309
dynamic views, for complex access patterns 310 – 312
row-Level security, with row filters 309, 310
**fine-tuning**
versus Retrieval-Augmented Generation (RAG) 354
**front-end Private Link** **302**

## G

**GenAI applications, tracing and observability**
reference link 359
**General Data Protection Regulation (GDPR)** **318**
data classification and discovery 320, 321
right to erasure, with Delta Lake 318, 319
**Generative AI Platform** **334**
**Genie** **334, 349**
Mosaic AI platform, overview 350
use case, example 349, 350
use cases 350
working 349
**Git Integration for Databricks**
branching strategy 252
environment separation 254
setting up 250, 251
**GitFlow** **252**
**Glue** **13**
**Google Cloud Platform (GCP)** **5**
**Grouped map UDFs** **122**
**garbage collection** **109**
optimization 127
**gold layer** **206**

## H

**Health Insurance Portability and Accountability Act (HIPAA)** **318**
health information, protecting 321, 322
**hash joins** **119**
**hash partitioning** **112**
**high-availability configuration** **127**
**high-performance streaming systems**
Autoloader optimization 159
building 158
caching 159
data partitioning 158
resource tuning 159
**horizontal scaling** **123**

## I

**IP access lists** **305**
**Infrastructure as Code** **128**
**Infrastructure as Code, with Terraform** **255, 256**
prerequisites 256
project resources, creating 257
**Iterator UDFs** **122**
**identity management** **45**
access control 50
groups 47
permission matrix 52 – 54
privileges, setting up 51
SQL warehouse 57

user groups, creating 48
user provisioning 46, 47
**identity management, access control**
ACLs, managing 51
entitlements, managing 50
**incremental loading pattern 70, 71, 83**
**instance types 127**

# J

**JDBC connectivity 79**
**JSONPlaceholder API ingestion 90, 94**
**Java Database Connectivity (JDBC) 79**
**Java Virtual Machine (JVM) 102**
**JavaScript Object Notation (JSON) 68**
advantages 68
**join hints 120**
**join ordering 120**
**join-heavy workloads 113**

# L

**LakeFlow Spark Declarative Pipelines 194**
**Lakeflow Jobs for task scheduling**
advanced job patterns and optimization 231, 232
architecture 229
job creation and configuration 229 – 231
using 229
**Lakeflow SDP**
benefits 195
core concepts 196, 197
usage 196
**Lakeflow pipeline**
bronze layer 200 – 202
building 197
configuring 210
creating 210
data quality metrics, viewing 213, 214
execution, monitoring 211, 212
gold layer 206 – 208
results, verifying 213
running 210
scenarios 197, 198
silver layer 202 – 205
starting 211
tables, versus materialized views 209
**Lakeflow pipeline, performance**
automatic optimization 220
clustering strategy 221
joins and aggregations, optimizing 222
optimizing 220
pipeline configuration 221
pipeline monitoring 223
**Lakeflow pipeline, scenarios**
code building 199
data, setting up 198, 199
imports and configuration 200
**Lakehouse Monitoring 334**
**Large Language Models (LLMs) 6**
**Least Recently Used (LRU) 116**
**Lineage Flow**
example 335
**Liquid Clustering**
advanced processing layer, tuning with Spark 285
User-Defined Functions (UDFs) 286
**late data and fault tolerance**
checkpointing 158
exactly-once processing 158
handling 157
watermarks 157, 158
**lazy evaluation optimization 127**
**liquid clustering 113, 221, 284**

# M

**MERGE INTO**
approach, using 187
**MLflow 333 – 336**
core concepts 336
use cases 342
**MLflow, core concepts**
experiment tracking 336 – 339
model registry 337
**MLflow, model registry and deployment 339**
model serving deployment 341
model versions, managing with Aliases 340
model, registering 340
**Machine Learning (ML) 333**
**Massively Parallel Processing (MPP) 3**
**Microsoft Entra ID 110**
**Microsoft-managed keys (MMKs) with AES-256 304**

**Model Serving** **334**
**Mosaic AI Agent Framework** **334**
**materialized views**
versus tables 209
**memory configuration** **108, 125**
**memory optimization** **109**
**metastore** **21, 22**
**micro partitioning** **12**
**mixed workloads** **113**
**model routing** **357**

## N

**Near Real-Time streaming** **136, 137**
type, selecting 137, 138
versus Real-Time streaming 135
**Network Security Groups (NSGs)** **301**
**Notebook Modularization**
best practices 238
comprehensive monitoring infrastructure 243
debugging techniques and tools 243, 244
operational excellence and continuous improvement 245, 246
principles 238
production notebooks patterns and standards 241, 242
refactoring techniques 239, 240

## O

**OAuth 2.0**
service principal 65, 66
**off-heap storage** **109**
**optimal partitioning strategies**
selecting 113
**optimized connectors** **110**

## P

**Pandas UDF** **122, 286**
**Parquet** **69**
advantages 69
**Personal Access Token (PAT)** **251**
**Photon** **57**
**Photon Engine** **109**
**Platform-as-a-Service (PaaS)** **17**
**Protected Health Information (PHI)** **321**
**Python** **5**
**pagination patterns** **88**
**partition size analysis** **126**
**performance optimization**
tips 85, 86
**performance tuning** **280**
**performance tuning for Spark and Delta Lake** **280**
advanced storage layer optimization 281
data skipping 282
file compaction and file problem 281, 282
generated columns, partitioning 284
optimization 281
strategy, partitioning 283, 284
Z-Ordering 283
**permission matrix** **52 – 54**
classic compute access mode 56
compute option 55
dedicated (Single-User) access mode 56
Photon 57
serverless compute 55
standard (shared) access mode 56
**persistence** **116**
**personal access tokens (PATs)** **305**
**predicate pushdown** **119**
**production cluster sizing** **127**
**production deployment patterns** **127**

## Q

**quality metrics**
monitoring 217, 218

## R

**R language** **5**
**RAG function**
building 354, 355
Conversation Memory, adding 355
RAG quality, evaluating 355, 356
**Real-Time streaming** **135, 136**
versus Near Real-Time streaming 135
**Resilient Distributed Dataset (RDD)** **7, 102**
**Retrieval-Augmented Generation (RAG)** **353**
versus fine-tuning 354

**range partitioning** **112**
**rate limiting** **89**
**relational databases** **79**
**resource optimization** **126, 127**
**resource tuning** **159**
**resource utilization monitoring** **126**
**right-sizing** **291**
**robust error handling** **128**
**row filters**
used, for row-level security 309, 310
**rule-based optimizations** **119**

# S

**SFTP/FTP sources** **95**
**SQL / DataFrame tab** **296**
**SQL optimization techniques** **119 – 121**
**SQL warehouse** **57**
classic type 57
pro type 57
resources, deleting 58
serverless type 57
system tables 58
**Serverless SQL** **13**
**Service Level Agreement (SLA)** **245**
**Spark**
advanced processing layer, tuning 285
Join and Shuffle optimization 285
Shuffle Partitions, tuning 286
**Spark Cache** **289**
storage levels 289
**Spark Declarative Pipelines (SDP)** **193**
**Spark Driver** **101**
**Spark UI** **294**
identifying 294
monitoring tools 295
navigating 294
Skewed Join Key, salting 296, 297
Skewed Join, debugging 295, 296
**Spark pools** **13**
**SparkContext** **101**
**Structured English Query Language (SEQUEL)** **3**
**Symmetric multiprocessing (SMP)** **4**
**salting** **127**
**scale-in policies** **125**
**scale-out policies** **125**
**scaling strategies** **122 – 125**
**schema**
management, importances 173
**schema enforcement** **173**
Delta Lake's approach to 174 – 176
**schema evolution** **173**
automated approach 178, 179
best practices 179, 180
Delta Lake's approach to 177, 178
overwrite schema approach 178
**separate processing paths** **127**
**serverless compute** **55**
limitations 55
usage 55
versus classic compute 56
**silver layer** **202 – 205**
**smart cache eviction** **116**
**sort-merge joins** **119**
**spot instances** **292**
best practices 293
using, for cost savings 292
**standard (shared) access mode** **56**
**stateful streaming**
managing 156
need for 157
Spark Structured Streaming operations 157
state 156
**storage optimization** **126**
**streaming applications**
Databricks UI, using 154
debugging 154
monitoring 154
performance tips 155, 156
problems and solutions 155
**streaming data**
Event Hubs connection, configuring 147
notebook, creating 147
reading, from Event Hubs 147
**streaming job**
streaming data, reading from Event Hubs 147
writing 147
**streaming pipeline**
Azure account, setting up 144
building 144

**subqueries** **120**
**system tables** **58, 314**

## T

**Terraform project resources**
configuration 257, 258
creating 257
deploying 258, 259
project structure 257
**Trunk-Based Development** **252**
**tables**
versus materialized views 209
**task execution time analysis** **126**
**time-to-live (TTL)** **157**
**token management** **305**
**traditional data pipeline development** **194**
Lakeflow SDP, benefits 195
**transformation chaining** **118**

## U

**Unity Catalog** **18, 334**
data lineage 324
data lineage, for compliance 326
data lineage, querying 324, 325
impact analysis, of schema changes 325
key features 20
metastore 21, 22
need for 18, 19
storage integration, overview 29 – 31
used, for fine-grained access control 306
**user groups**
assigning, to workspaces (UI) 49
creating, at account level (UI) 48
**user-defined functions (UDFs)** **118, 121, 286**

## V

**VACUUM**
impact, on change data feed 190, 191
**Vector Index**
creating 351, 352
searching 352
**Vector Search** **334, 351**
need for 353
**Vectorized UDFs** **121**
**vectorized UDF** **286**
**vertical scaling** **124**

## Z

**Z-Ordering** **283**